

Iterative Algorithmen

- Schleifenkonstrukte wie `for`- und `while`-Schleifen führen zu **iterativen Algorithmen**
- beschreiben Berechnungen über einer Menge von Variablen, die in jeder Iteration aktualisiert werden
- `for`-Schleifen alleine reichen im Allgemeinen nicht aus, um alles Berechenbare zu abzudecken (`while`-Schleifen notwendig).

Mehr hierzu in Berechenbarkeitstheorie!

- **Rekursion** (nächstes Thema) erlaubt uns einige Probleme viel einfacher und eleganter zu lösen.

Rekursion im Alltag

Ein Mensch ist **natürlich geborene(r) US-Staatsangehörige(r)**, falls sie/er

- in den USA geboren ist, oder **(Basisfall)**
- Kind zweier Eltern ist, die jeweils entweder eingebürgerte oder **natürlich geborene US-Staatsangehörige** sind und mindestens ein Elternteil bereits in den USA gelebt hat.

(Rekursiver Fall)

Rekursionsprinzip

(Induktionsprinzip)

Basisfall

- Definiere/Löse triviale Instanzen direkt.

(Induktionsanfang)

Rekursiver Fall

Definiere/Löse **nicht-triviale Instanzen I**, indem

- **einfachere Instanz(en) J** definiert/gelöst wurde(n),

(Induktionsvoraussetzung)

- **I** mittels Definition(en)/Lösung(en) von **J** und einfachen Operationen definiert/gelöst wird

(Induktionsschritt)

Rekursion in der Informatik

- I) Eine Art und Weise **Objekte/Probleme** zu **beschreiben**

Beispiel wohlgeklammerte Ausdrücke.

Wohlgeklammerte Ausdrücke	Klammerpaare
()	1
()() (())	2
()()() ()(()) (())() (())() ((()))	3

Rekursive Definition?

Wohlgeklammerte Ausdrücke rekursiv definiert

Basisfall. Es gibt nur einen wohlgeklammerten Ausdruck mit $n=0$ **Klammerpaaren**, nämlich **das leere Wort**.

Rekursiver Fall.

Sei $n \geq 1$. Ein **wohlgeklammerter Ausdruck** e mit n **Klammerpaaren** ist ein Ausdruck der Art

$$e = (f) g$$

wobei f ein wohlgekl. Ausdruck mit i **Klammerpaaren**
und g ein wohlgekl. Ausdruck mit j **Klammerpaaren**
ist, sodass $i + j = n-1$ gilt.

Rekursion in der Informatik

II) Eine Art und Weise **Lösungen** zu **entwerfen**

Für $n > 0$ ist die **Fakultät von n** folgendermassen definiert:

$$n! = \prod_{i=1}^n i = 1 \cdot 2 \cdot 3 \cdots n$$

Rekursive Berechnung von $n!$

Basisfall. Für $n=1$ gilt $n!=1!=1$

Induktiver Fall.

Berechnung von $I=n!$ für $n > 1$:

- Berechne $J=(n-1)!$ rekursiv

In Python typischerweise durch
rekursiven Methodenaufruf

- Berechne $I=n \cdot J$

Ist korrekt, denn $I=n \cdot J=n \cdot (n-1)!=n!$

Rekursion

- algorithmisch: Eine Art und Weise Lösungen zu beschreiben mittels
 - Teile und Herrsche (**Divide & Conquer**) oder
 - Verkleinere und Herrsche (**Decrease & Conquer**)
 - **Idee**: Reduziere ein Problem in Unterprobleme (desselben grundsätzlichen Problems) und führe deren Lösungen zu einer Lösung des Gesamtproblems
- Semantisch: Eine Programmiertechnik bei der sich **Funktionen selbst aufrufen**
 - Beim Programmieren sollte keine Endlosrekursion entstehen
 - Basisfall/Basisfälle müssen direkt gelöst werden
 - Induktiver Fall muss auf einfacheren Fall überführbar sein

Multiplikation iterativ

ohne Beschränkung der Allgemeinheit

- Die Multiplikation $a * b$ ist äquivalent dazu a mit sich selbst b mal zu addieren (wir nehmen hier oBdA an, dass a und b nicht negativ sind)
- Der aktuelle **Zustand** der Multiplikation wird beschrieben durch
 - Eine **Iterationszahl** i , die bei b beginnt
 $i \leftarrow i - 1$ und stoppe sobald i gleich 0
 - einen aktuellen Berechnungswert (Zwischensumme) `ergebnis`
 $\text{ergebnis} \leftarrow \text{ergebnis} + a$

```
def mult_iterativ(a,b):  
    ergebnis=0  
    while b>0:  
        ergebnis+=a  
        b-=1  
    return ergebnis
```

Iteration
aktueller Berechnungswert, Zwischensumme
aktueller Wert der Iterationsvariablen

Multiplikation rekursiv

- **Rekursiver Fall**

- Man überlegt sich wie man das Problem auf eine **kleinere/einfachere Version** des Problems reduzieren kann

...vereinfachte Problem wiederholt...

- **Basisfall**

- ...bis einfacher Fall entsteht, der **direkt gelöst** werden kann
- **Beispiel:** Falls $b = 1$ dann gilt
$$a \cdot b = a$$

$$a \cdot b = \underbrace{a + a + a + \dots + a}_{b \text{ mal}}$$

b mal

$$= a + \underbrace{a + a + \dots + a}_{b-1 \text{ mal}}$$

b-1 mal

$$= a + \boxed{a \cdot (b - 1)}$$

*Rekursive
Reduktion*

```
def mult(a,b):  
    if b==1:  
        return a  
    else:  
        return a+mult(a,b-1)
```

Basisfall

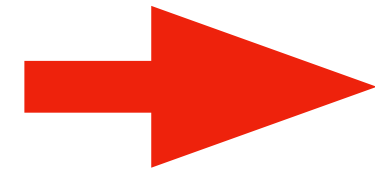
Rekursiver Fall

Fakultätsberechnung

$$n! = n \cdot (n - 1) \cdot (n - 2) \cdot (n - 3) \cdot \dots \cdot 1$$

- Für des welche n kennen wir die Fakultät?

$n = 1$



```
if n==1:
```

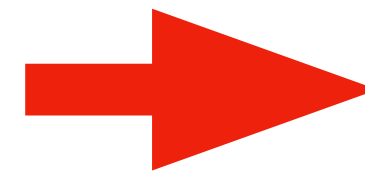
```
    return 1
```

Basisfall

- Wie kann das Fakultätsberechnungsproblem reduziert werden?

Falls $n > 1$ so gilt

$$n! = n \cdot (n - 1)!$$



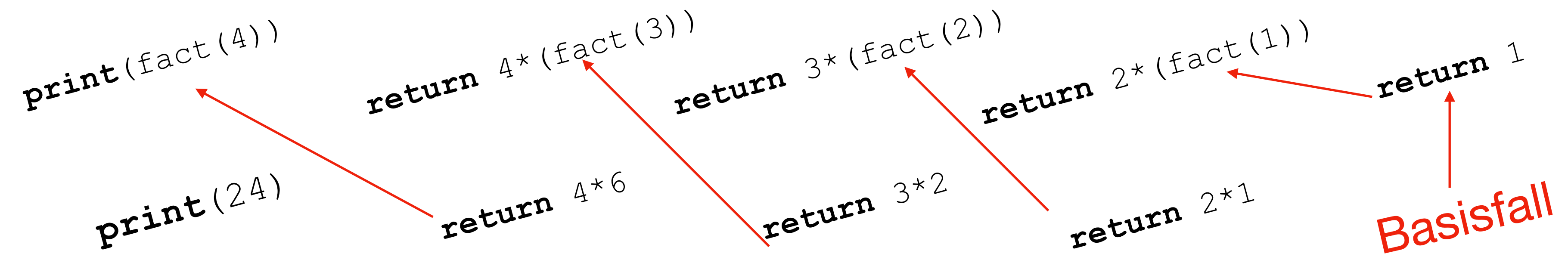
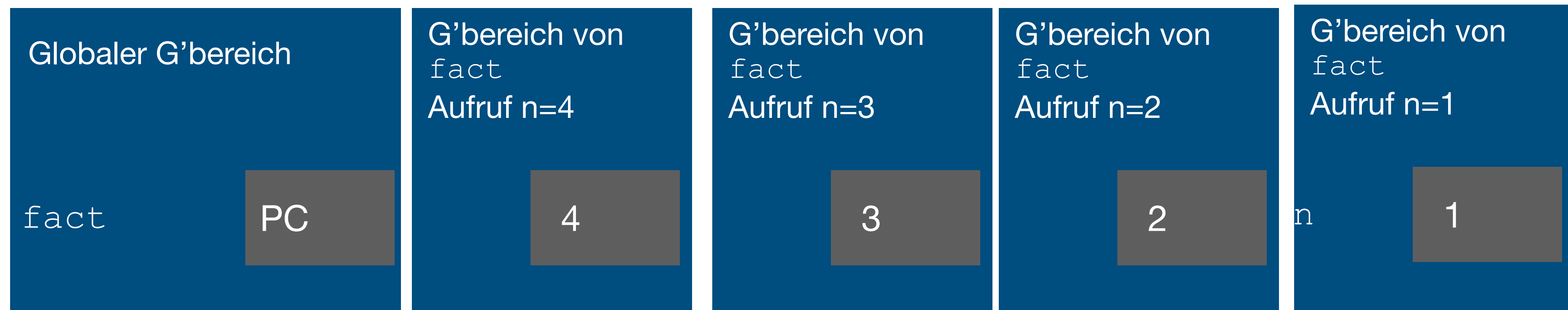
```
else:
```

```
    return n*fakultaet(n-1)
```

Rekursiver Fall

Geltungsbereich rekursiver Funktionen

```
def fact(n):  
    if n==1:  
        return 1  
    else:  
        return n*fact(n-1)  
print(fact(4))
```



Ein paar Beobachtungen

- Jeder rekursive Aufruf einer Funktion erzeugt einen **eigenen Geltungsbereich**
- **Variablenbelegungen** im Geltungsbereich der rekursiven aufgerufenen Funktion hat keine Auswirkung auf den Aufrufer
- Der **Kontrollfluß kehrt zum Aufrufer** zurück sobald der rekursiv berechnete Wert zugewiesen wird

obwohl u.U. derselbe Variablenname verwendet wird, handelt es sich um unterschiedliche Objekte in verschiedenen Geltungsbereichen

Iteration vs. Rekursion

```
def fakultaet_iterativ(n):  
    produkt=1  
    for i in range(1,n+1):  
        produkt*=i  
    return produkt
```

```
def fakultaet(n):  
    if n==1:  
        return 1  
    else:  
        return n*fakultaet(n-1)  
print(fakultaet(4))
```

- Rekursion kann u.U. einfacher bzw. intuitiver sein
- Iteration kann u.U. einfacher bzw. intuitiver sein
- Rekursion kann effizienter aus der Sicht der Programmiers sein
- Rekursion kann u.U. weniger effizient sein aus der Sicht des Computers

Induktives Schlussfolgern

- Woher wissen wir dass unser rekursiver Code auch wirklich korrekt ist?

```
def mult_iterativ(a,b):  
    ergebnis=0  
    while b>0:  
        ergebnis+=a  
        b-=1  
    return ergebnis
```

- `mult_iterativ` terminiert, da `b` zu Beginn positiv ist und in jedem Schleifendurchlauf um 1 dekrementiert wird, muss also irgendwann einen Wert annehmen, der nicht größer 1 ist

```
def mult(a,b):  
    if b==1:  
        return a  
    else:  
        return a+mult(a,b-1)
```

- `mult` hat bei `b=1` keinen rekursiven Aufruf und hält an
- `mult` macht bei `b>1` einen rekursiven Aufruf mit einer kleineren Version von `b`, muss also irgendwann den Aufruf mit `b=1` erreichen

Mathematische Induktion

Menge der natürlichen Zahlen in der Informatik: $\mathbb{N} = \{0,1,2,\dots\}$

- Um zu zeigen, dass eine Aussage über allen natürlichen Zahlen n gilt, zeigt man zwei Dinge:

1. Man zeigt, dass die Aussage für das kleinste n gilt (also für $n=0$)

Induktionsanfang

2. Dann zeigt man, dass für alle natürlichen Zahlen n folgende Folgerung gilt:

Induktionsvoraussetzung

Wenn die Aussage für n gilt, **dann** gilt sie auch für $n+1$

Induktionsschritt

Um 2. zu beweisen, dürfen (und müssen) sogar annehmen, dass die Aussage für n **tatsächlich gilt**, um die Aussage für $n+1$ zu beweisen. Wir reduzieren den Fall für $n+1$ also auf den Fall n .

Induktion Beispiel

Aussage: Für alle $n \in \mathbb{N}$ gilt folgende Gleichheit

$$1 + 2 + 3 + \dots + n = (n \cdot (n + 1))/2$$

Beweis durch Induktion:

- Falls $n = 0$ so ist die linke Seite der Gleichung gleich 0 und die rechte Seite $(0 \cdot (0 + 1))/2 = 0$

Induktionsanfang

- Sei $n \geq 0$ eine beliebige aber feste natürliche Zahl größer 0

und nehmen wir an die obige Aussage gelte bereits für n ,

also $1 + 2 + 3 + \dots + n = (n \cdot (n + 1))/2$ (★)

Induktionsvoraussetzung

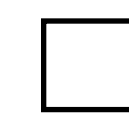
Unter der Annahme der Gleichung (★) müssen wir nun

$1 + 2 + 3 + \dots + n + (n + 1) = ((n + 1) \cdot (n + 2))/2$ zeigen:

Induktionsschritt

Es gilt: $1 + 2 + 3 + \dots + n + (n + 1) \stackrel{(\star)}{=} (n \cdot (n + 1))/2 + (n + 1)$

$$= (n \cdot (n + 1))/2 + 2 \cdot (n + 1)/2 = ((n + 2)(n + 1))/2$$



Relevanz bei Korrektheit von Programmen

- Wir wenden ebenfalls das Induktionsprinzip an.
- Zur Erinnerung: Wir haben angenommen, dass $a > 0$ und $b > 0$ gilt.

```
def mult(a,b):  
    if b==1:  
        return a  
    else:  
        return a+mult(a,b-1)
```

Keine Zuweisung sondern
mathematische Gleichheit gemeint!

- **Aussage:** Für alle $a > 0$ und alle $b > 0$ liefert `mult(a,b)` das Produkt $a * b$ zurück.

Beweis durch Induktion nach b

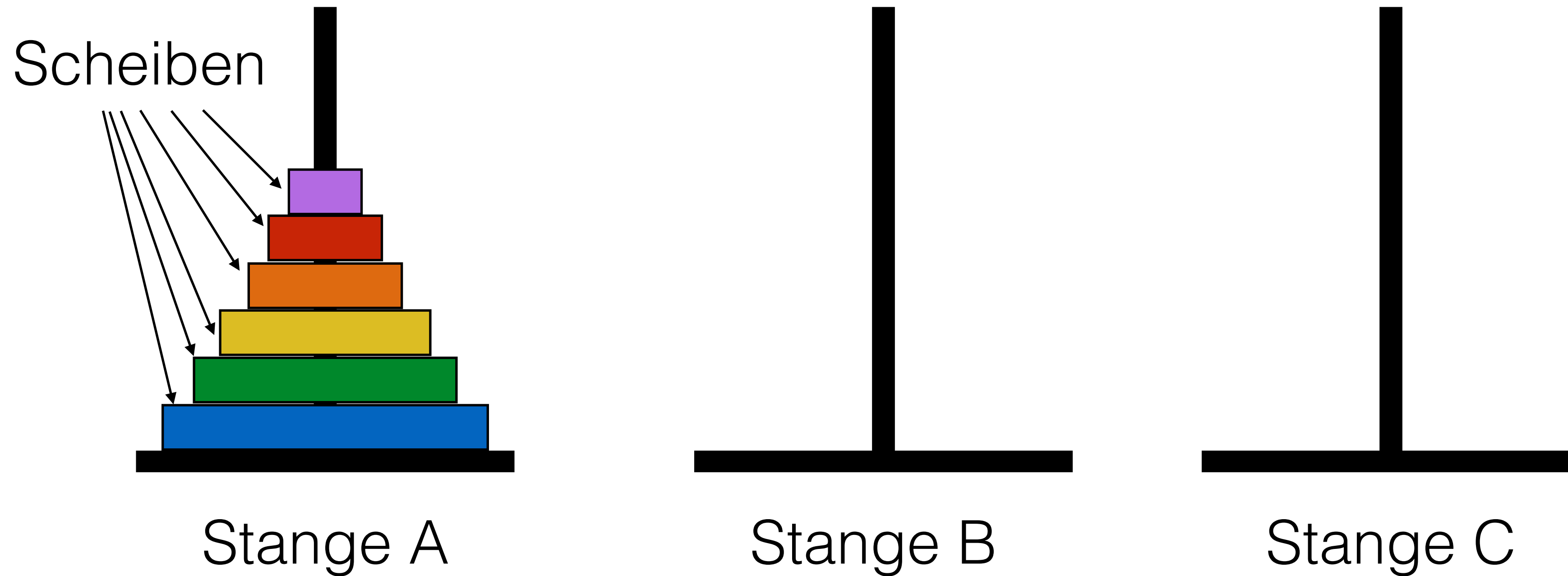
- **Induktionsanfang:** Für b gleich 1 gibt `mult` für alle a den Wert a zurück, korrekt da, wenn b gleich 1, für alle a gilt: $a * b = a$
- **Induktionsschritt:** Für alle a gibt `mult(a, b+1)` den Wert $a + \text{mult}(a, b+1-1)$ zurück

Es gilt $a + \text{mult}(a, b+1-1) = a + \text{mult}(a, b) = a + a * b = a * (b+1)$

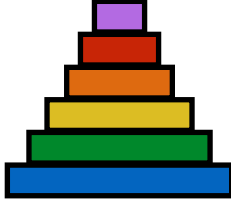
Induktionsvoraussetzung

Induktionsschritt

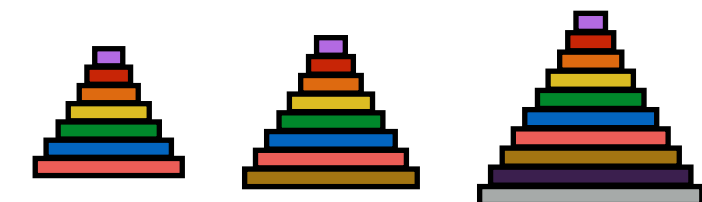
Die Türme von Hanoi



Die Regeln:

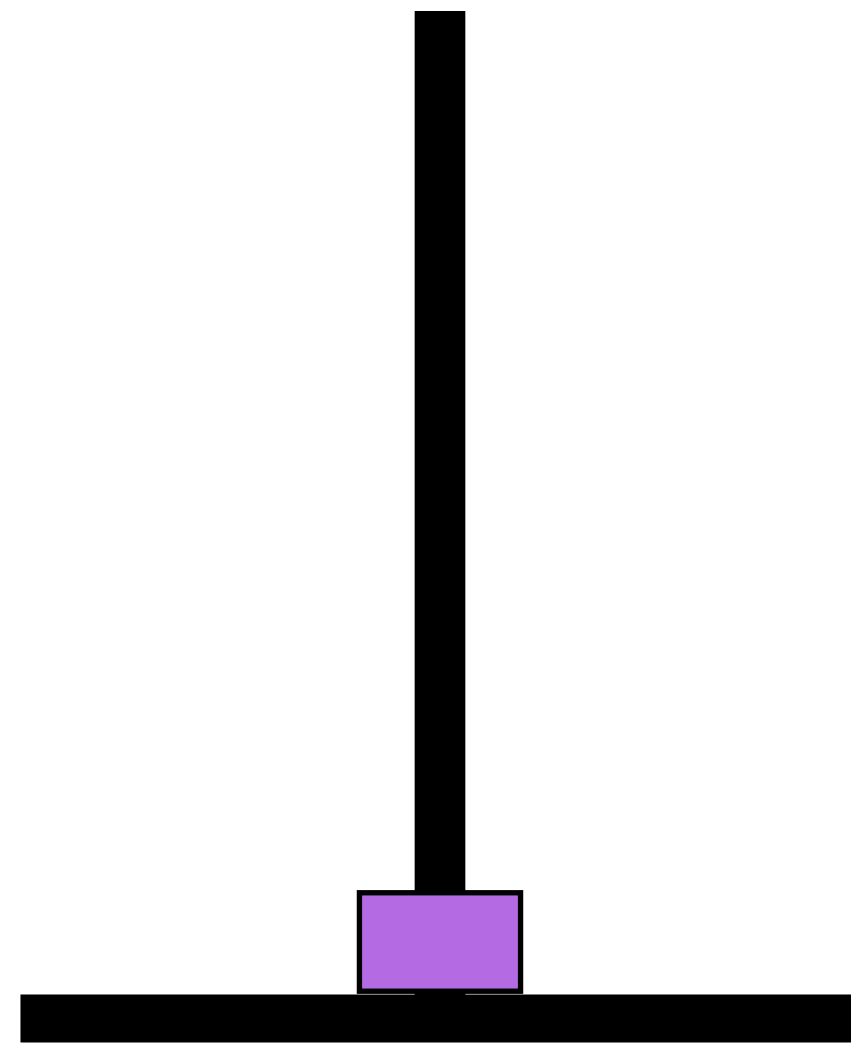
- Bewege einzelne Scheiben zwischen den Stangen, sodass sich  schliesslich auf Stange C befindet.
- **Bedingung:** Eine gerade gelegte Scheibe muss immer die kleinste der Stange sein, auf die sie gelegt wird!

Die Frage: Ist dies für jede Turmhöhe **n** möglich?

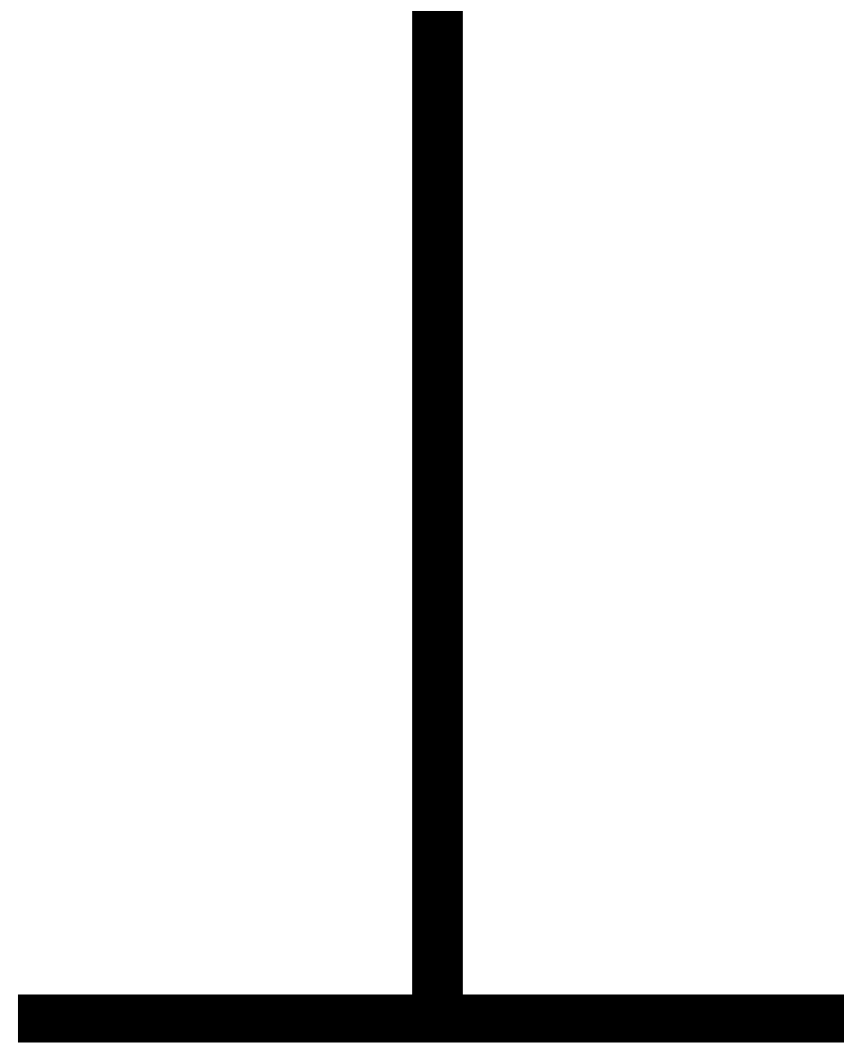


Kleine Türme von Hanoi

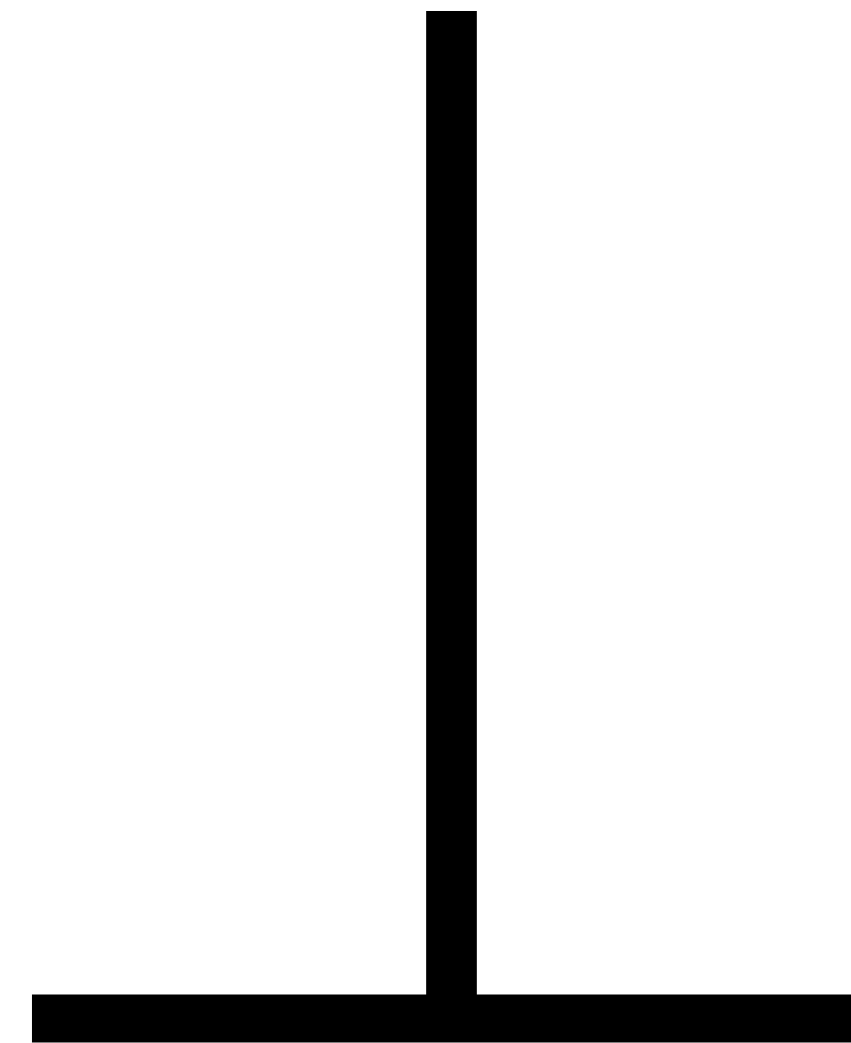
n=1



Stange A



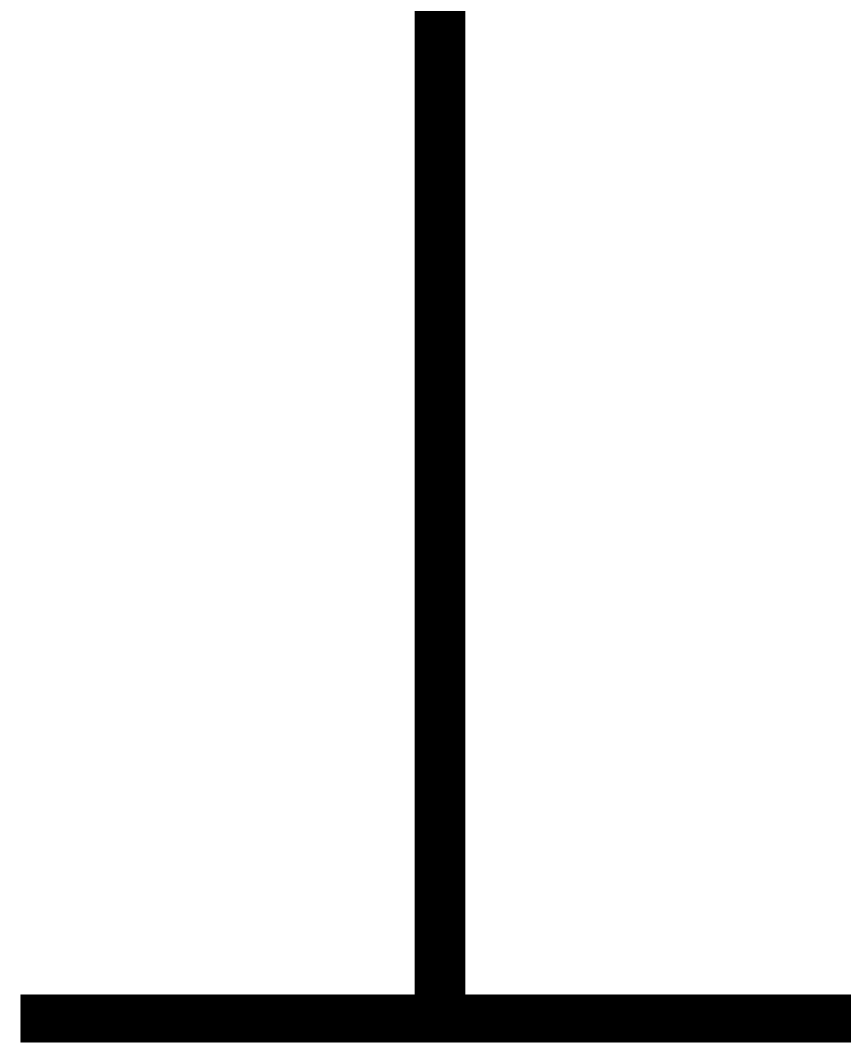
Stange B



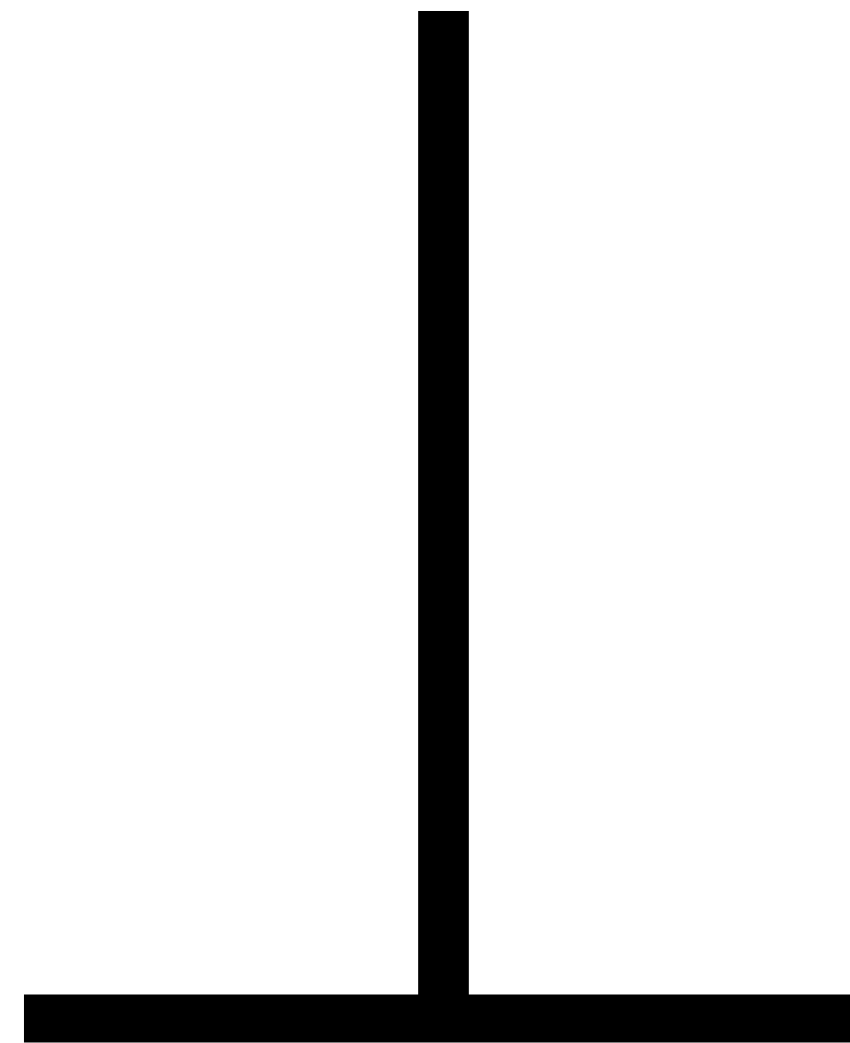
Stange C

Kleine Türme von Hanoi

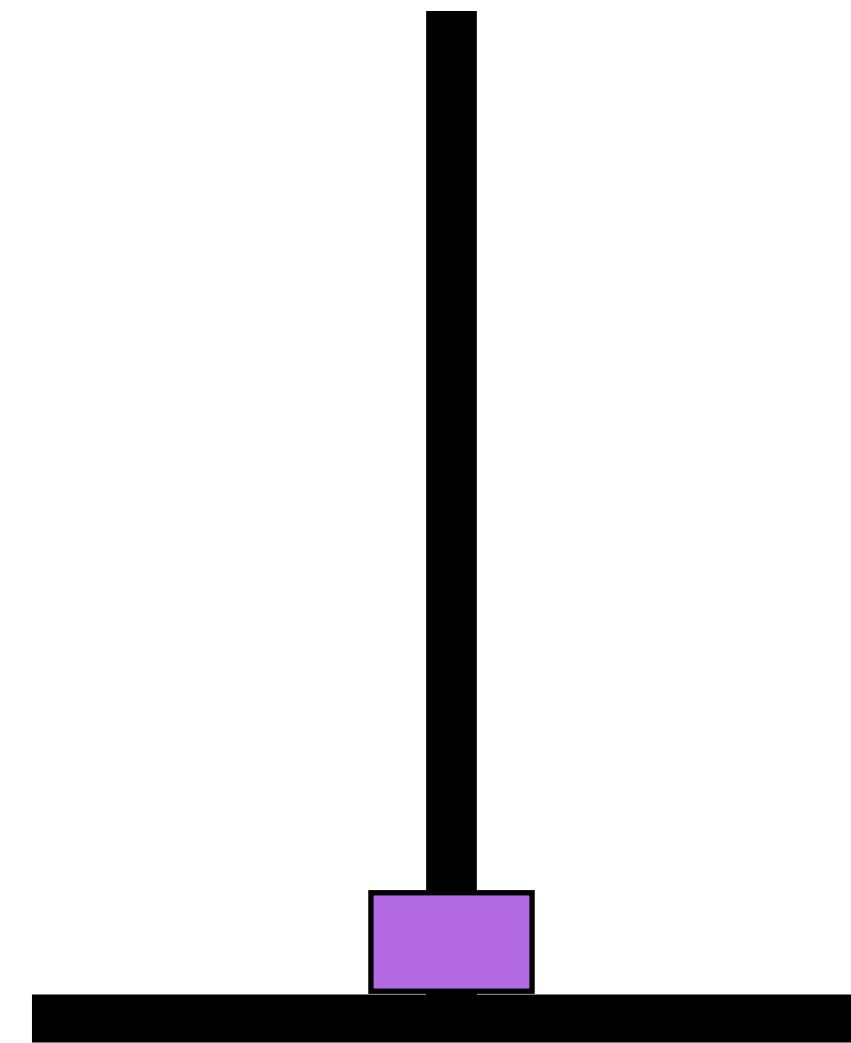
n=1



Stange A



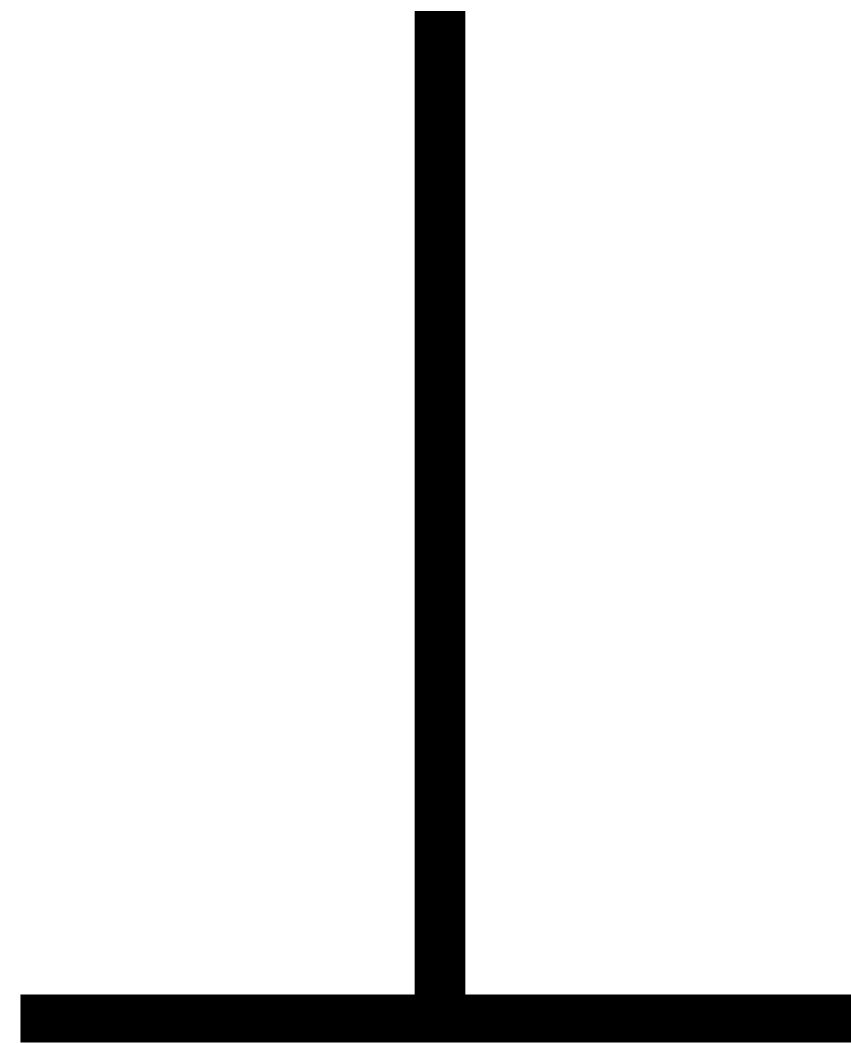
Stange B



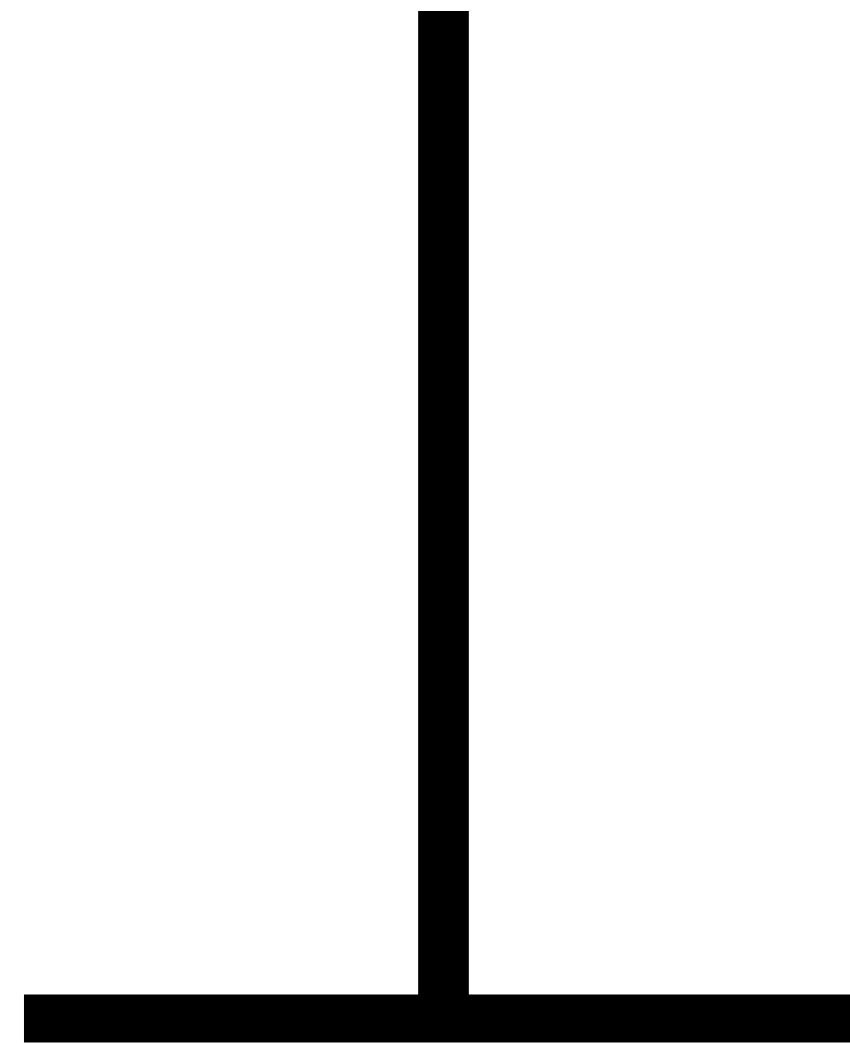
Stange C

Kleine Türme von Hanoi

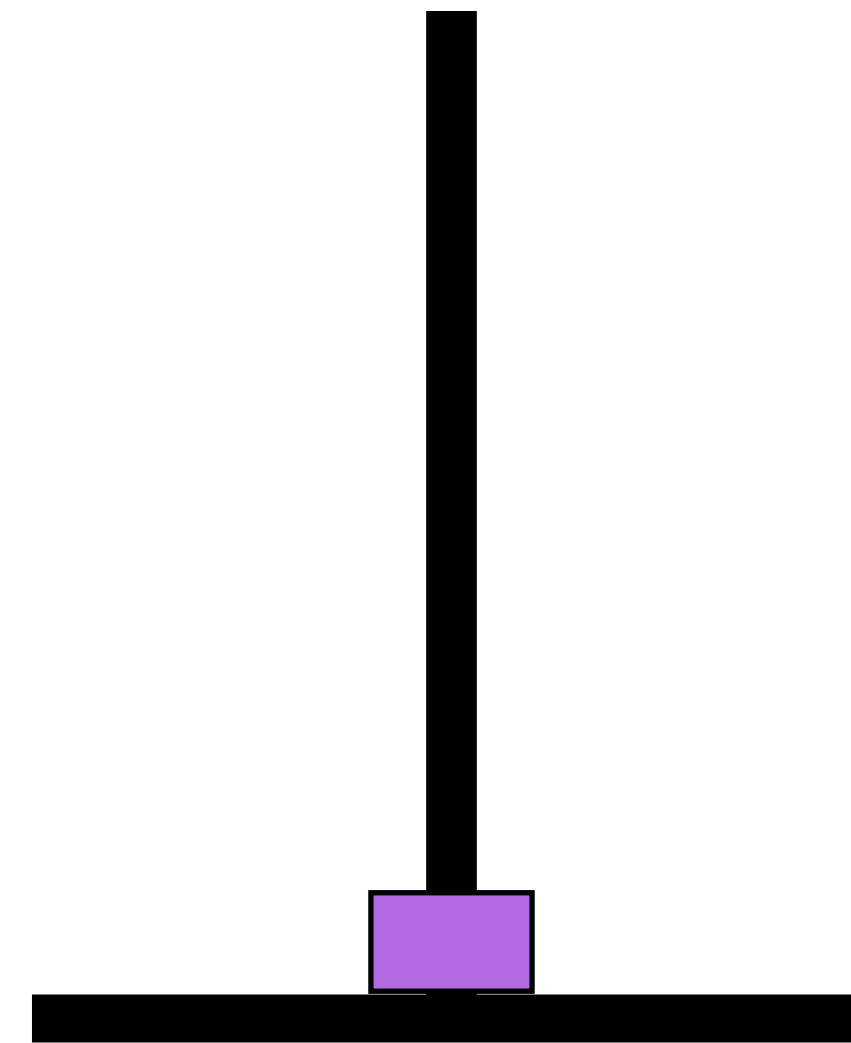
n=1



Stange A



Stange B

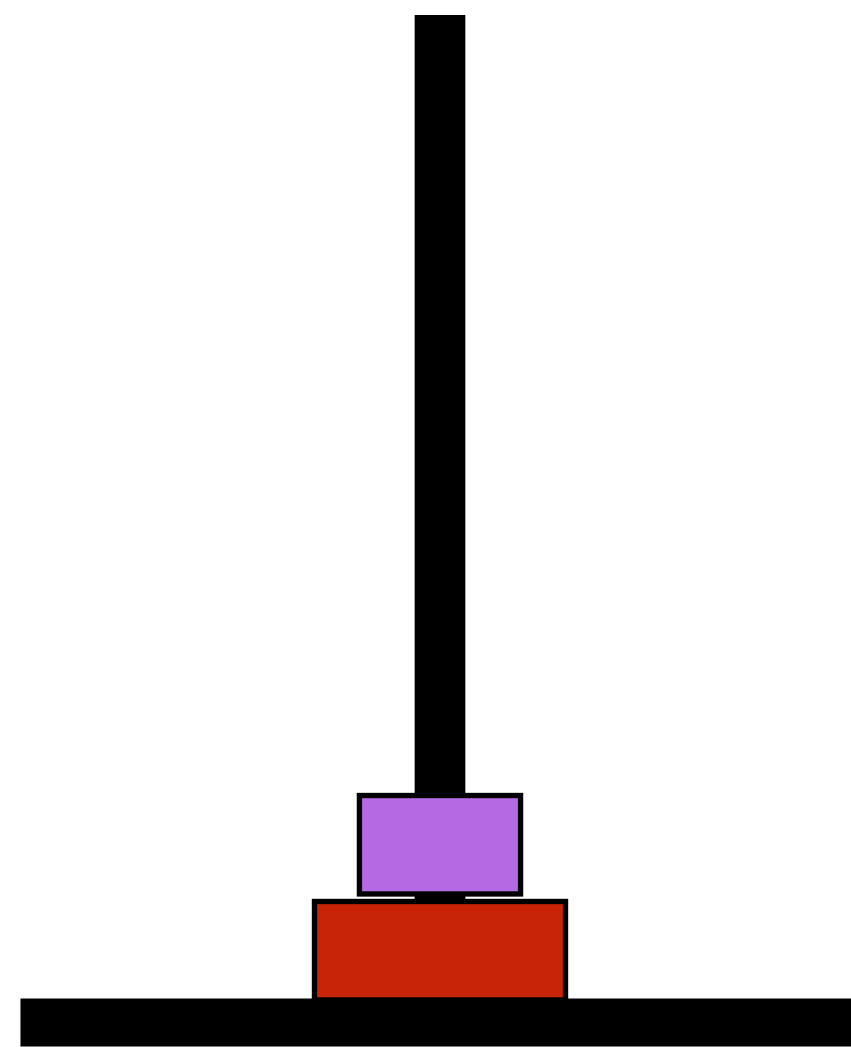


Stange C

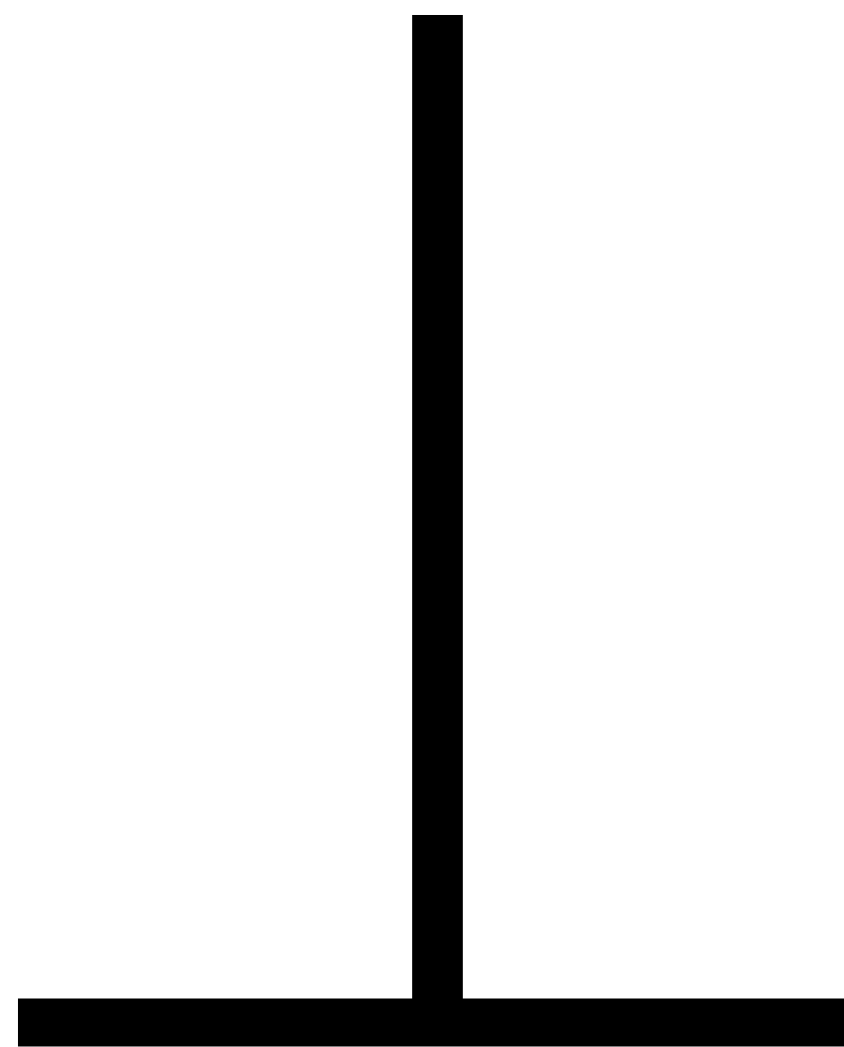


Kleine Türme von Hanoi

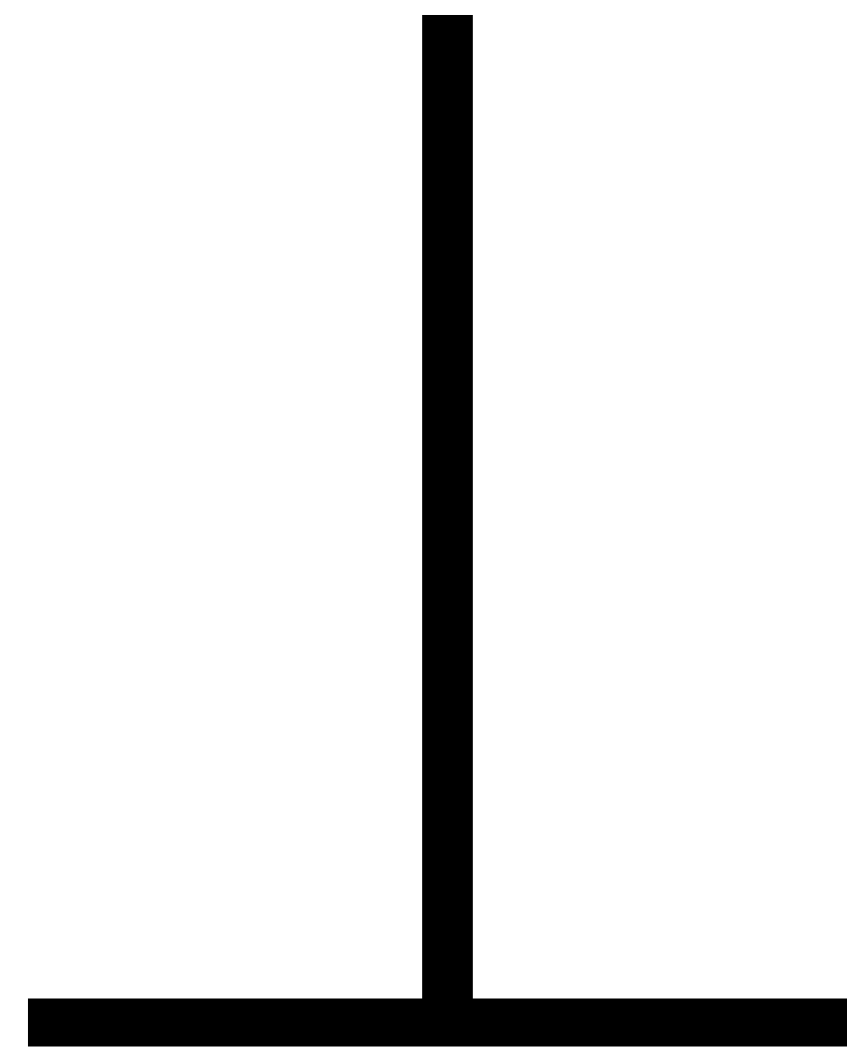
n=2



Stange A



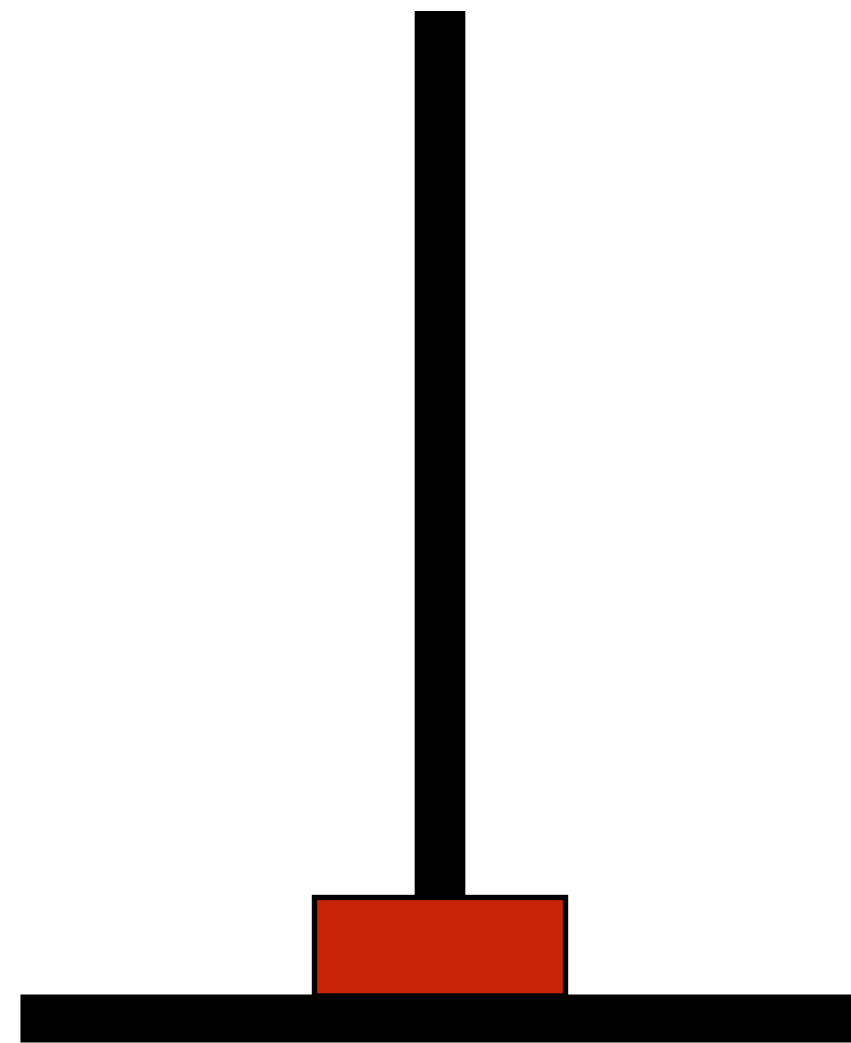
Stange B



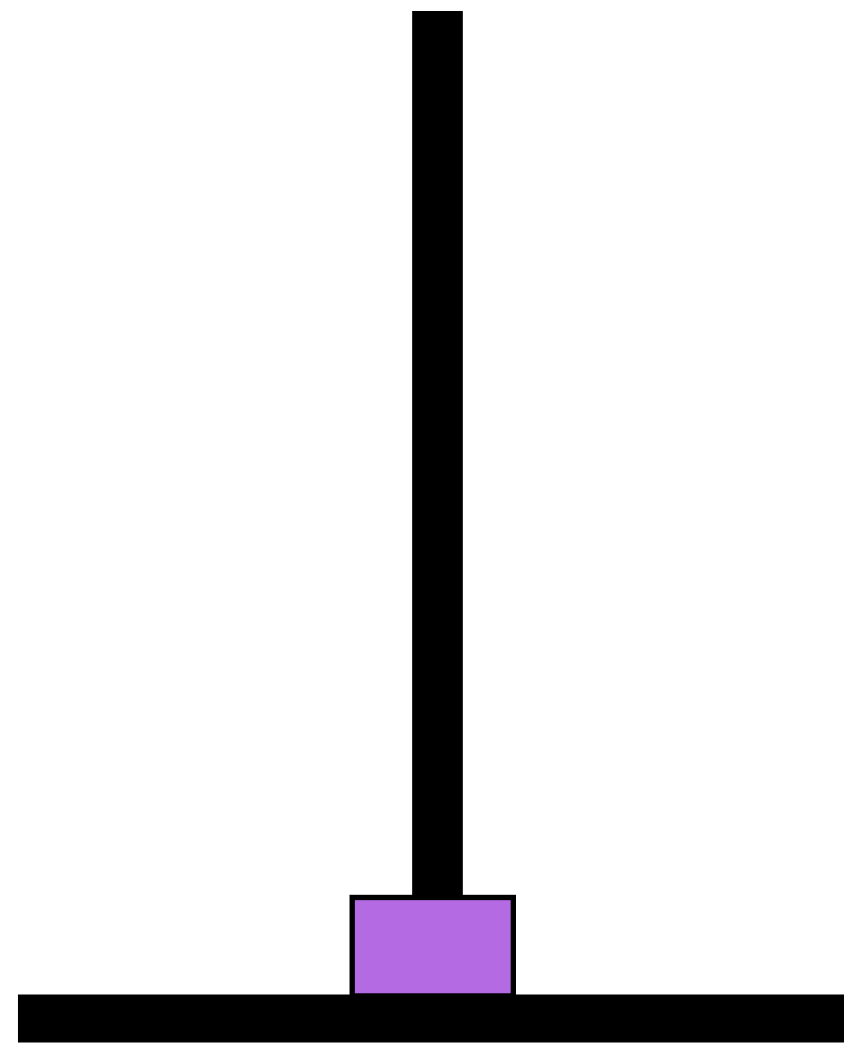
Stange C

Kleine Türme von Hanoi

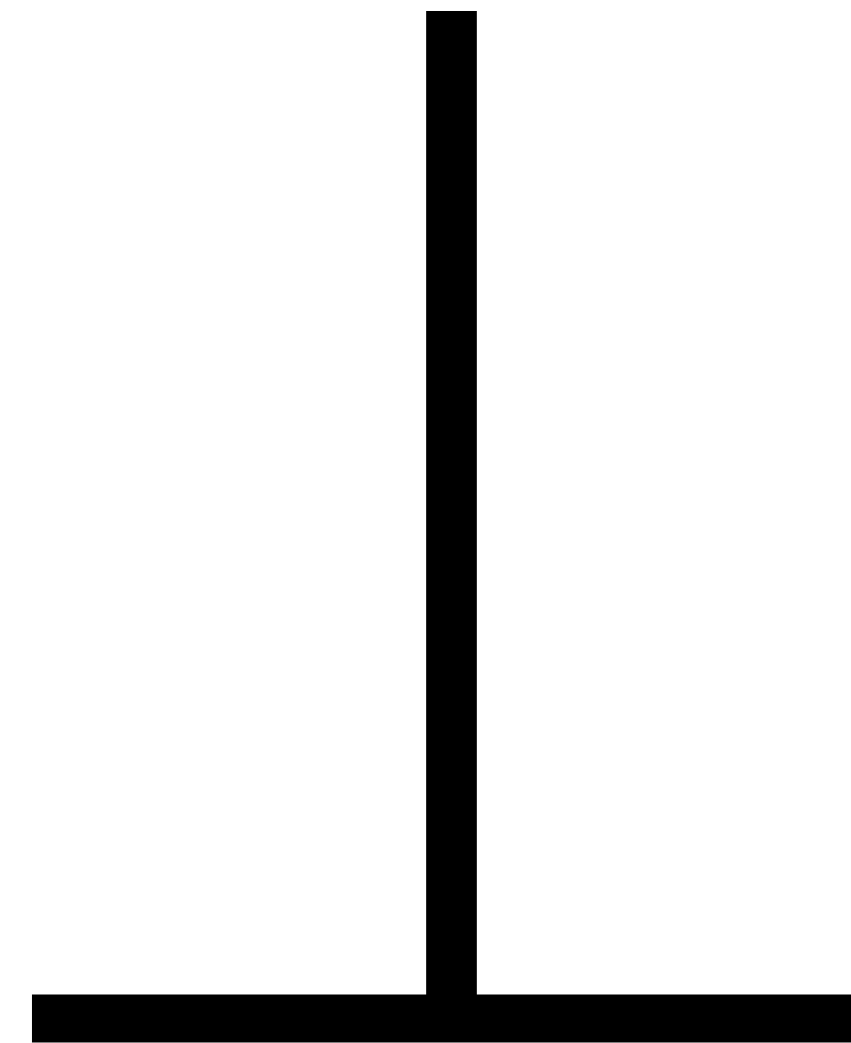
n=2



Stange A



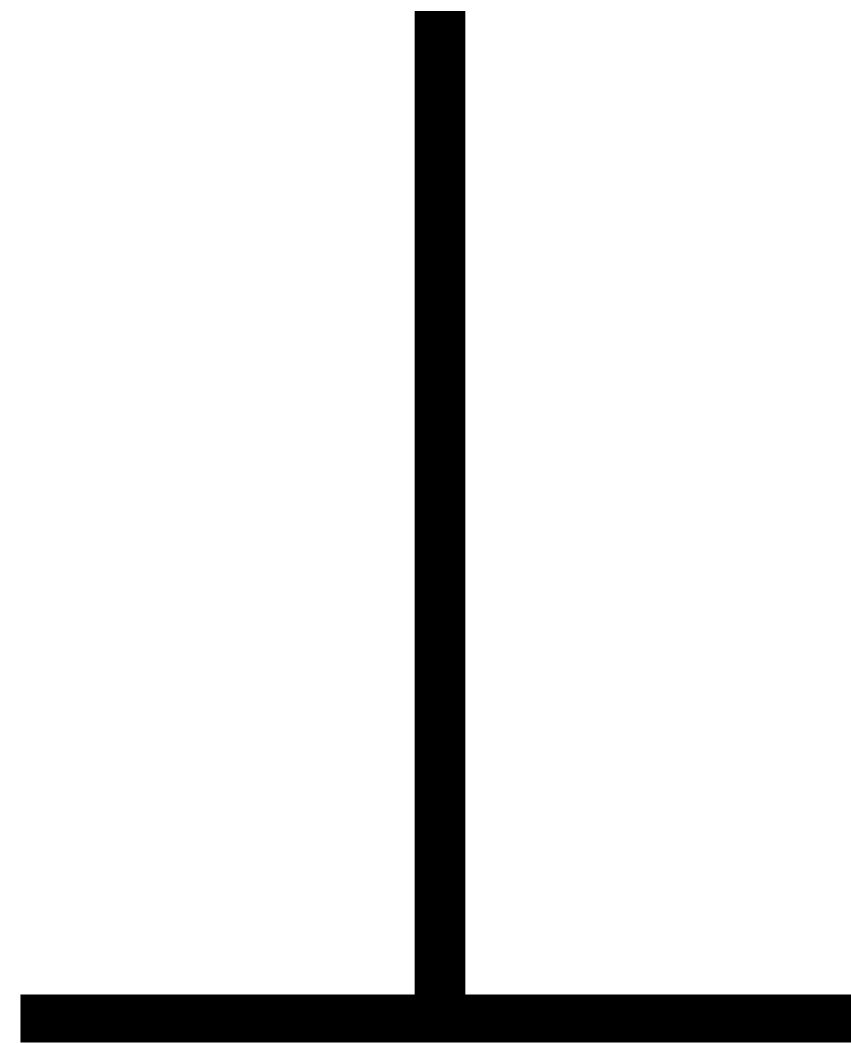
Stange B



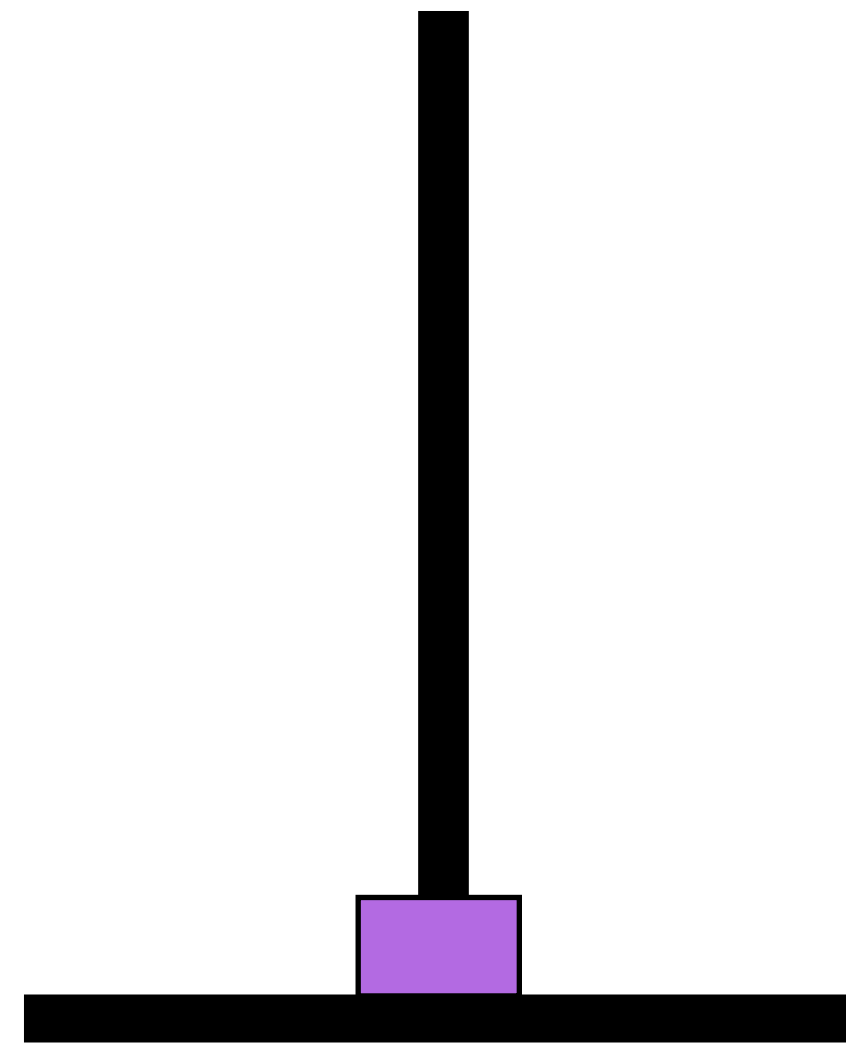
Stange C

Kleine Türme von Hanoi

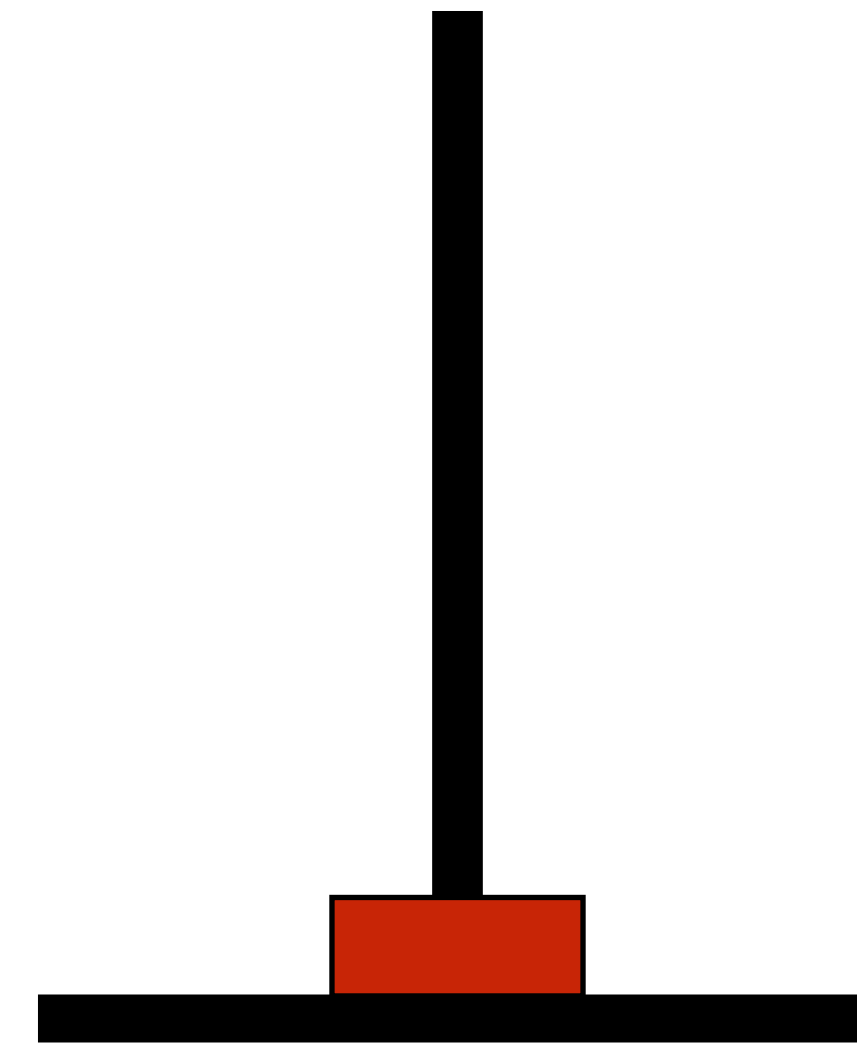
n=2



Stange A



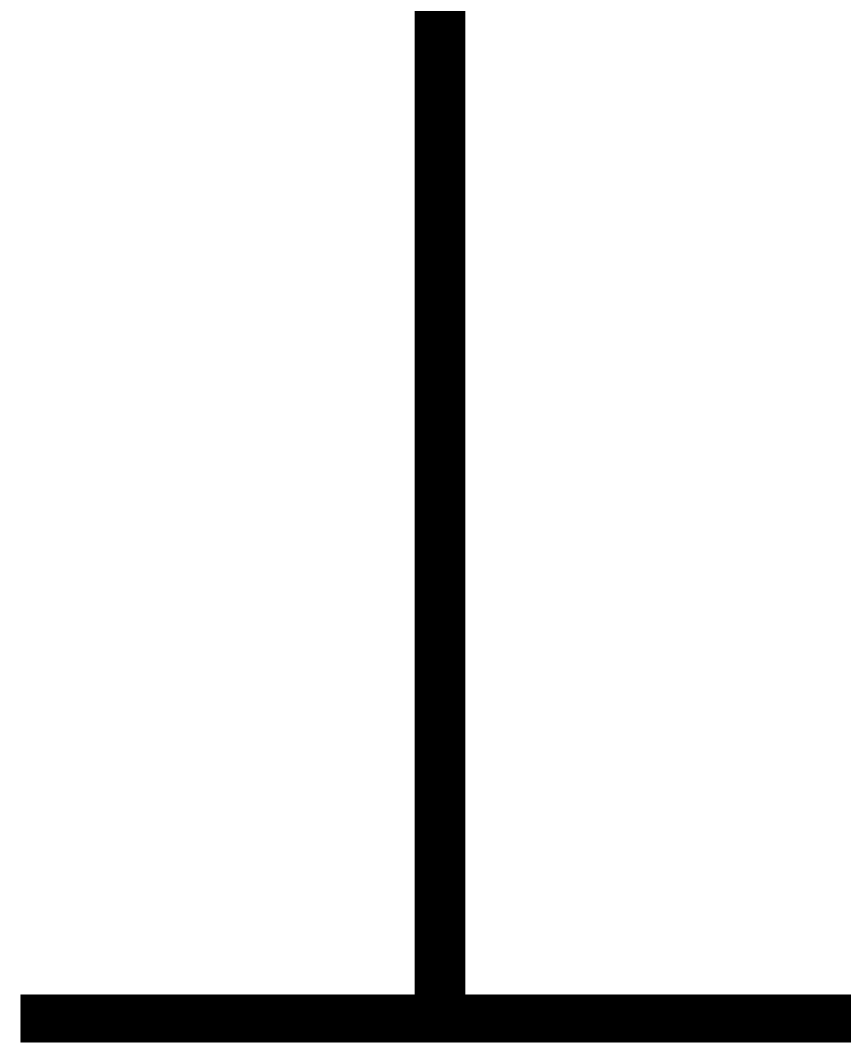
Stange B



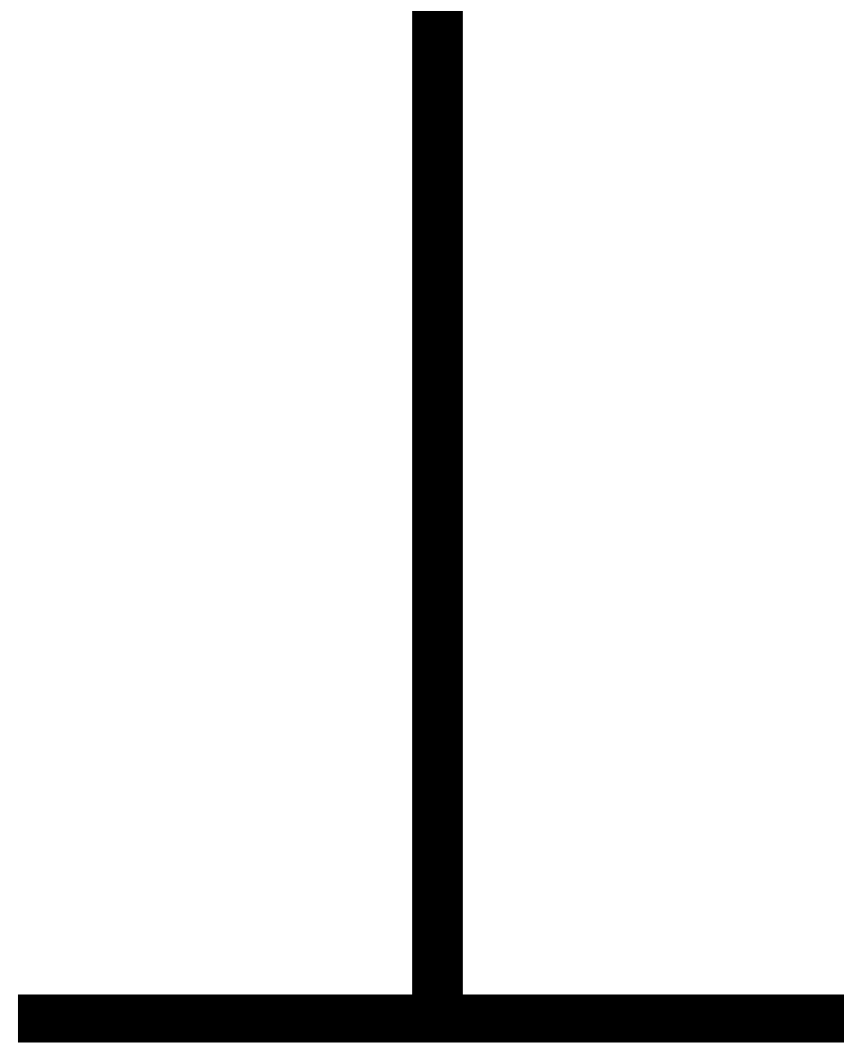
Stange C

Kleine Türme von Hanoi

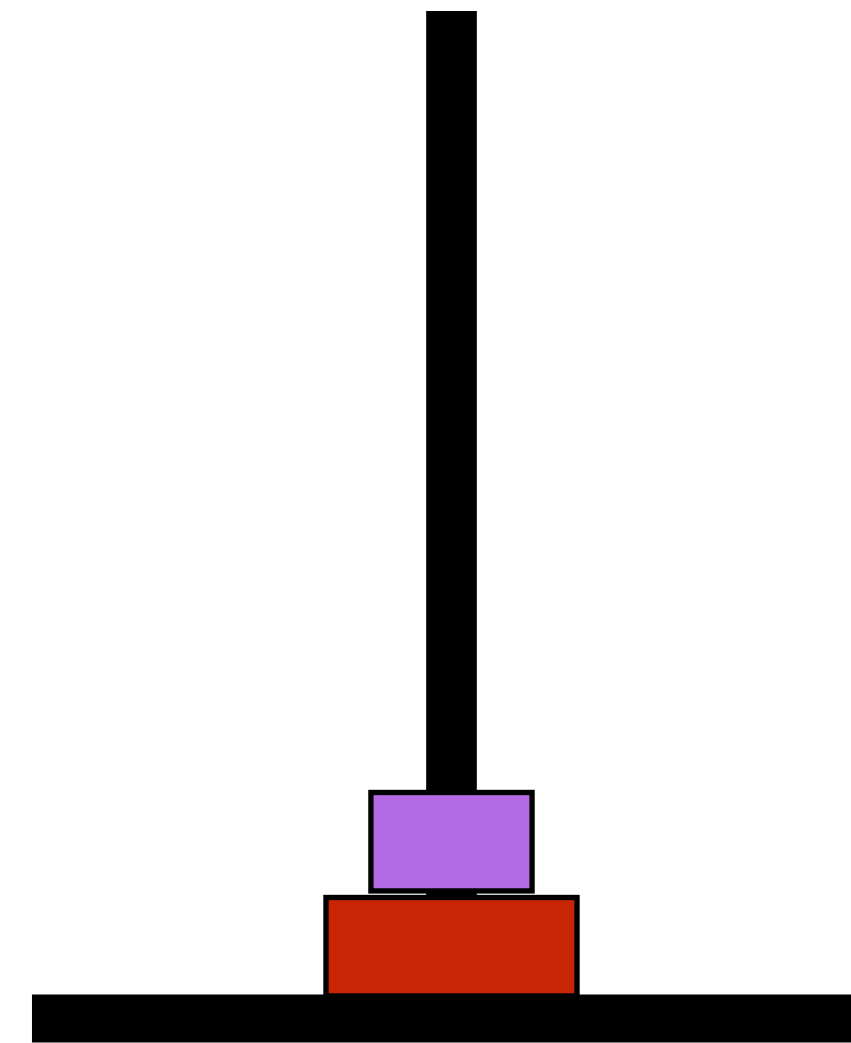
n=2



Stange A



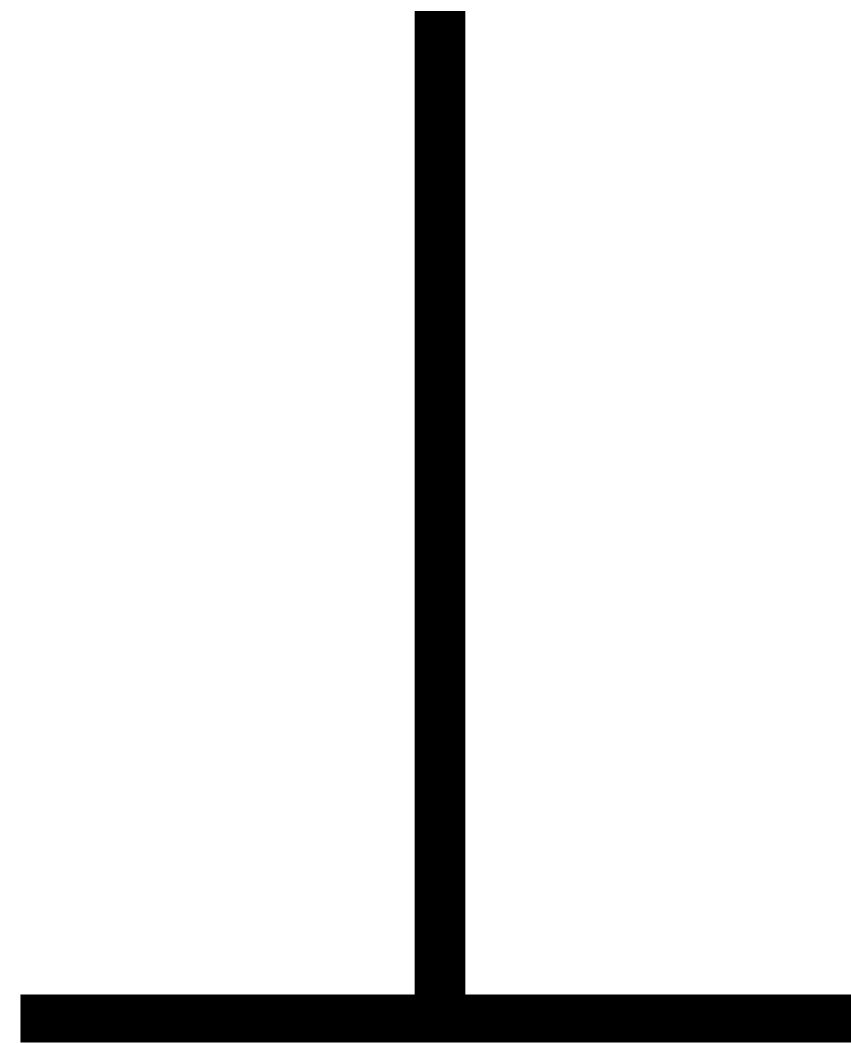
Stange B



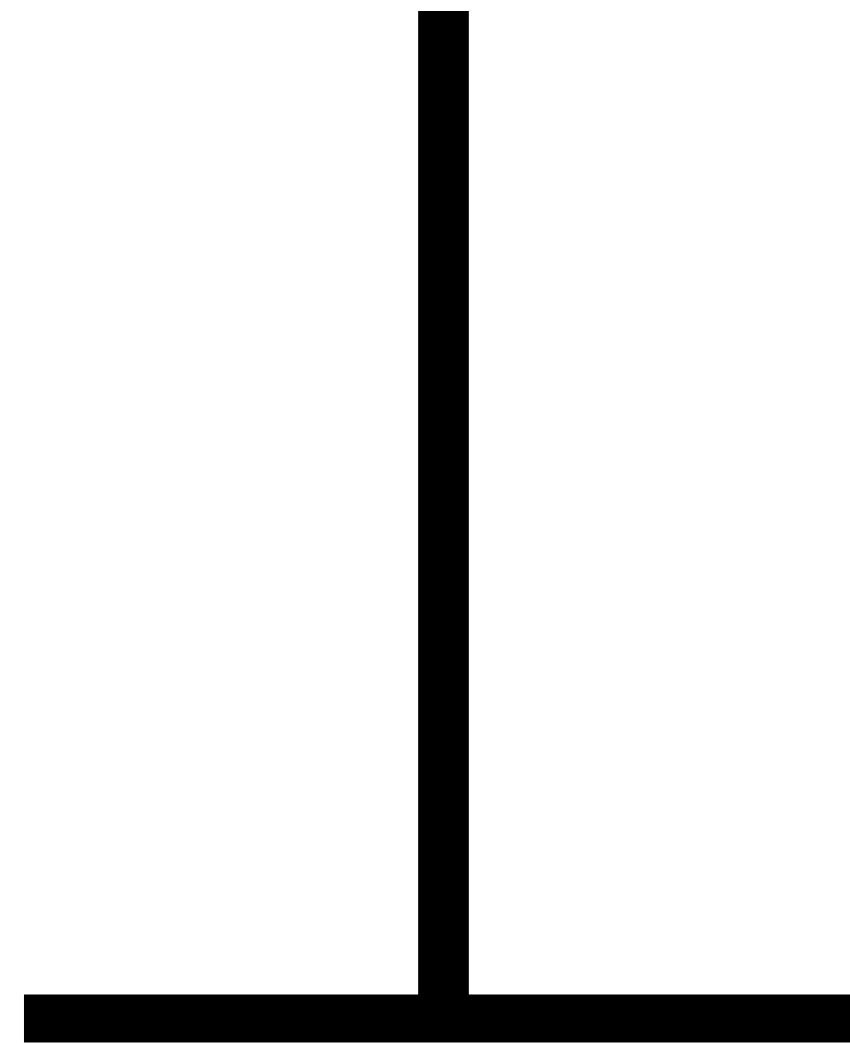
Stange C

Kleine Türme von Hanoi

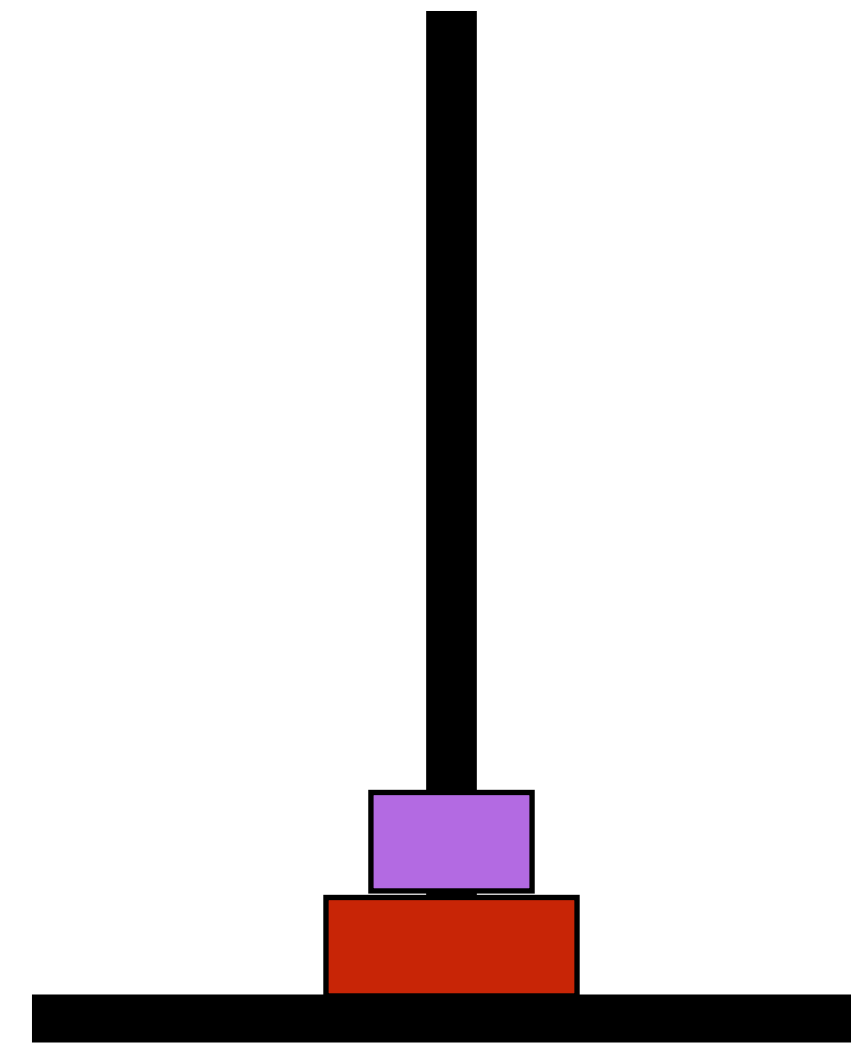
n=2



Stange A



Stange B

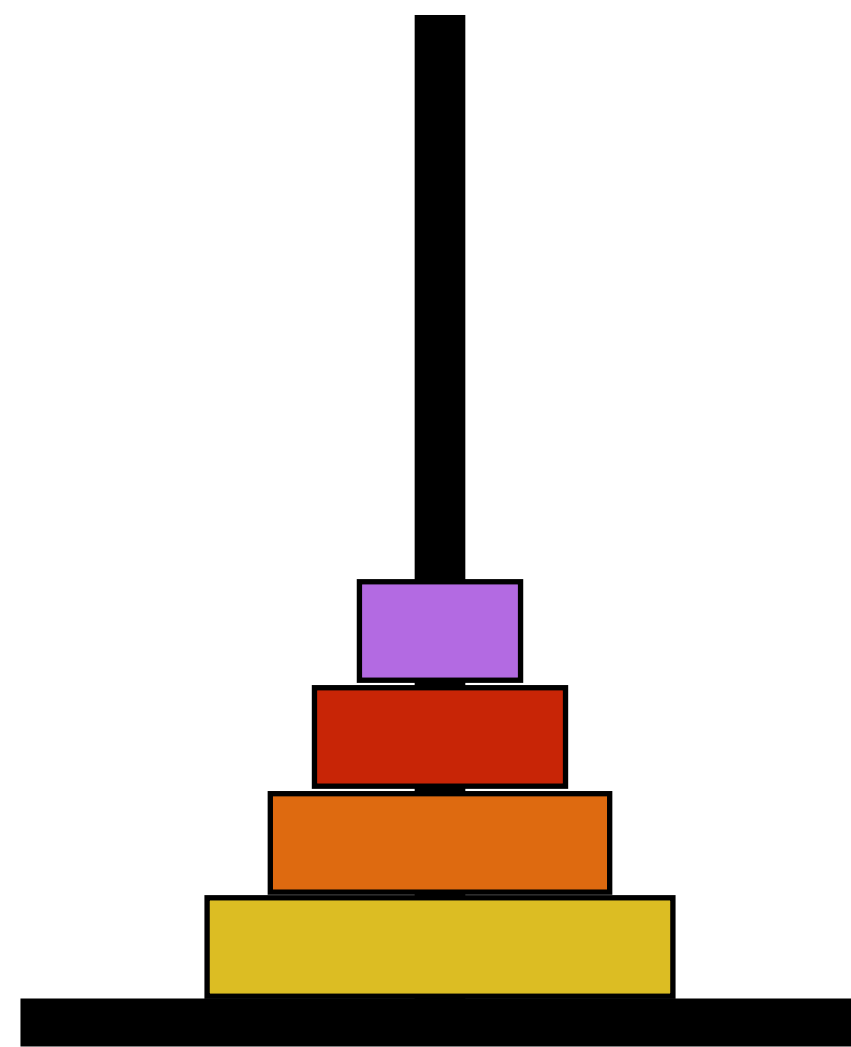


Stange C

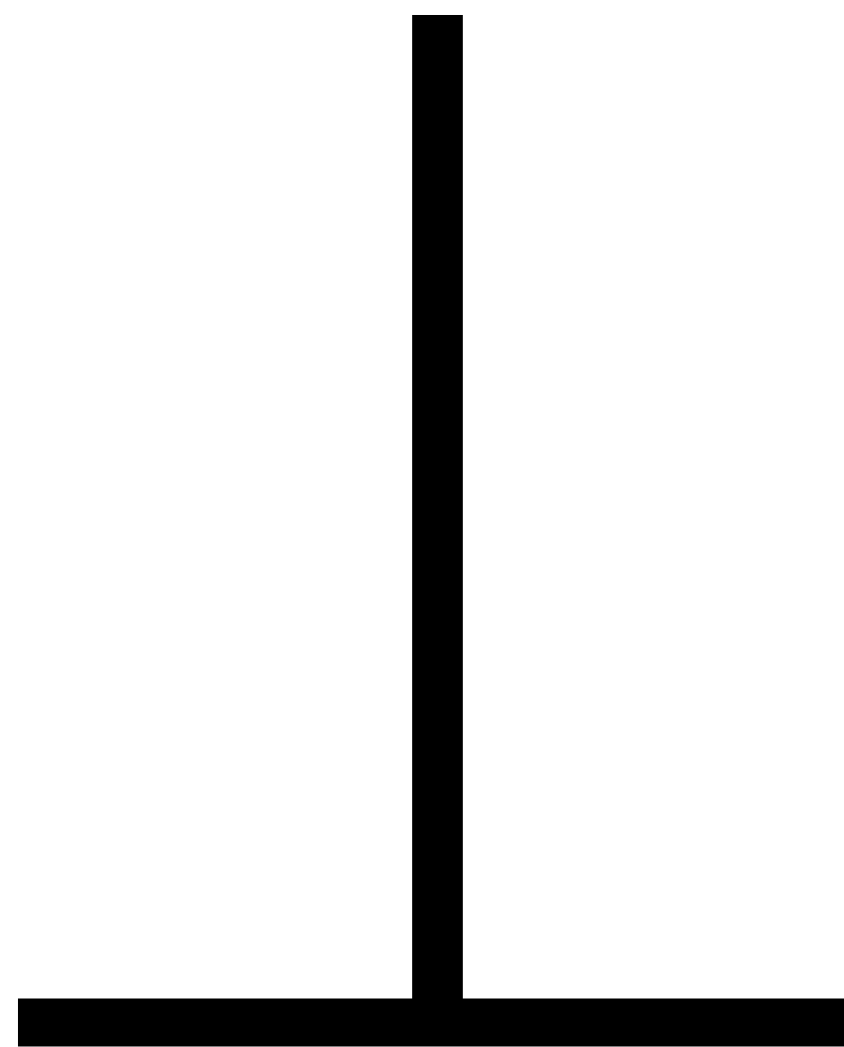


Kleine Türme von Hanoi

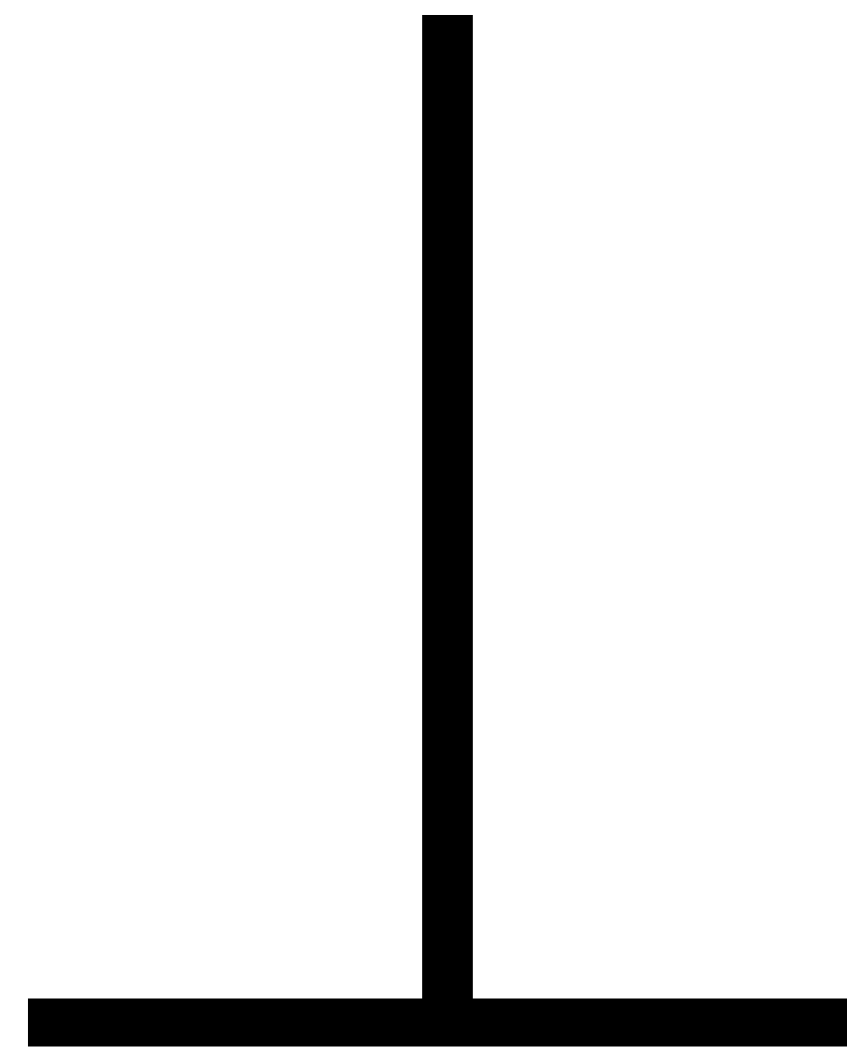
n=4



Stange A



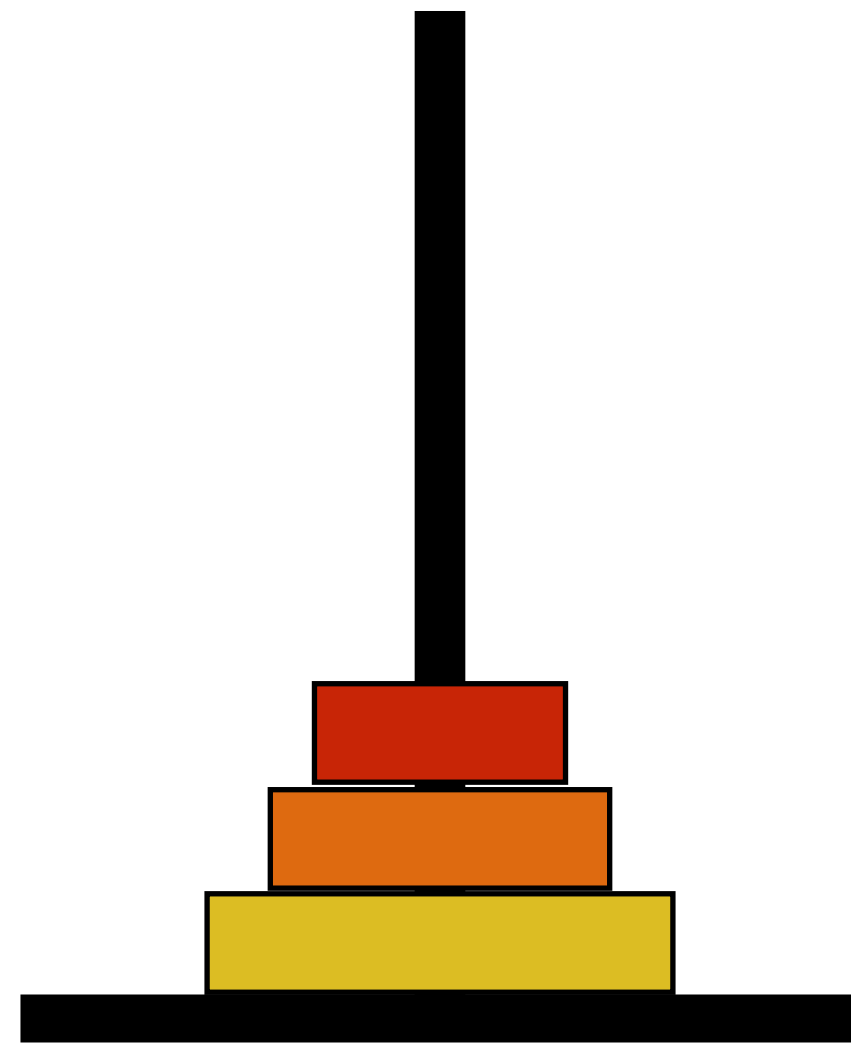
Stange B



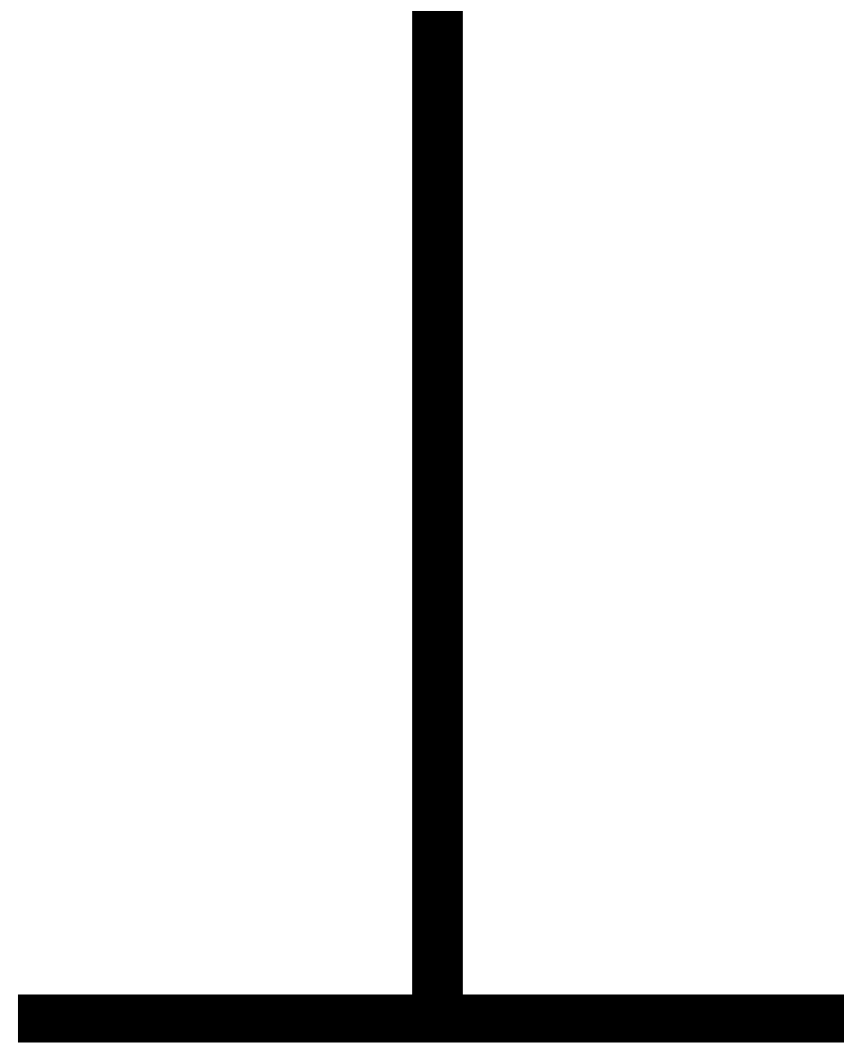
Stange C

Kleine Türme von Hanoi

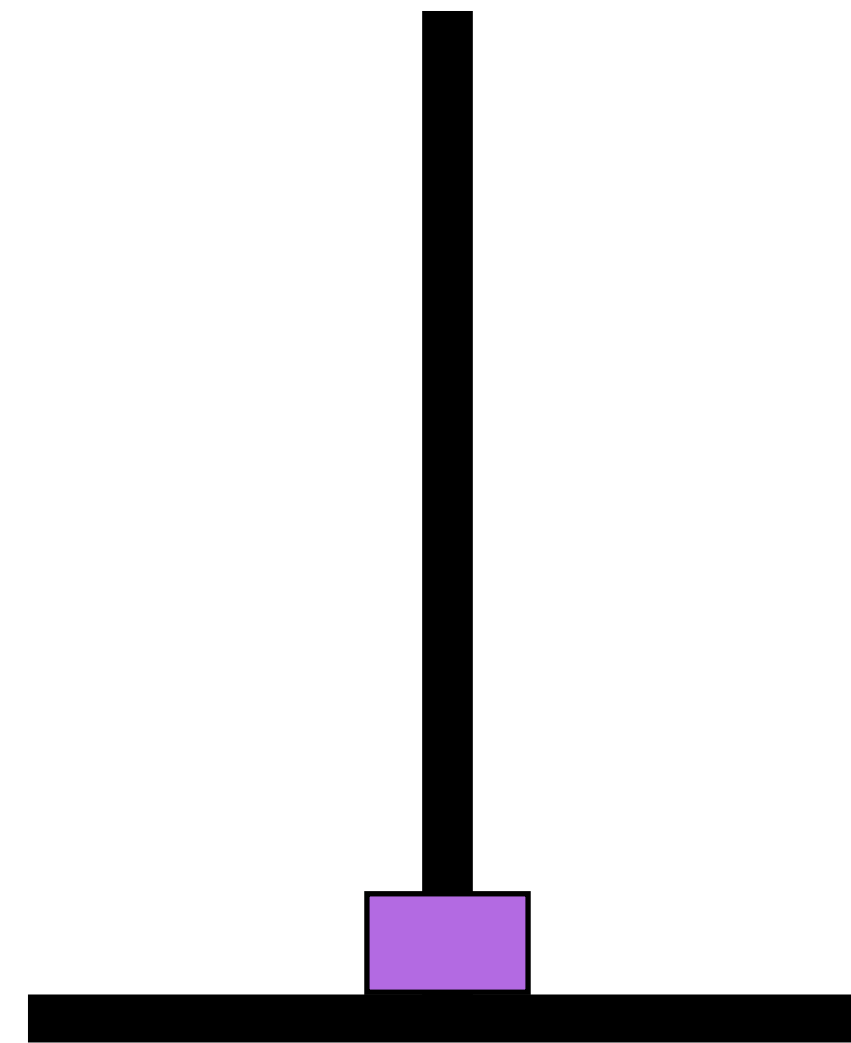
n=4



Stange A



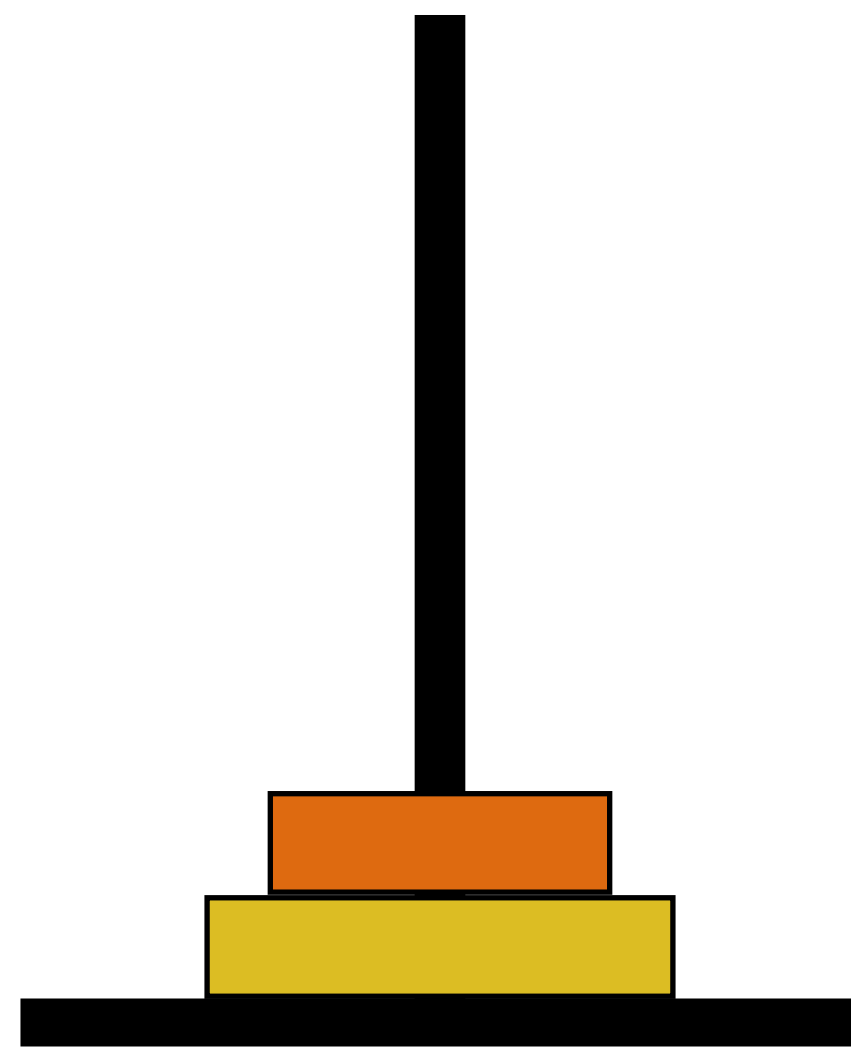
Stange B



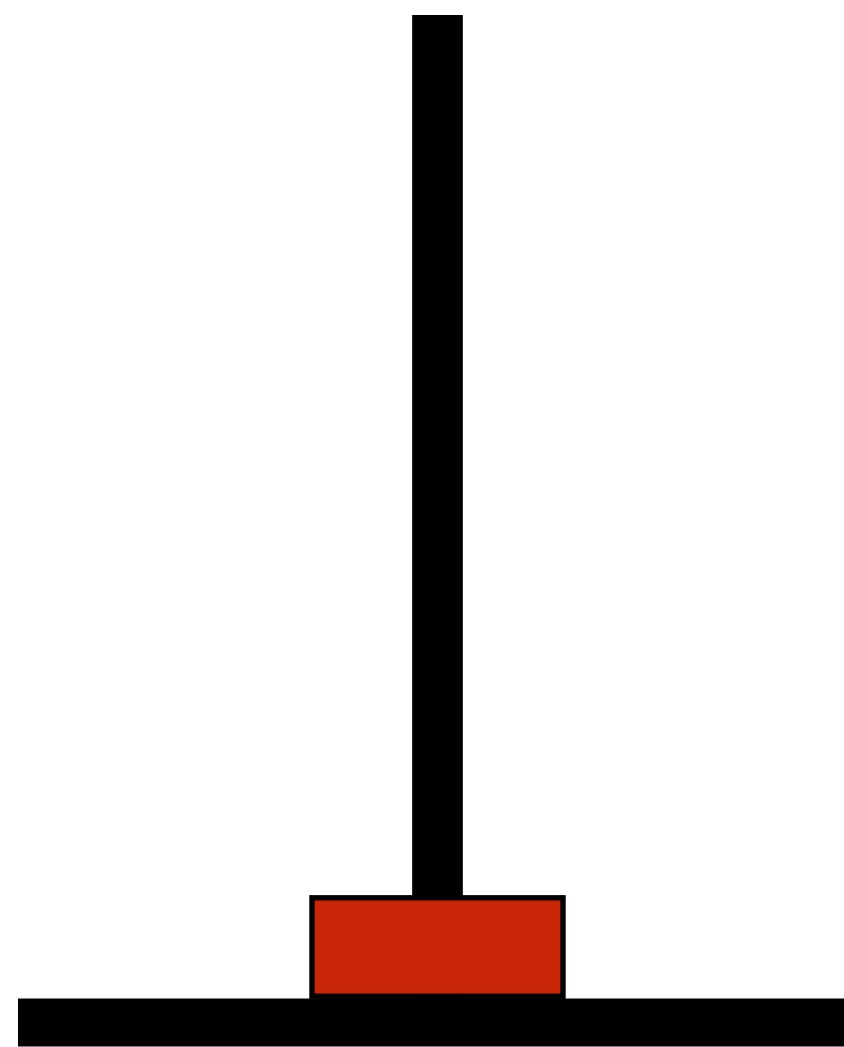
Stange C

Kleine Türme von Hanoi

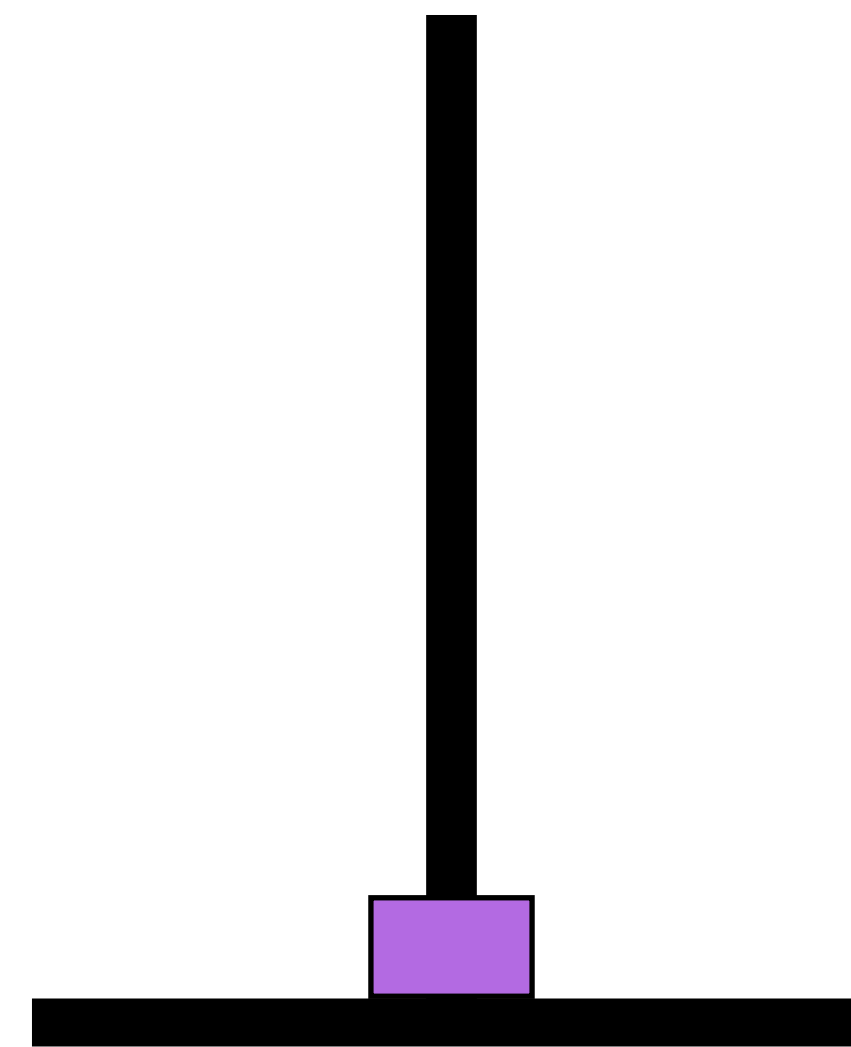
n=4



Stange A



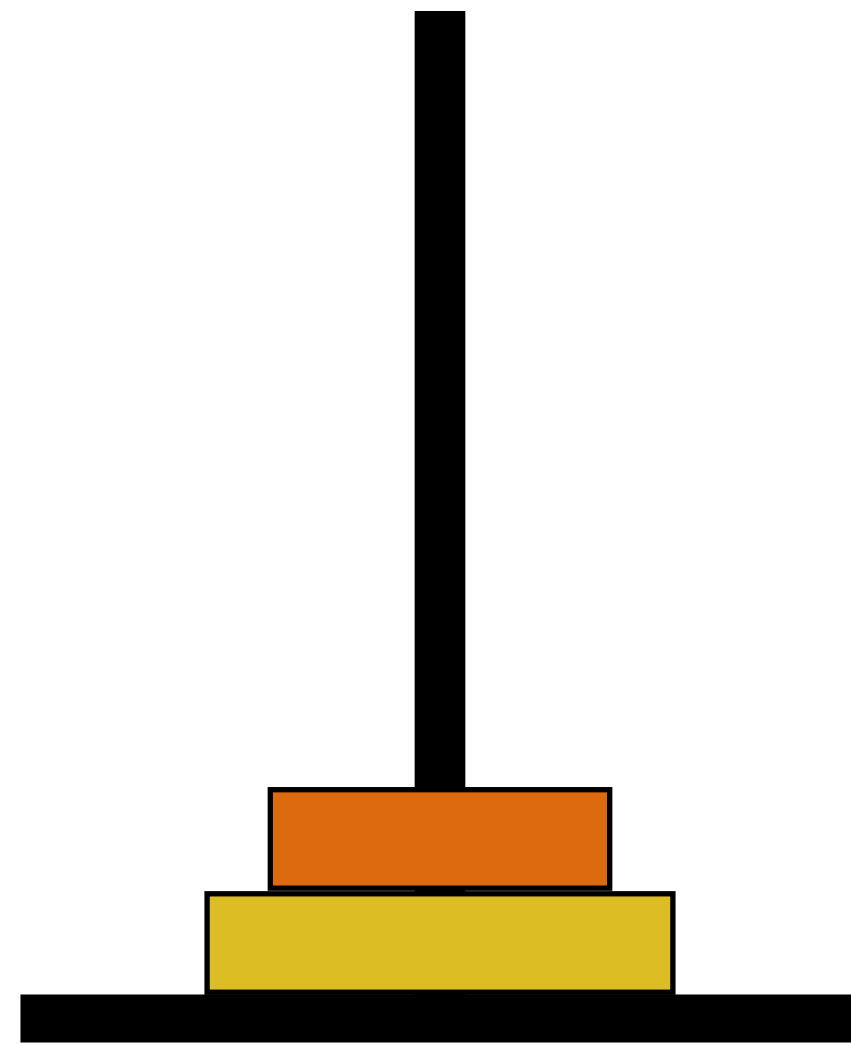
Stange B



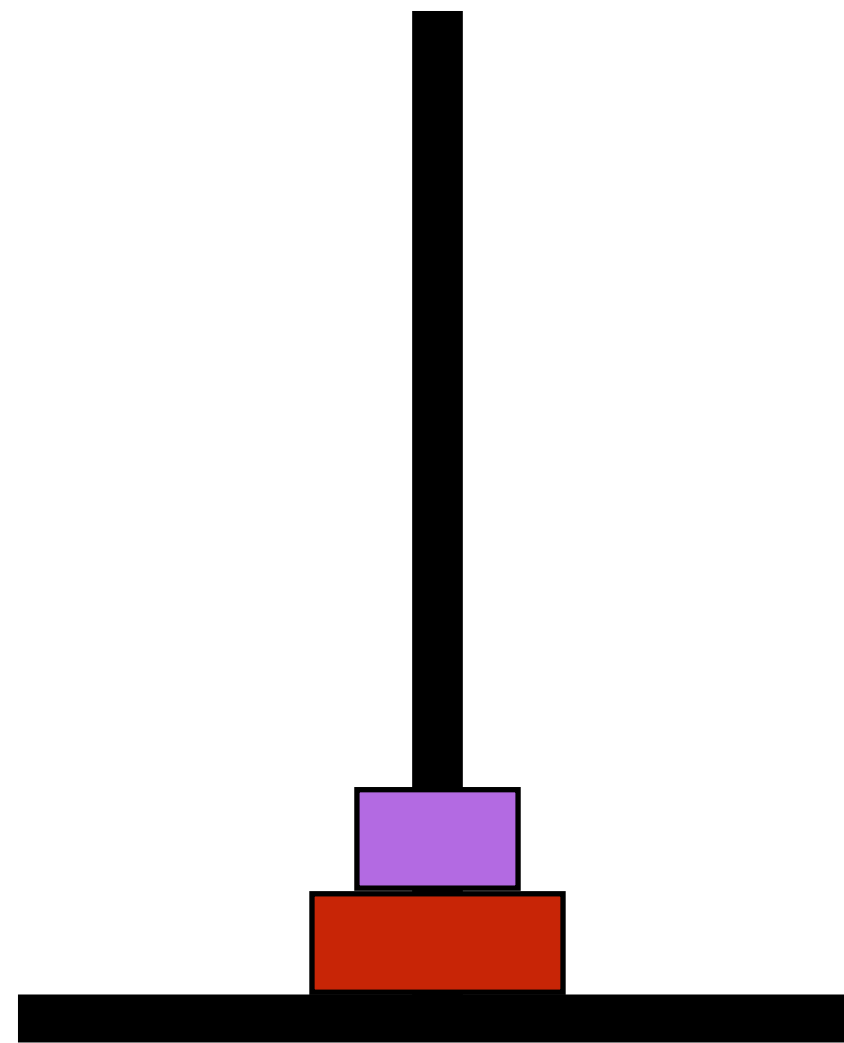
Stange C

Kleine Türme von Hanoi

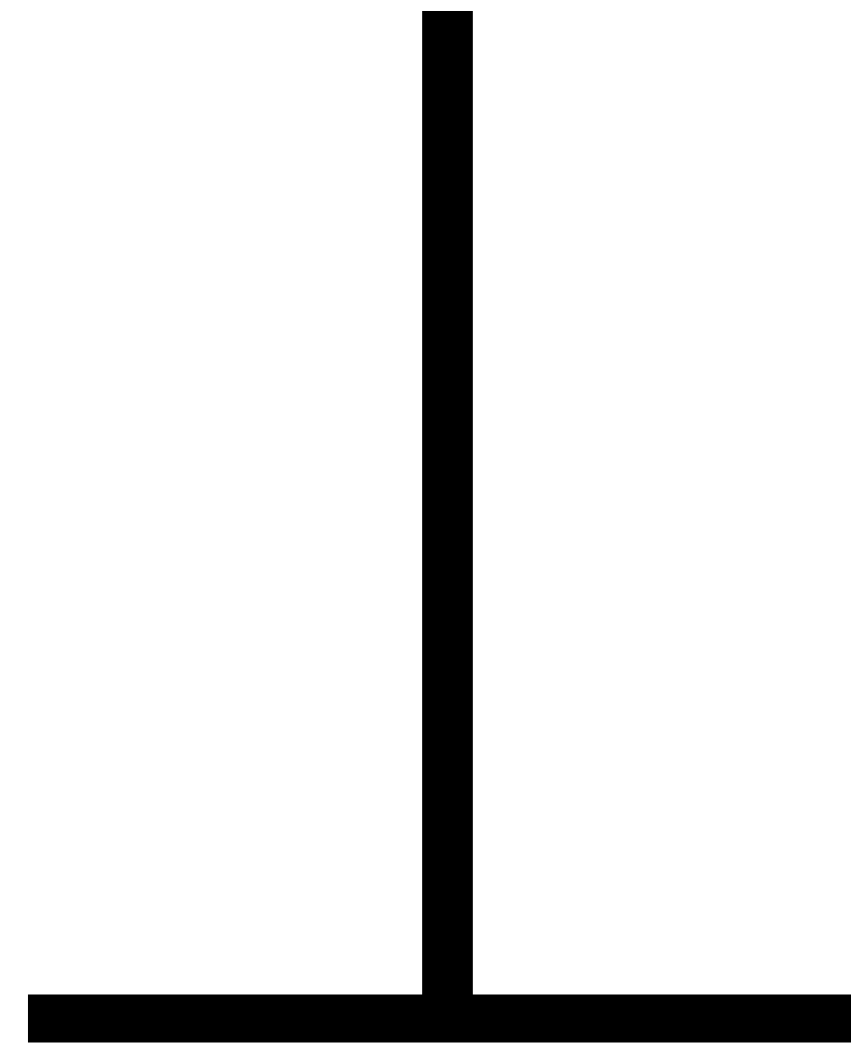
n=4



Stange A



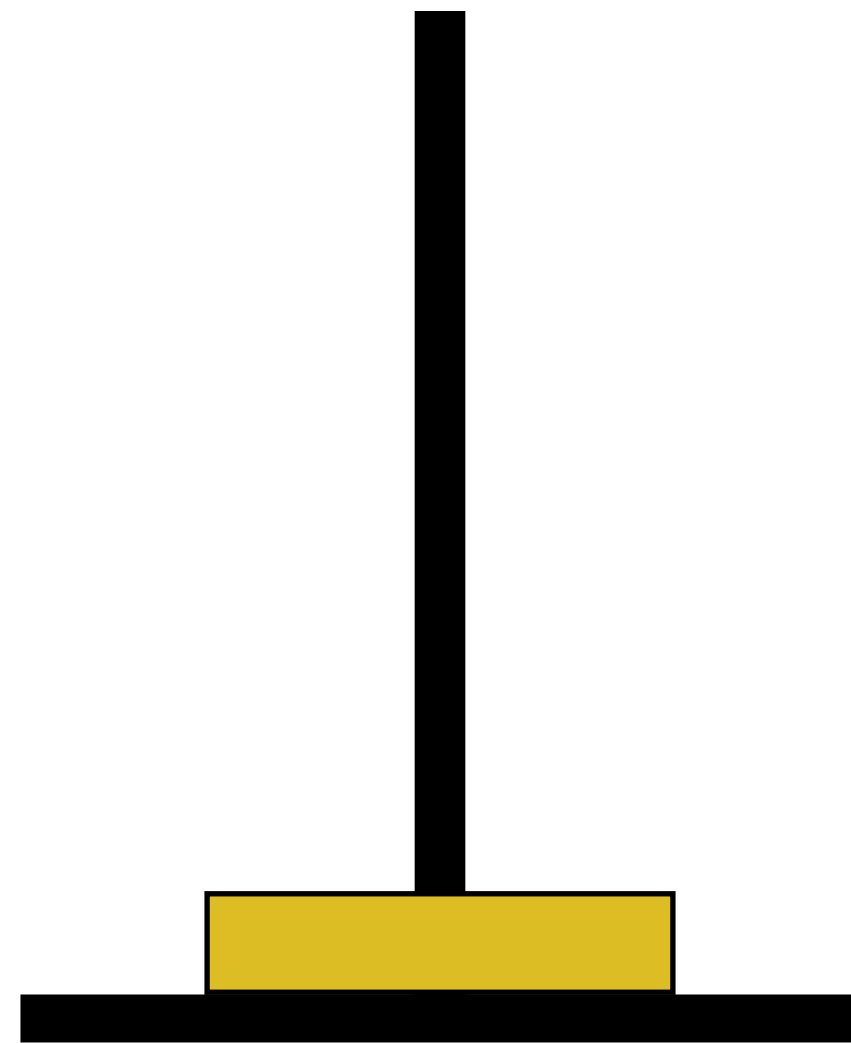
Stange B



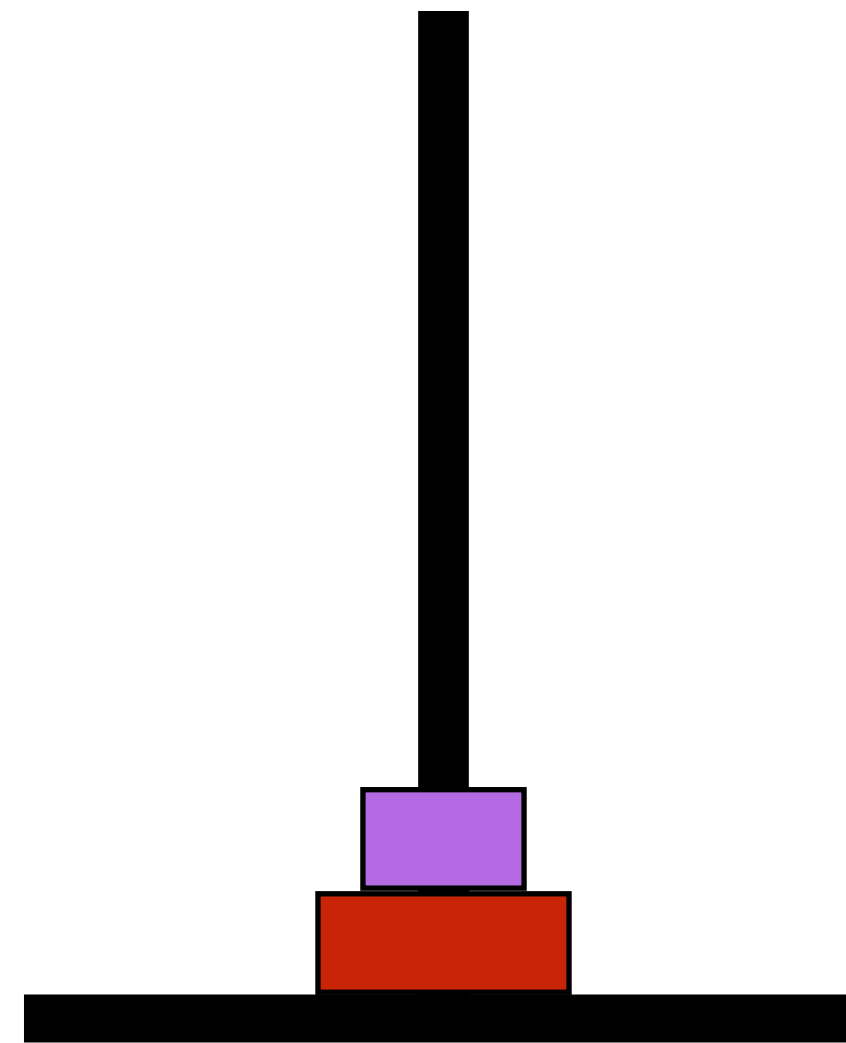
Stange C

Kleine Türme von Hanoi

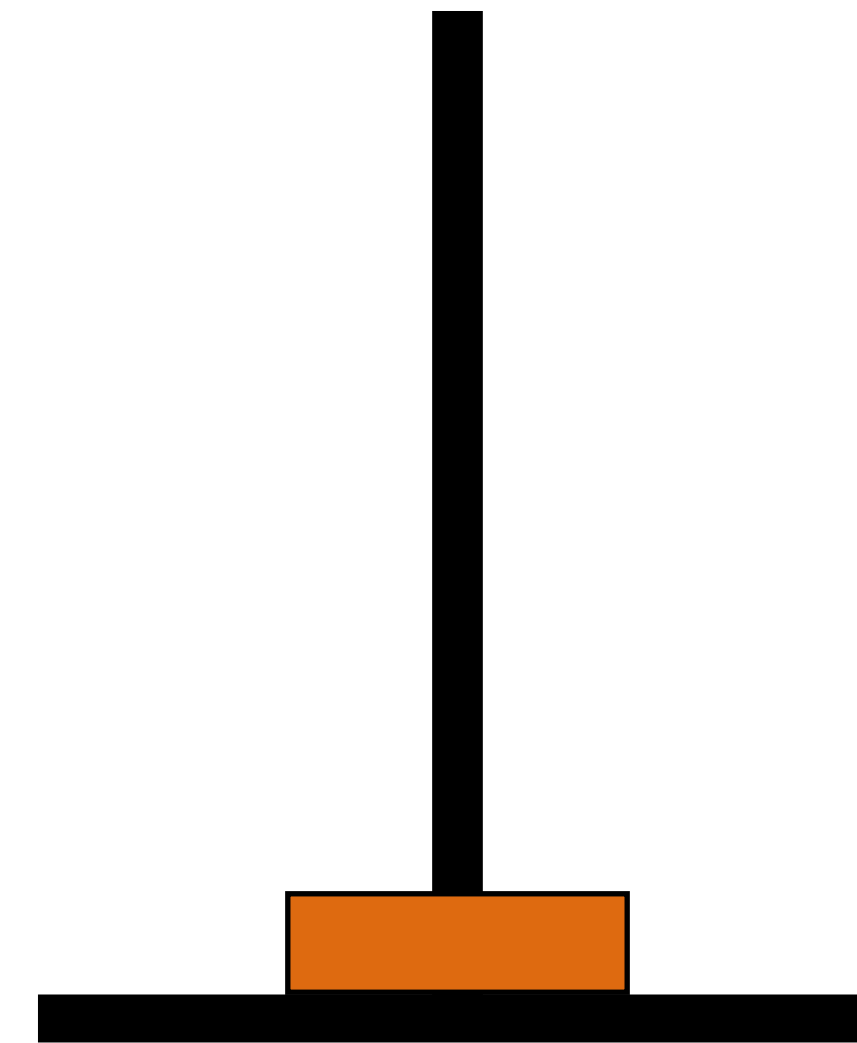
n=4



Stange A



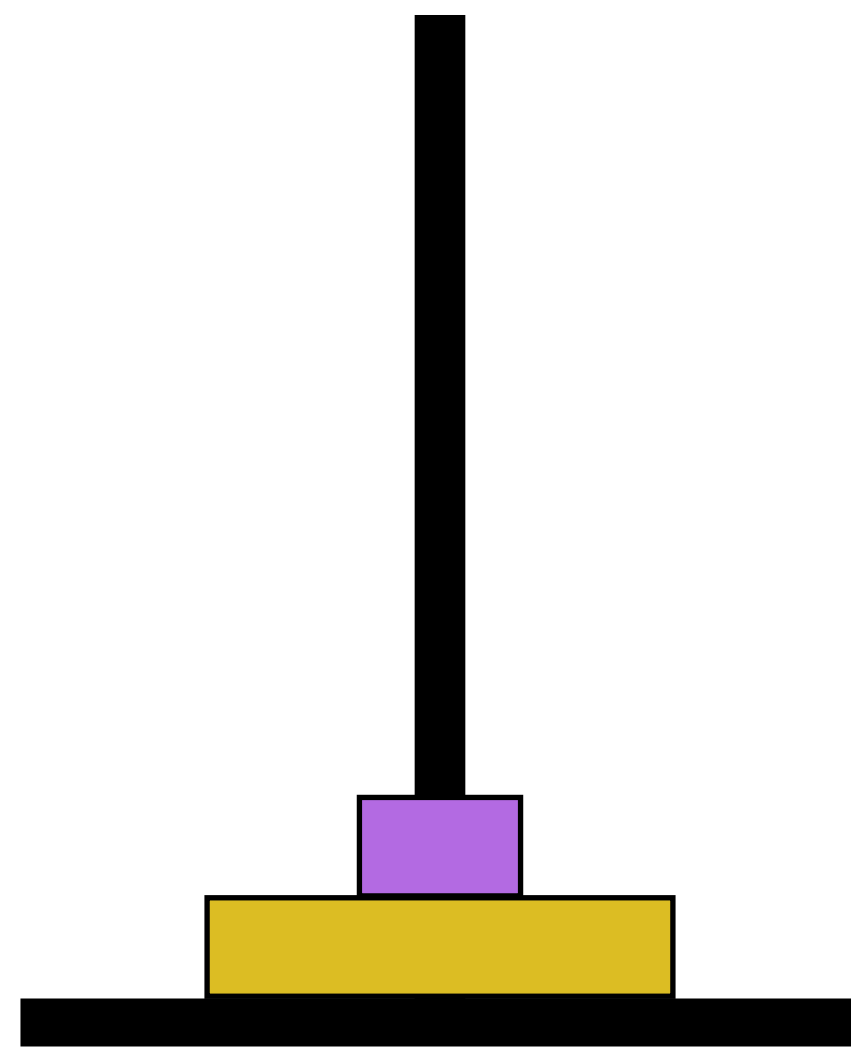
Stange B



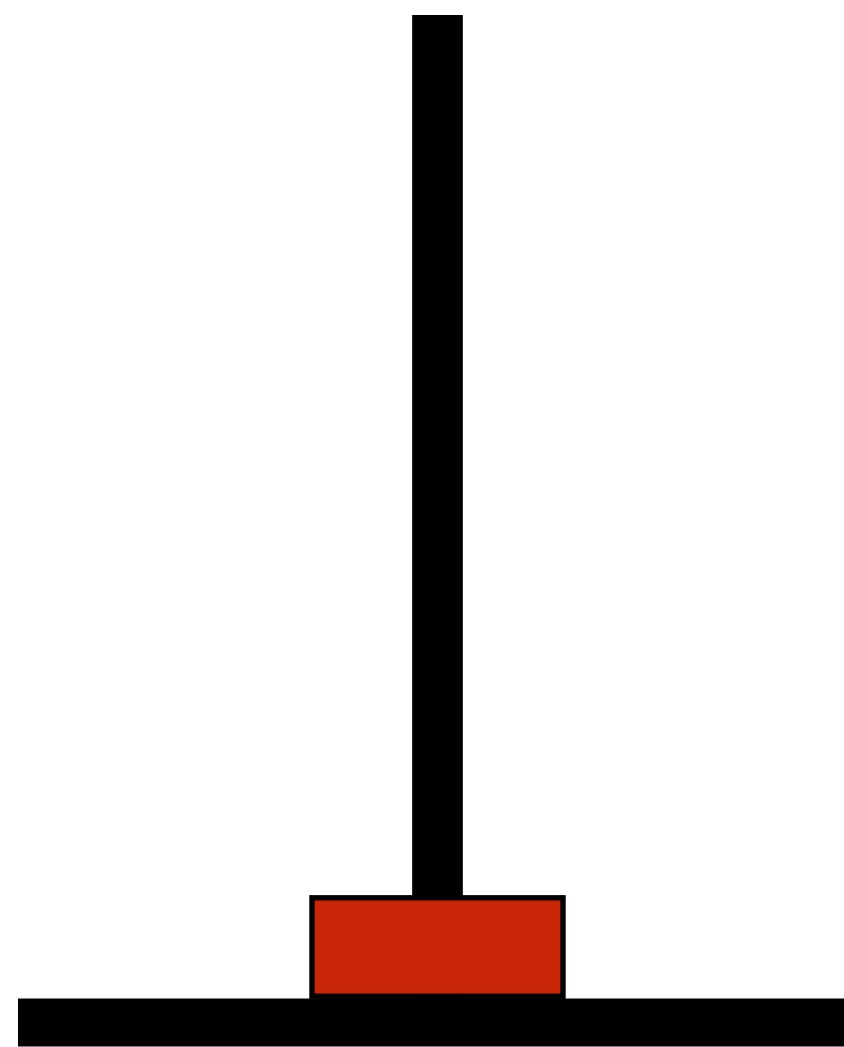
Stange C

Kleine Türme von Hanoi

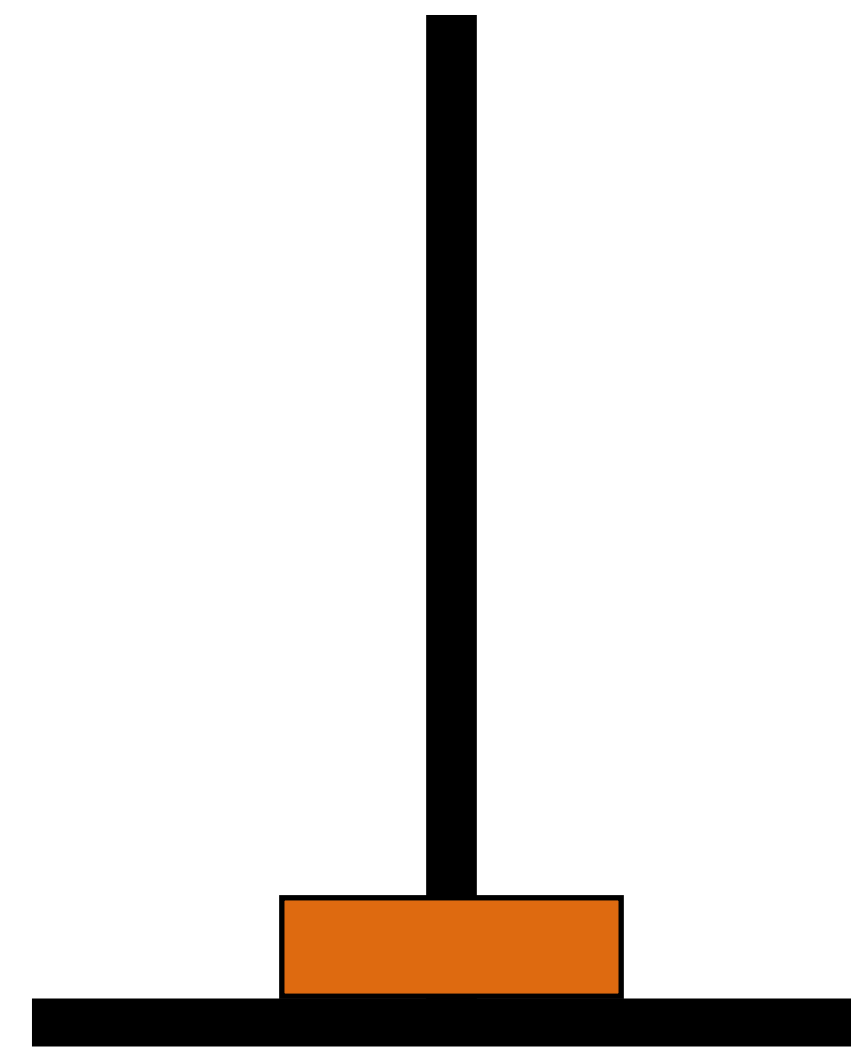
n=4



Stange A



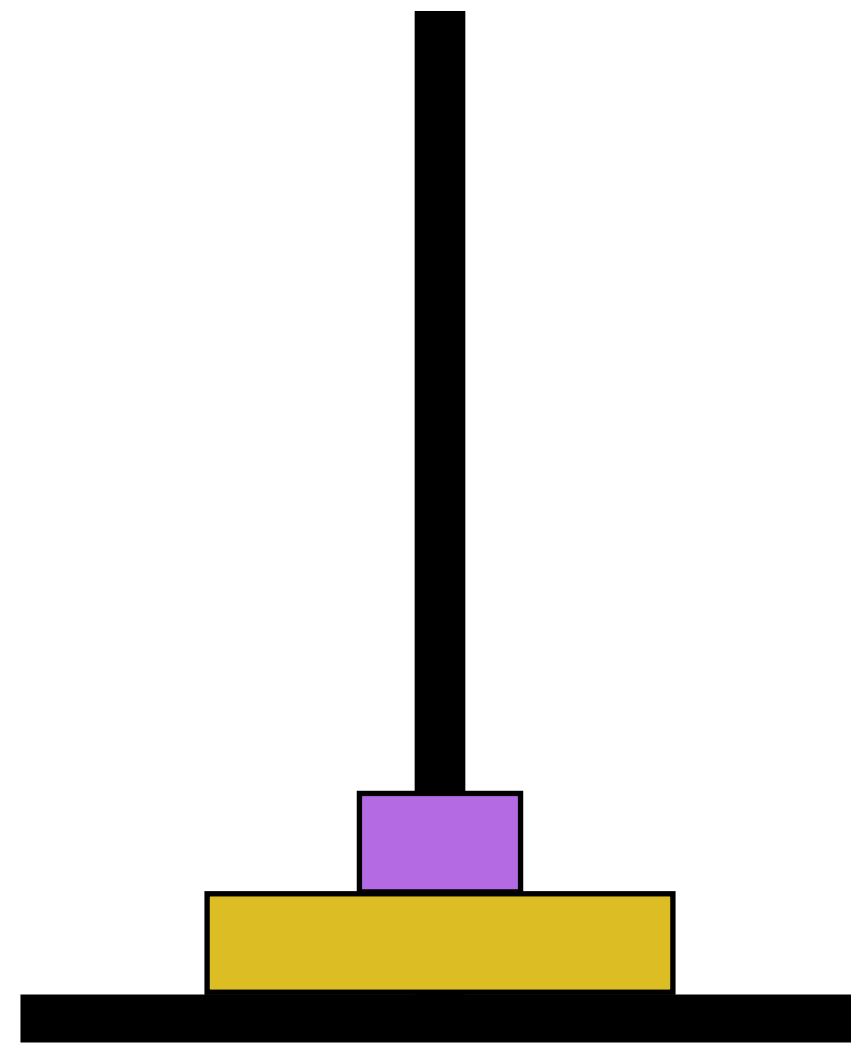
Stange B



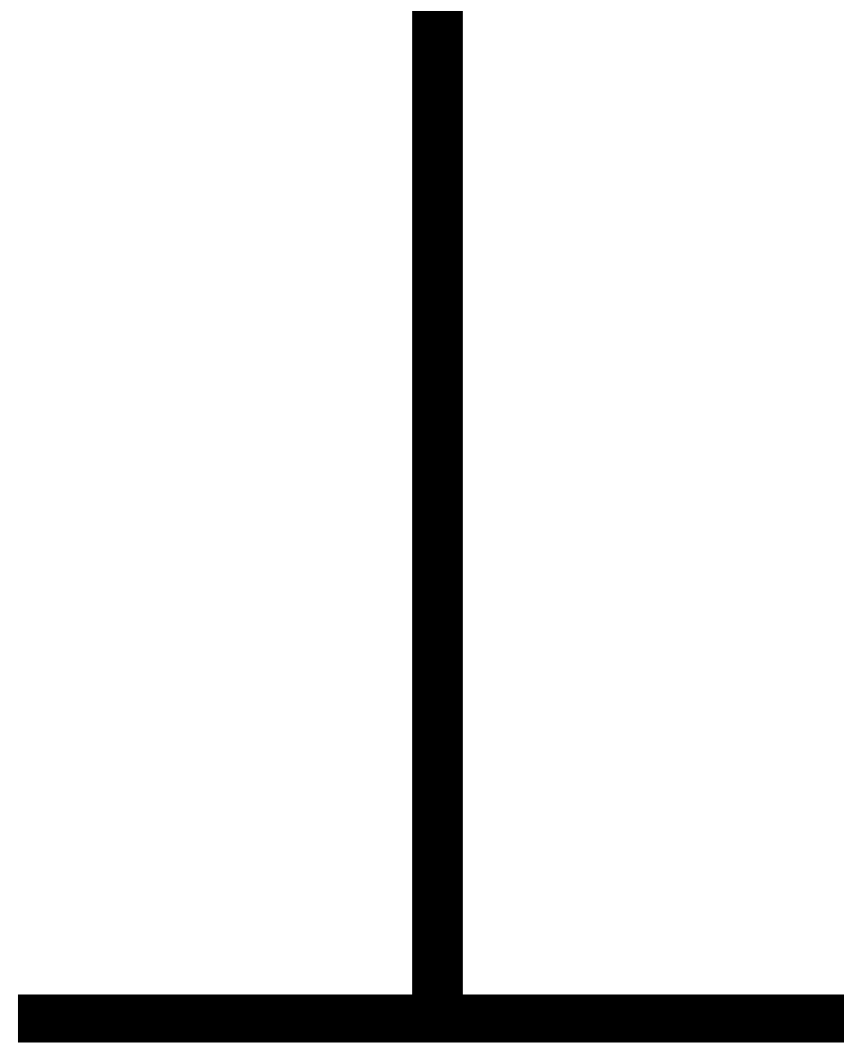
Stange C

Kleine Türme von Hanoi

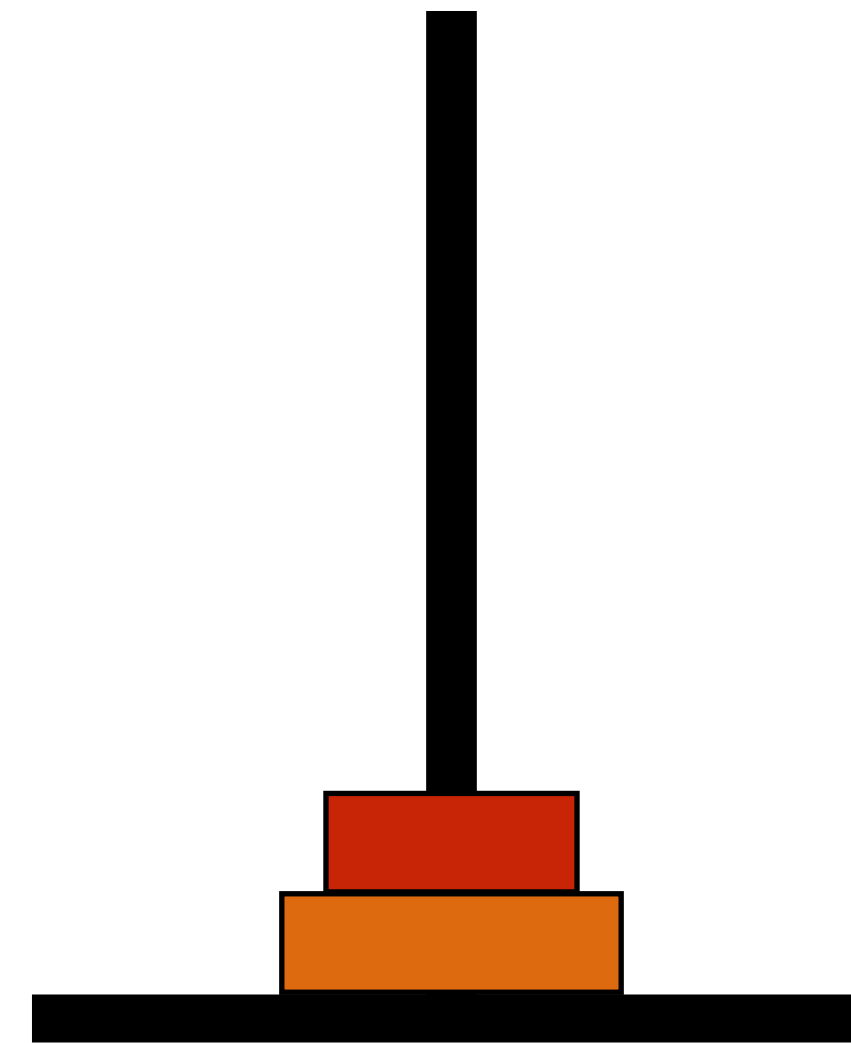
n=4



Stange A



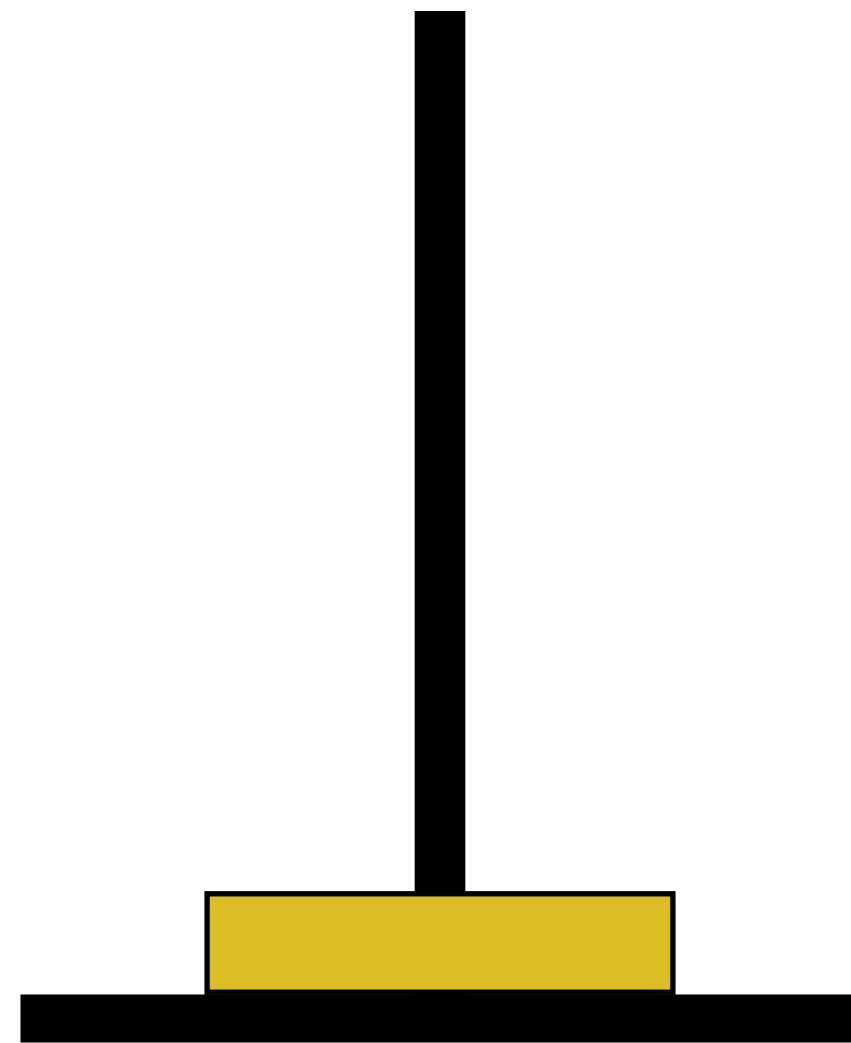
Stange B



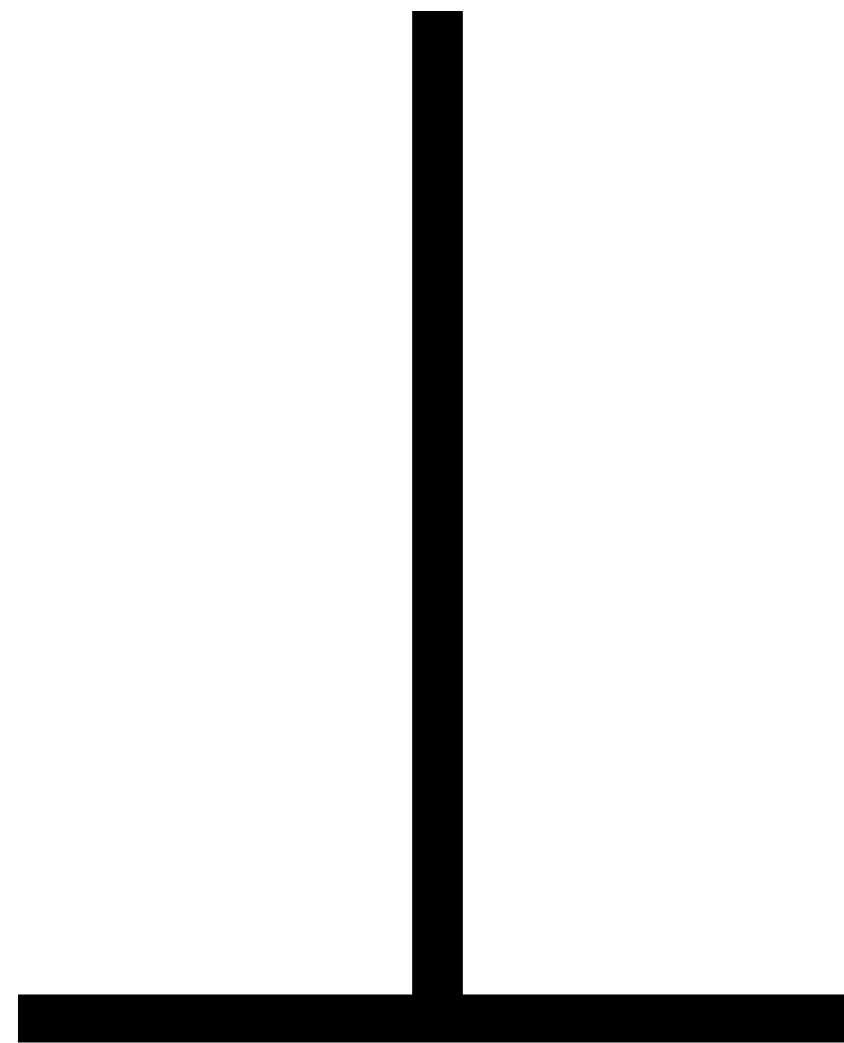
Stange C

Kleine Türme von Hanoi

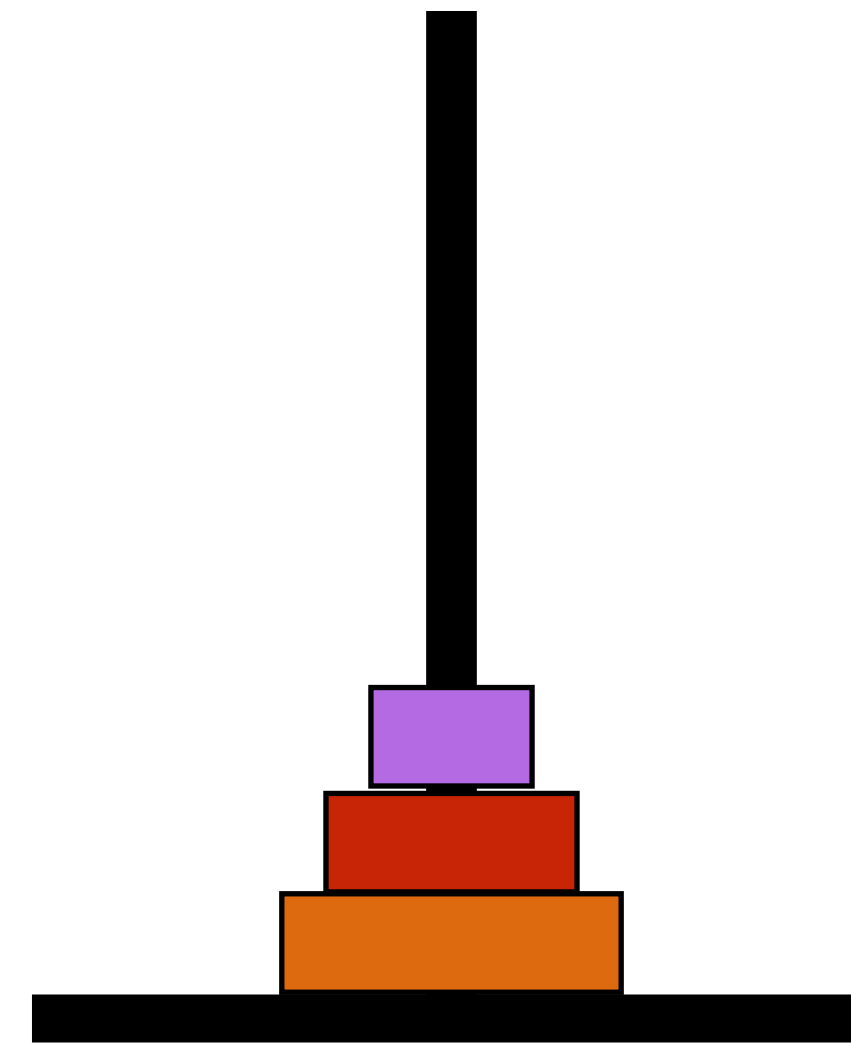
n=4



Stange A



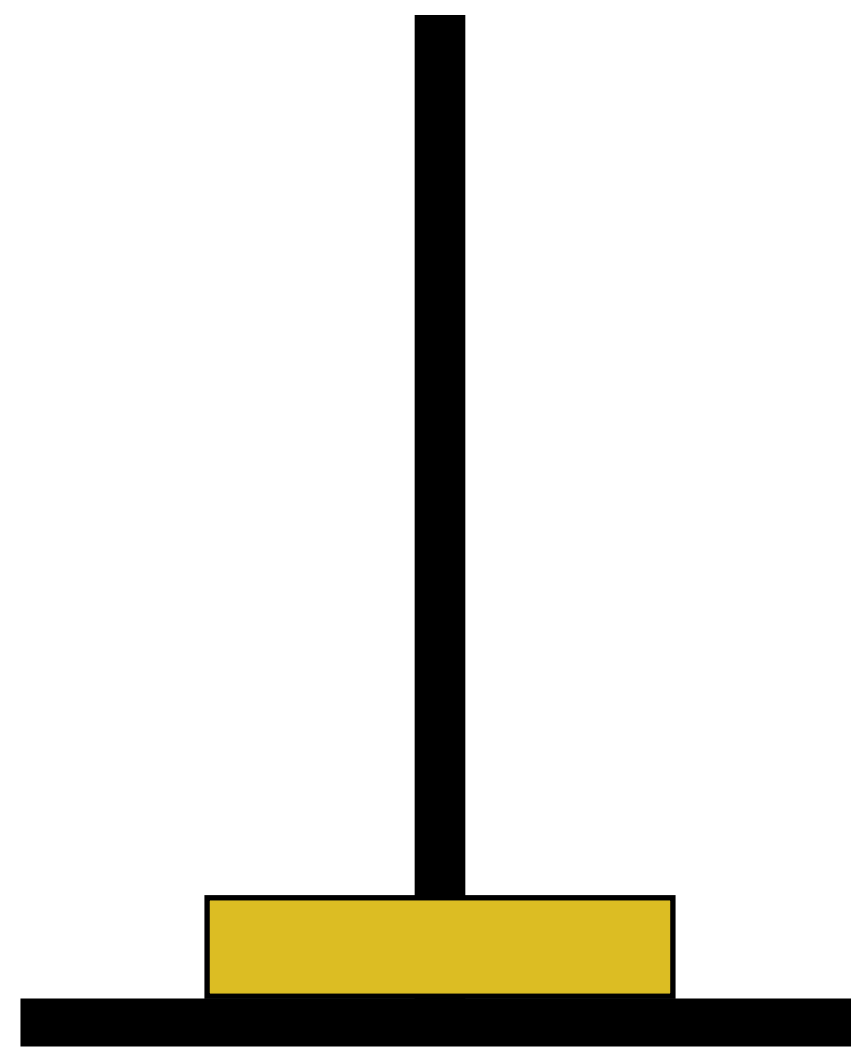
Stange B



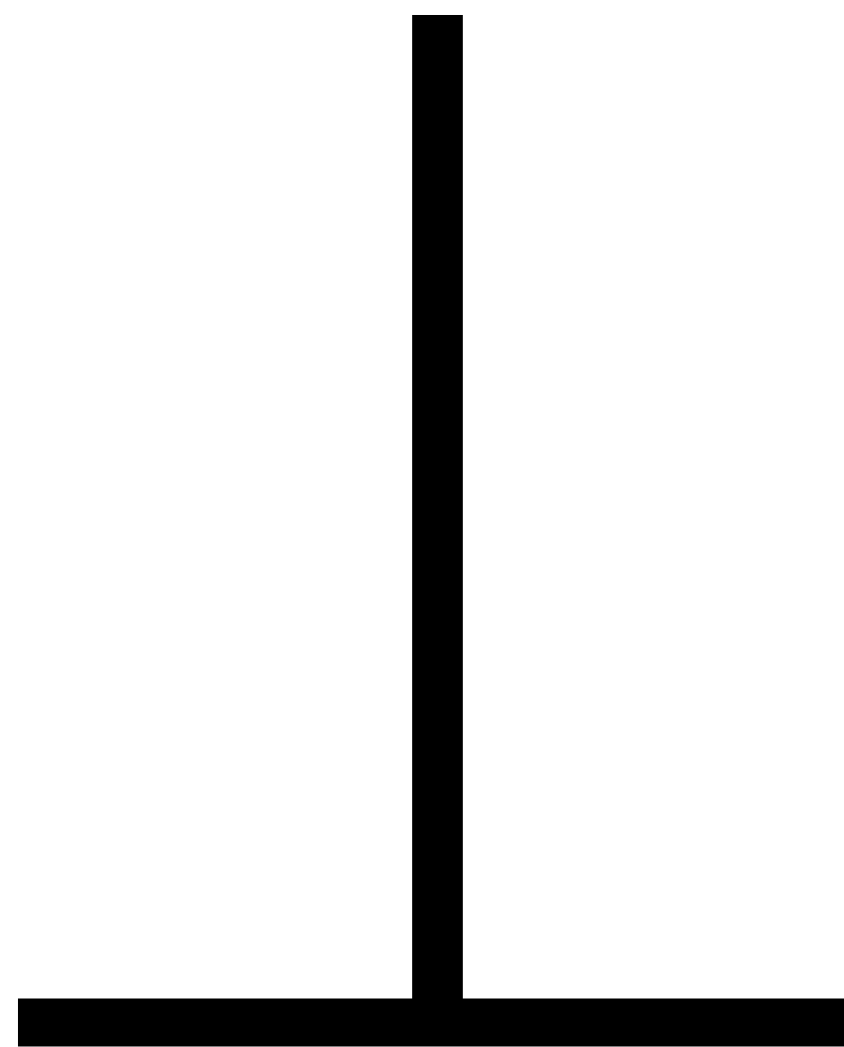
Stange C

Kleine Türme von Hanoi

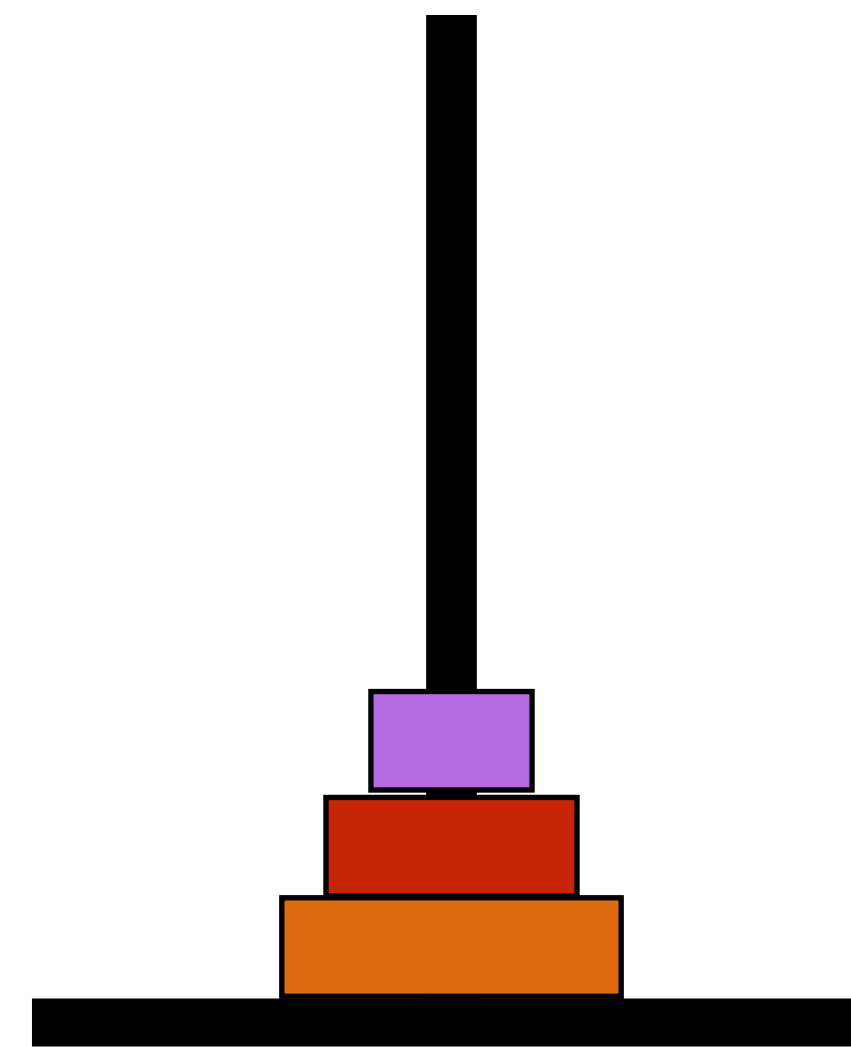
n=4



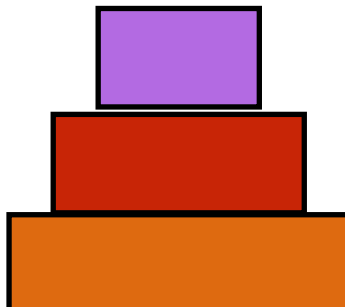
Stange A



Stange B

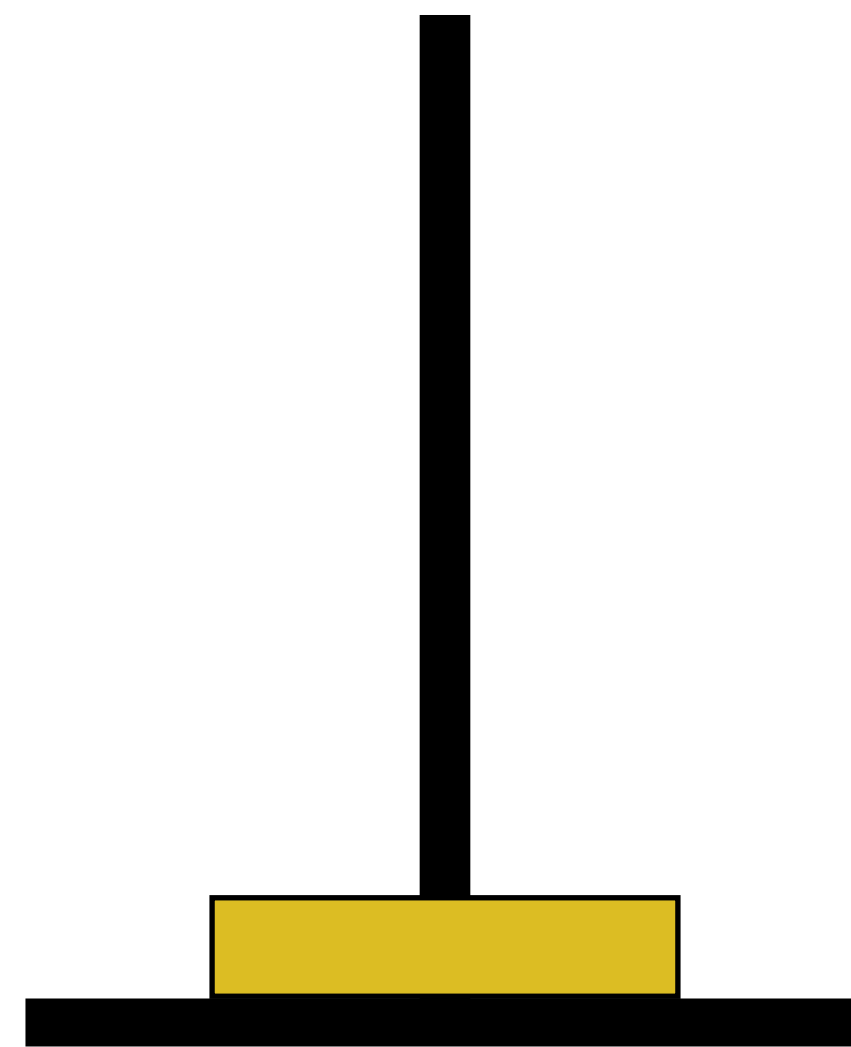


Stange C

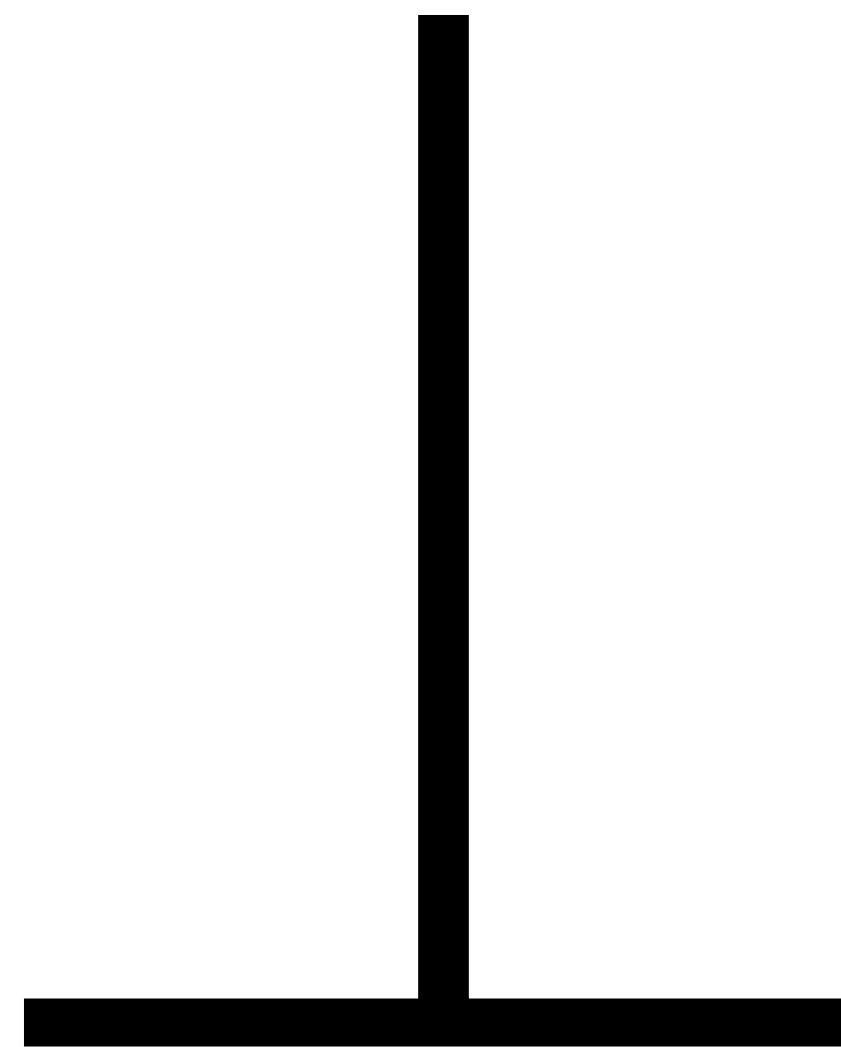
Eigentlich wollten wir  auf Stange C bewegen
direkt nachdem  auf Stange B bewegt wurde.

Kleine Türme von Hanoi

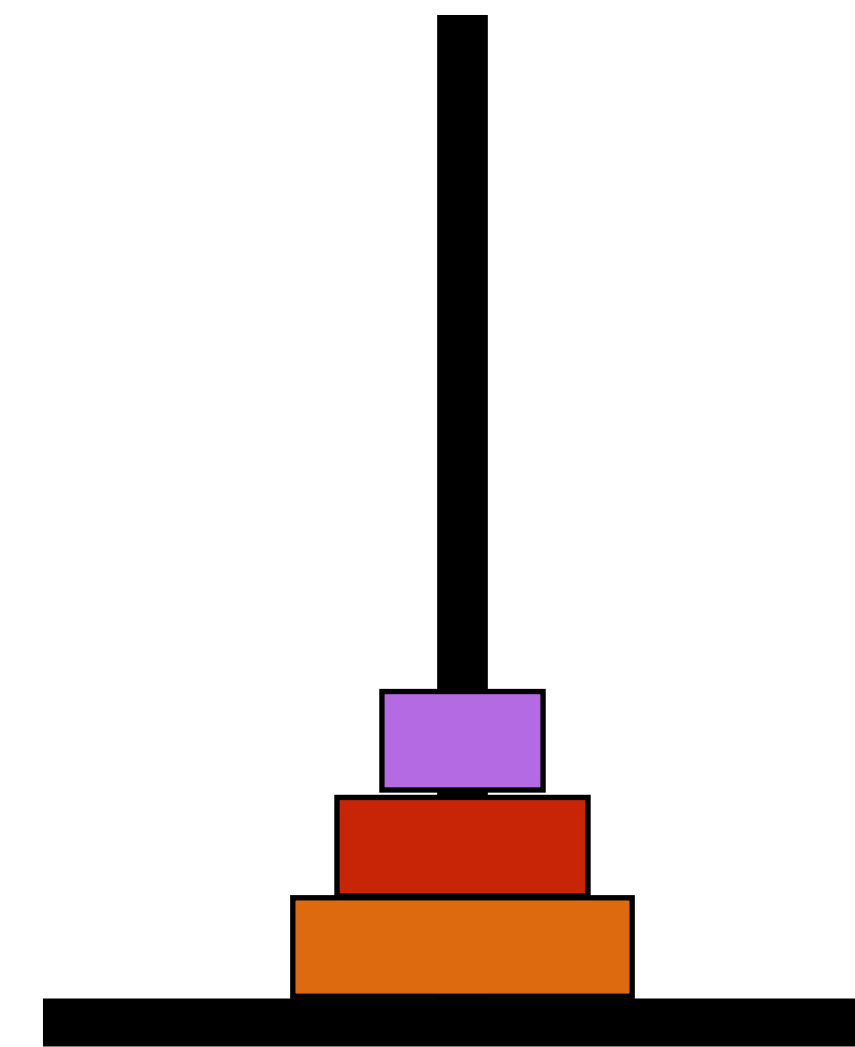
n=4



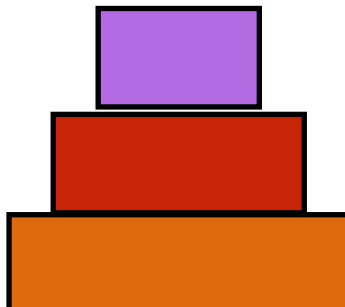
Stange A



Stange B



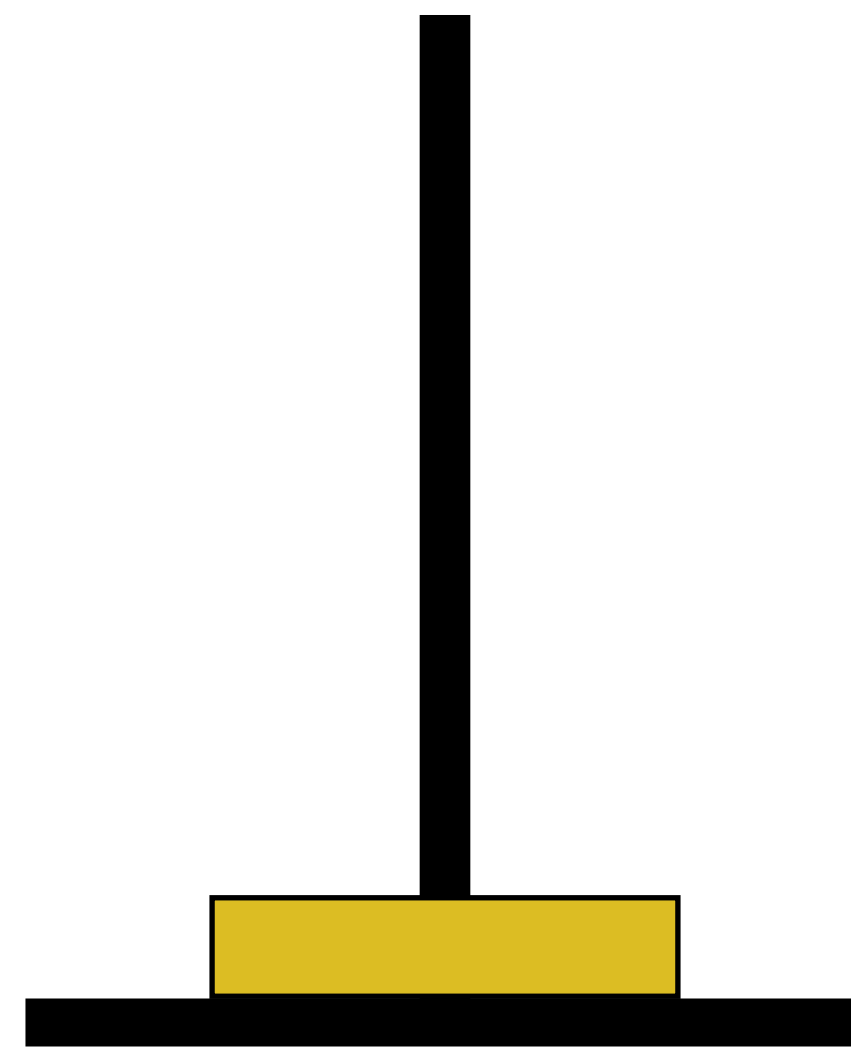
Stange C

Eigentlich wollten wir  auf Stange C bewegen
direkt nachdem  auf Stange B bewegt wurde.

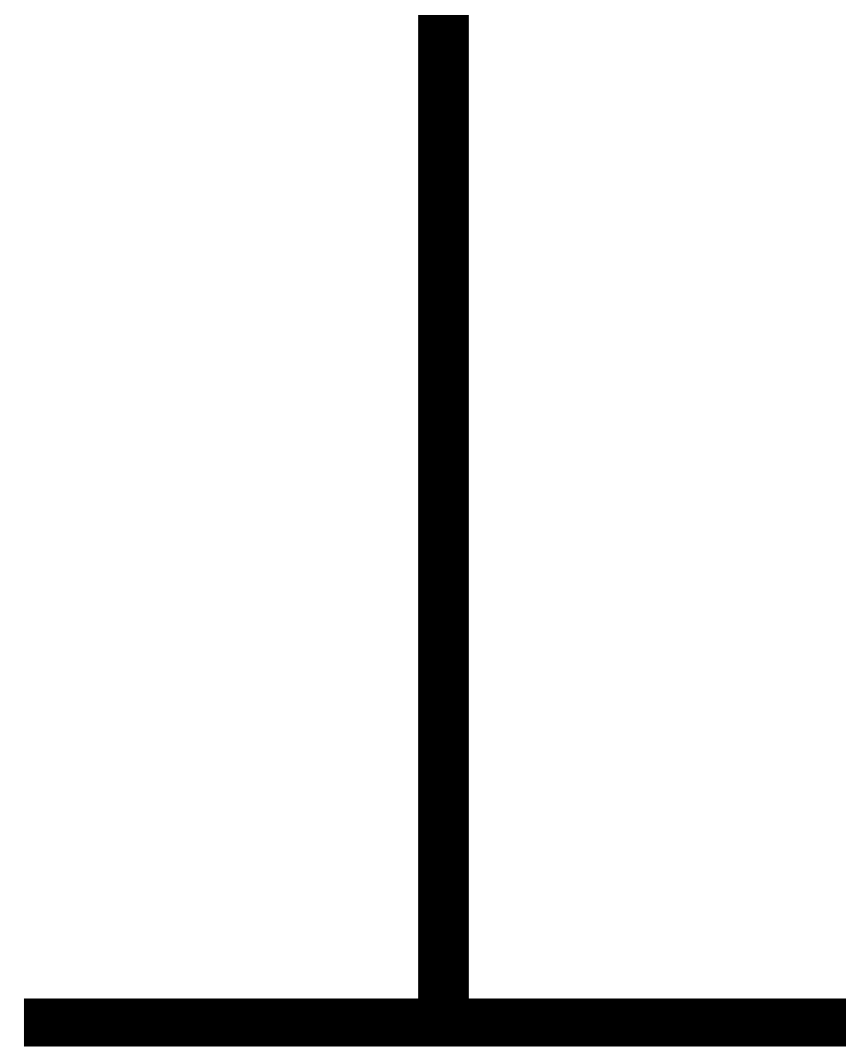


Kleine Türme von Hanoi

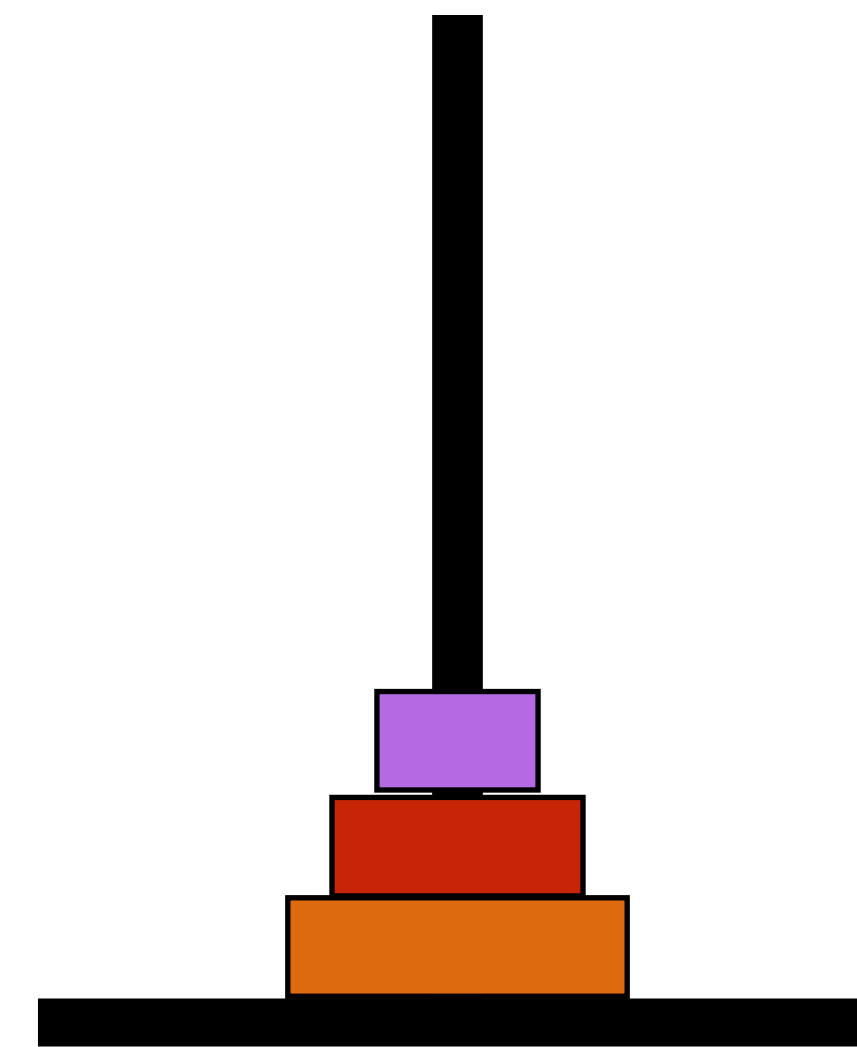
n=4



Stange A



Stange B

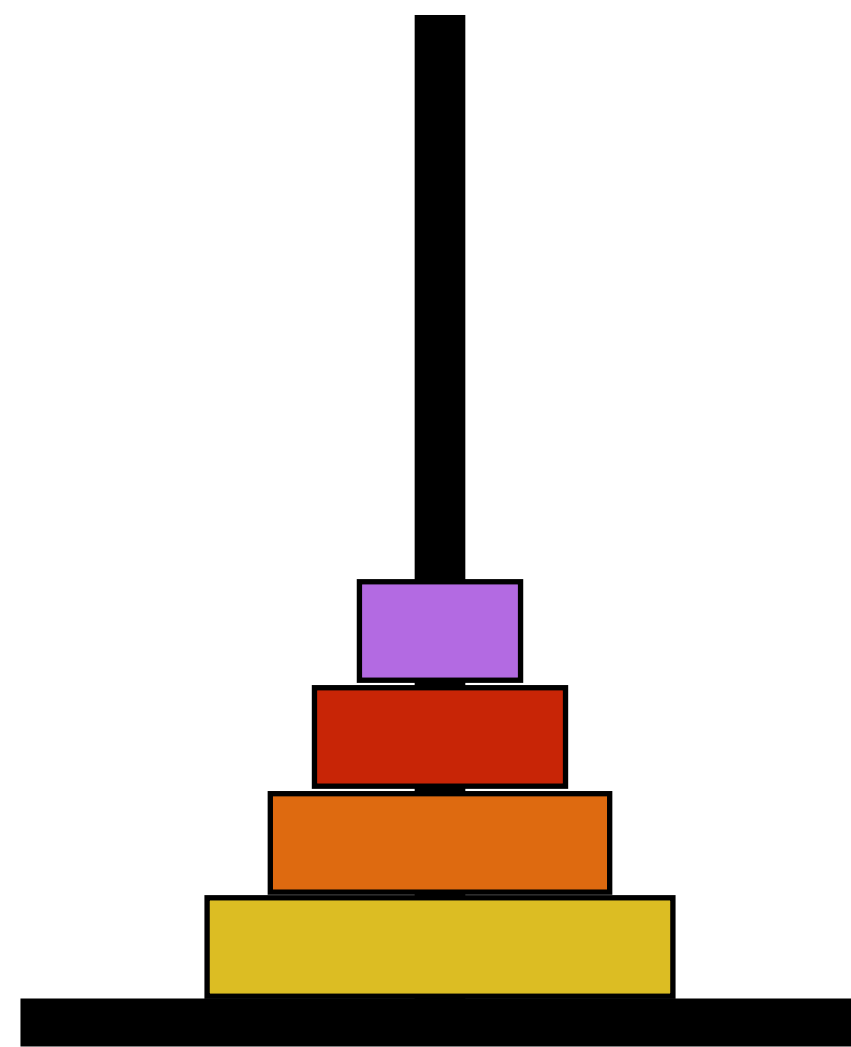


Stange C

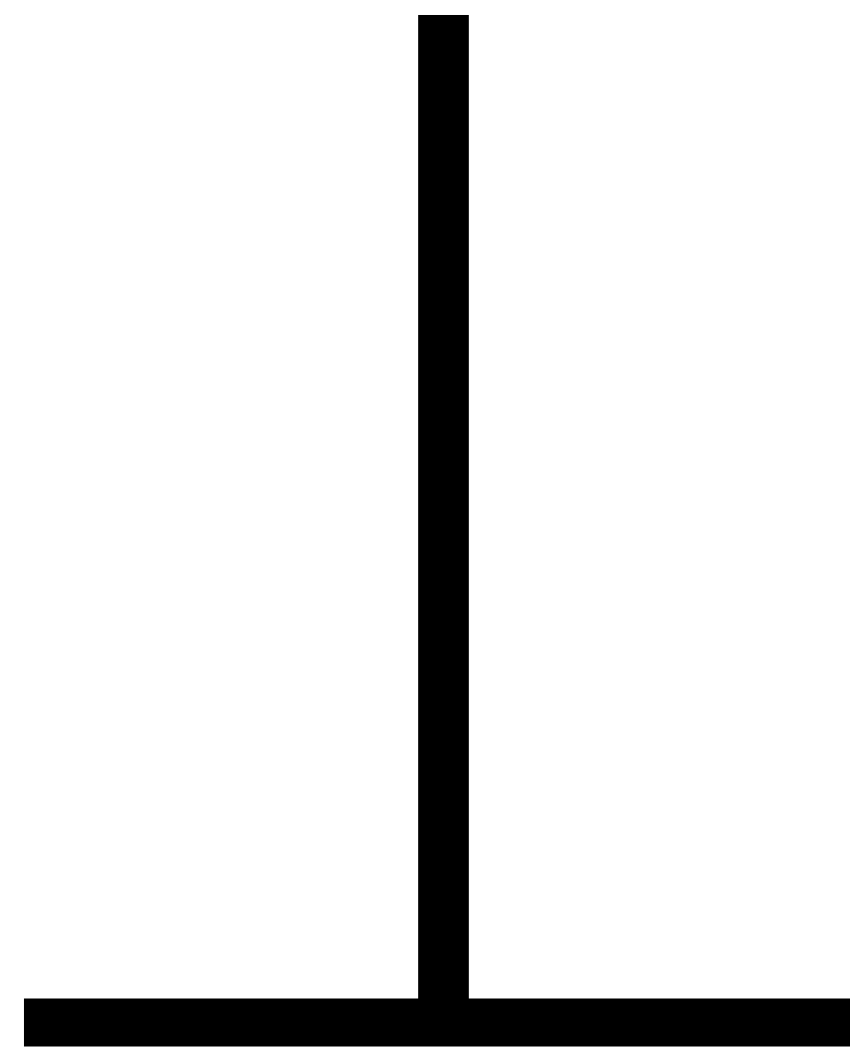
Versuchen wir es also erneut.

Kleine Türme von Hanoi

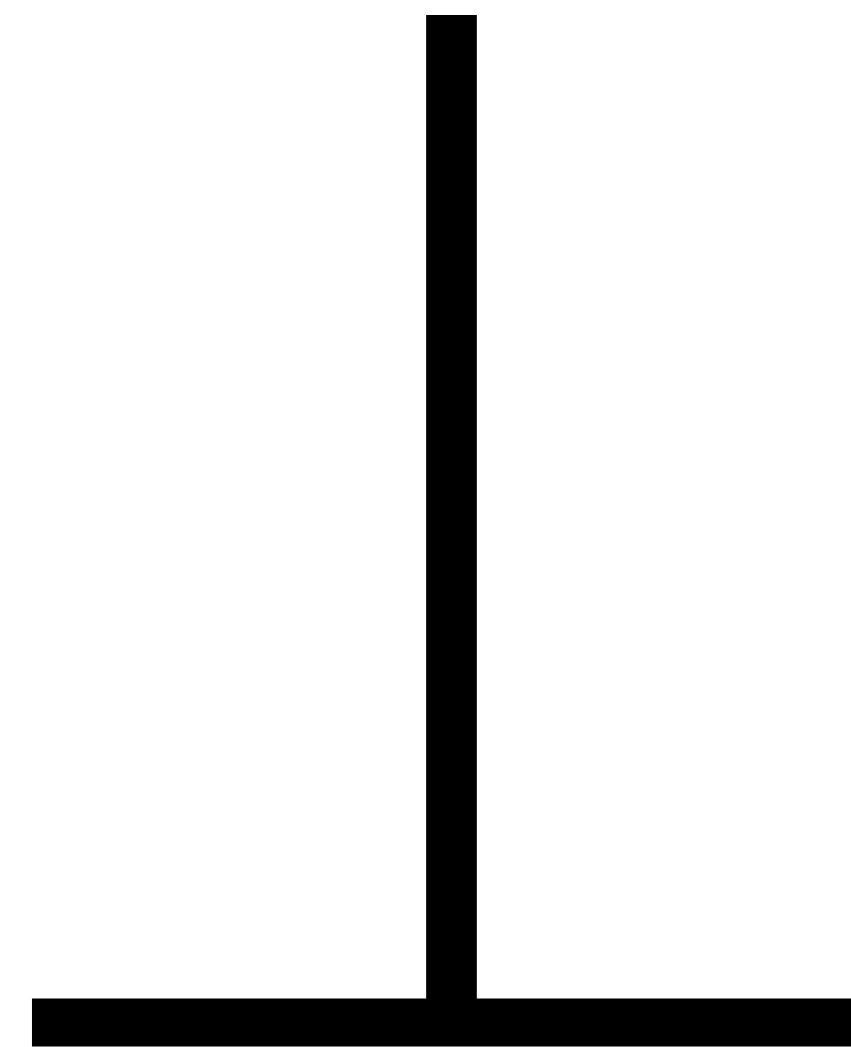
n=4



Stange A



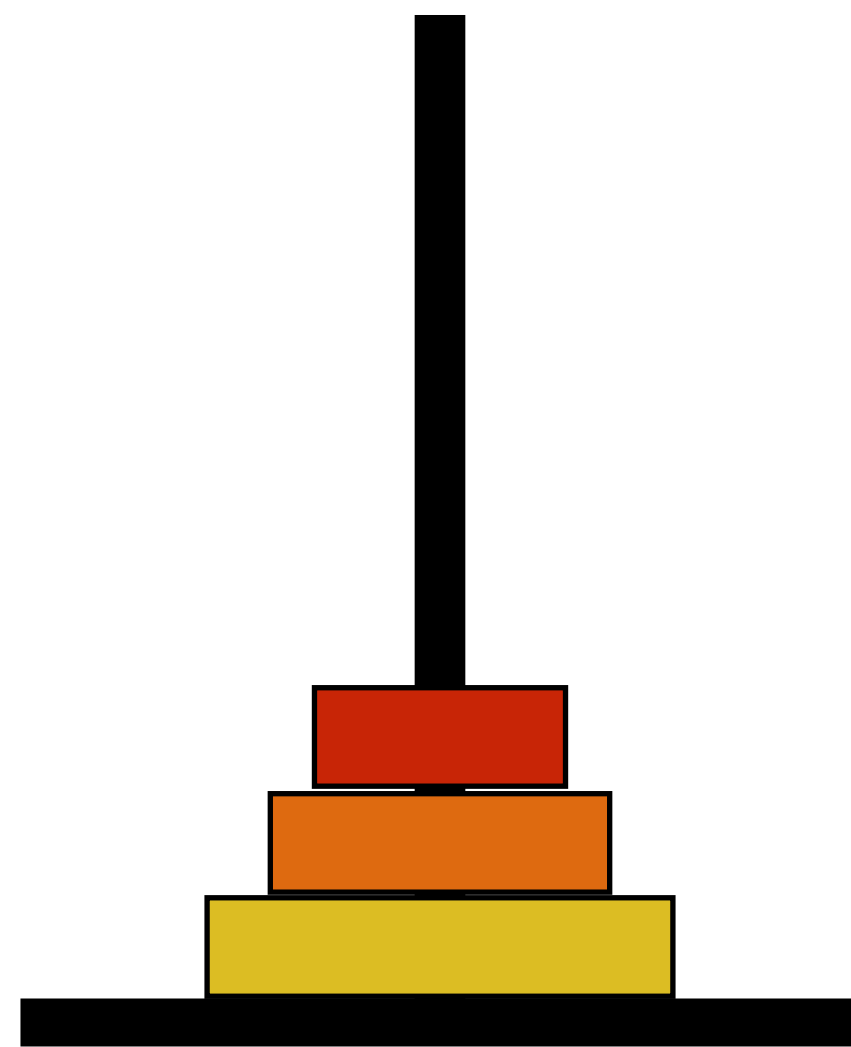
Stange B



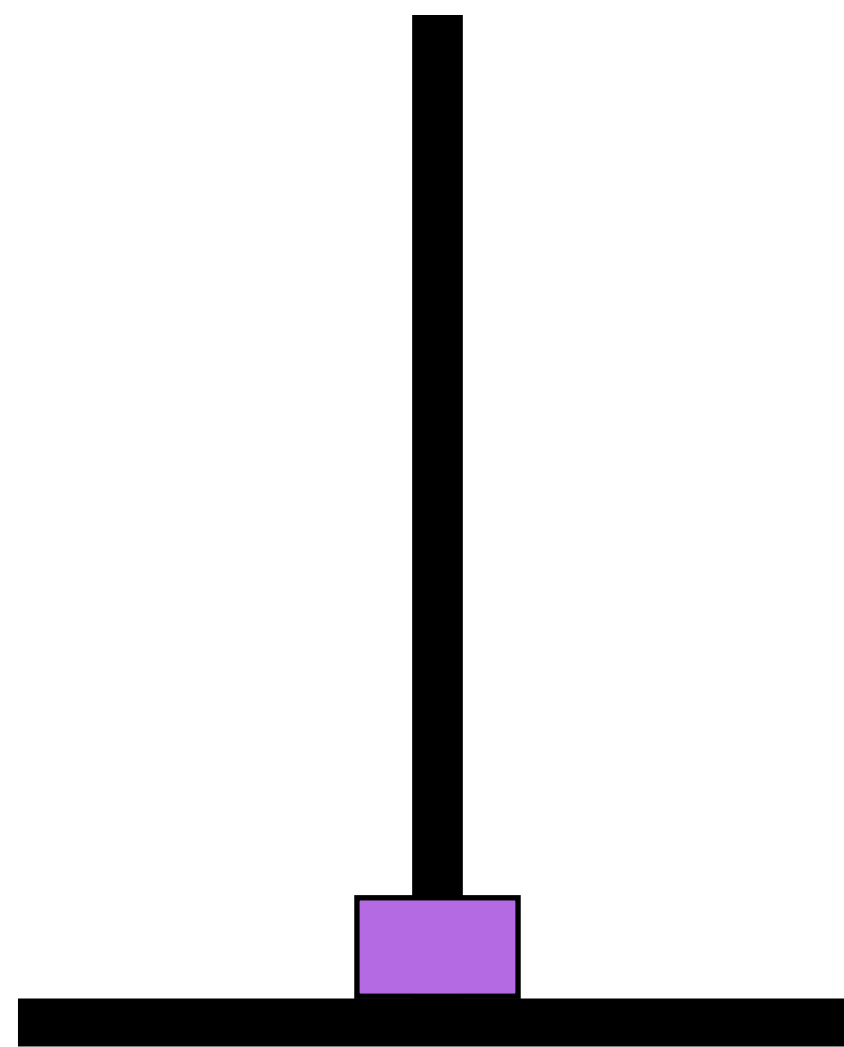
Stange C

Kleine Türme von Hanoi

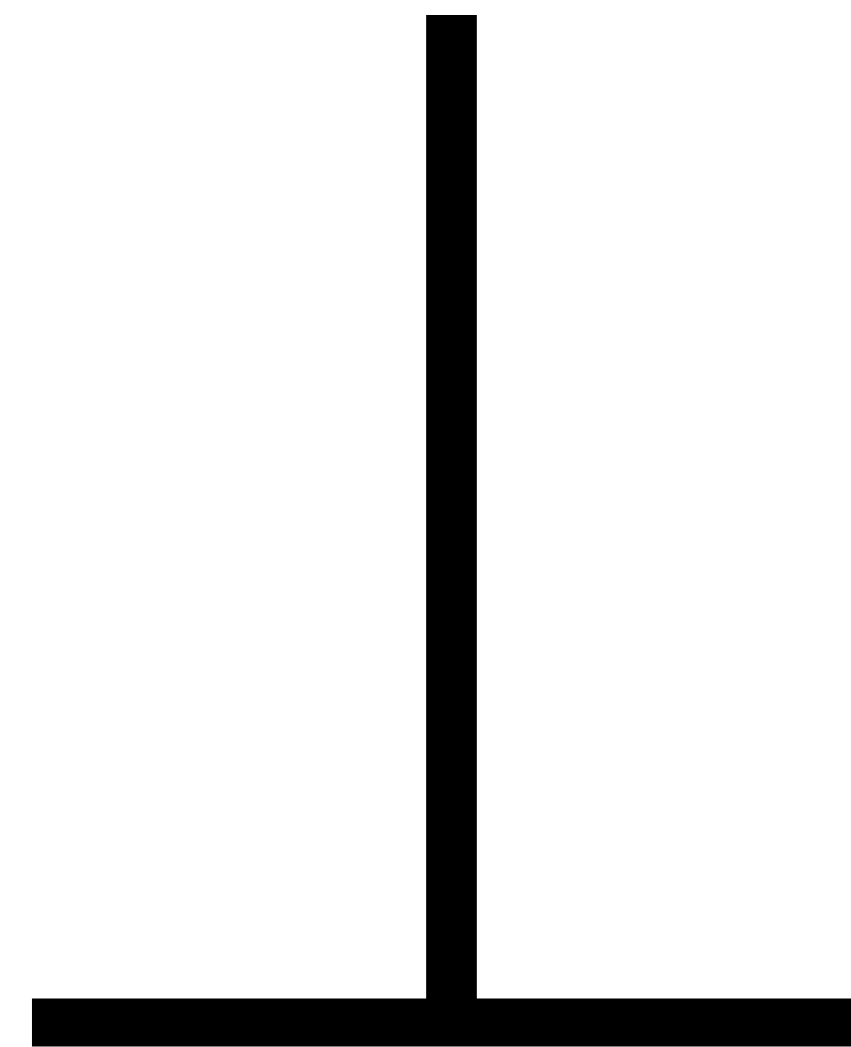
n=4



Stange A



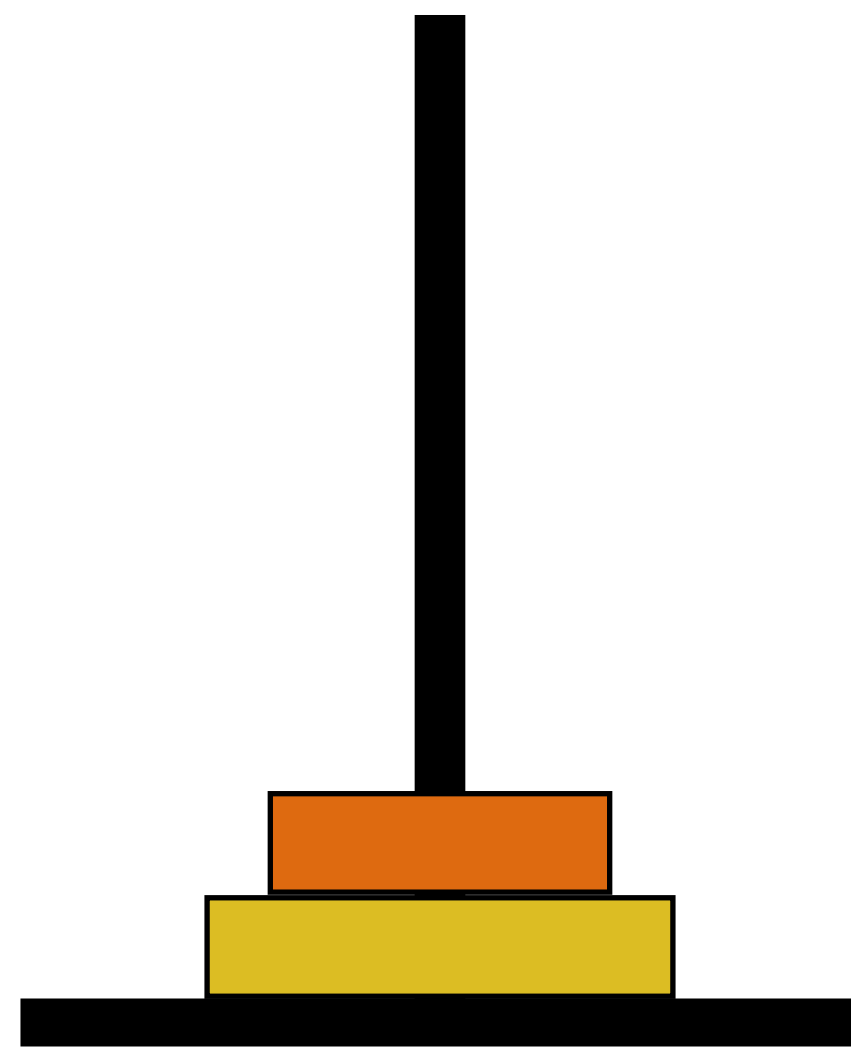
Stange B



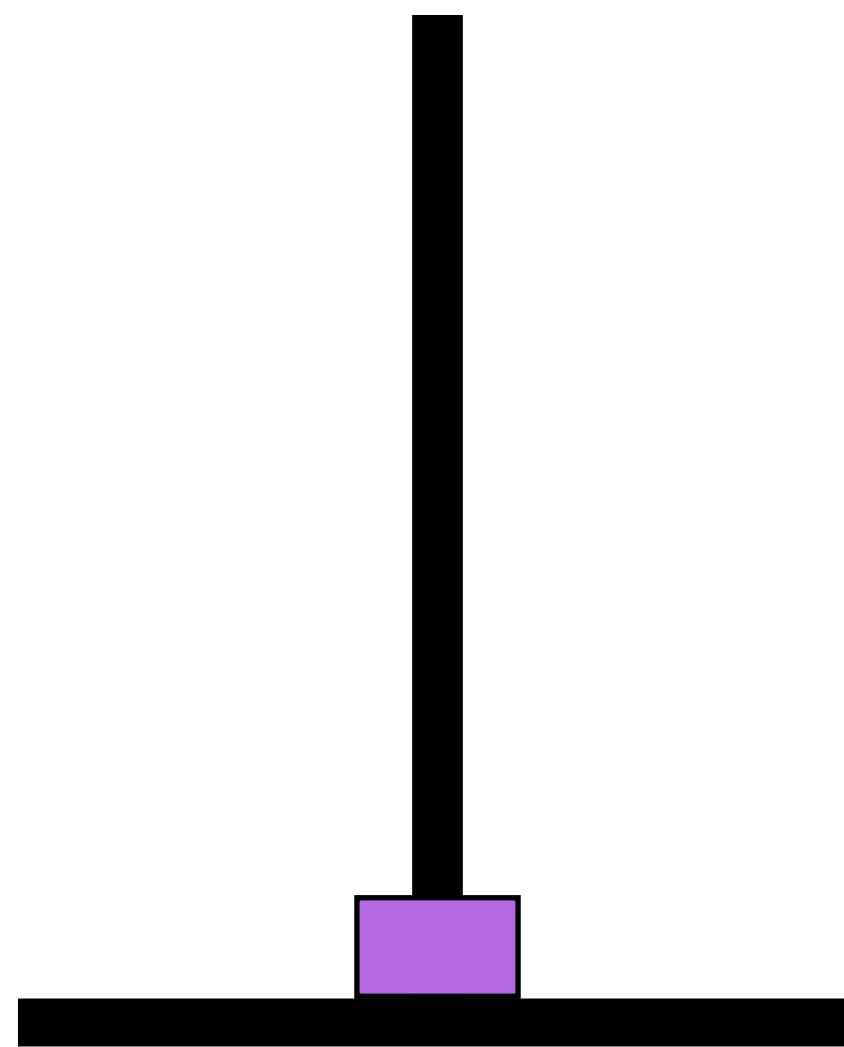
Stange C

Kleine Türme von Hanoi

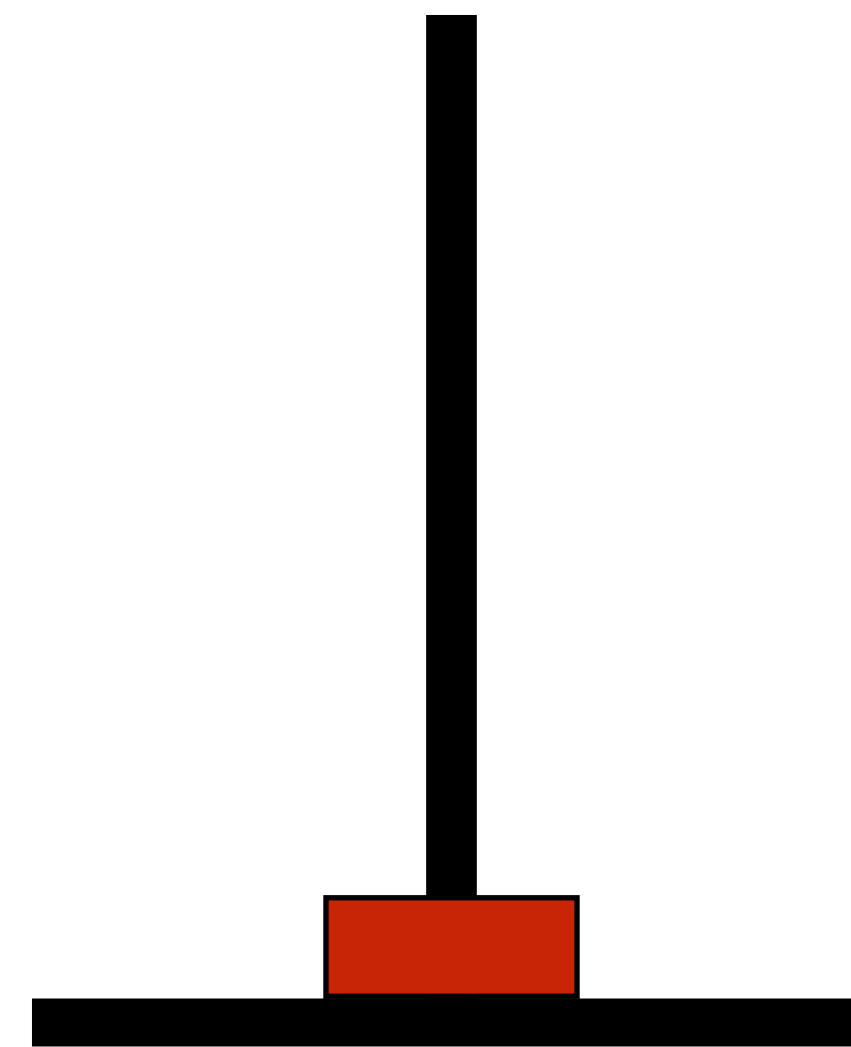
n=4



Stange A



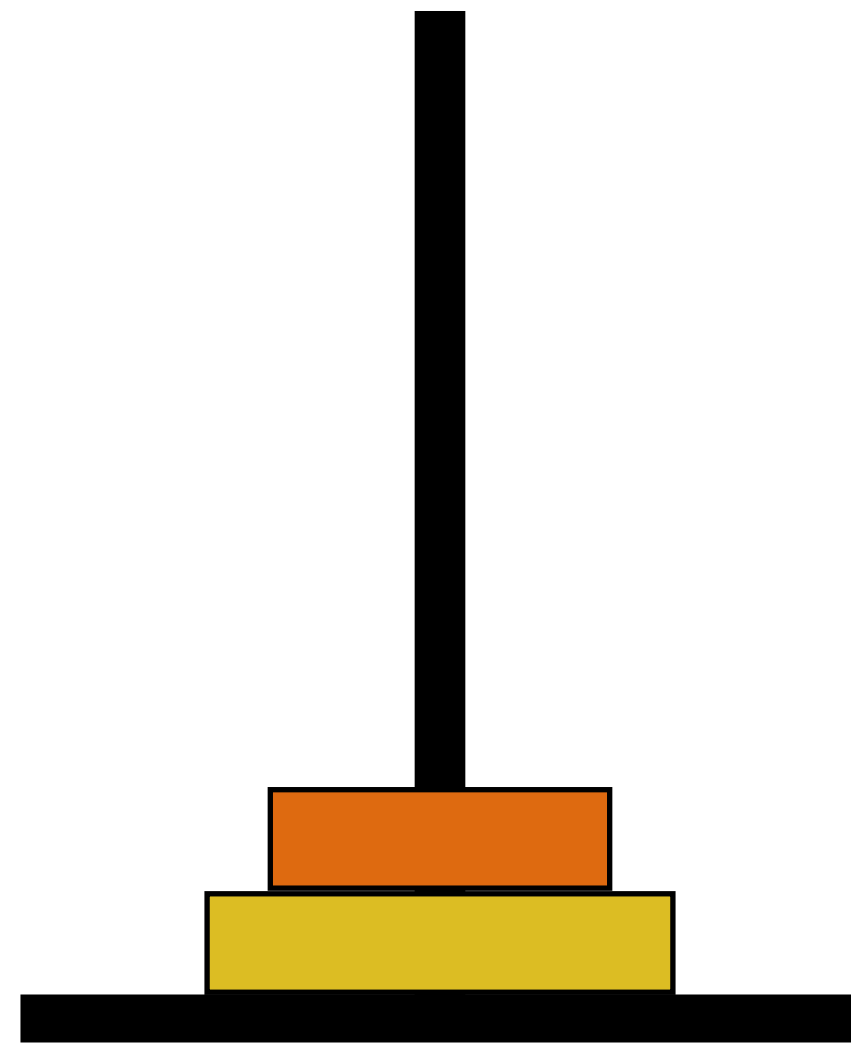
Stange B



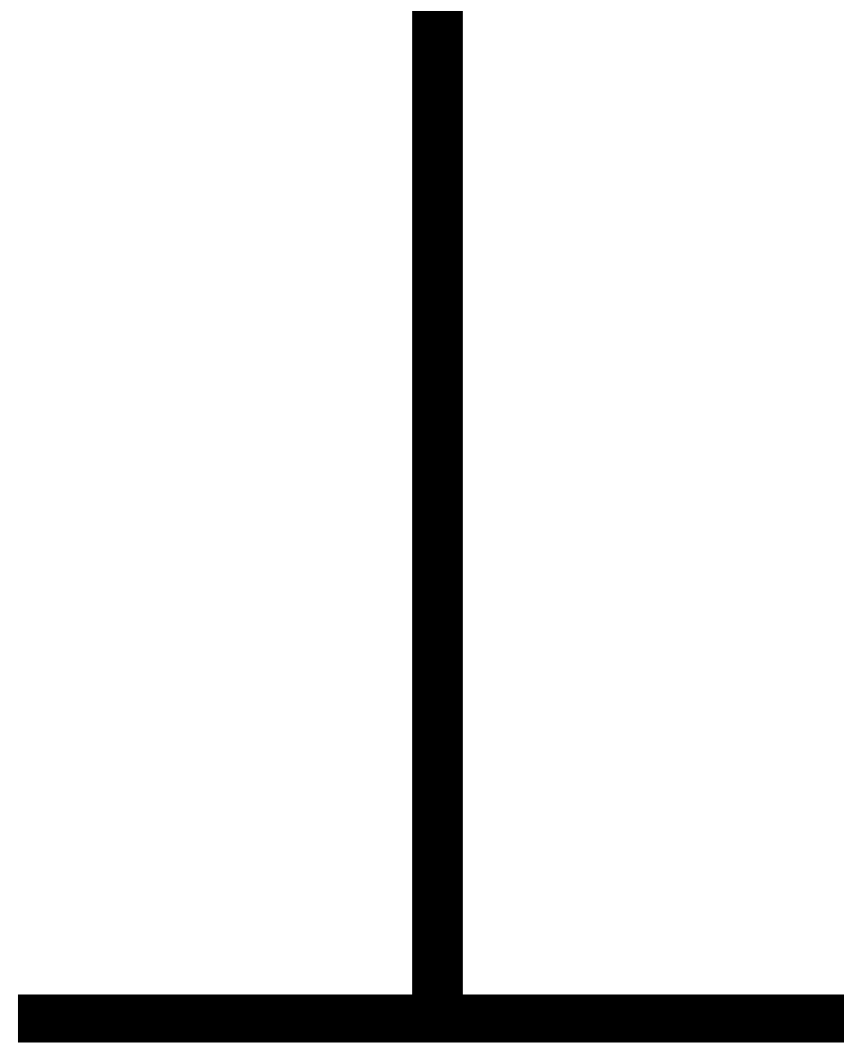
Stange C

Kleine Türme von Hanoi

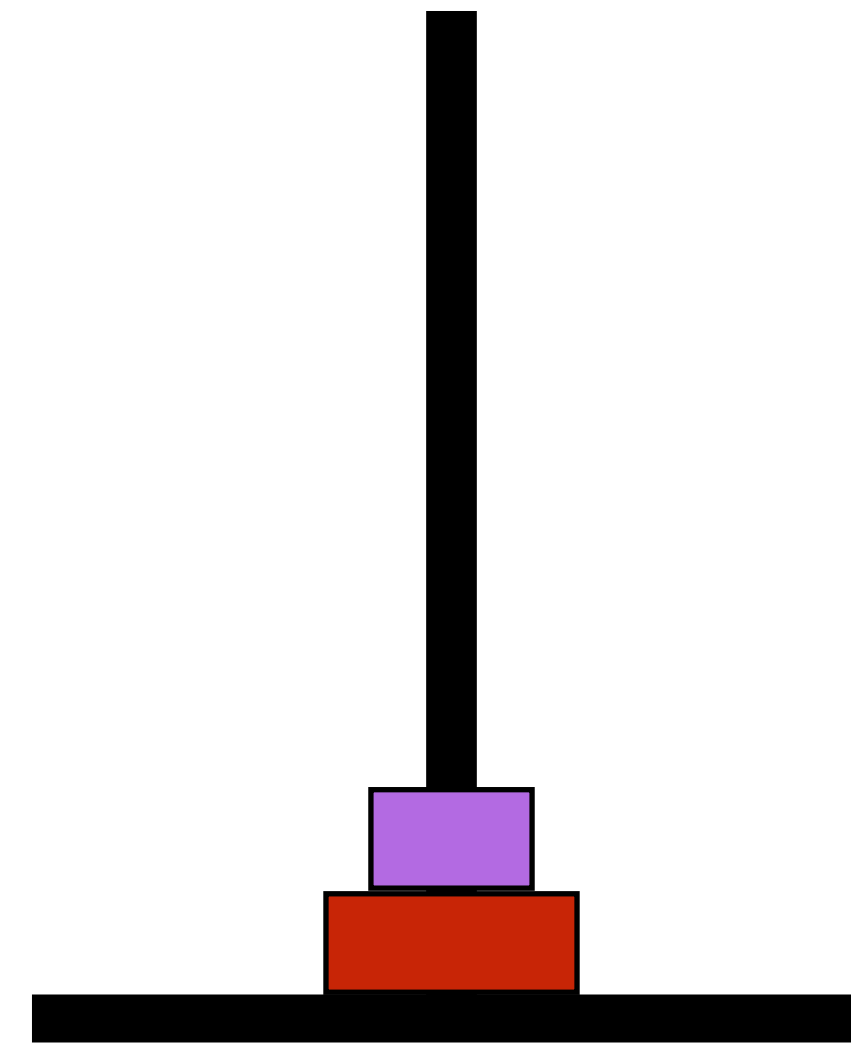
n=4



Stange A



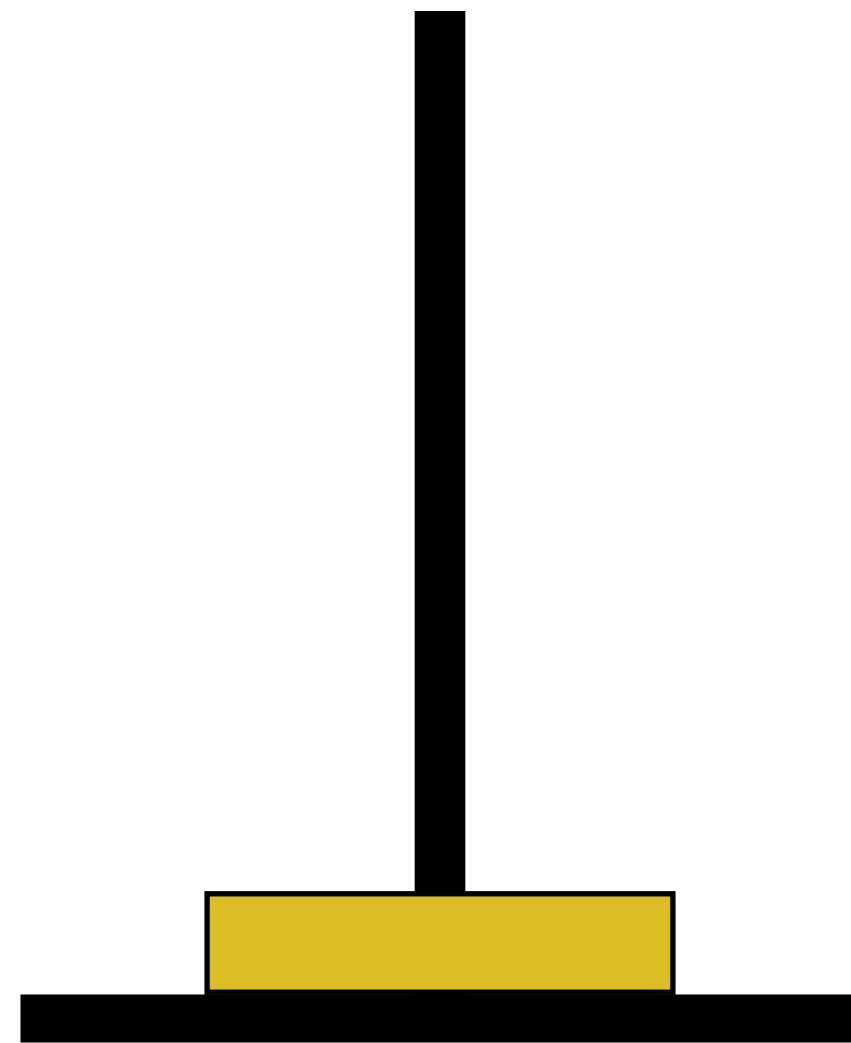
Stange B



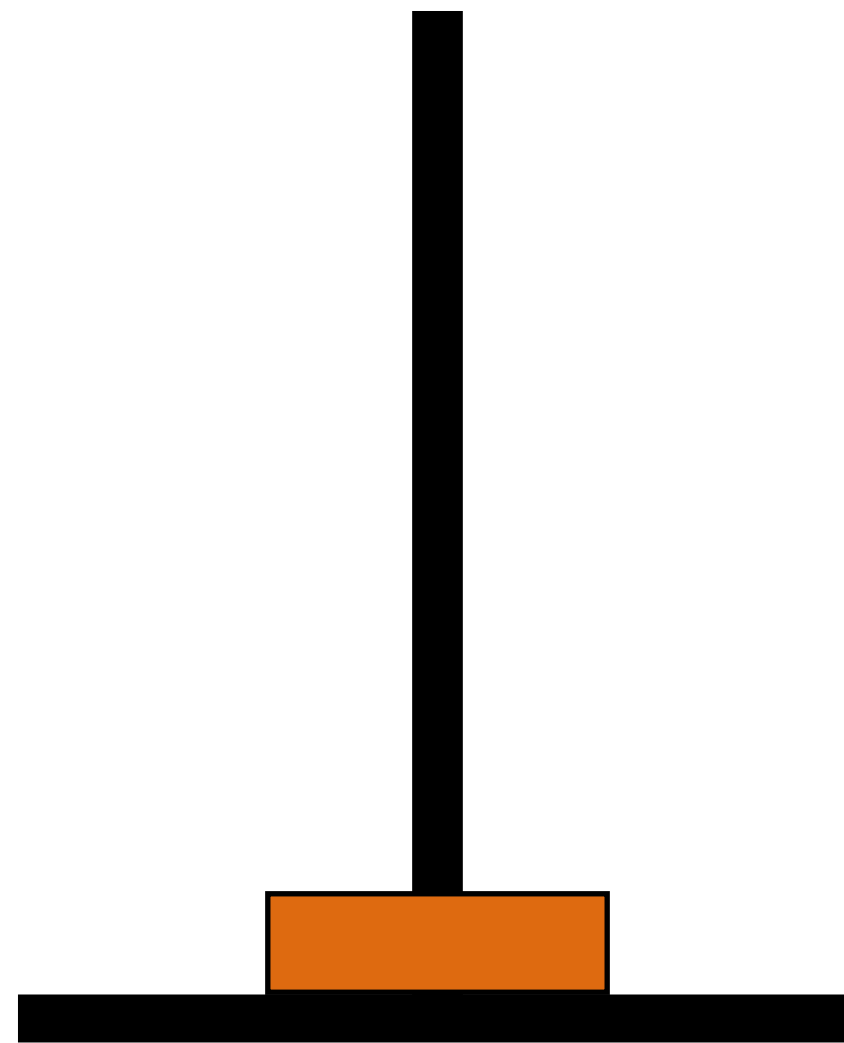
Stange C

Kleine Türme von Hanoi

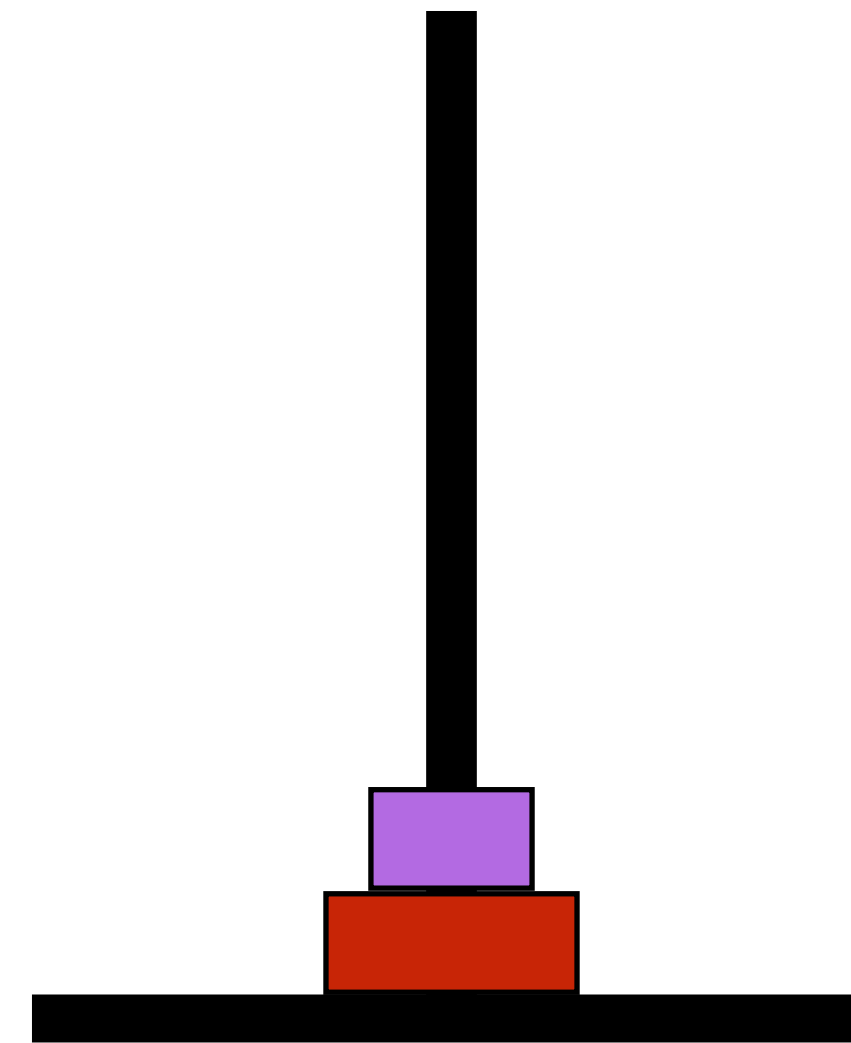
n=4



Stange A



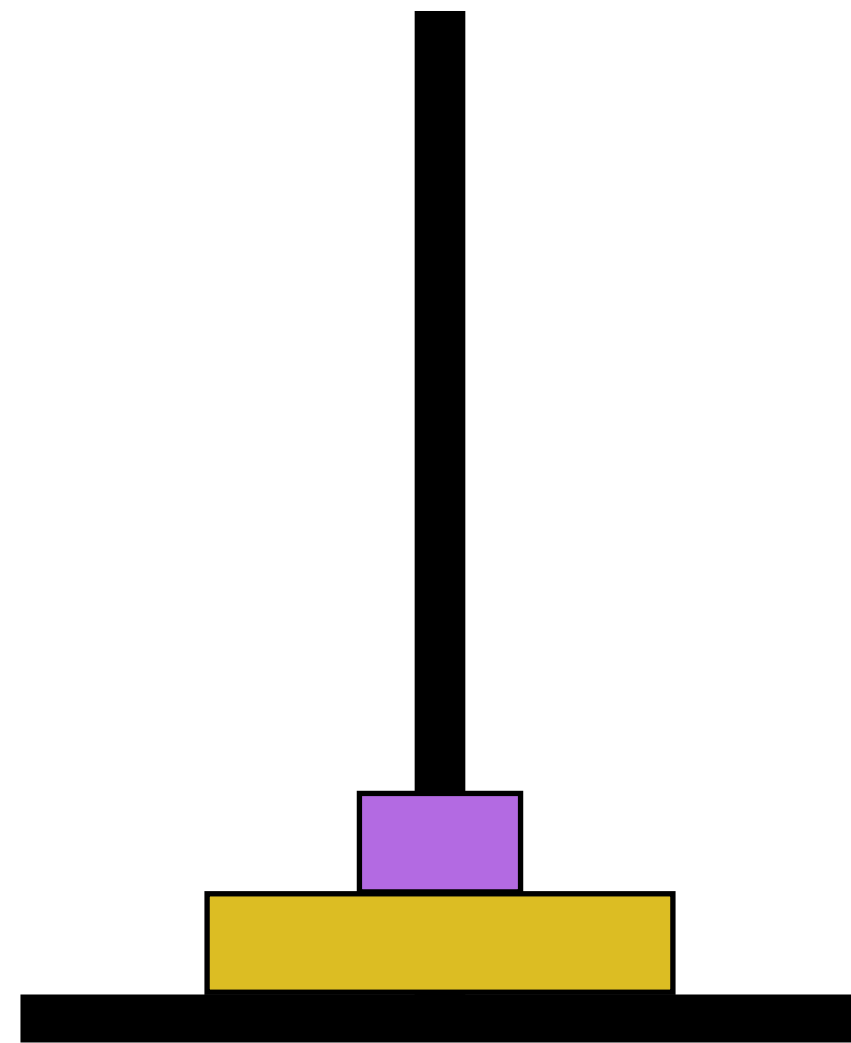
Stange B



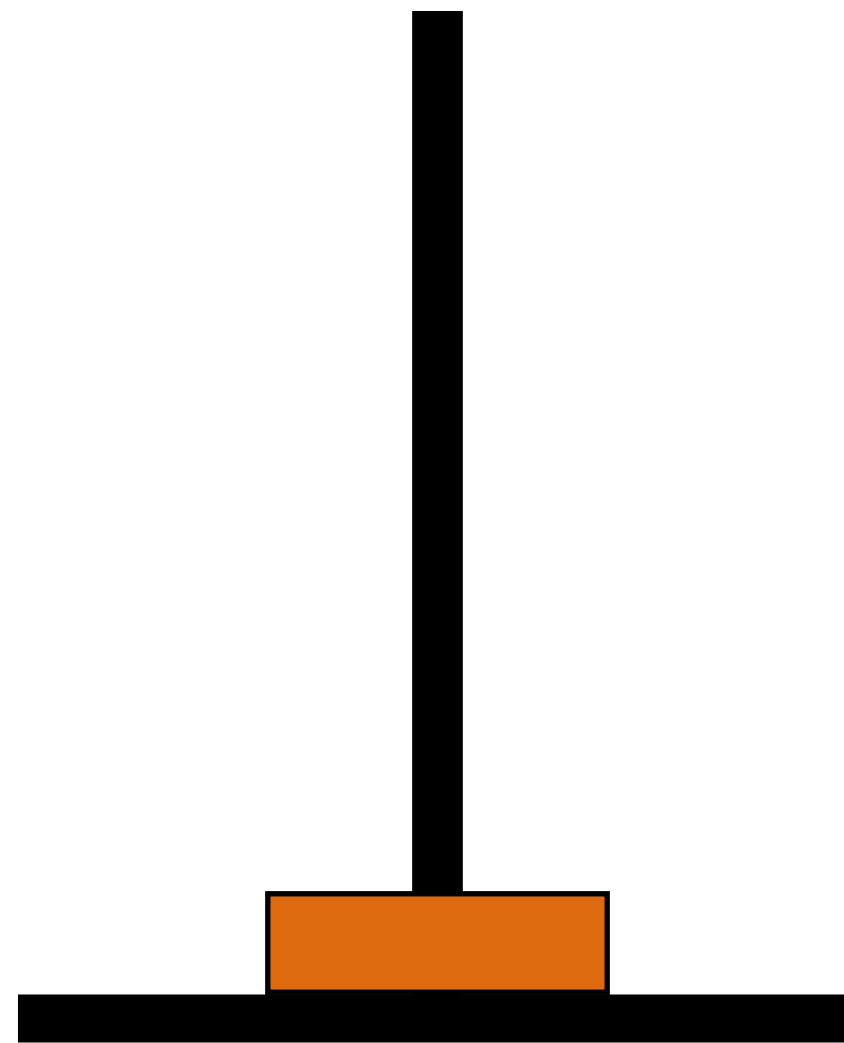
Stange C

Kleine Türme von Hanoi

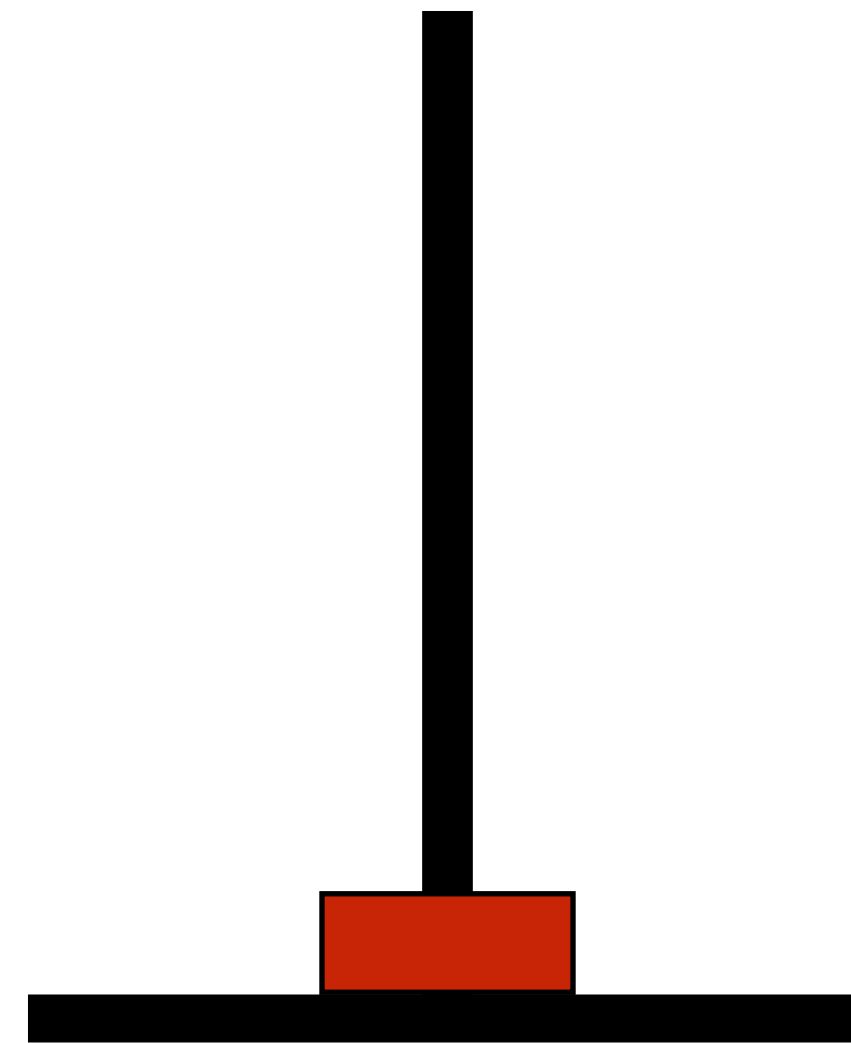
n=4



Stange A



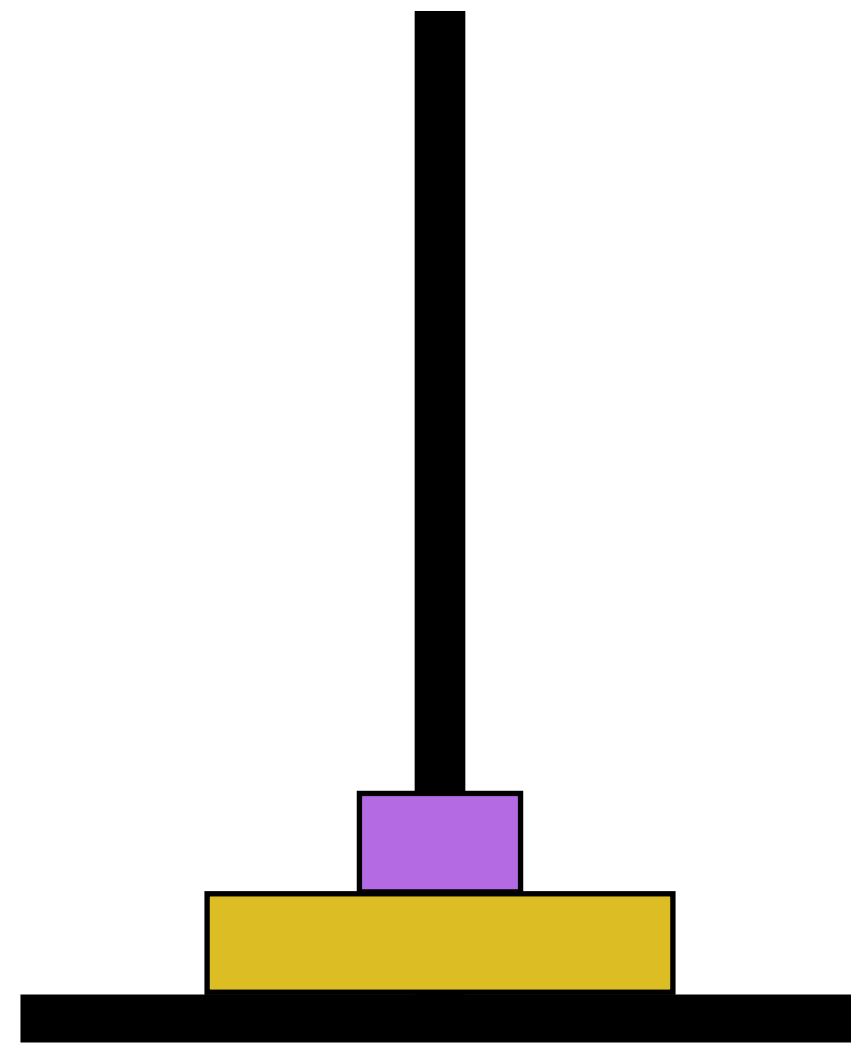
Stange B



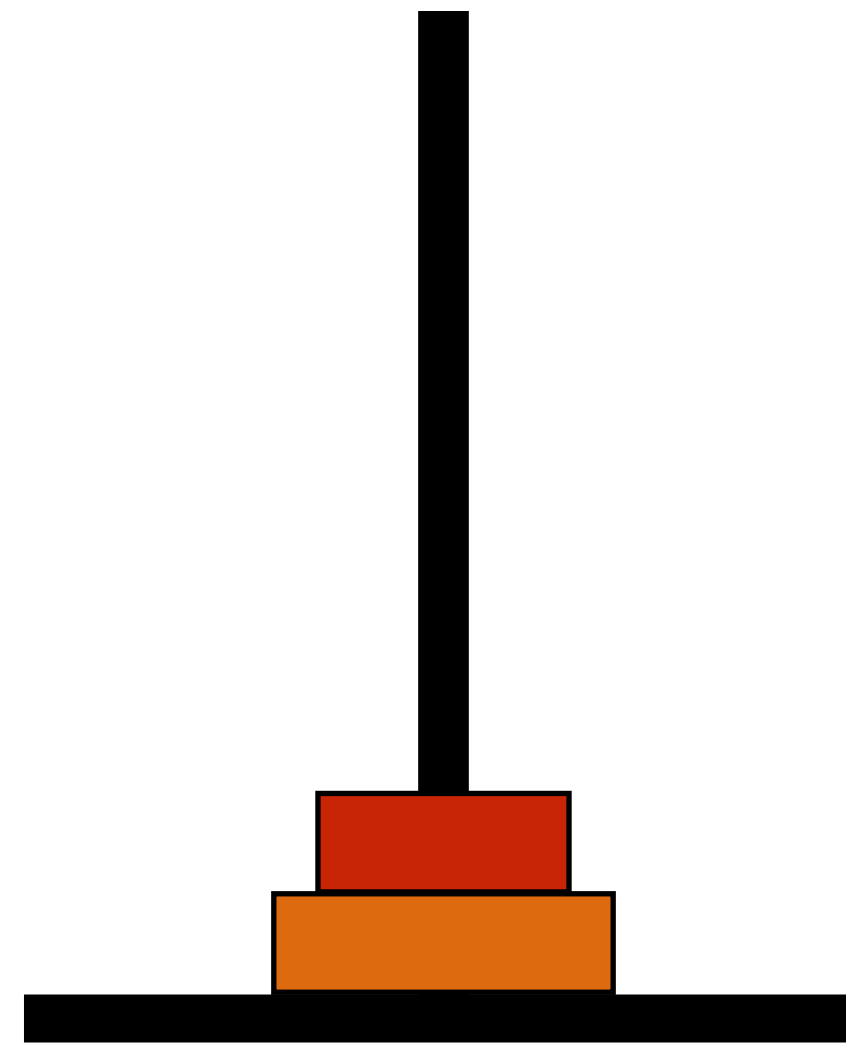
Stange C

Kleine Türme von Hanoi

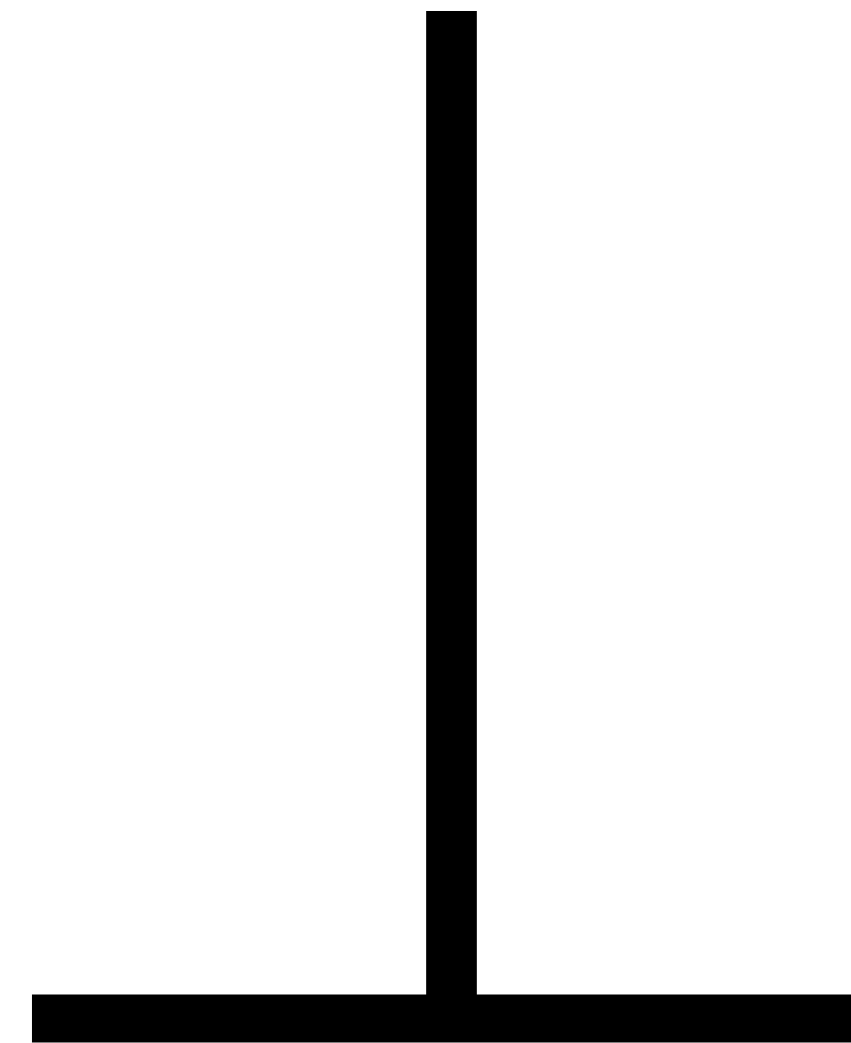
n=4



Stange A



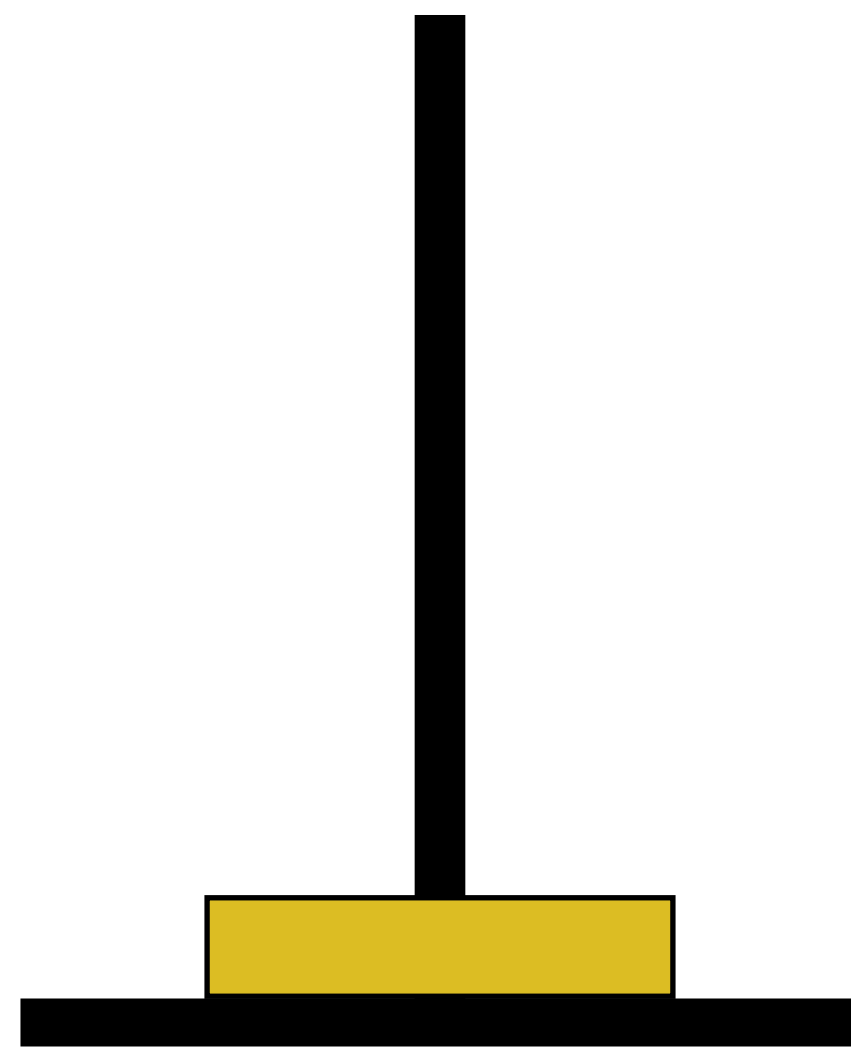
Stange B



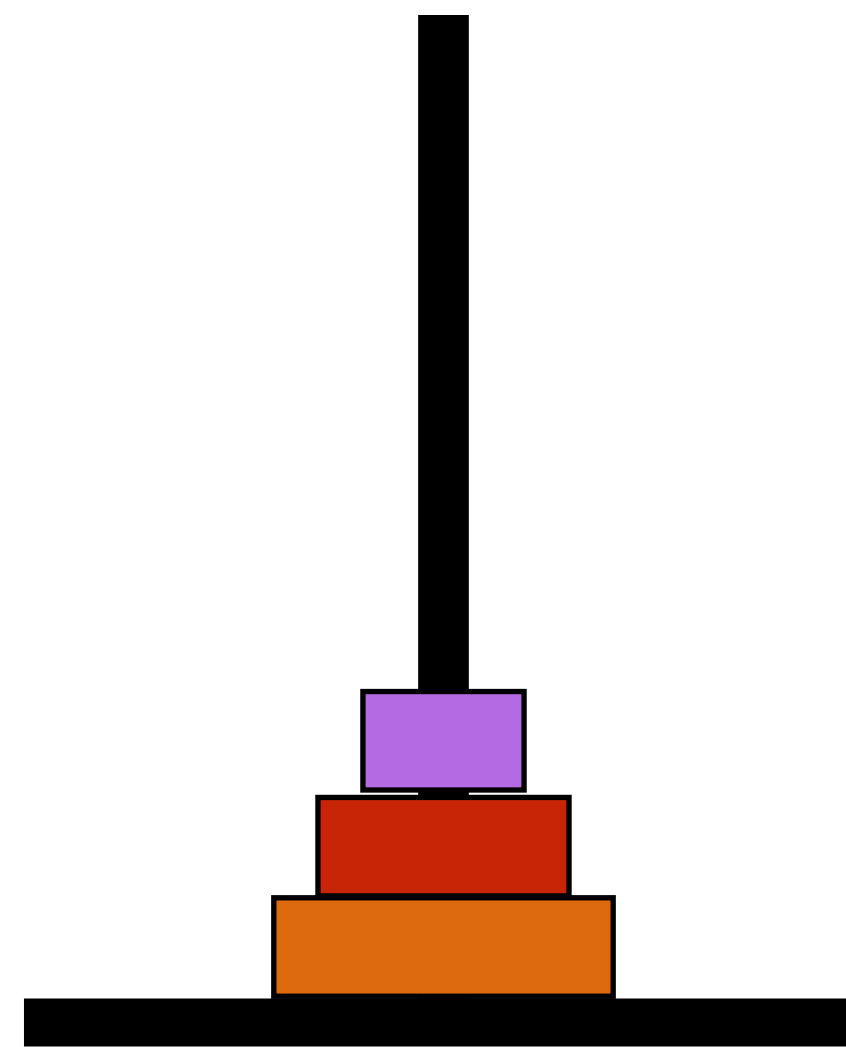
Stange C

Kleine Türme von Hanoi

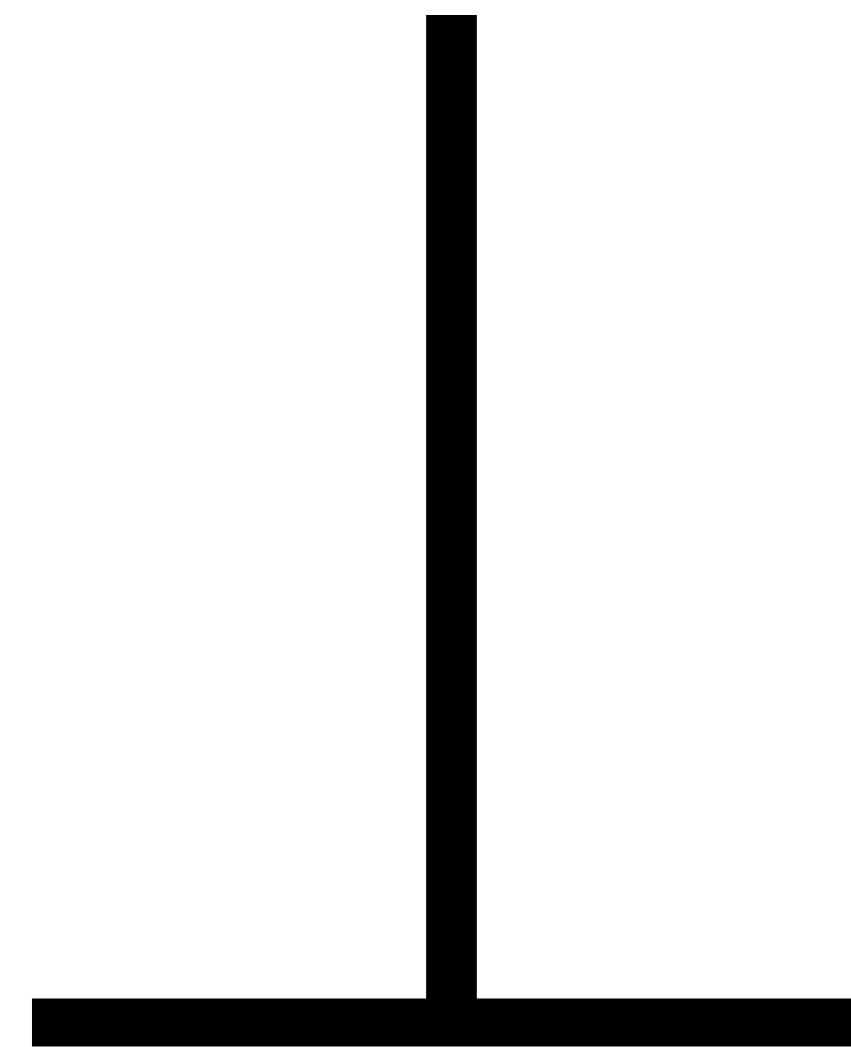
n=4



Stange A



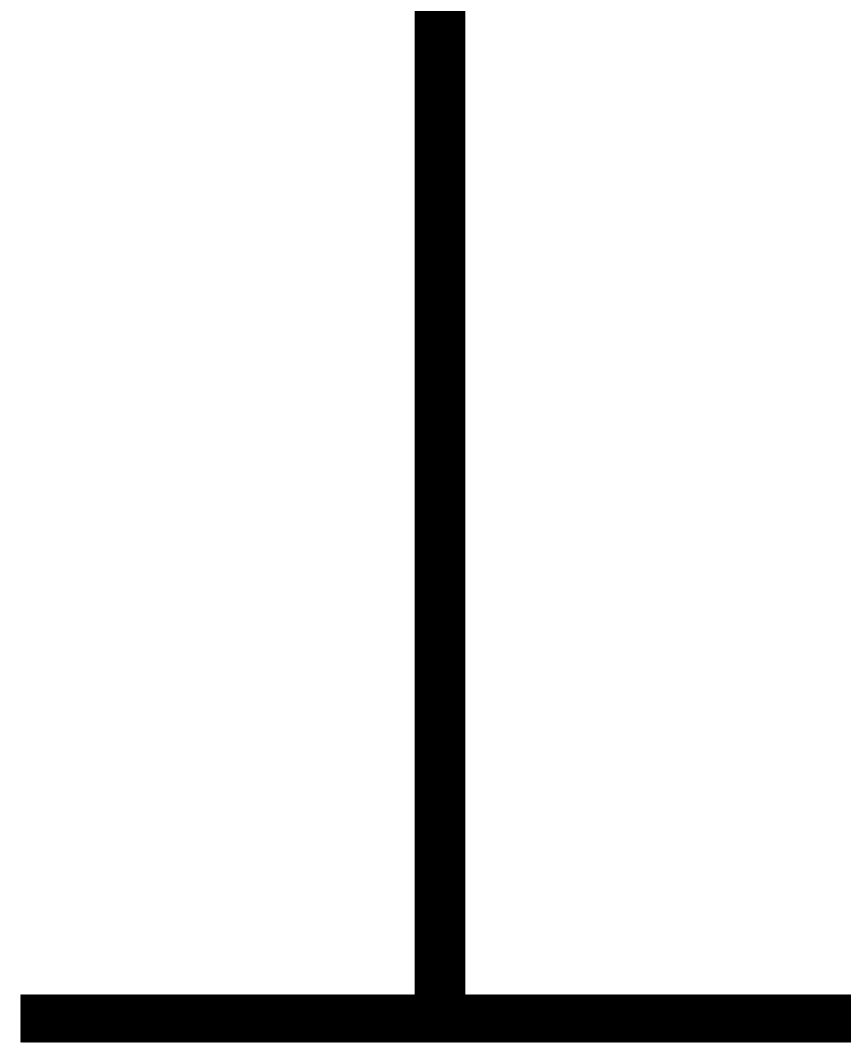
Stange B



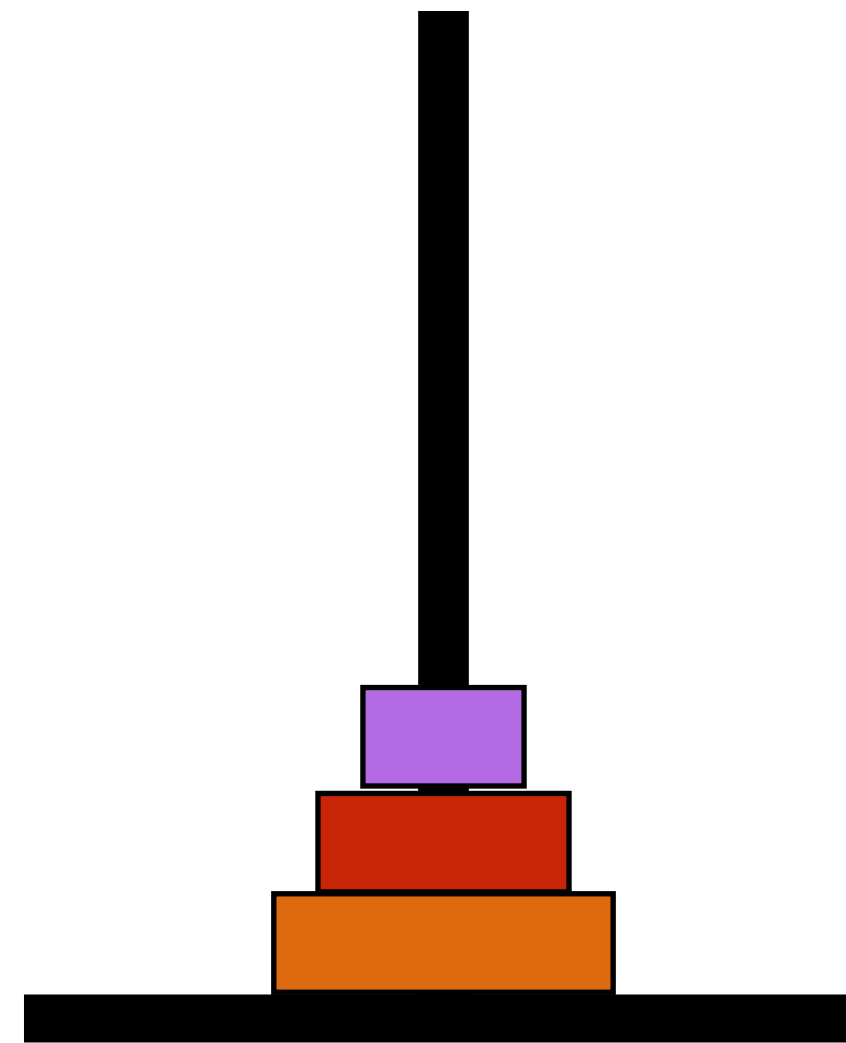
Stange C

Kleine Türme von Hanoi

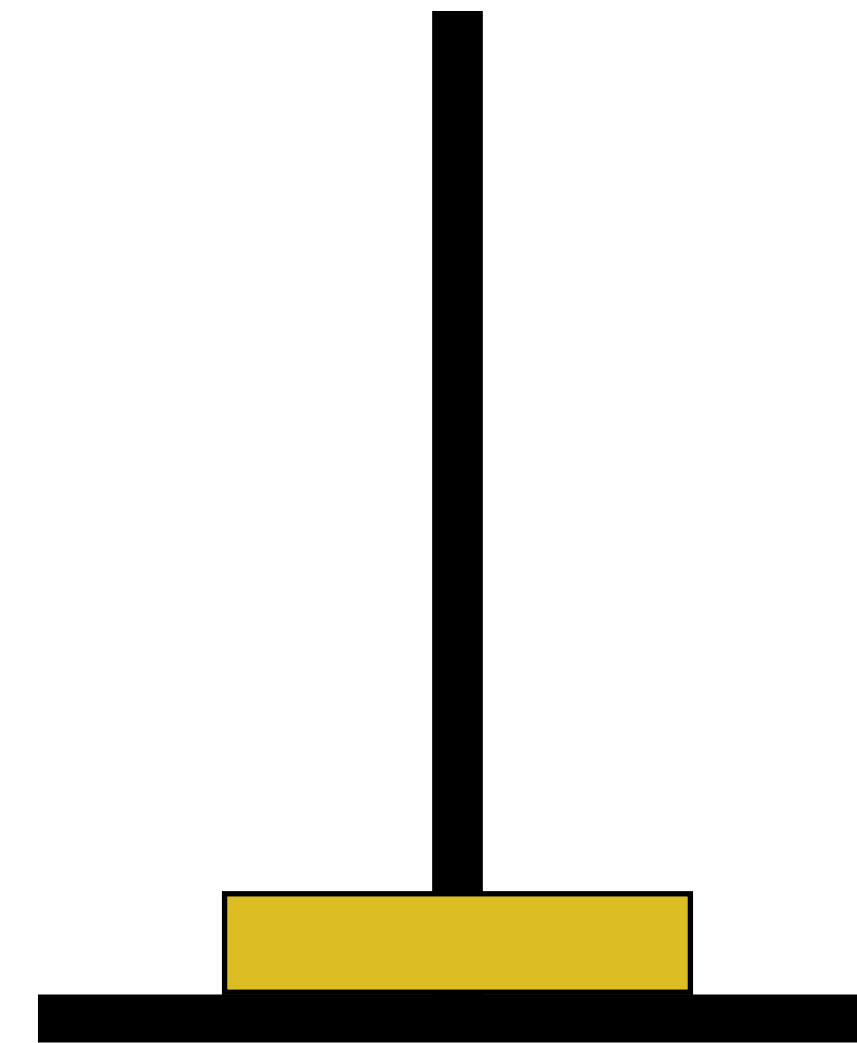
n=4



Stange A



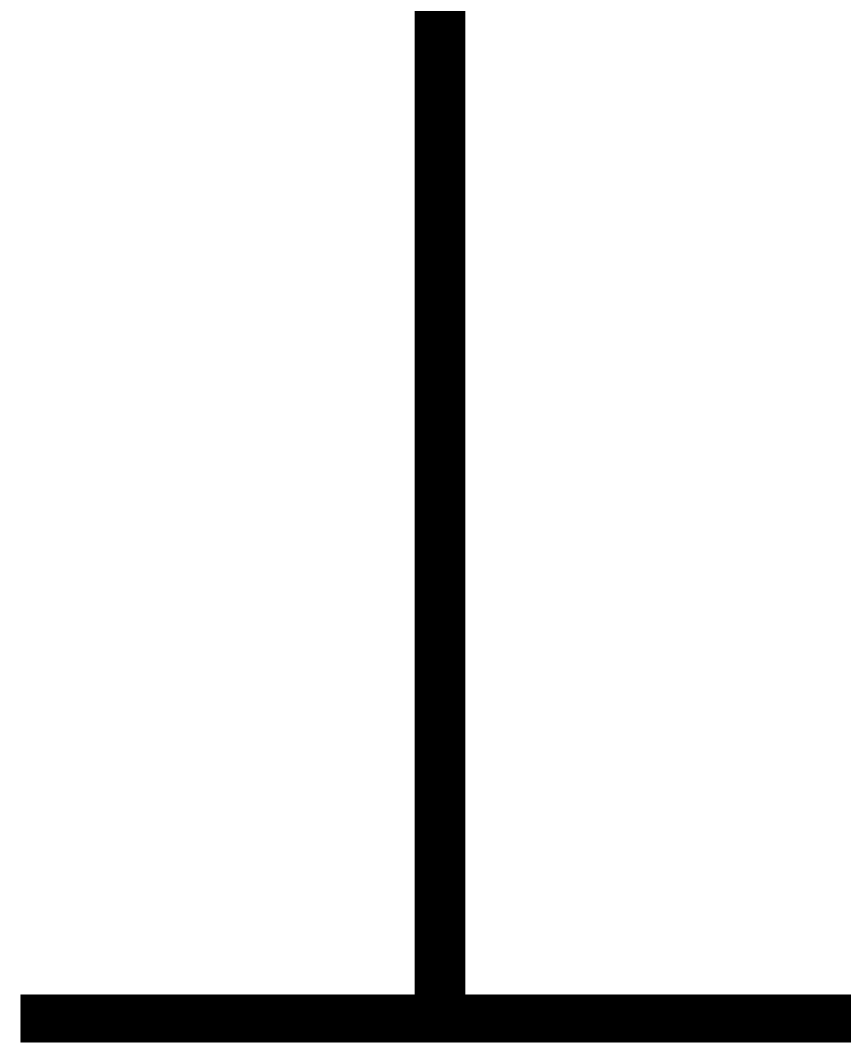
Stange B



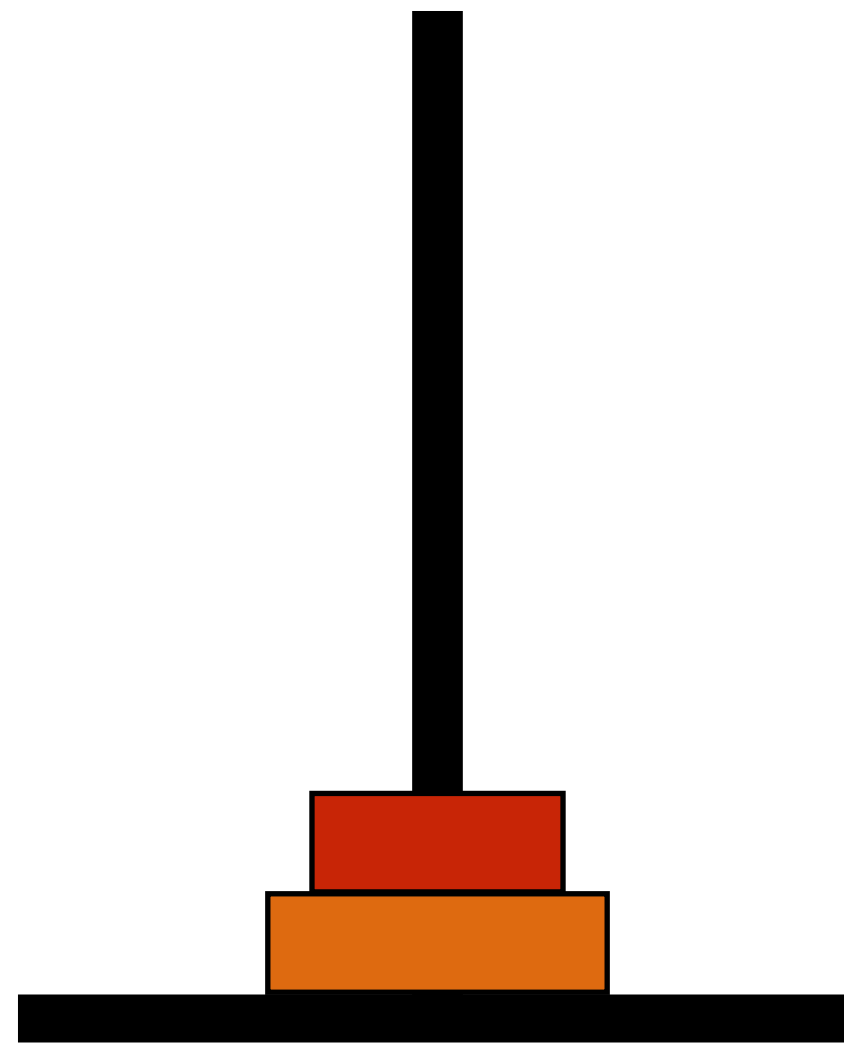
Stange C

Kleine Türme von Hanoi

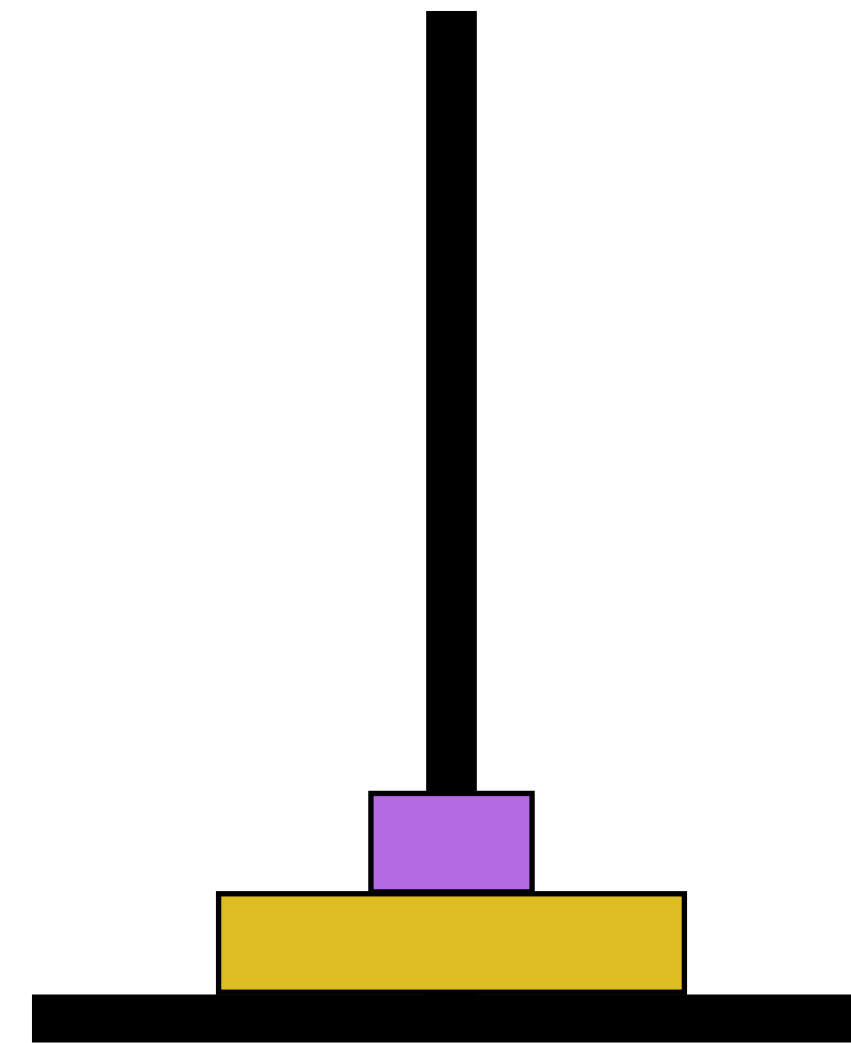
n=4



Stange A



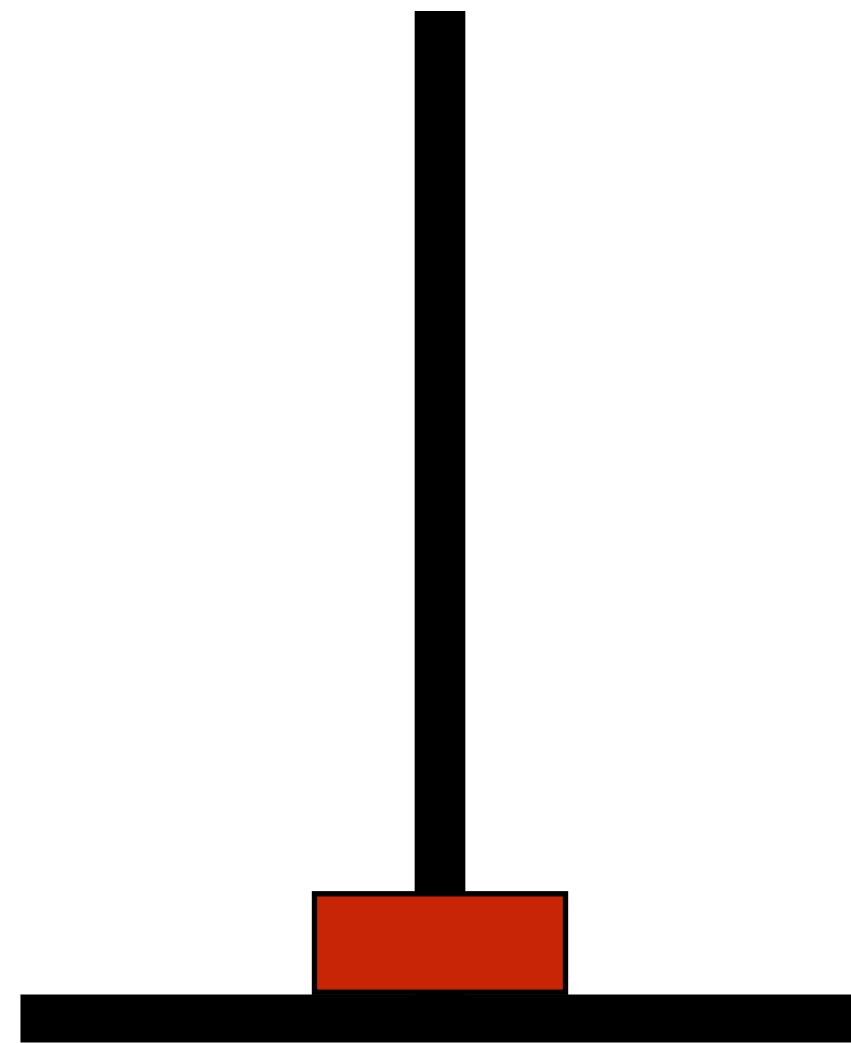
Stange B



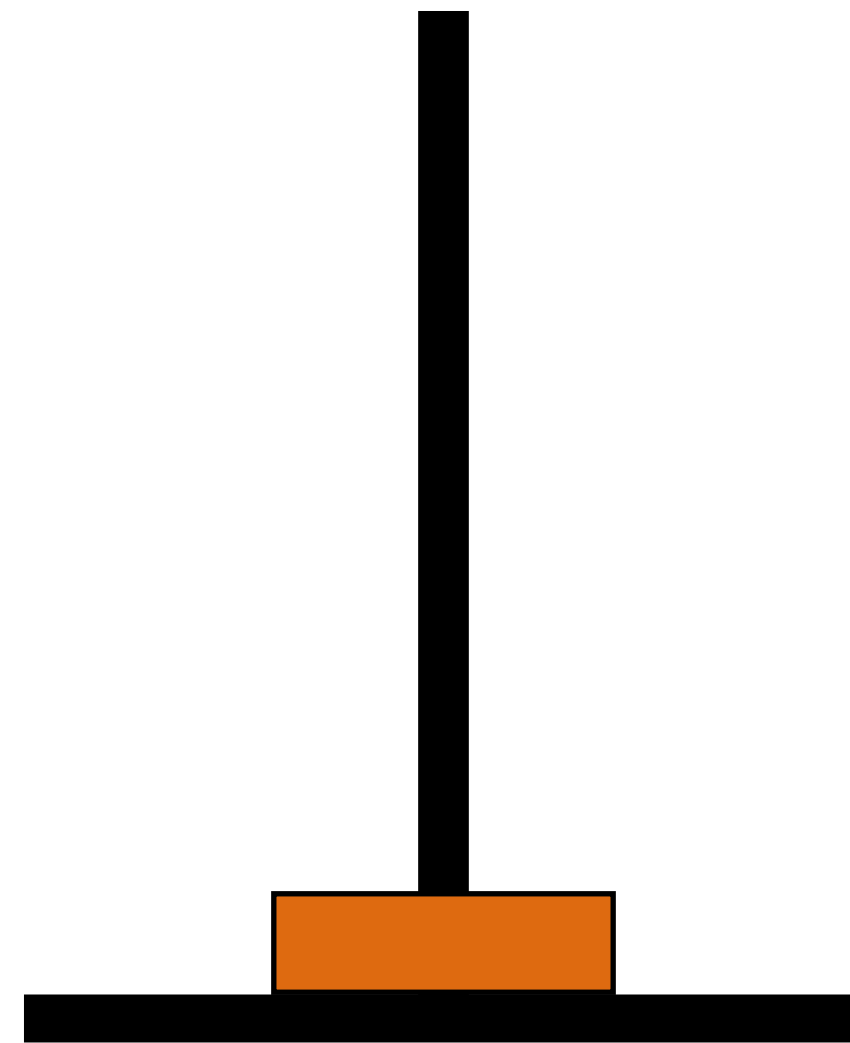
Stange C

Kleine Türme von Hanoi

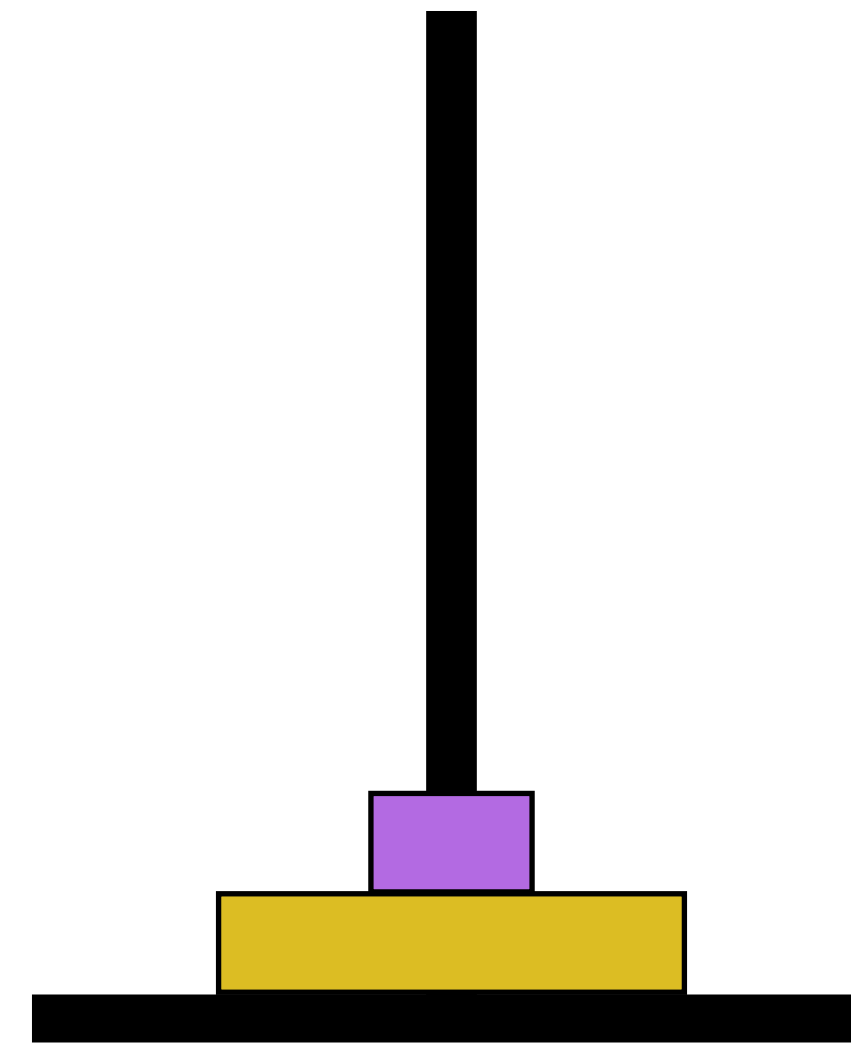
n=4



Stange A



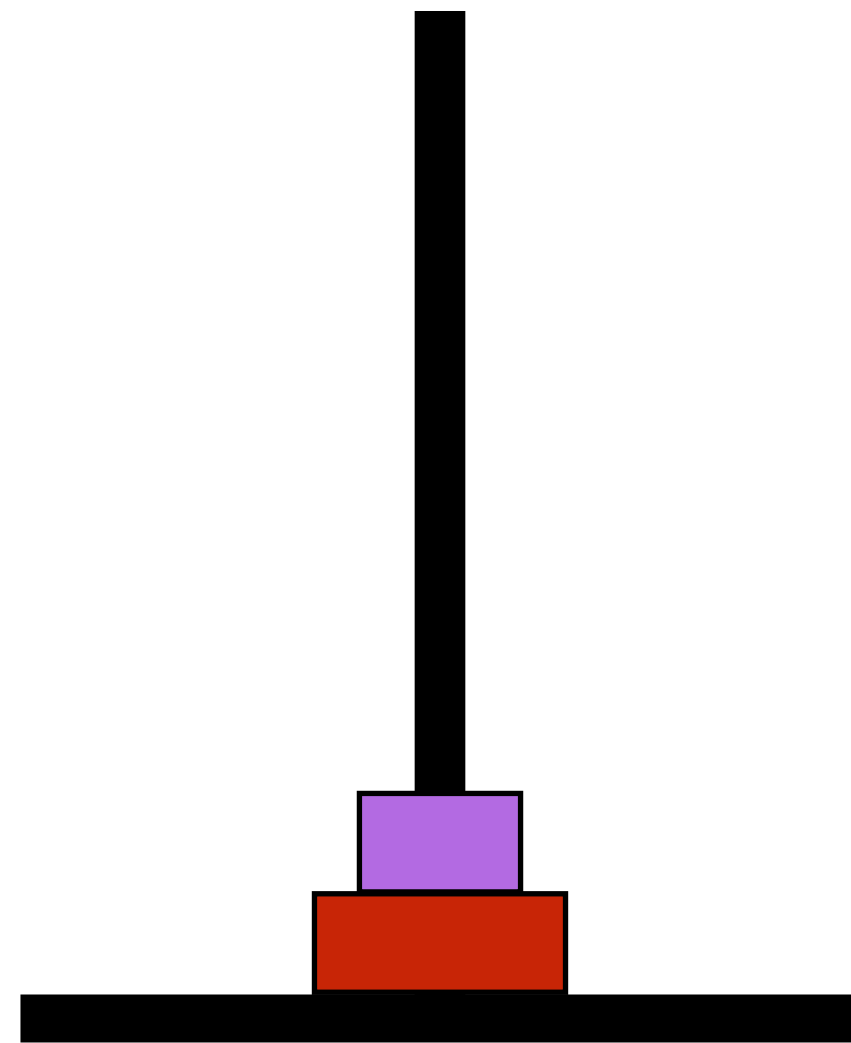
Stange B



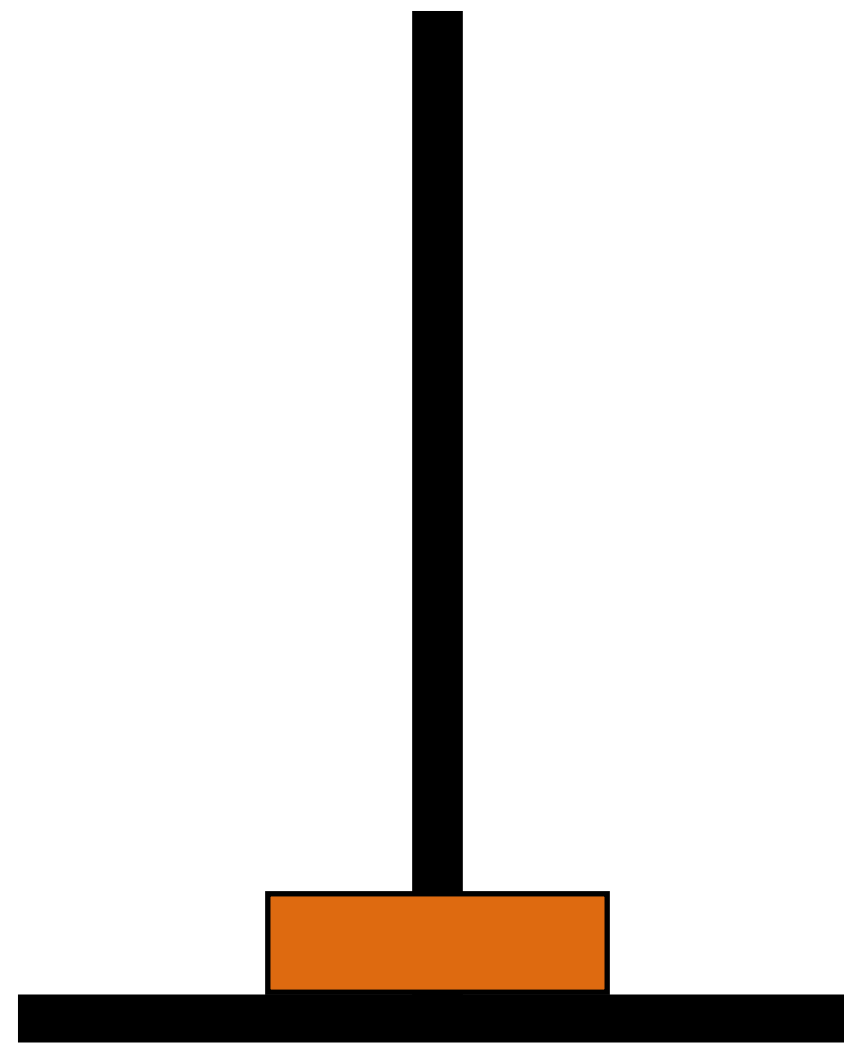
Stange C

Kleine Türme von Hanoi

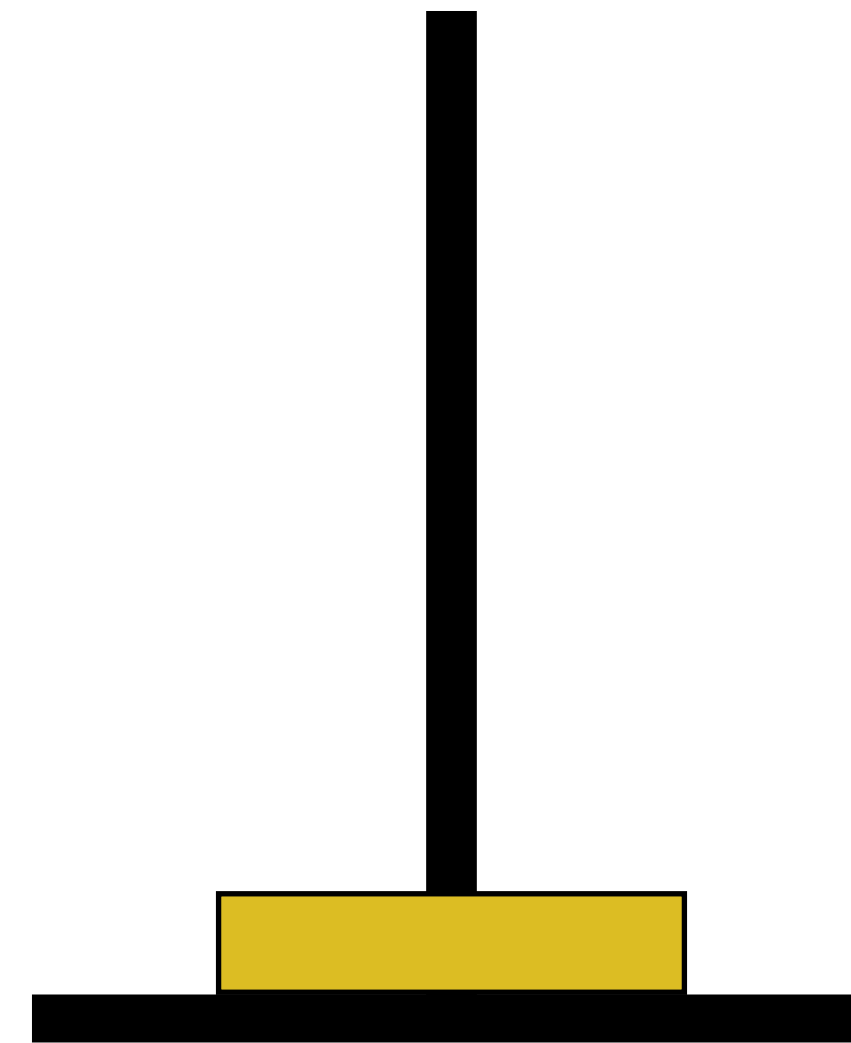
n=4



Stange A



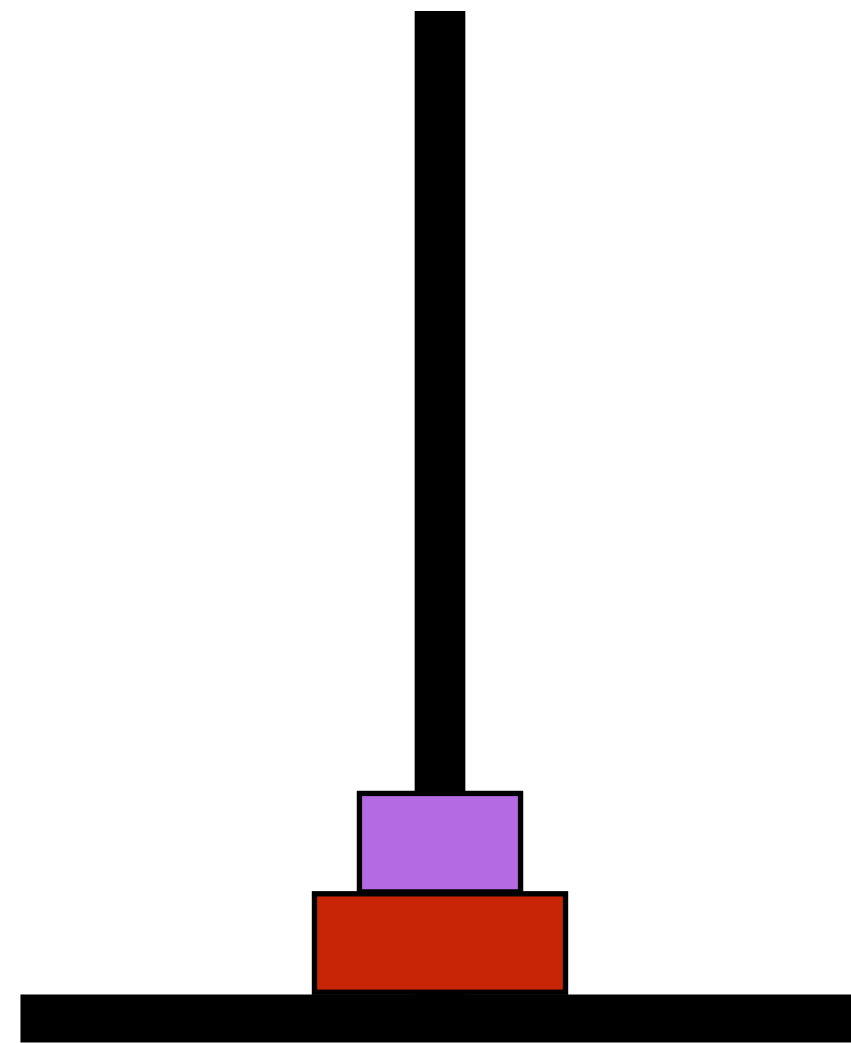
Stange B



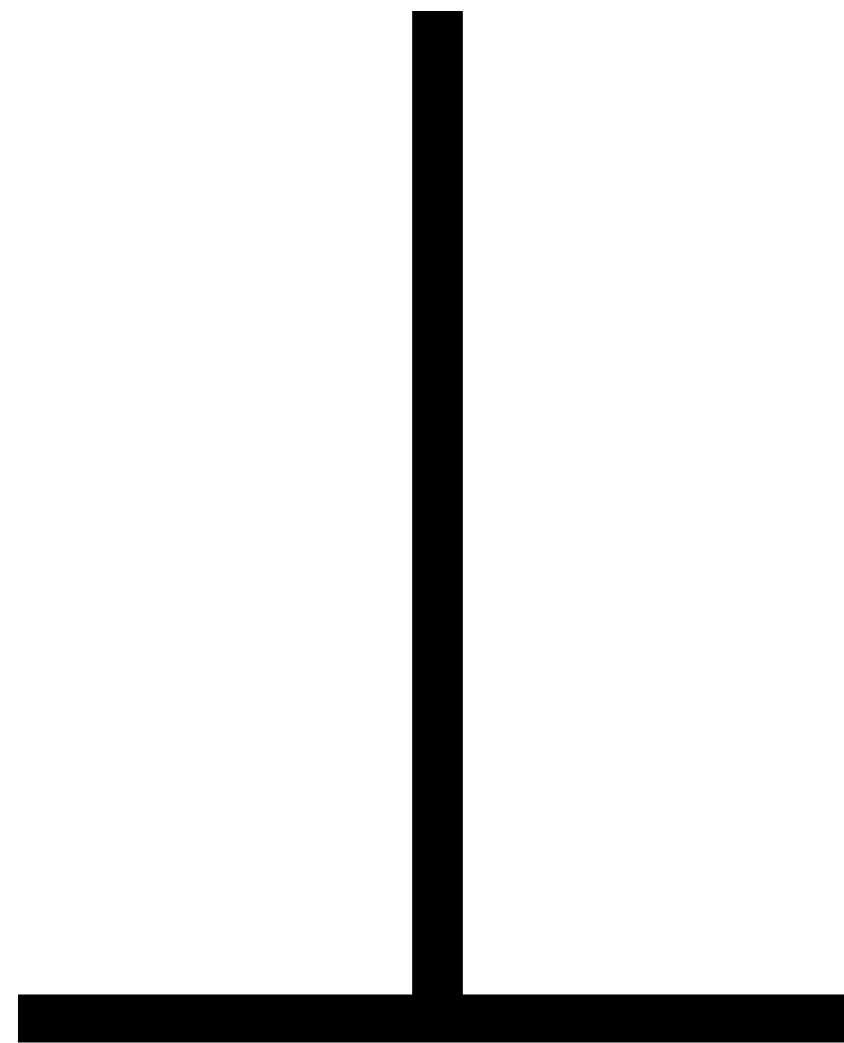
Stange C

Kleine Türme von Hanoi

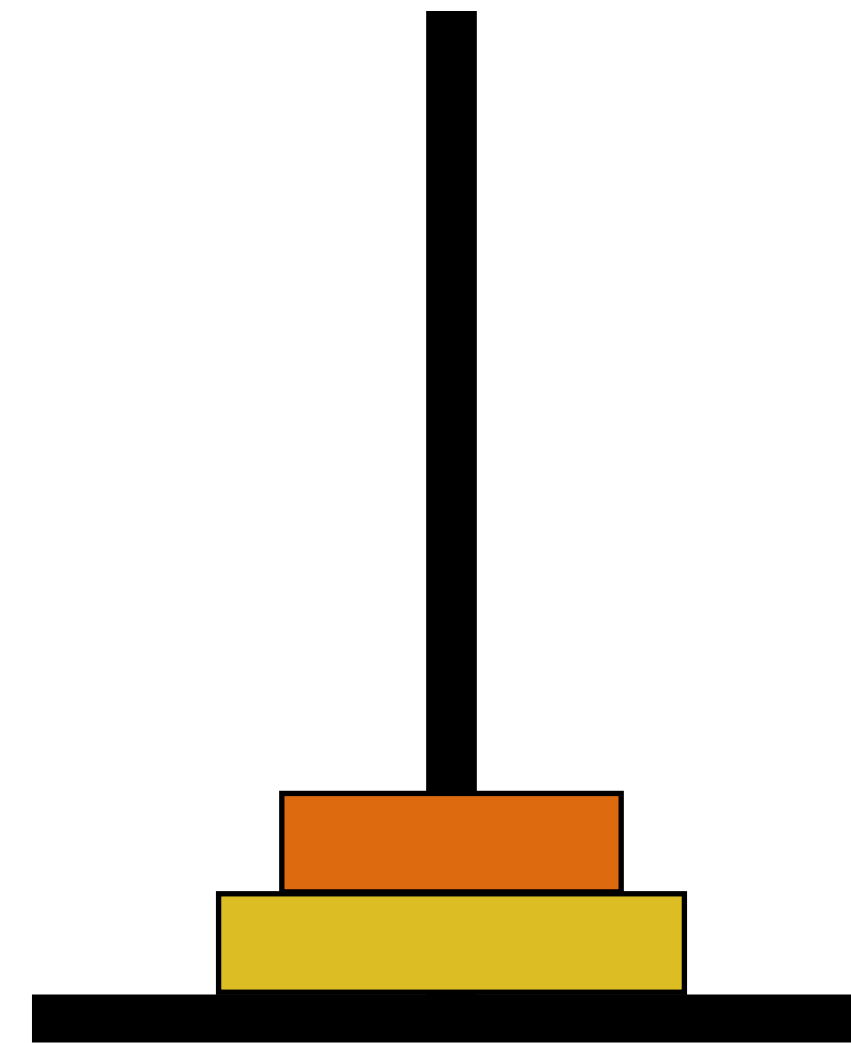
n=4



Stange A



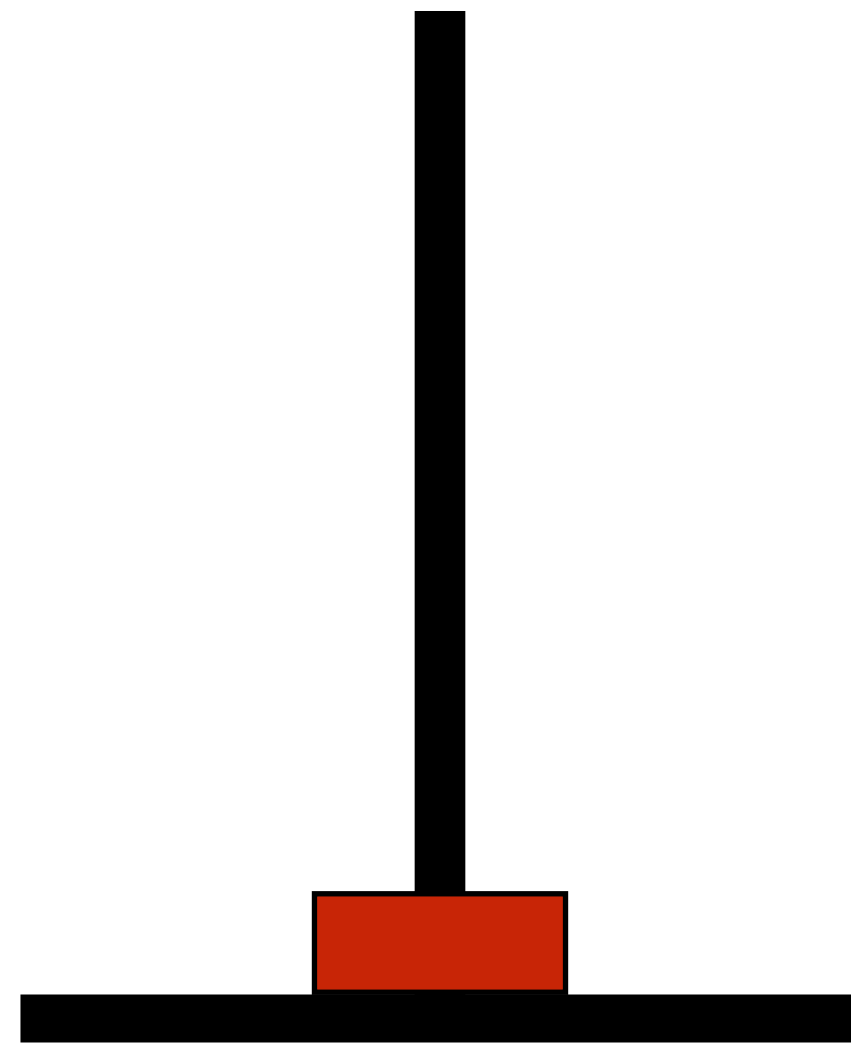
Stange B



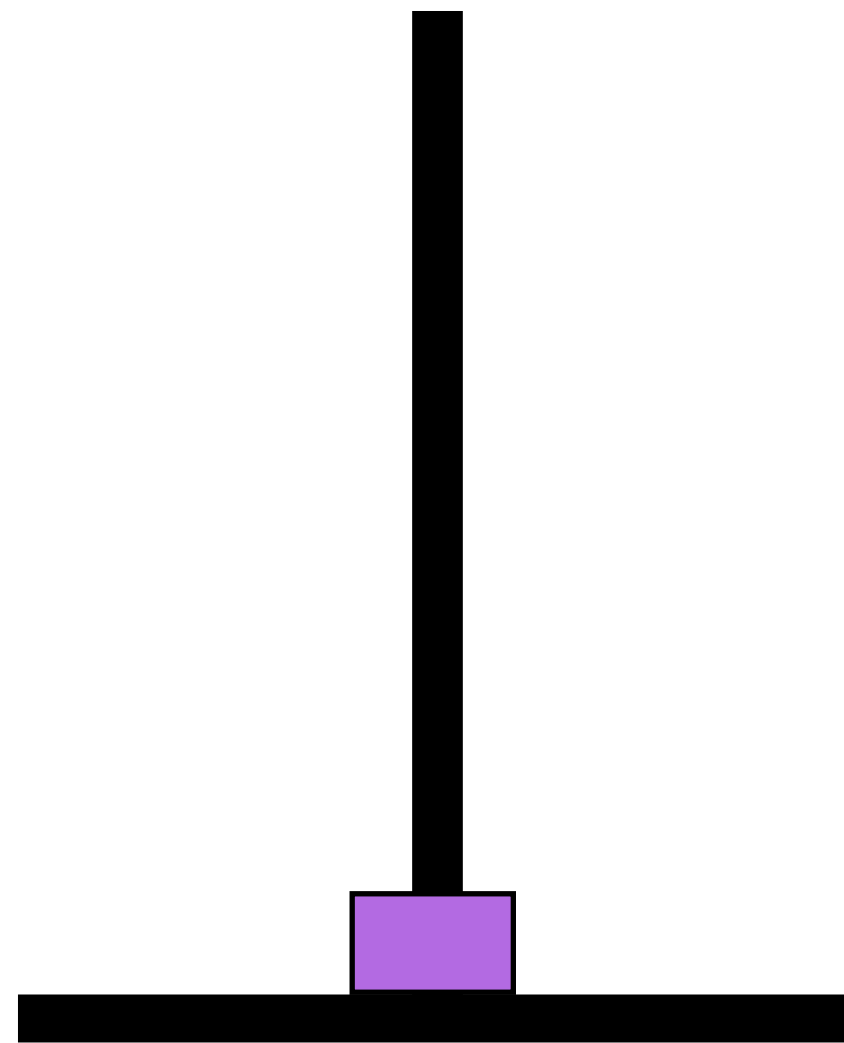
Stange C

Kleine Türme von Hanoi

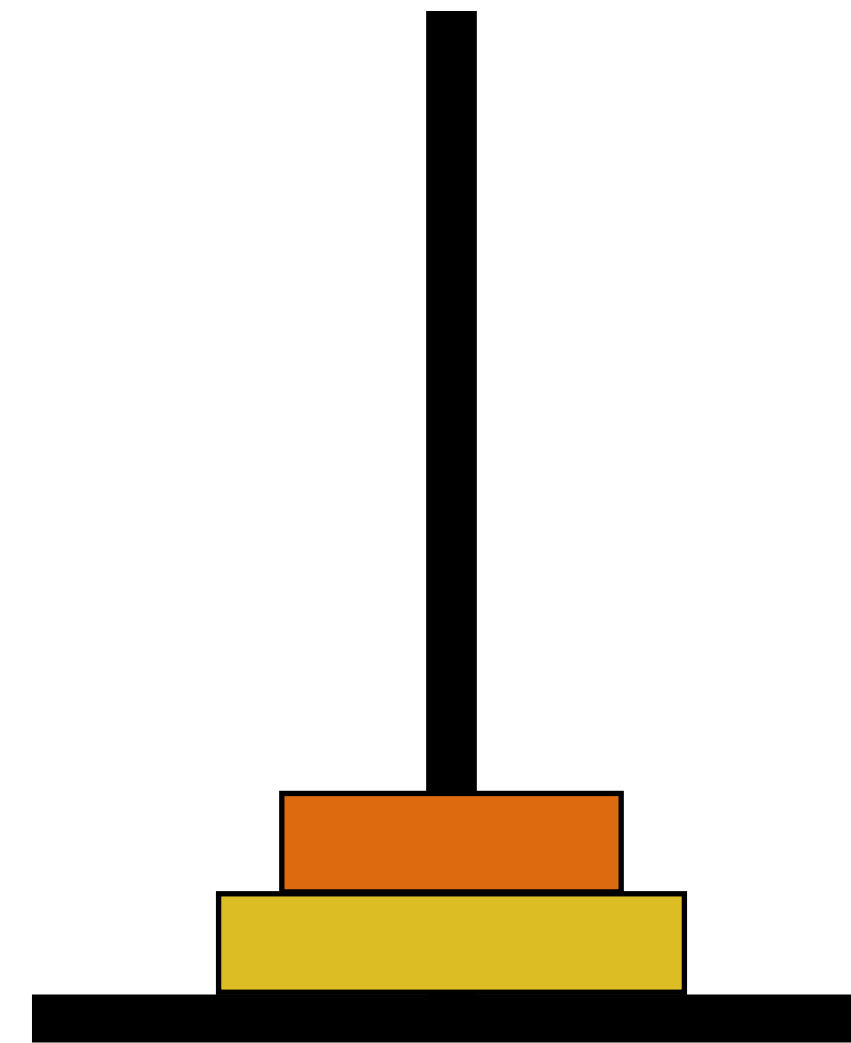
n=4



Stange A



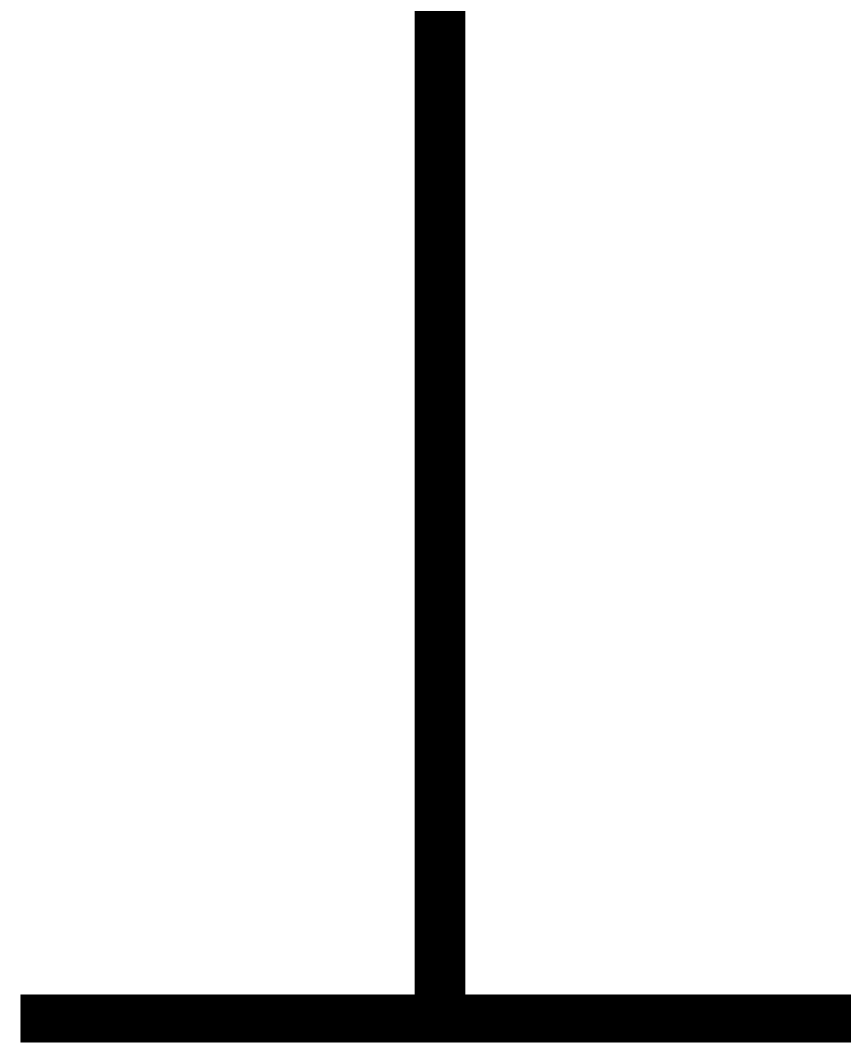
Stange B



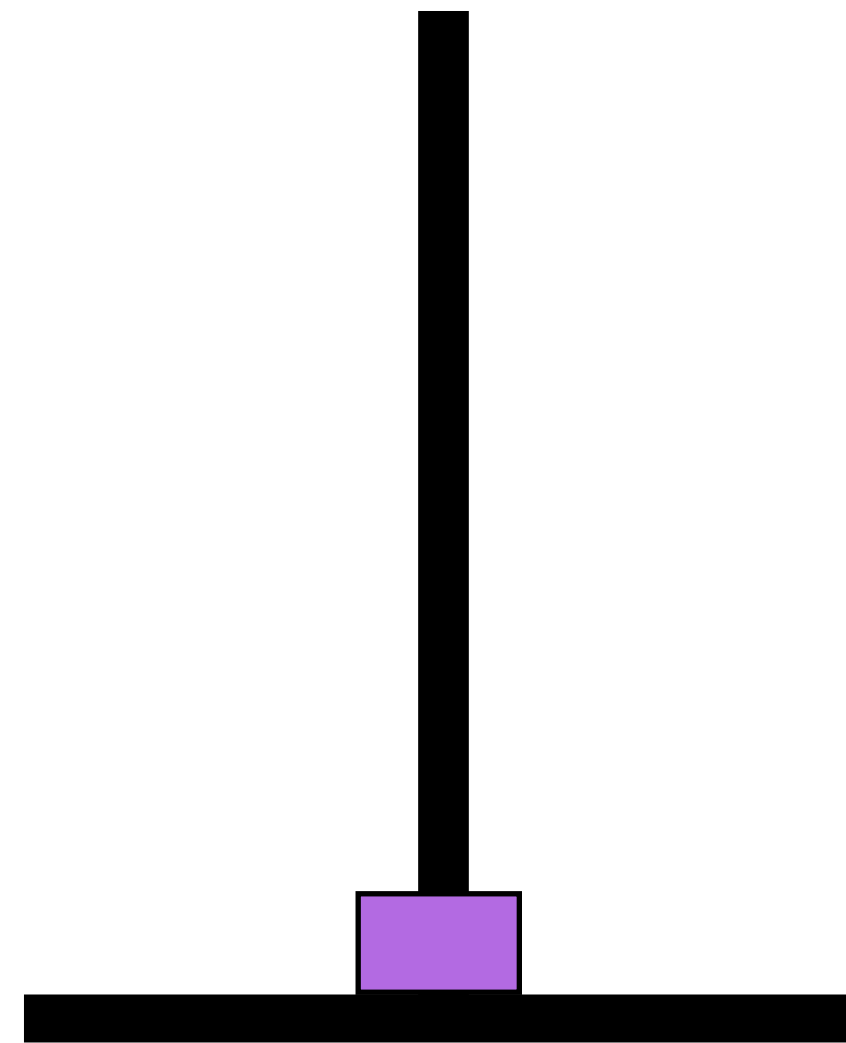
Stange C

Kleine Türme von Hanoi

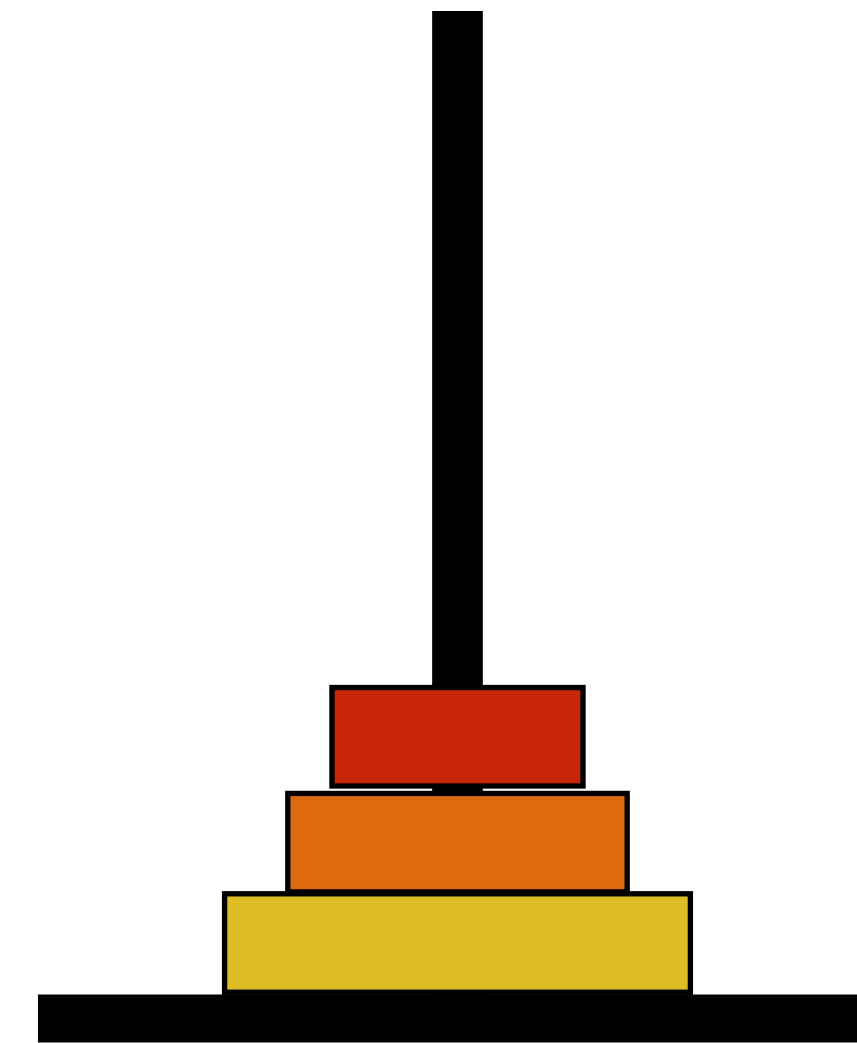
n=4



Stange A



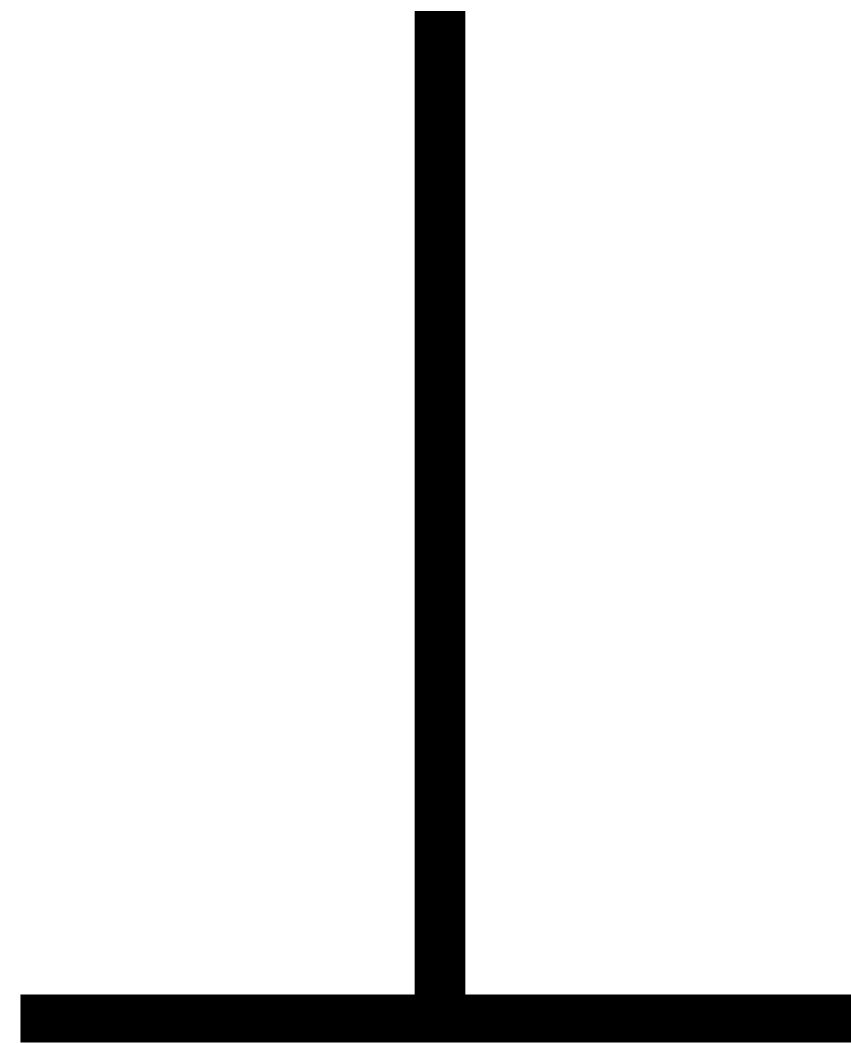
Stange B



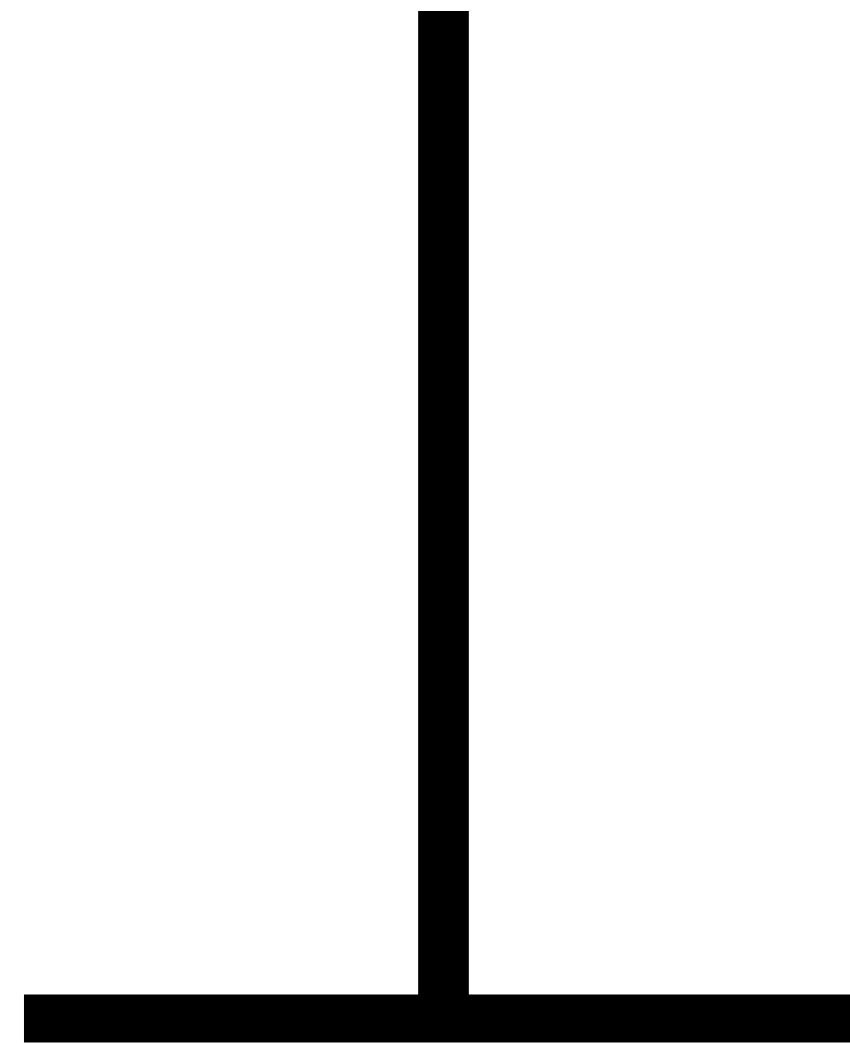
Stange C

Kleine Türme von Hanoi

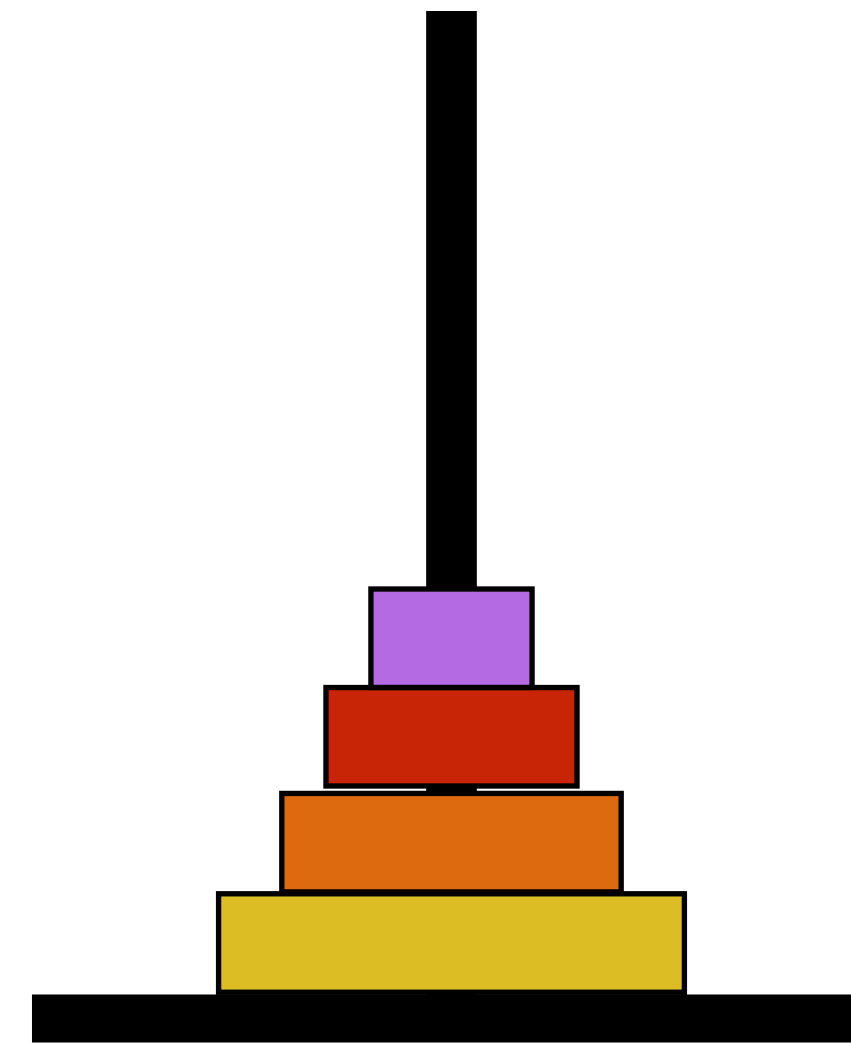
n=4



Stange A



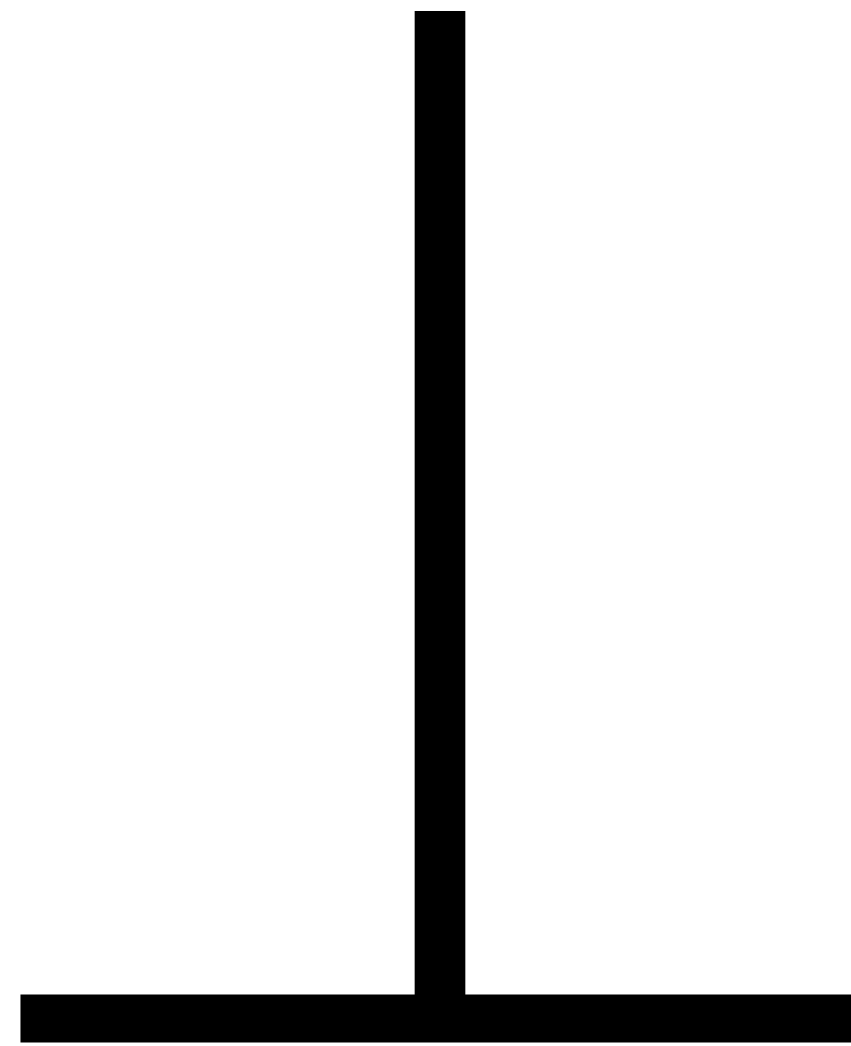
Stange B



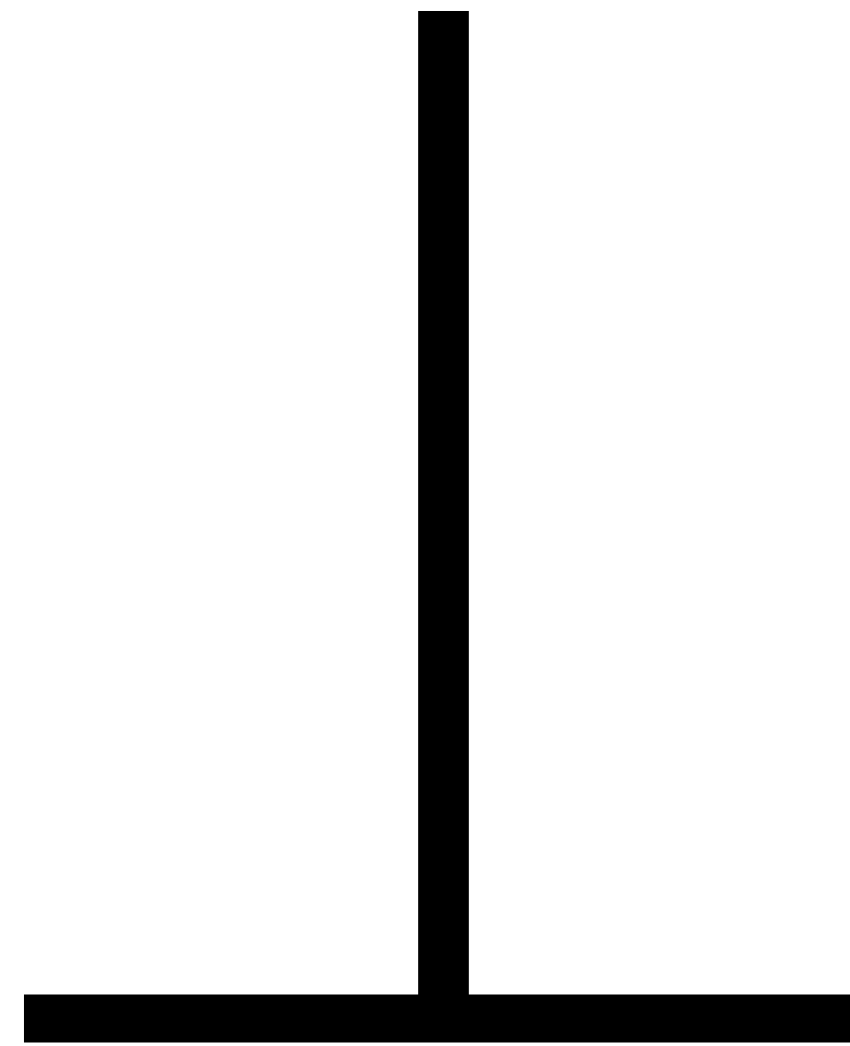
Stange C

Kleine Türme von Hanoi

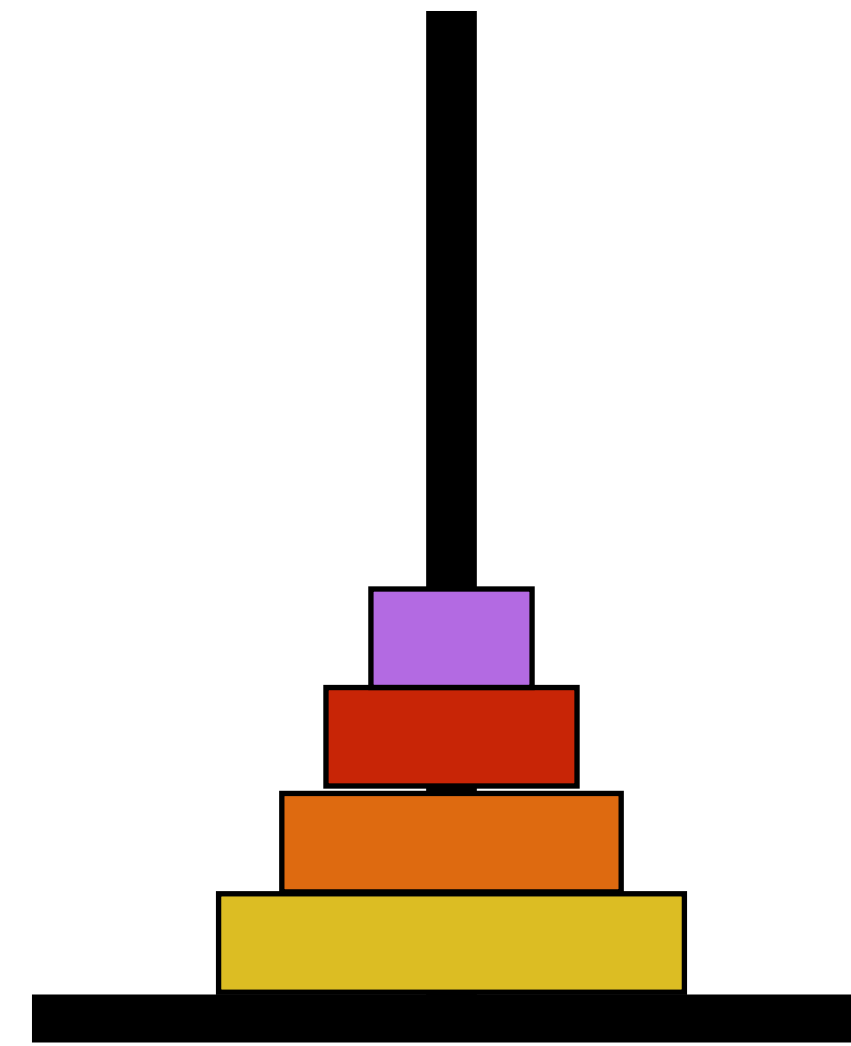
n=4



Stange A



Stange B

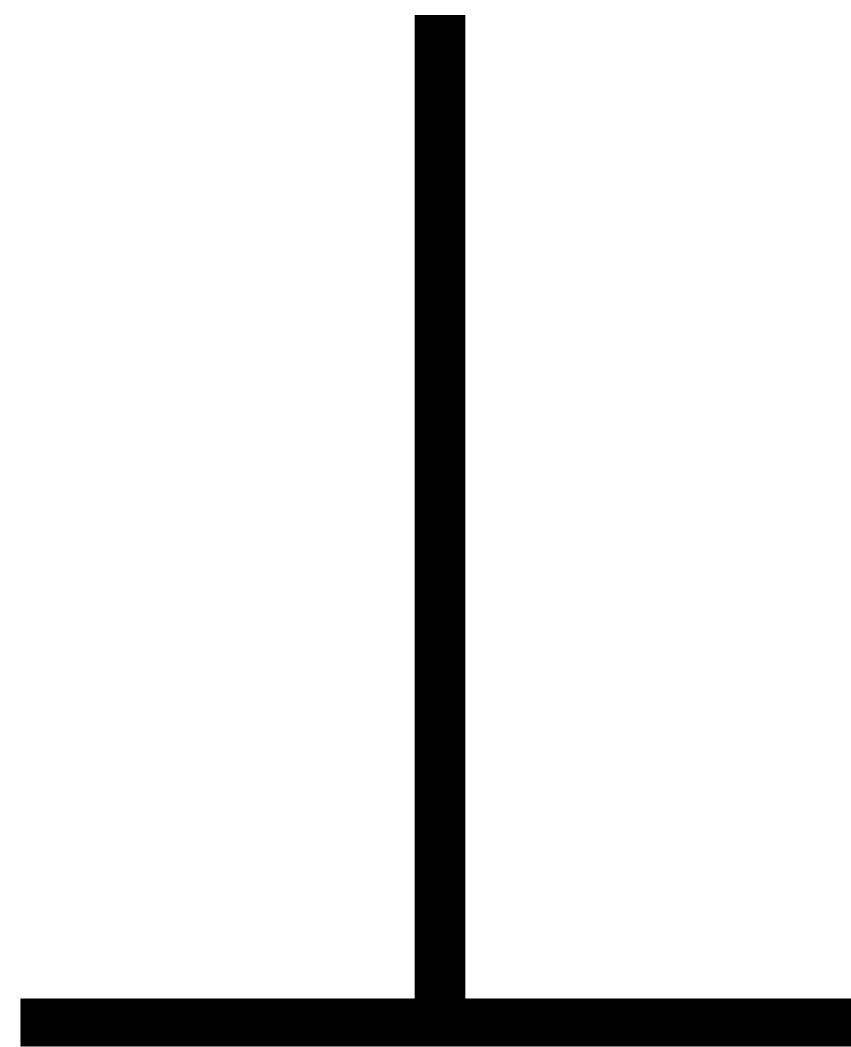


Stange C

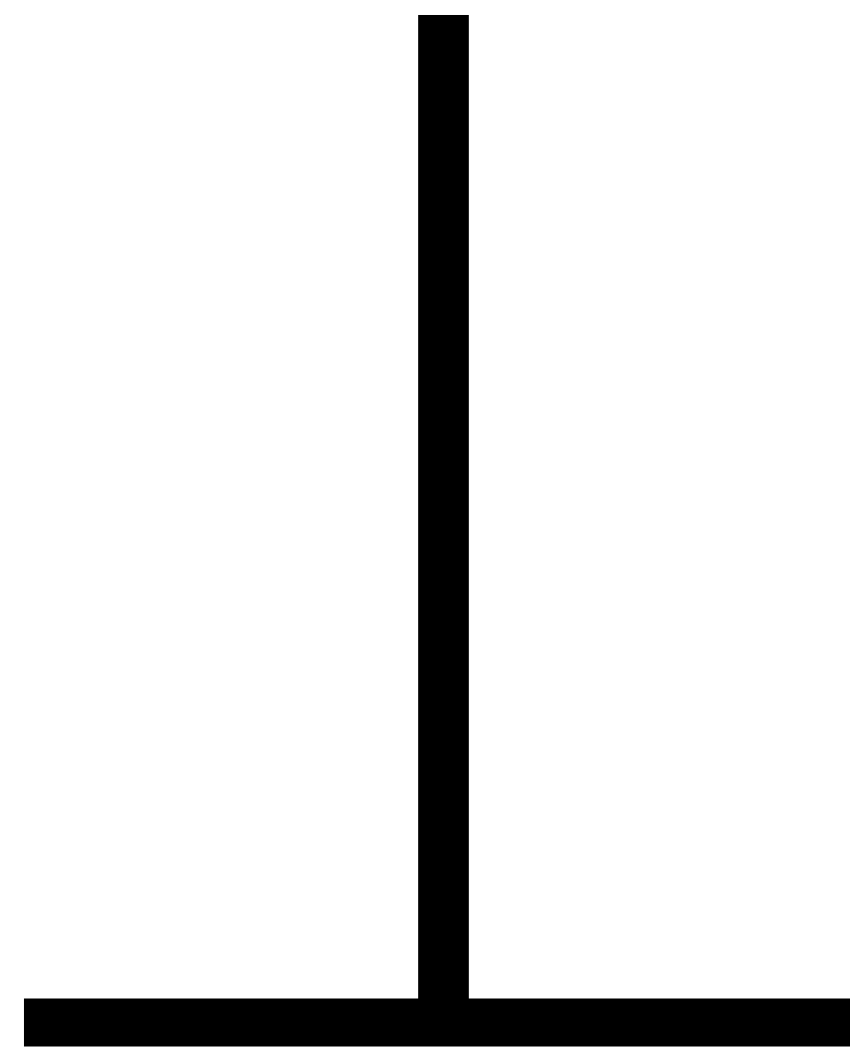


Kleine Türme von Hanoi

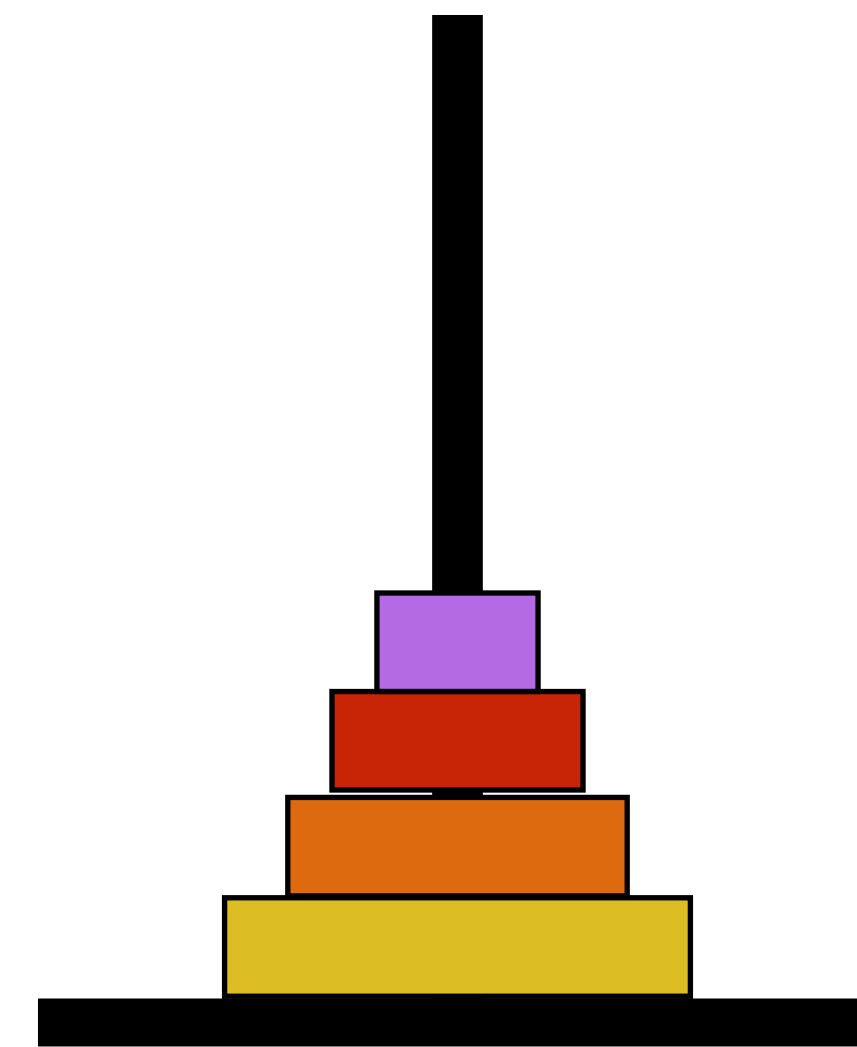
n=4



Stange A



Stange B

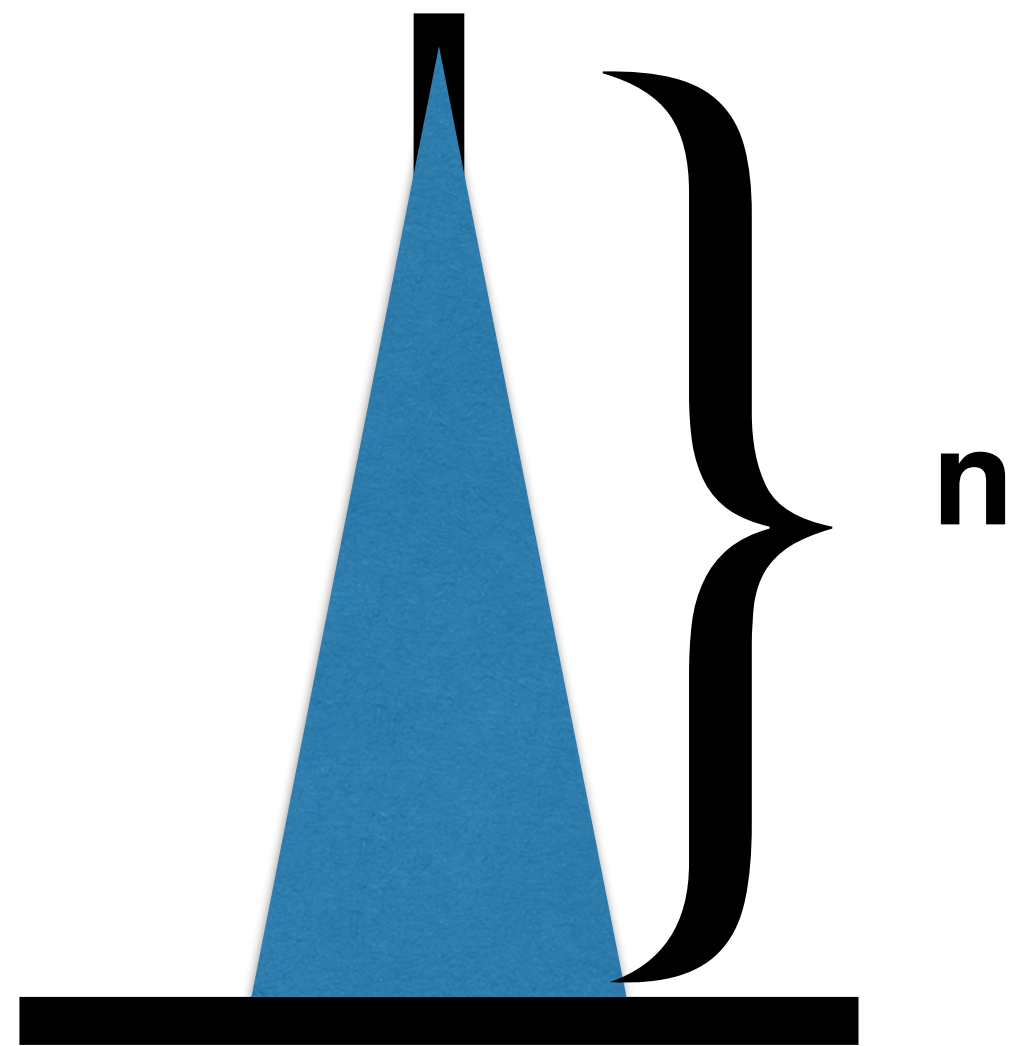


Stange C

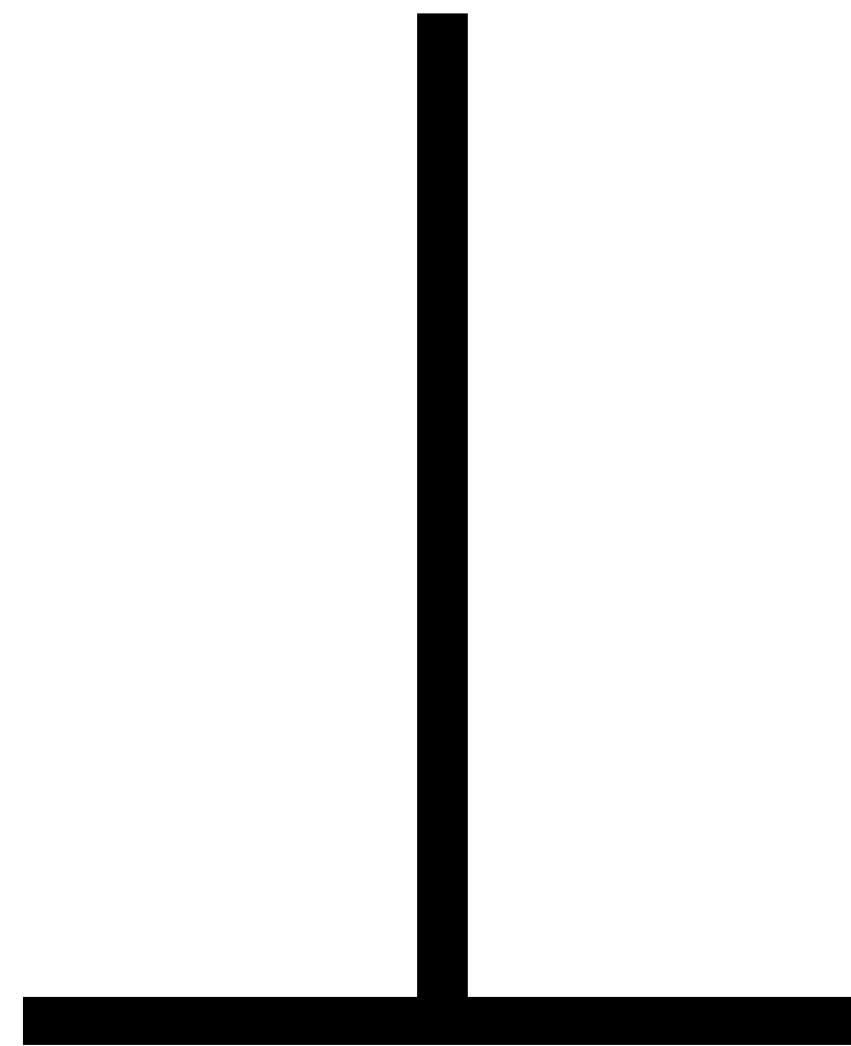
Kann dies für alle Turmhöhen **n** realisiert werden?

Leichte Verallgemeinerung

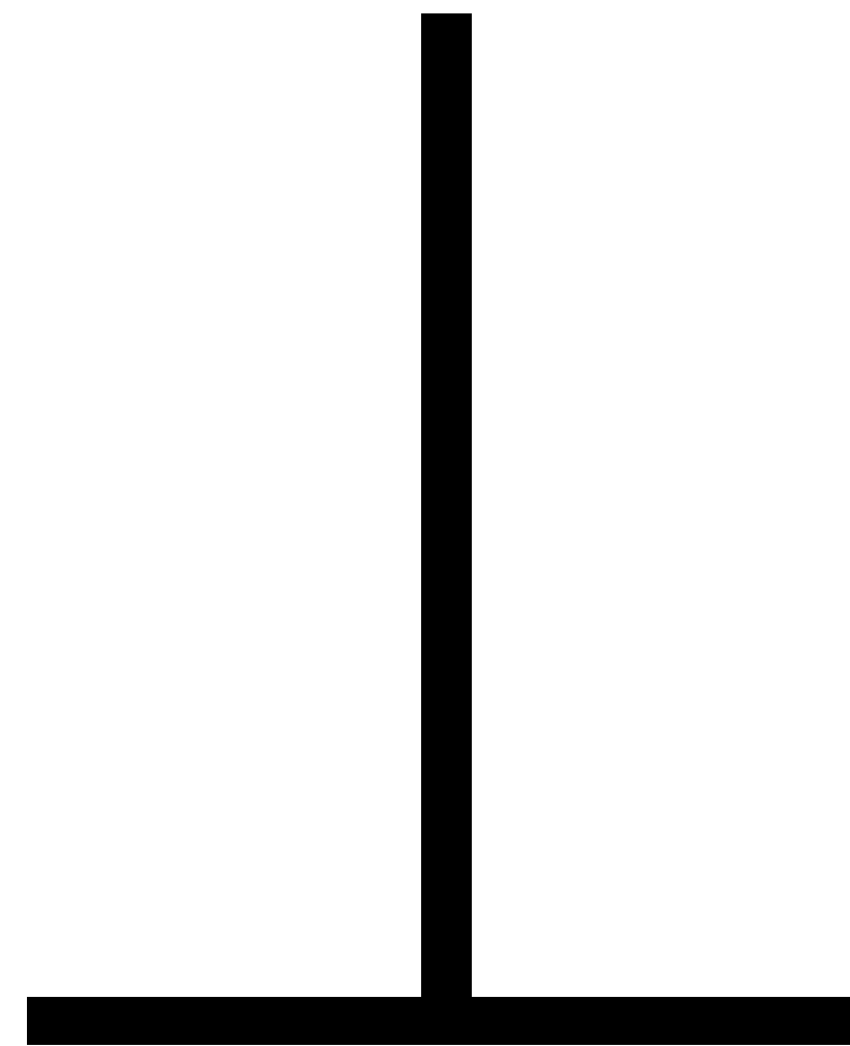
Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?



Startstange



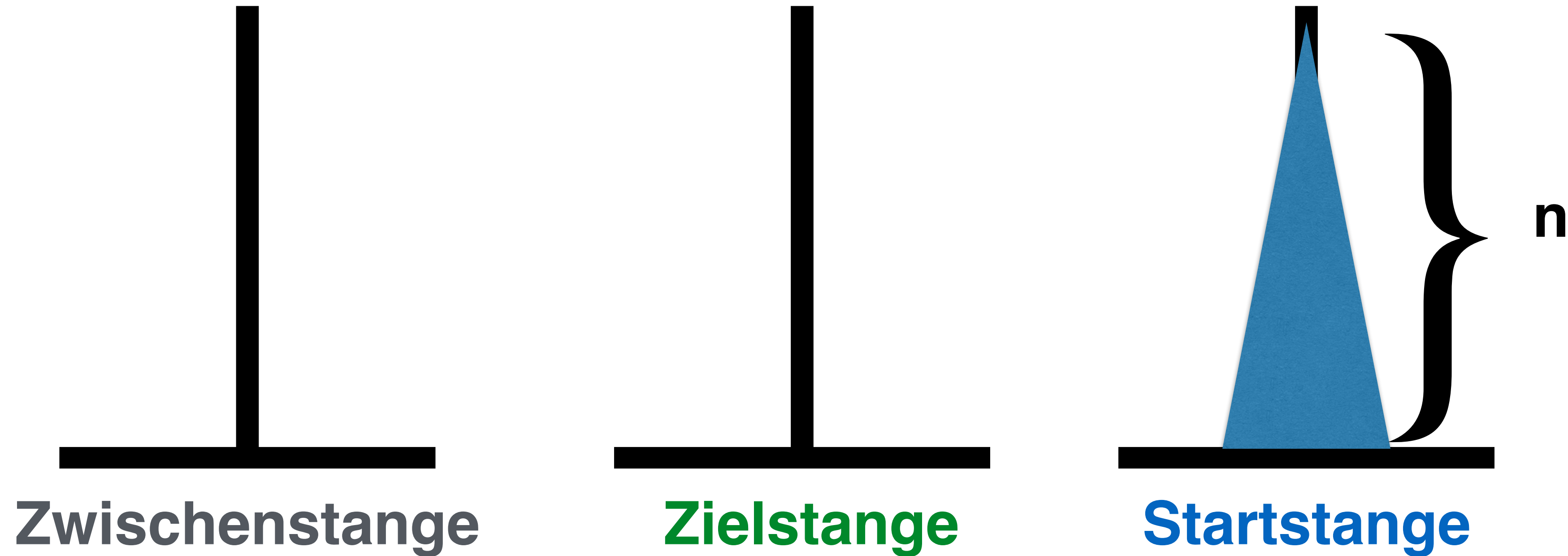
Zwischenstange



Zielstange

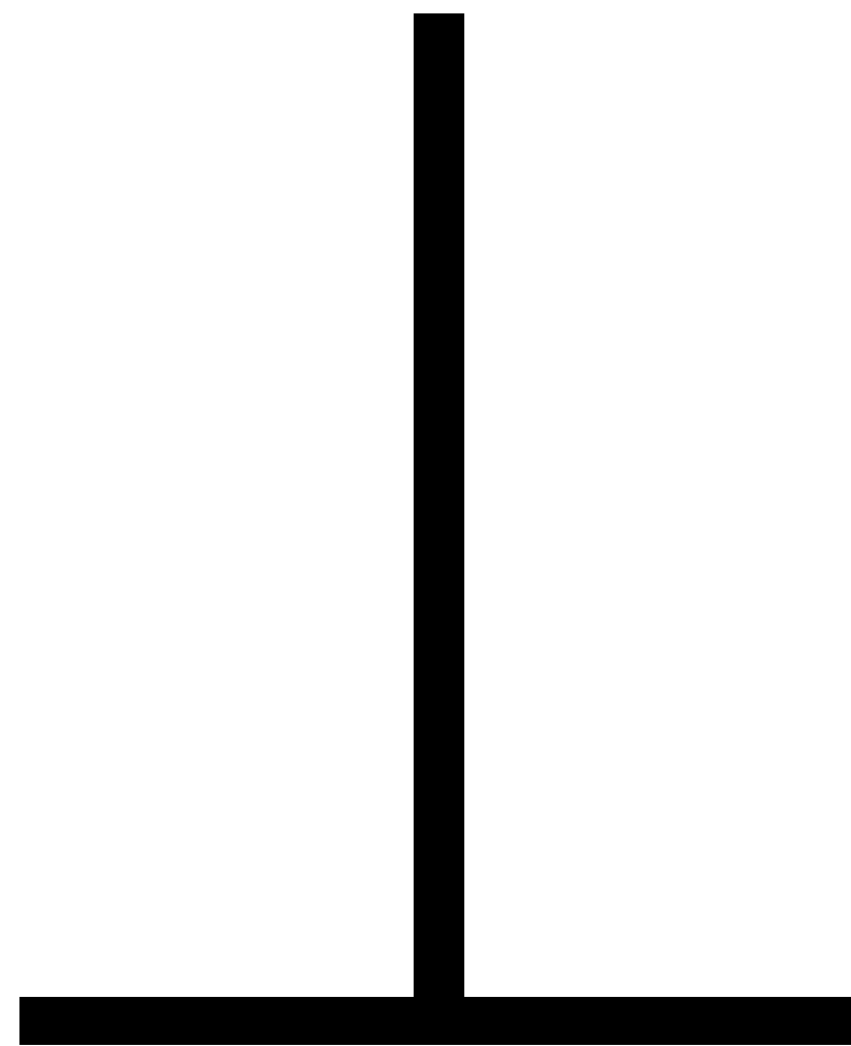
Leichte Verallgemeinerung

Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?

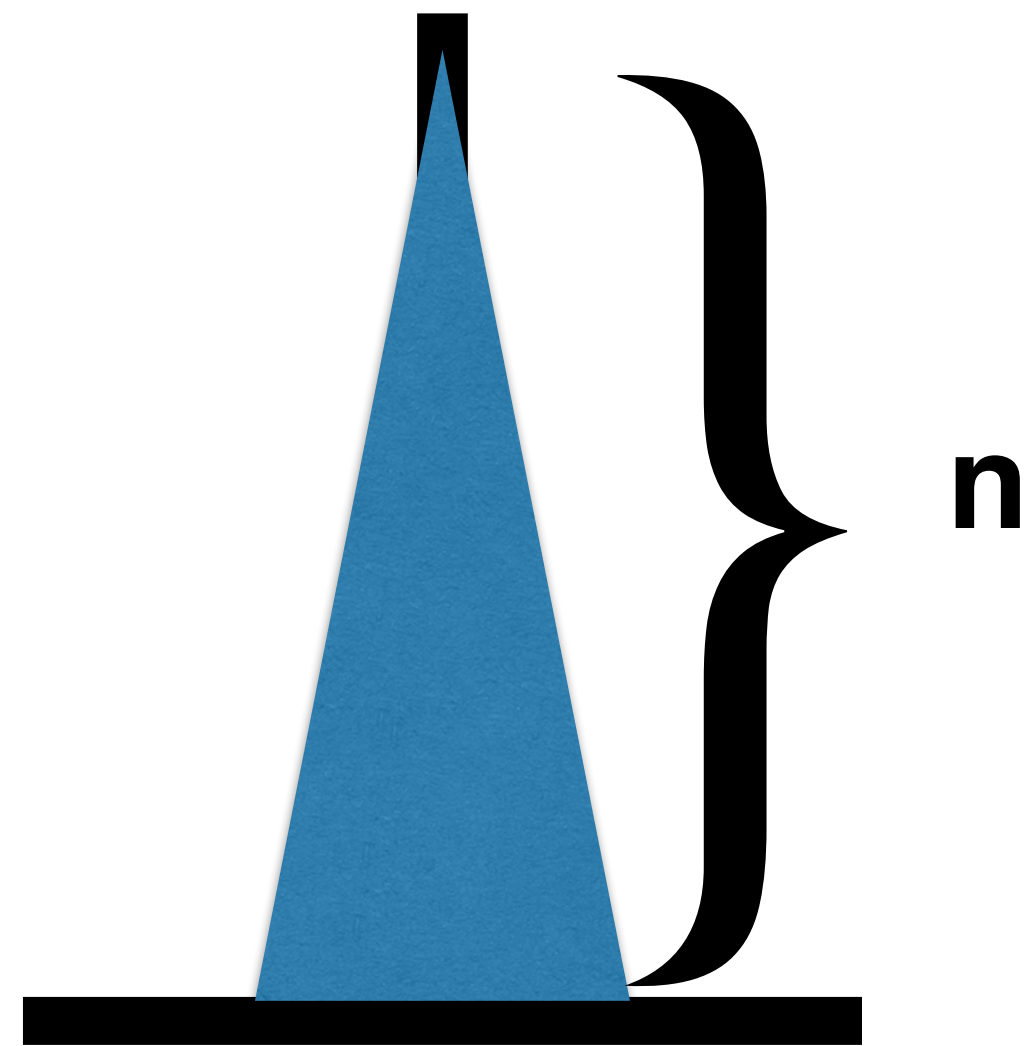


Leichte Verallgemeinerung

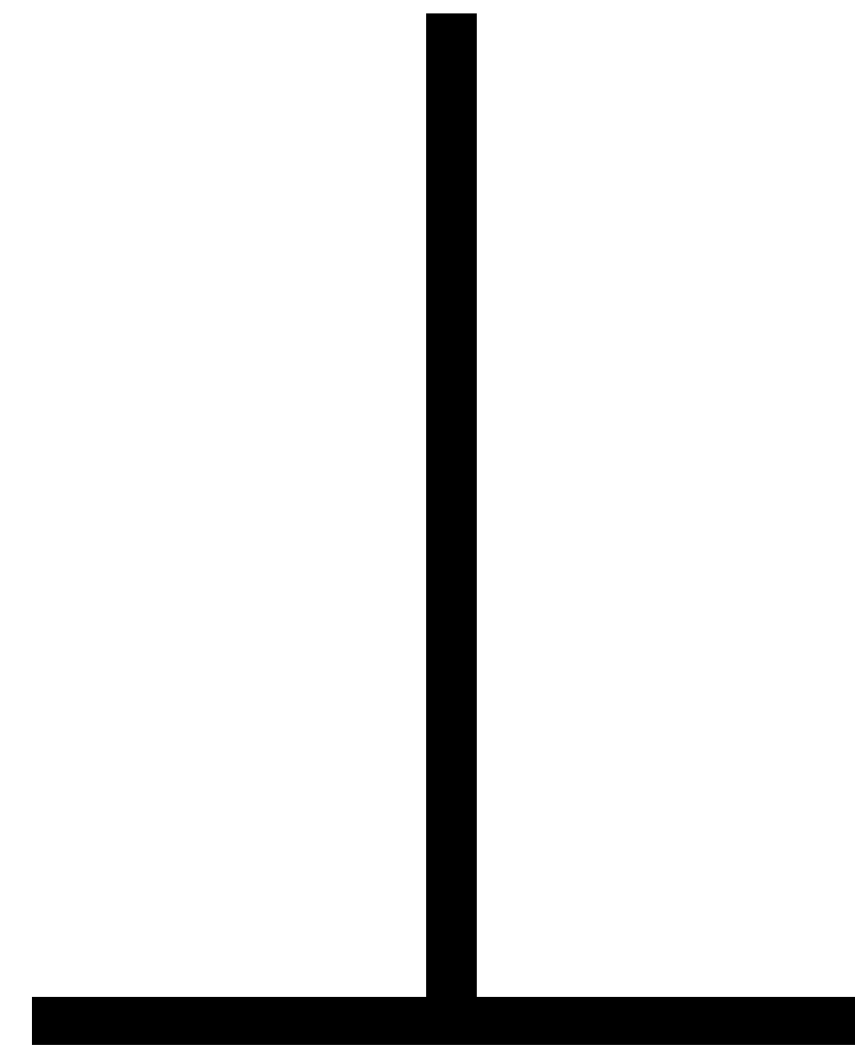
Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?



Zielstange



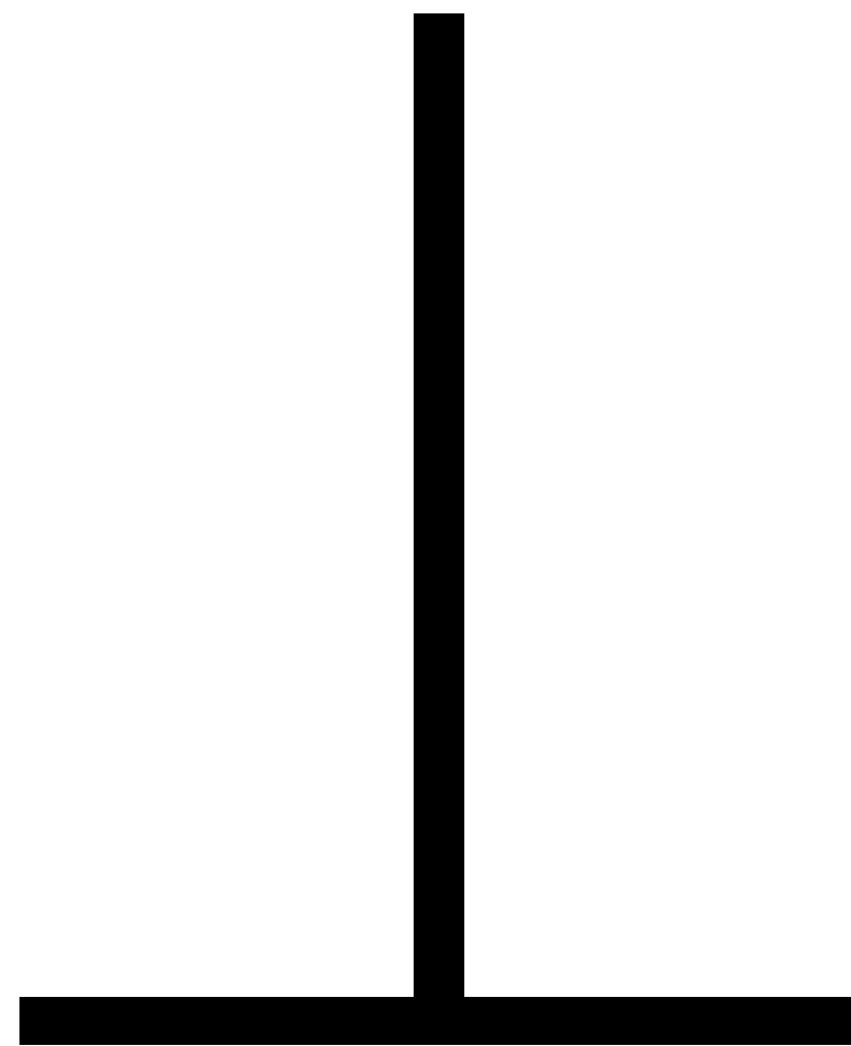
Startstange



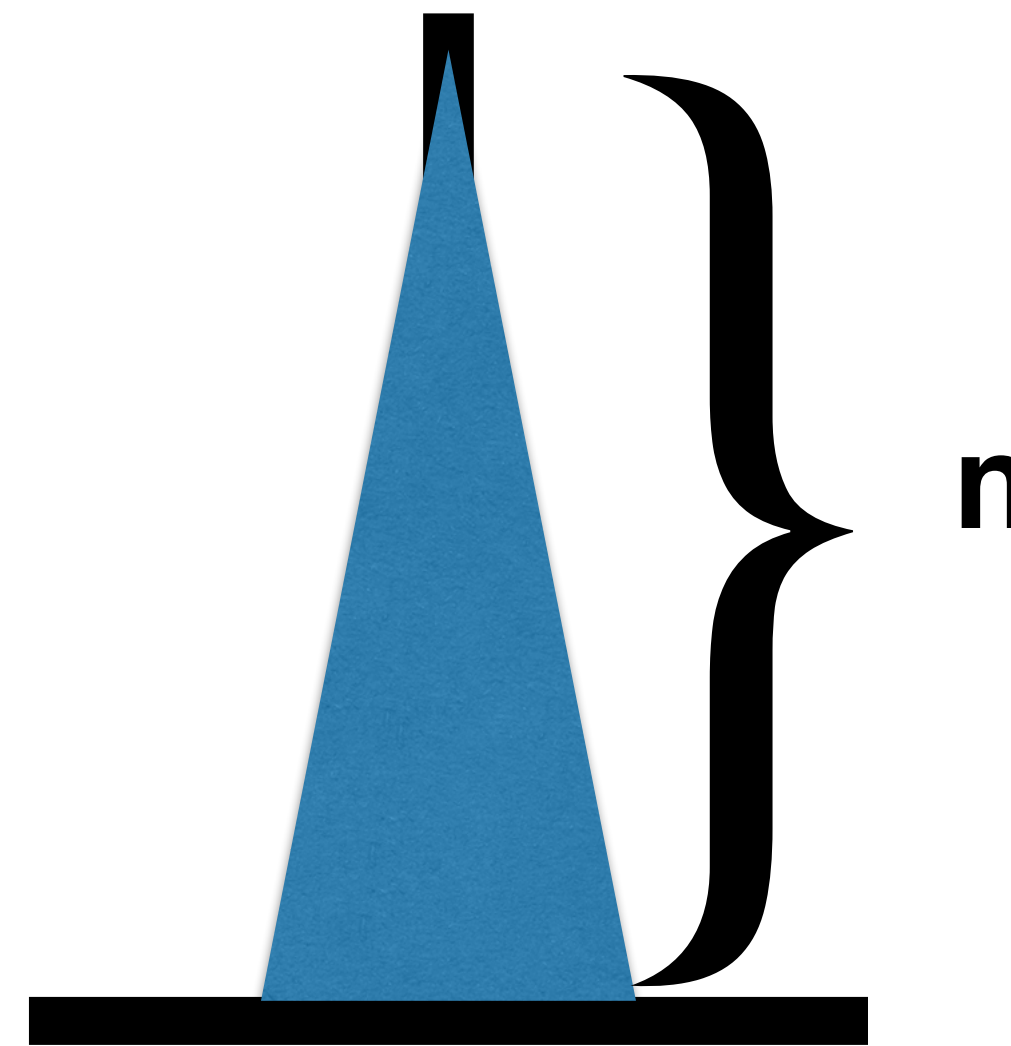
Zwischenstange

Leichte Verallgemeinerung

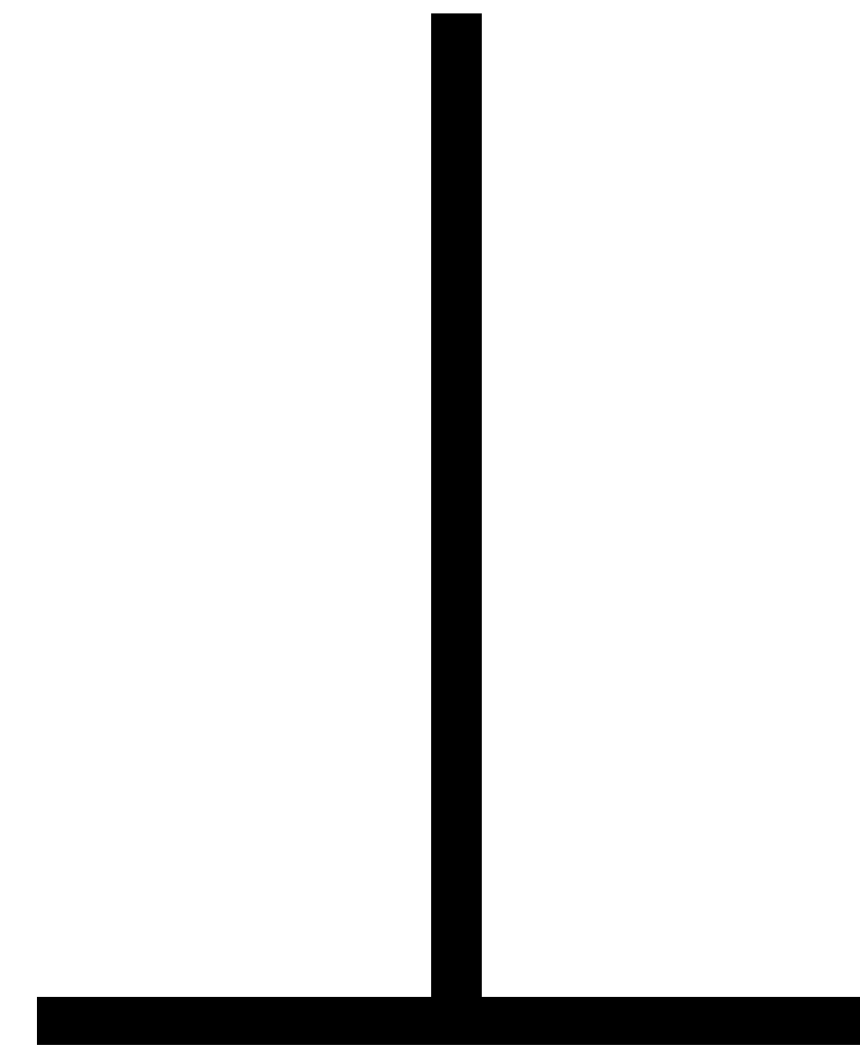
Wie bewegen wir (geordneten) Turm der Höhe **n** von beliebiger **Startstange** zu beliebiger **Zielstange**?



Zielstange



Startstange



Zwischenstange

Rekursion!

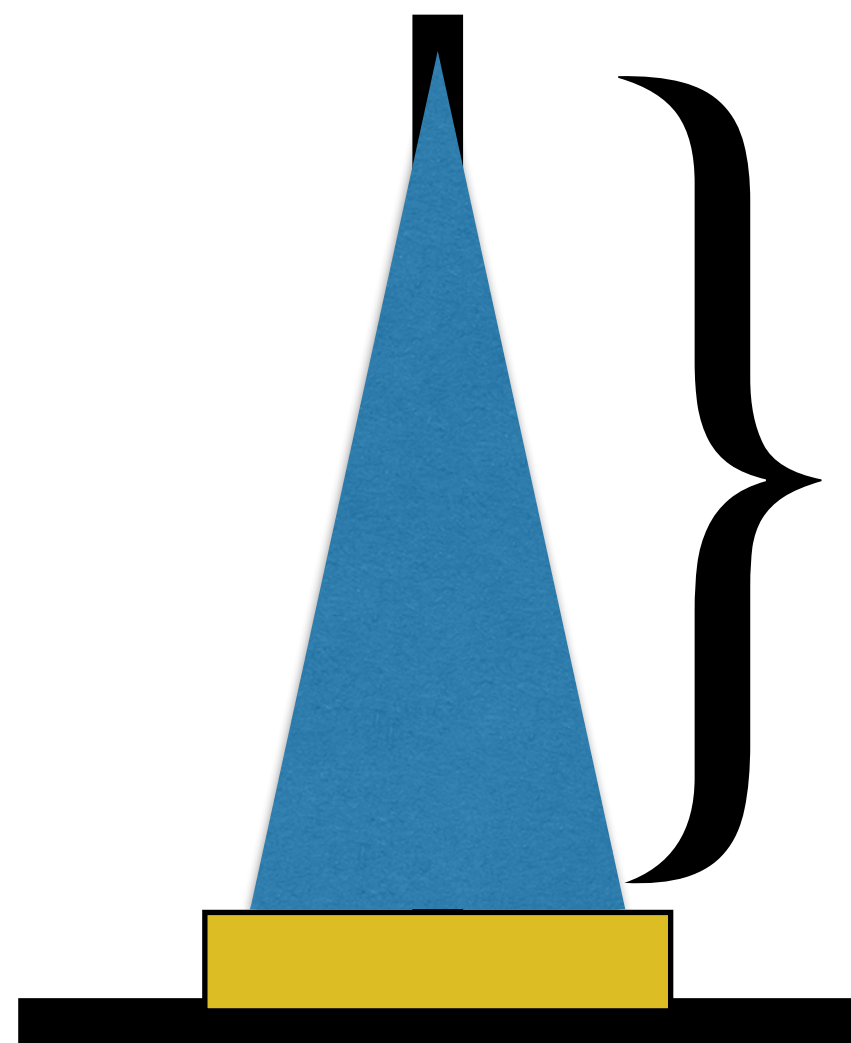
Leichte Verallgemeinerung

Die triviale Instanz **$n=1$** haben wir bereits gelöst!

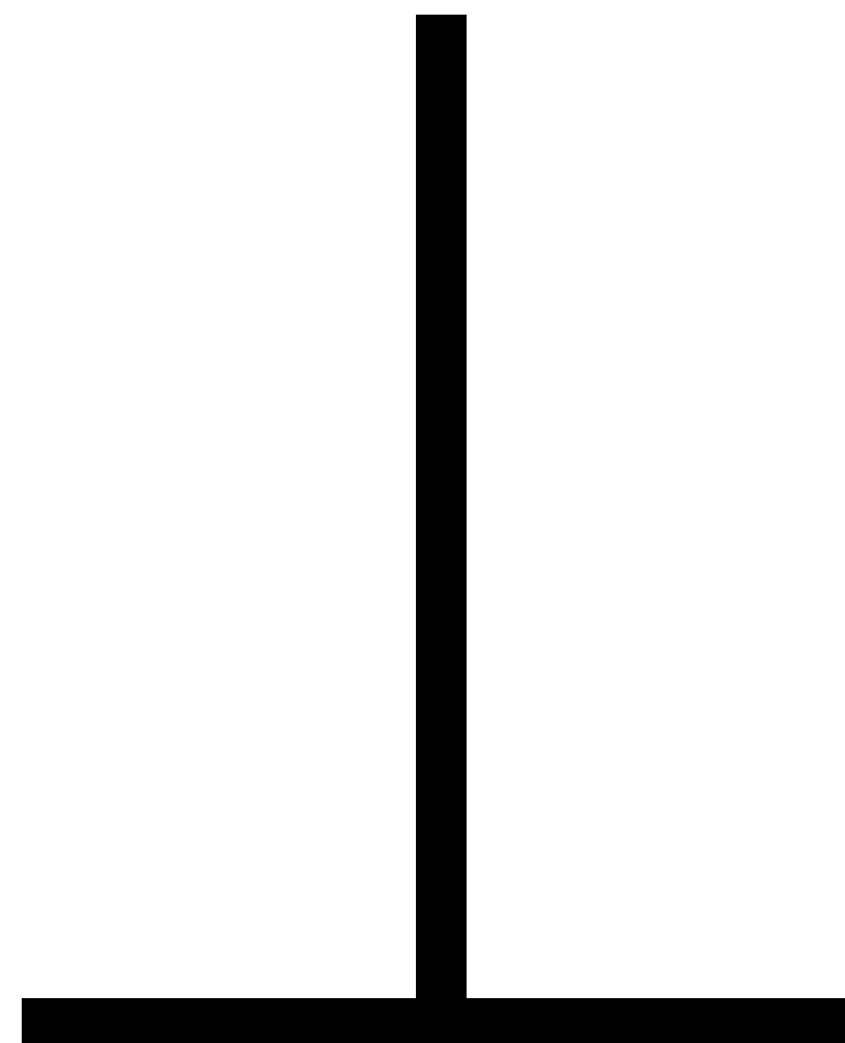
Leichte Verallgemeinerung

Die triviale Instanz **$n=1$** haben wir bereits gelöst!

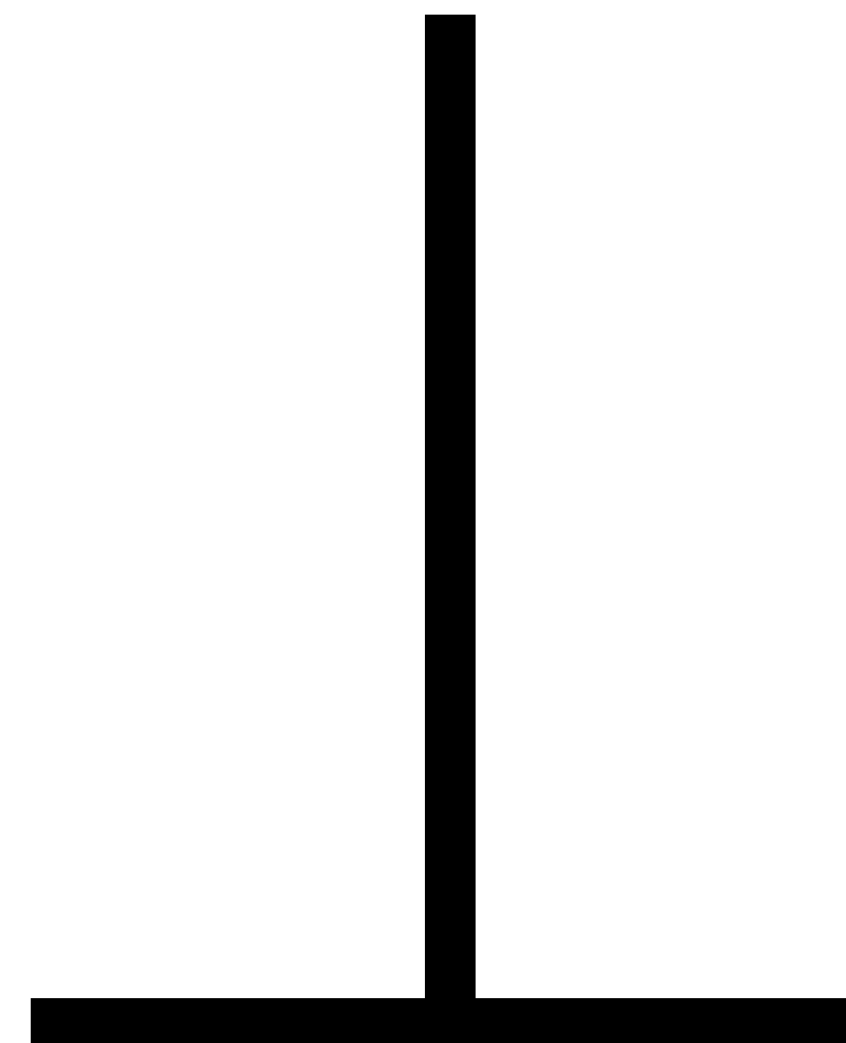
Sei also **$n > 1$** .



Startstange



Zwischenstange

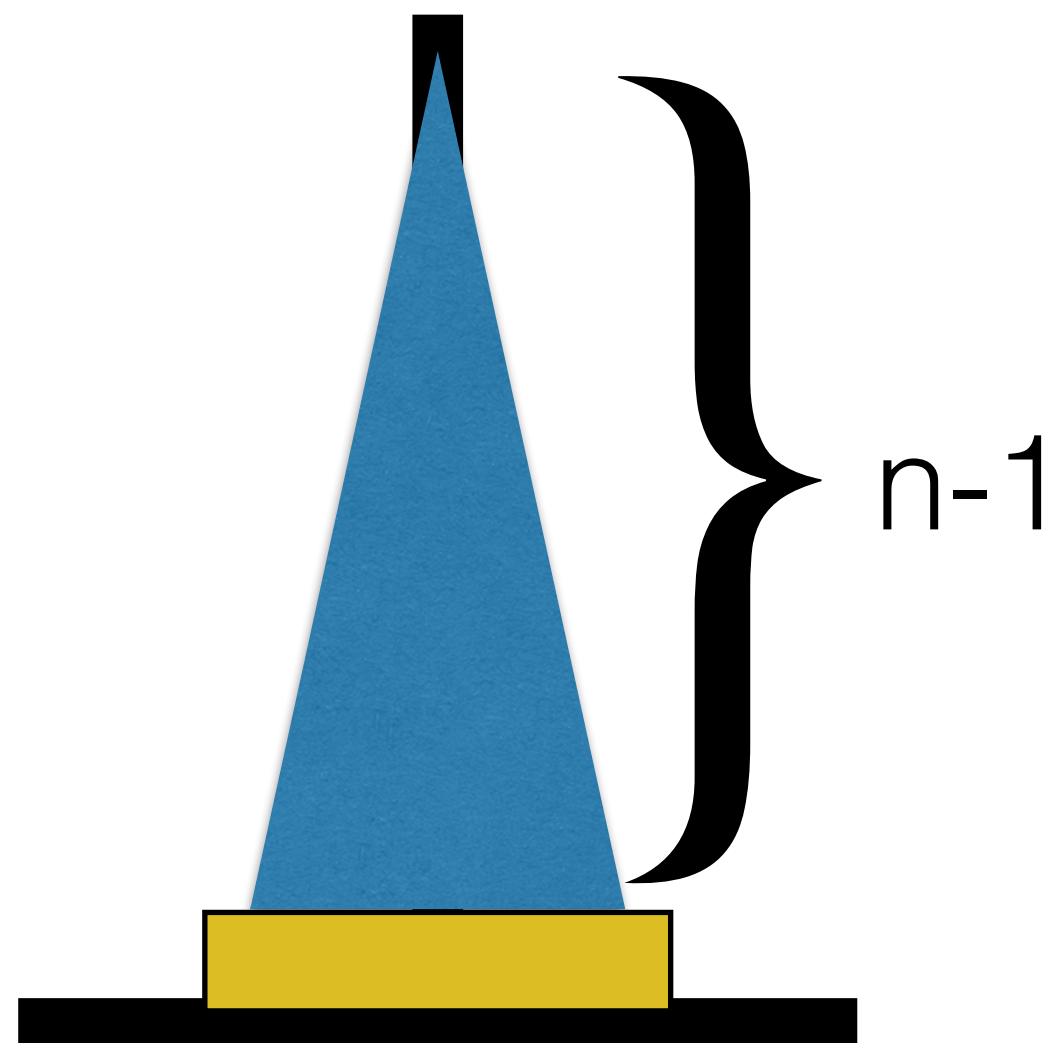


Zielstange

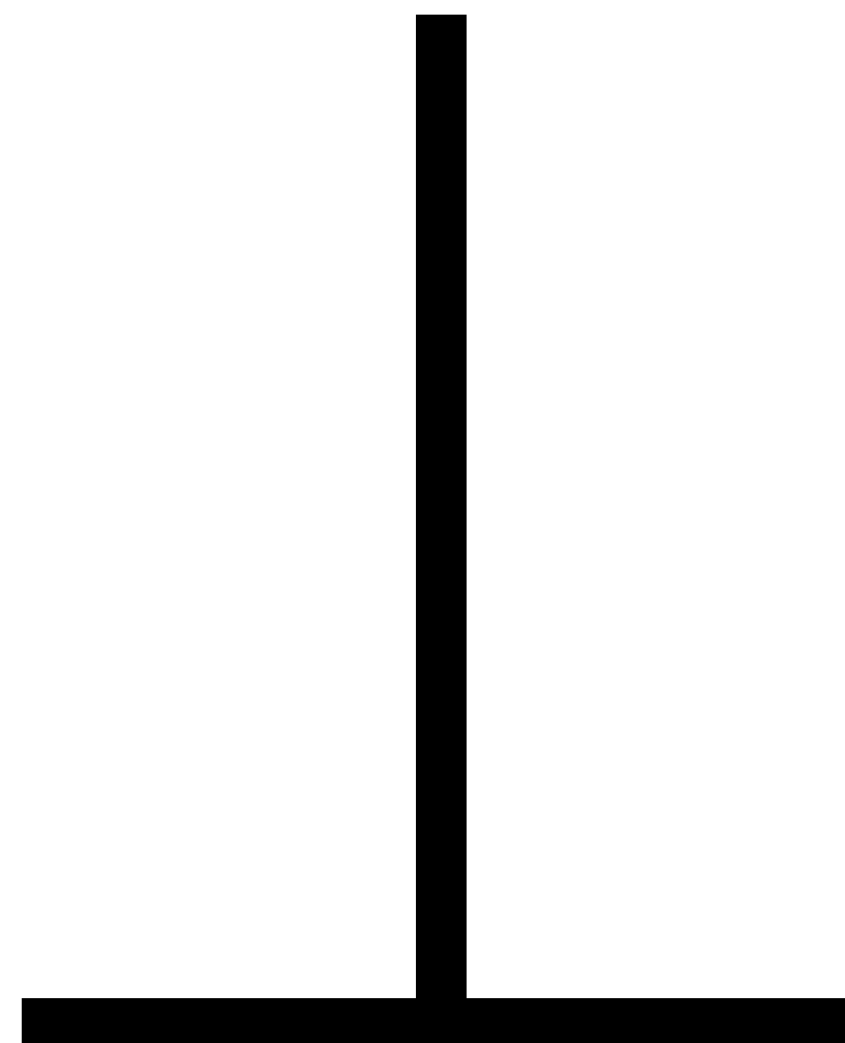
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

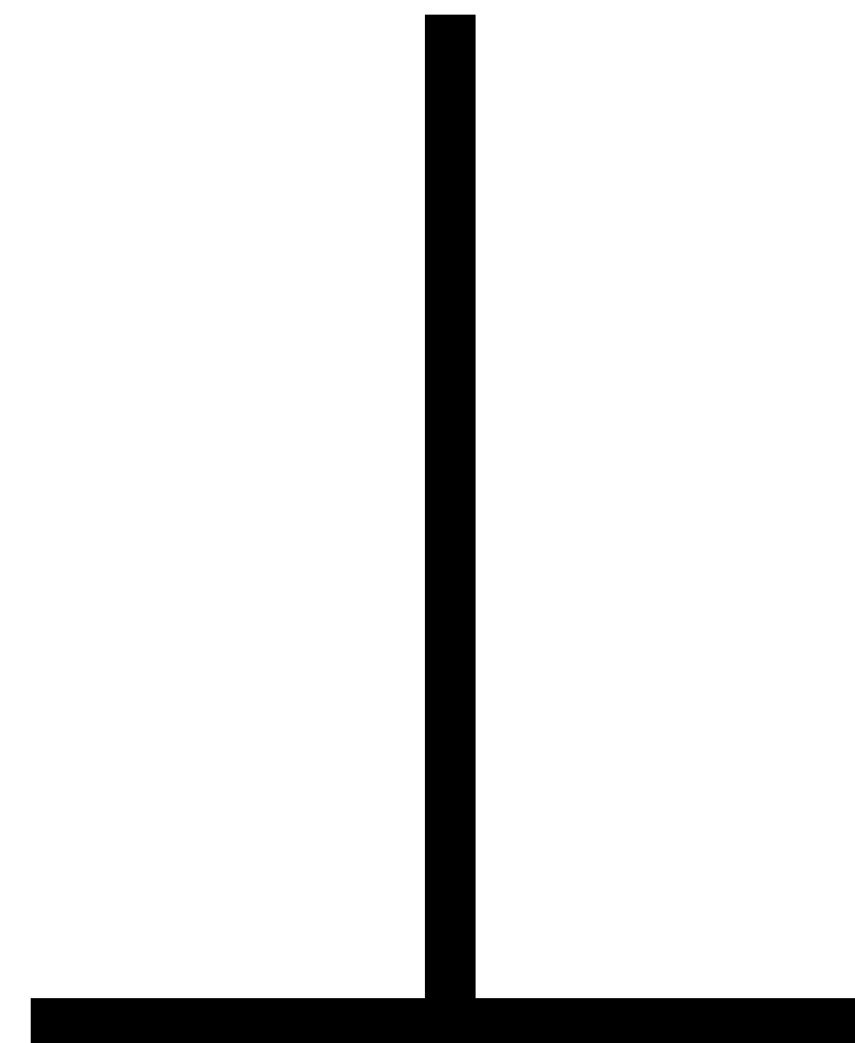
Sei also **n**>1.



Startstange



Zwischenstange



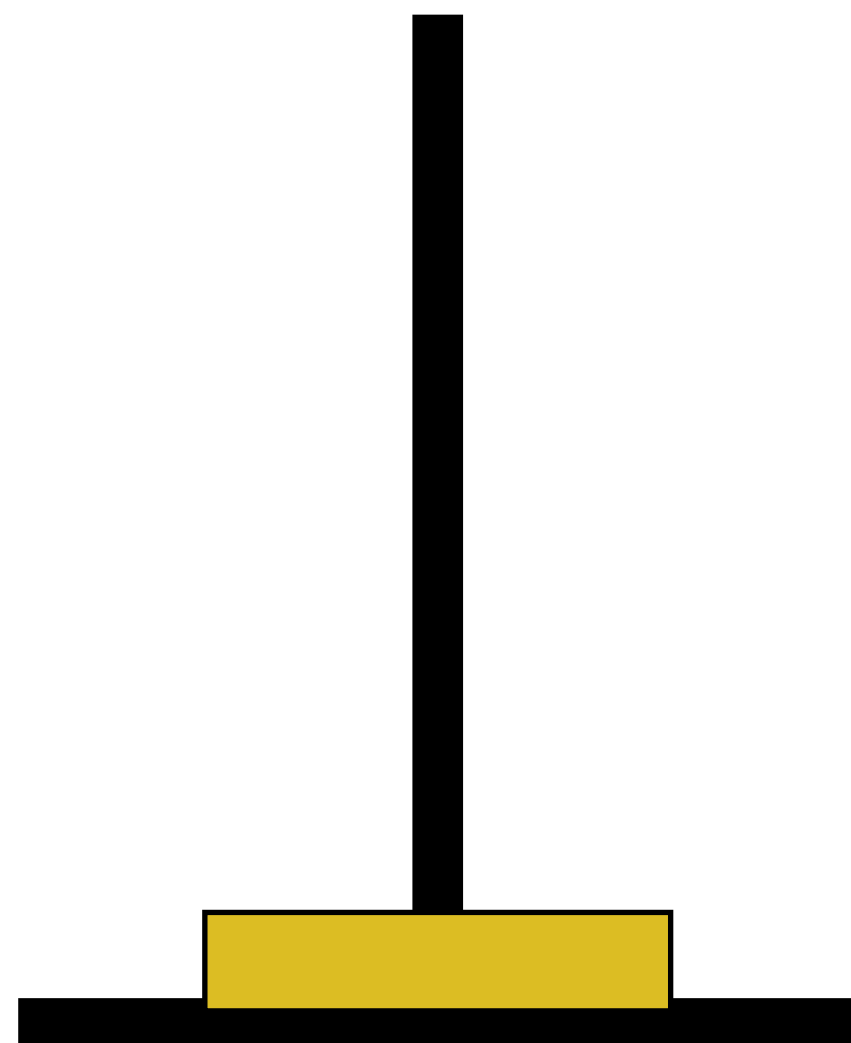
Zielstange

Verschiebe rekursiv  von **Startstange** zur **Zwischenstange**

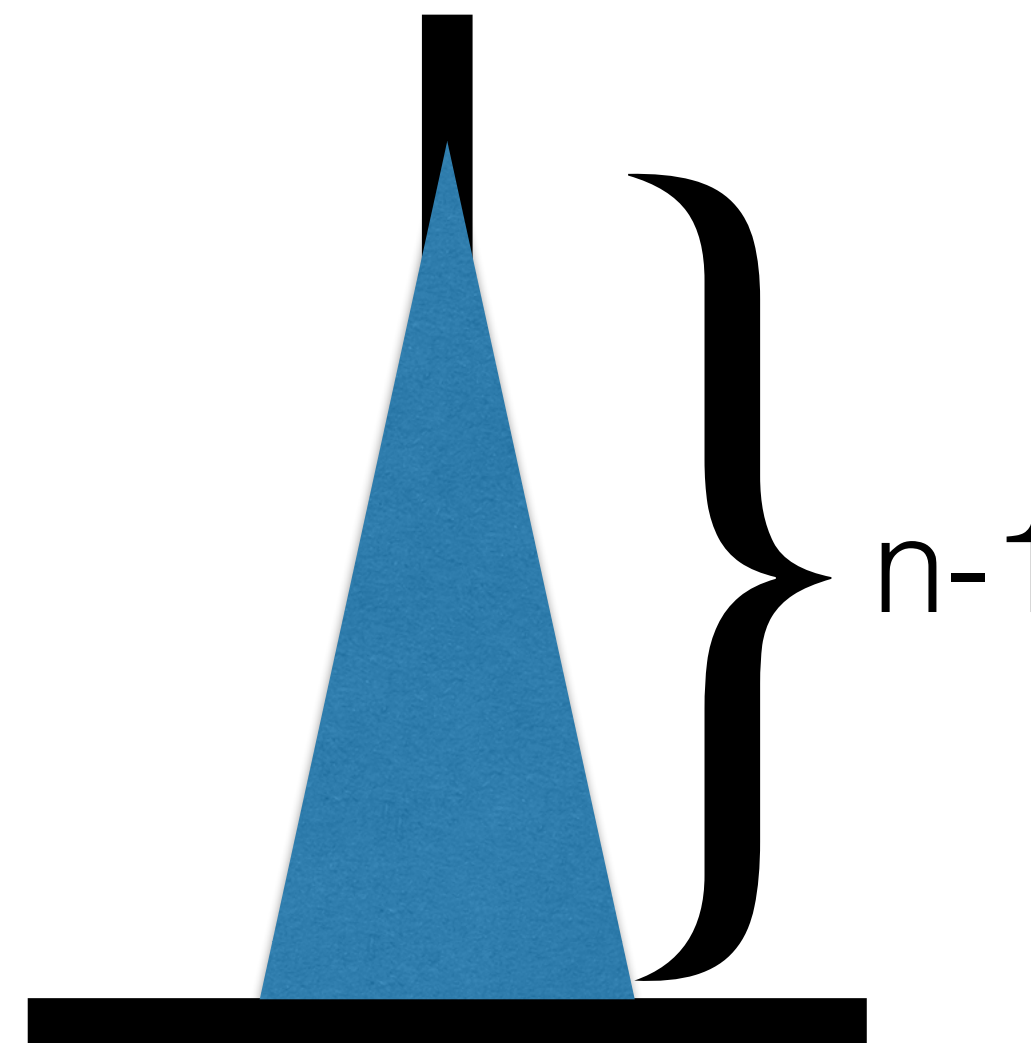
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

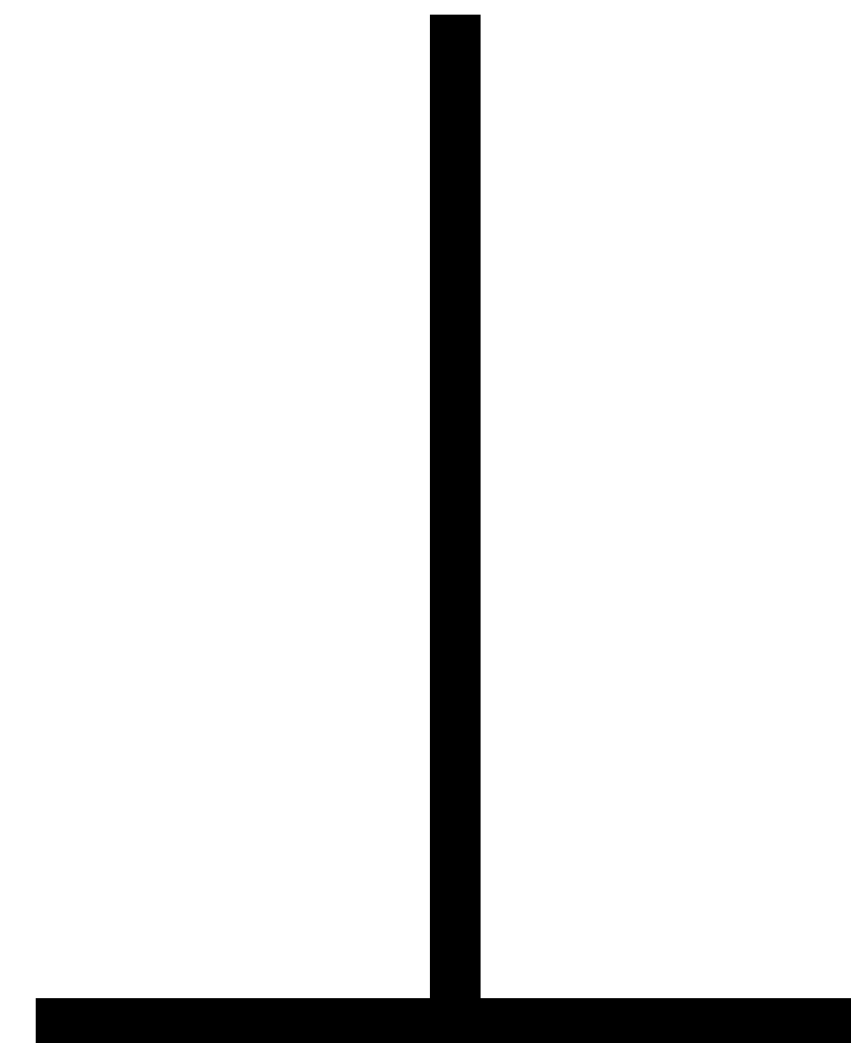
Sei also **n**>1.



Startstange



Zwischenstange



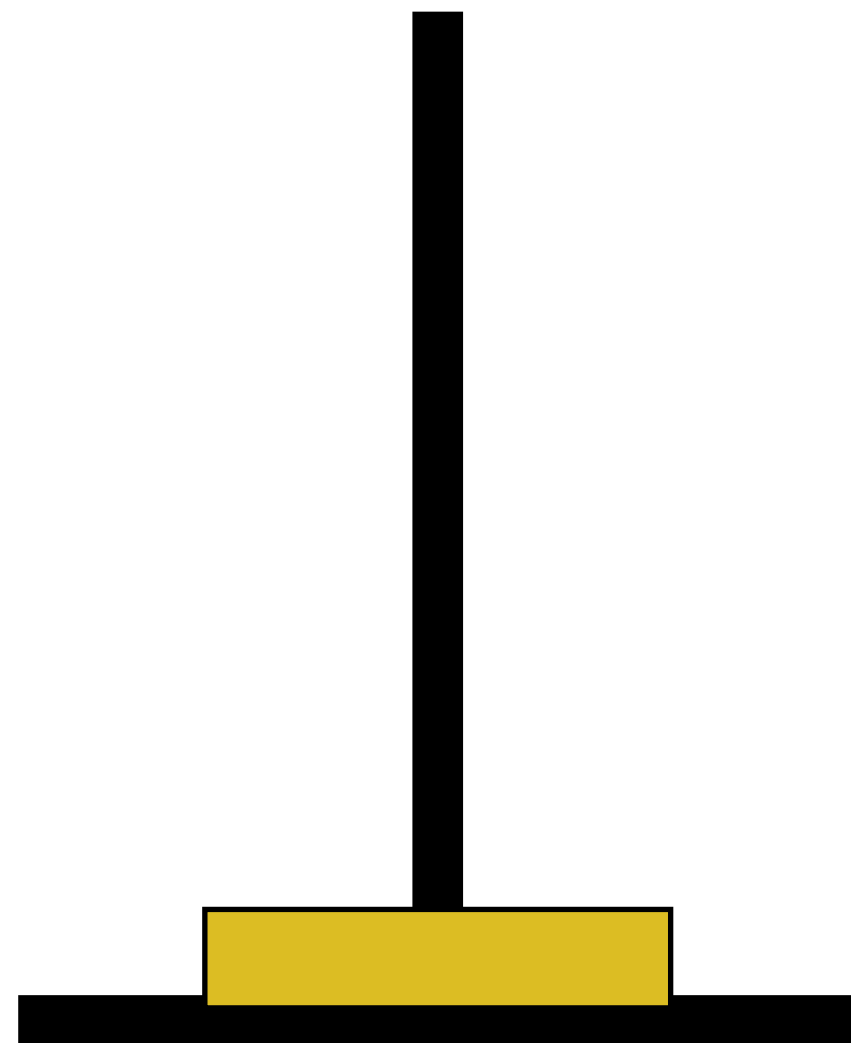
Zielstange

Verschiebe rekursiv  von **Startstange** zur **Zwischenstange**

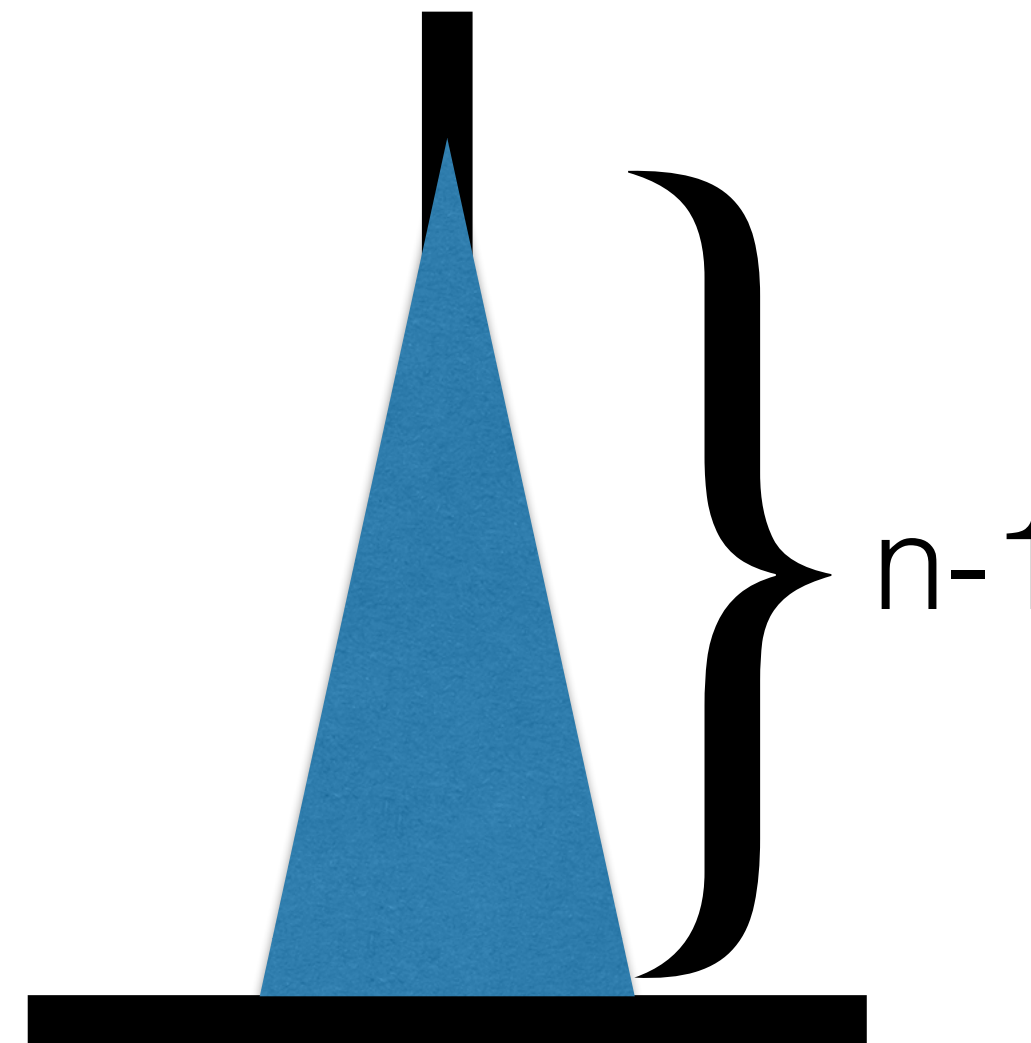
Leichte Verallgemeinerung

Die triviale Instanz $n=1$ haben wir bereits gelöst!

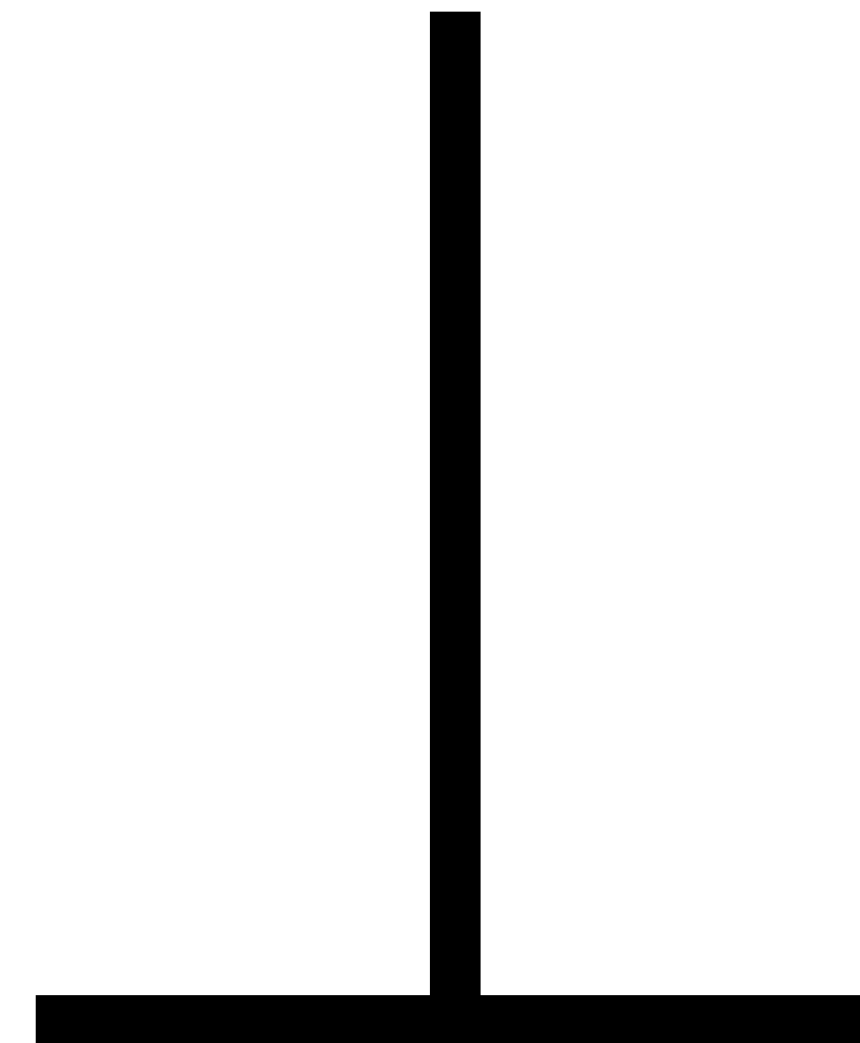
Sei also $n > 1$.



Startstange



Zwischenstange



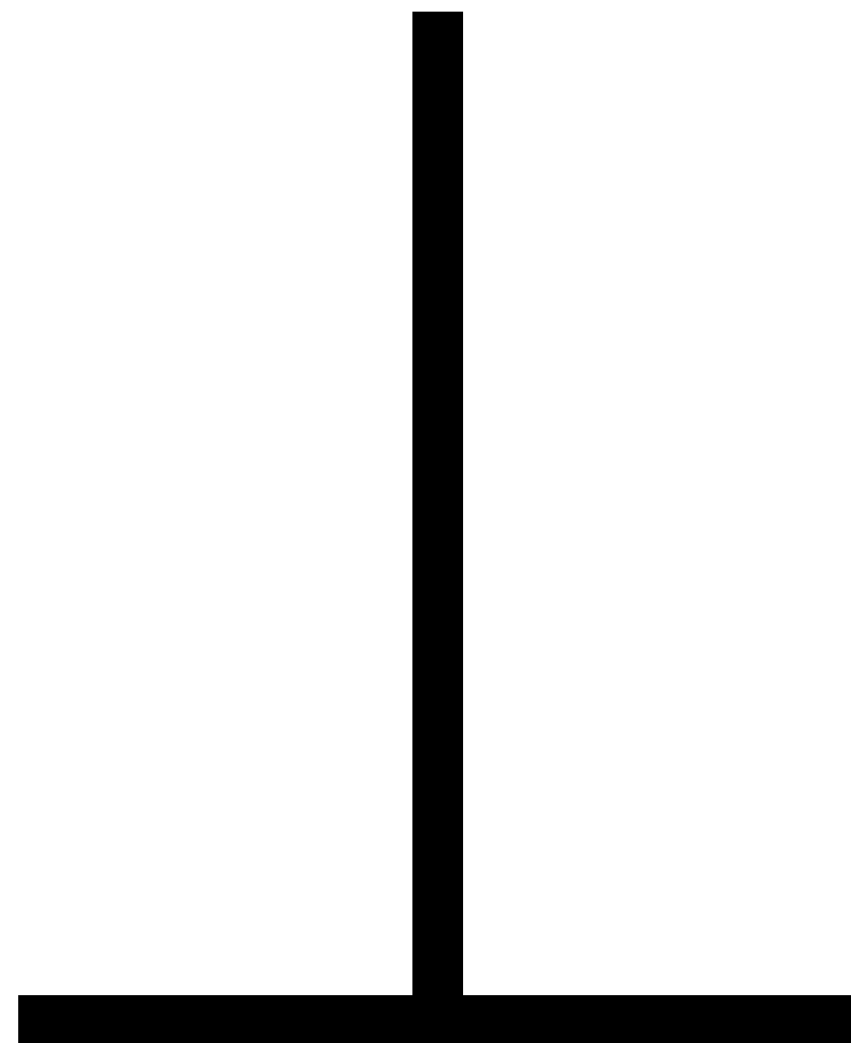
Zielstange

Verschiebe  direkt von **Startstange** zur **Zielstange**

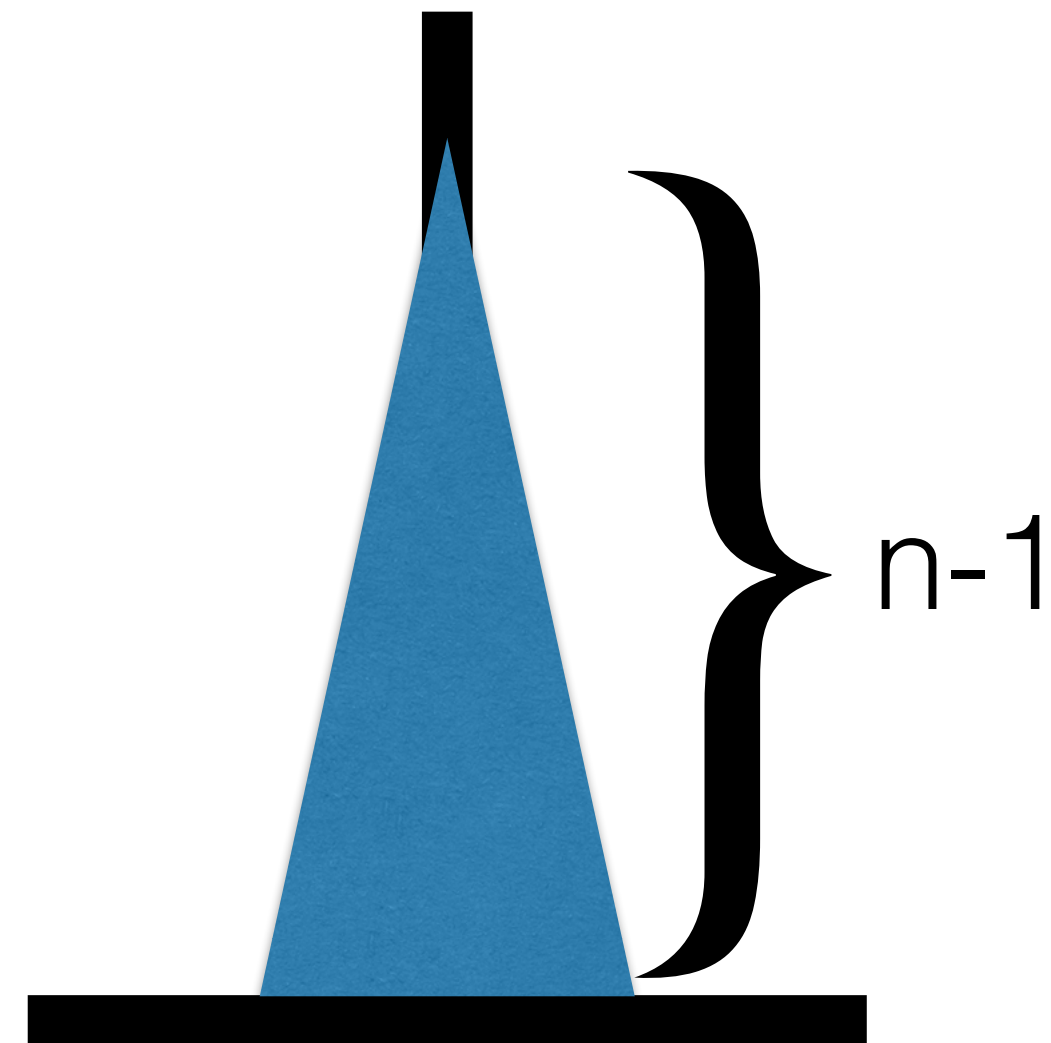
Leichte Verallgemeinerung

Die triviale Instanz **$n=1$** haben wir bereits gelöst!

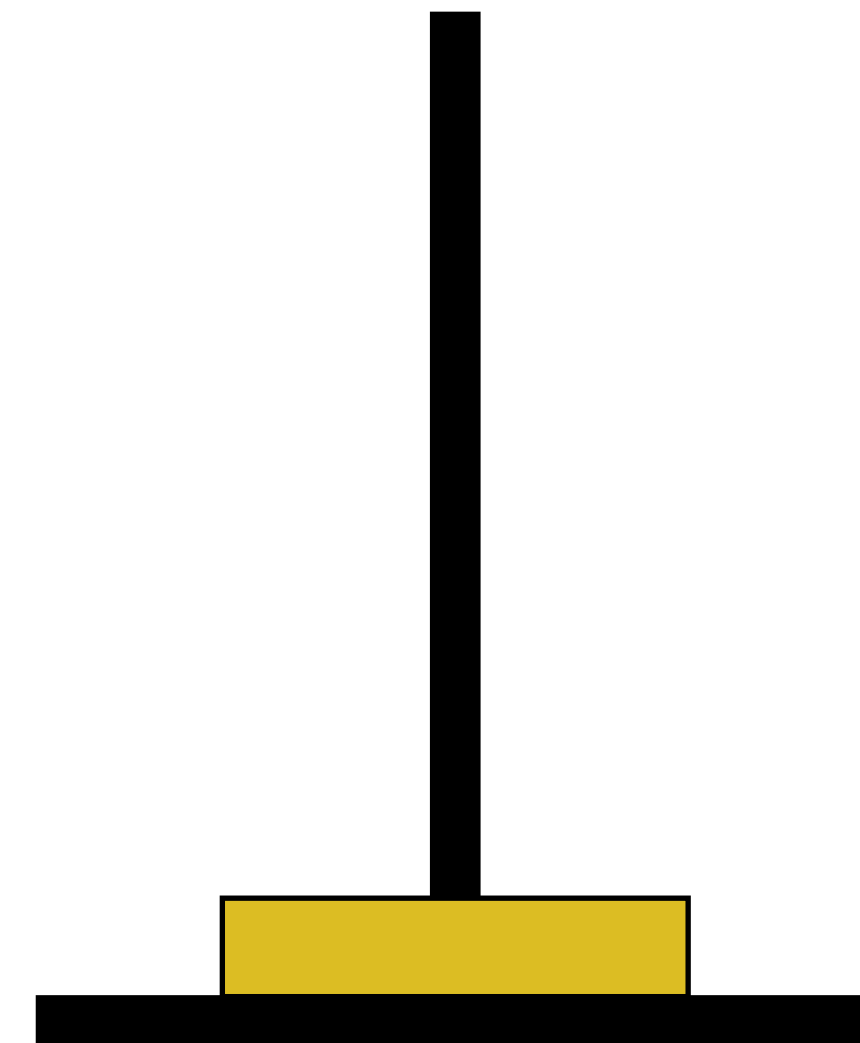
Sei also **$n > 1$** .



Startstange



Zwischenstange

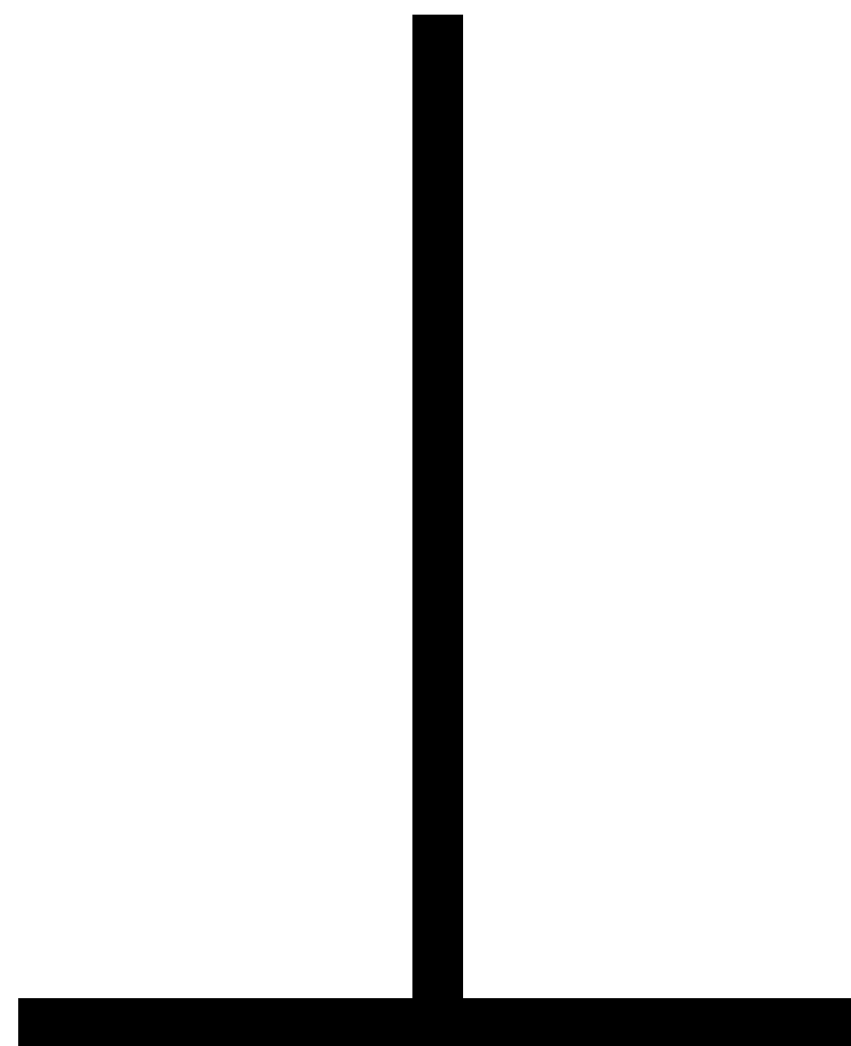


Zielstange

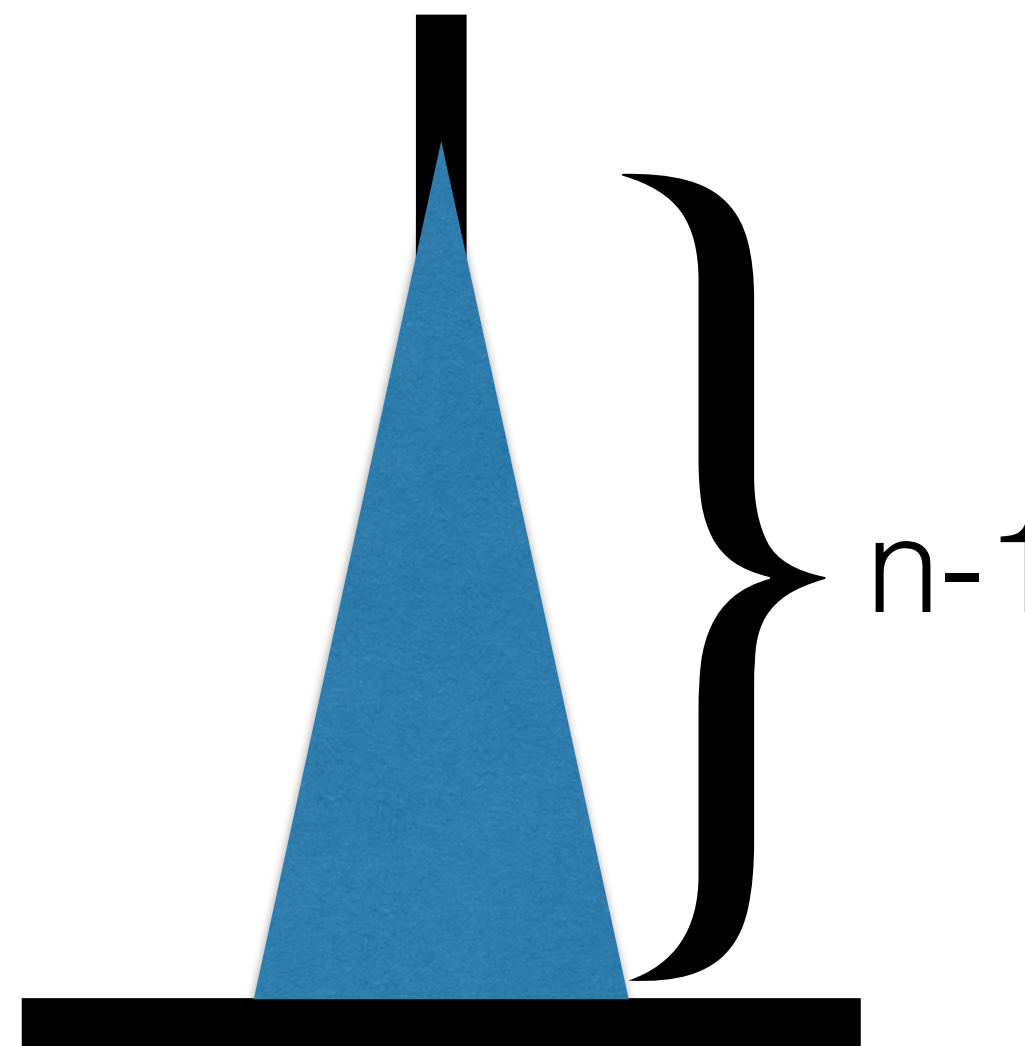
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

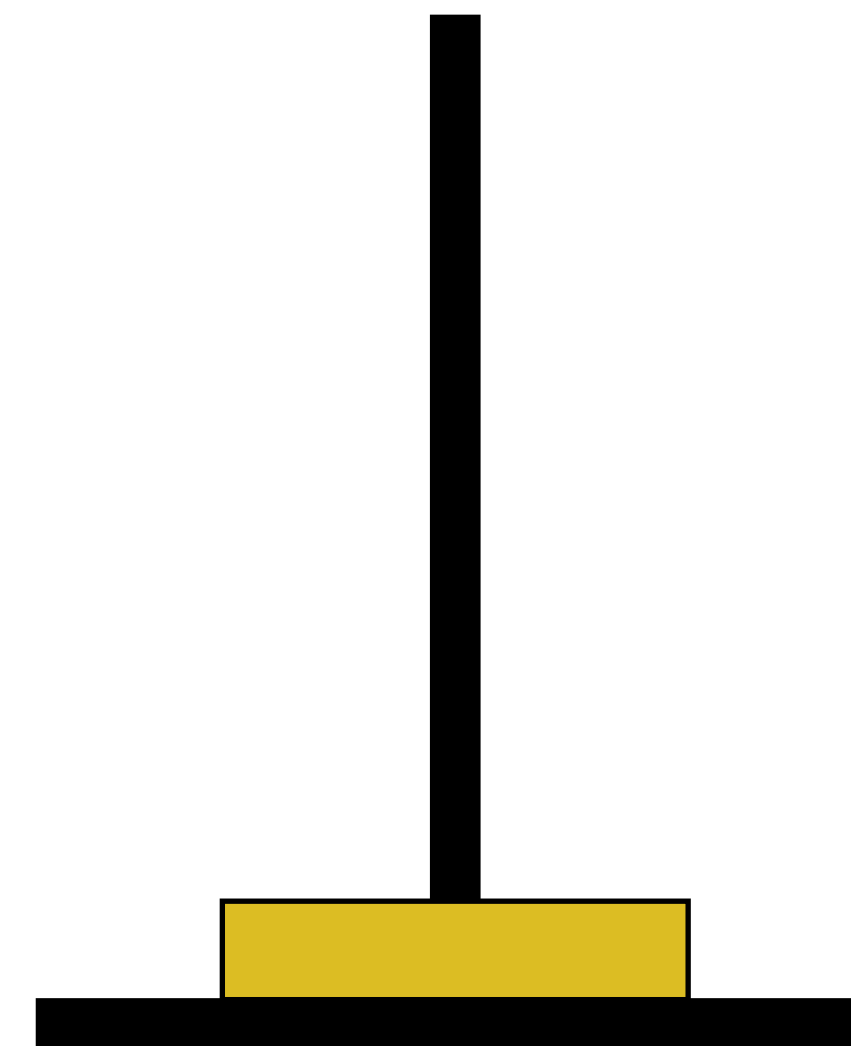
Sei also **n**>1.



Startstange



Zwischenstange



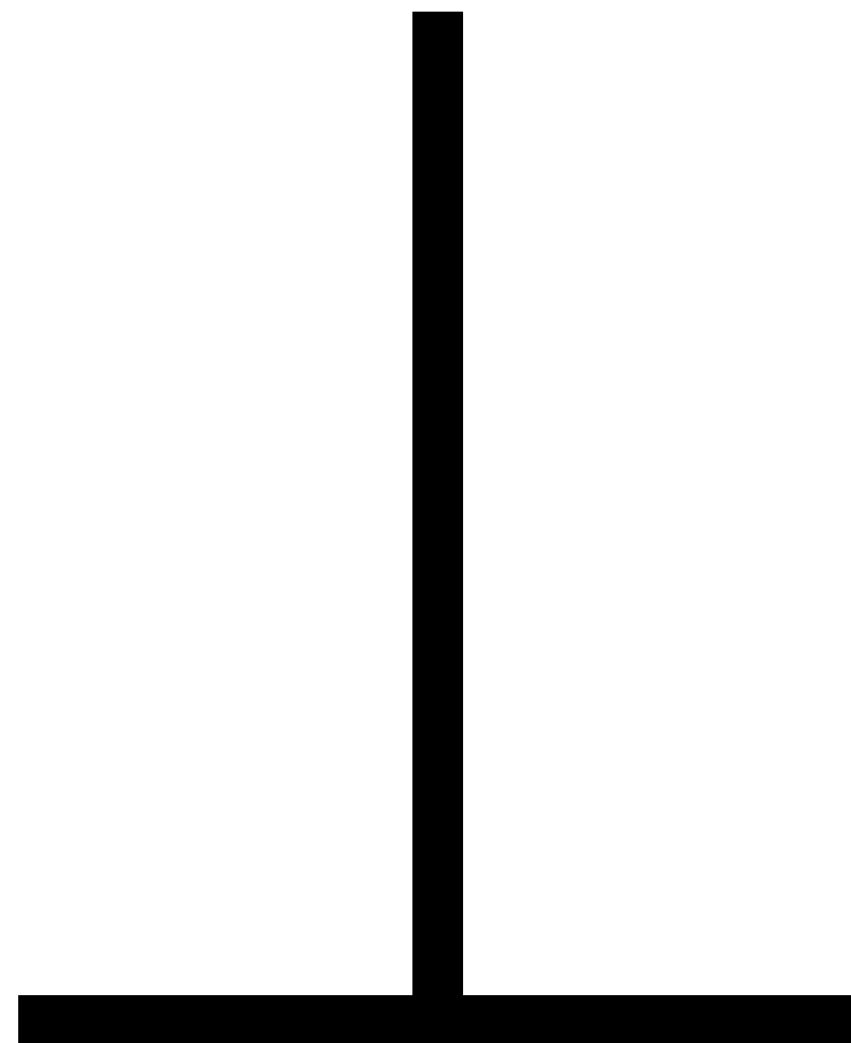
Zielstange

Verschiebe rekursiv  von **Zwischenstange** zur **Zielstange**

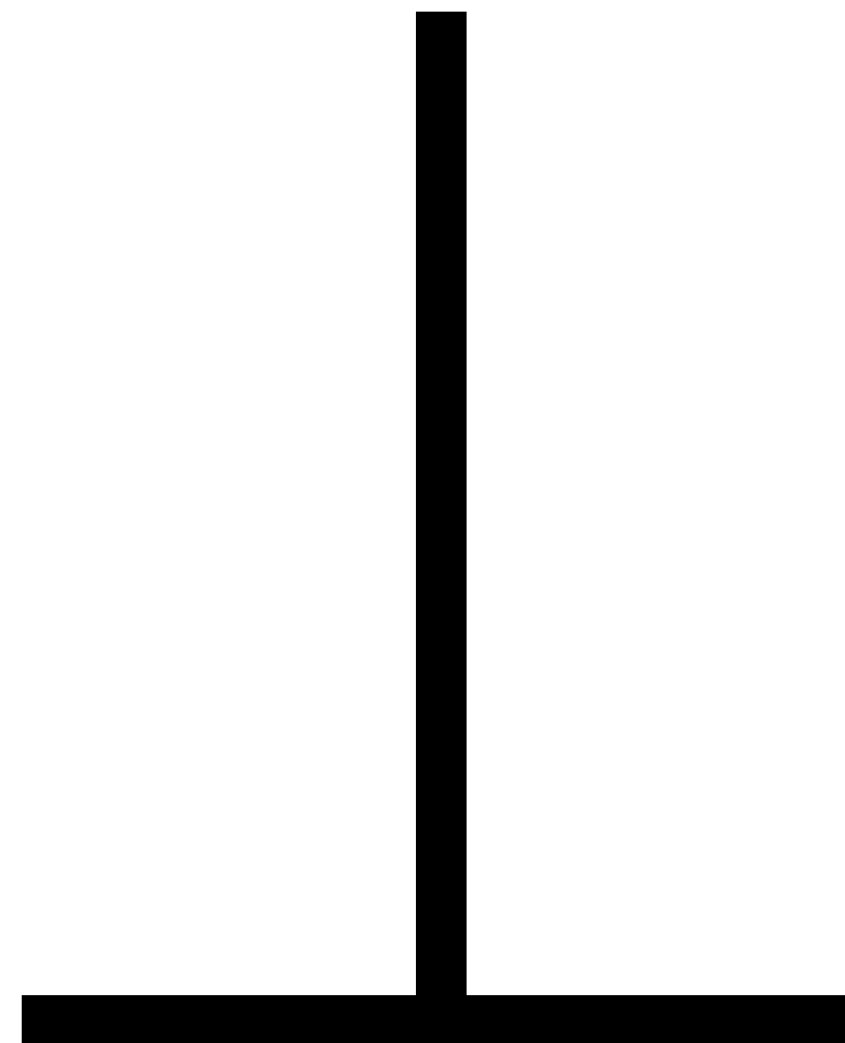
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

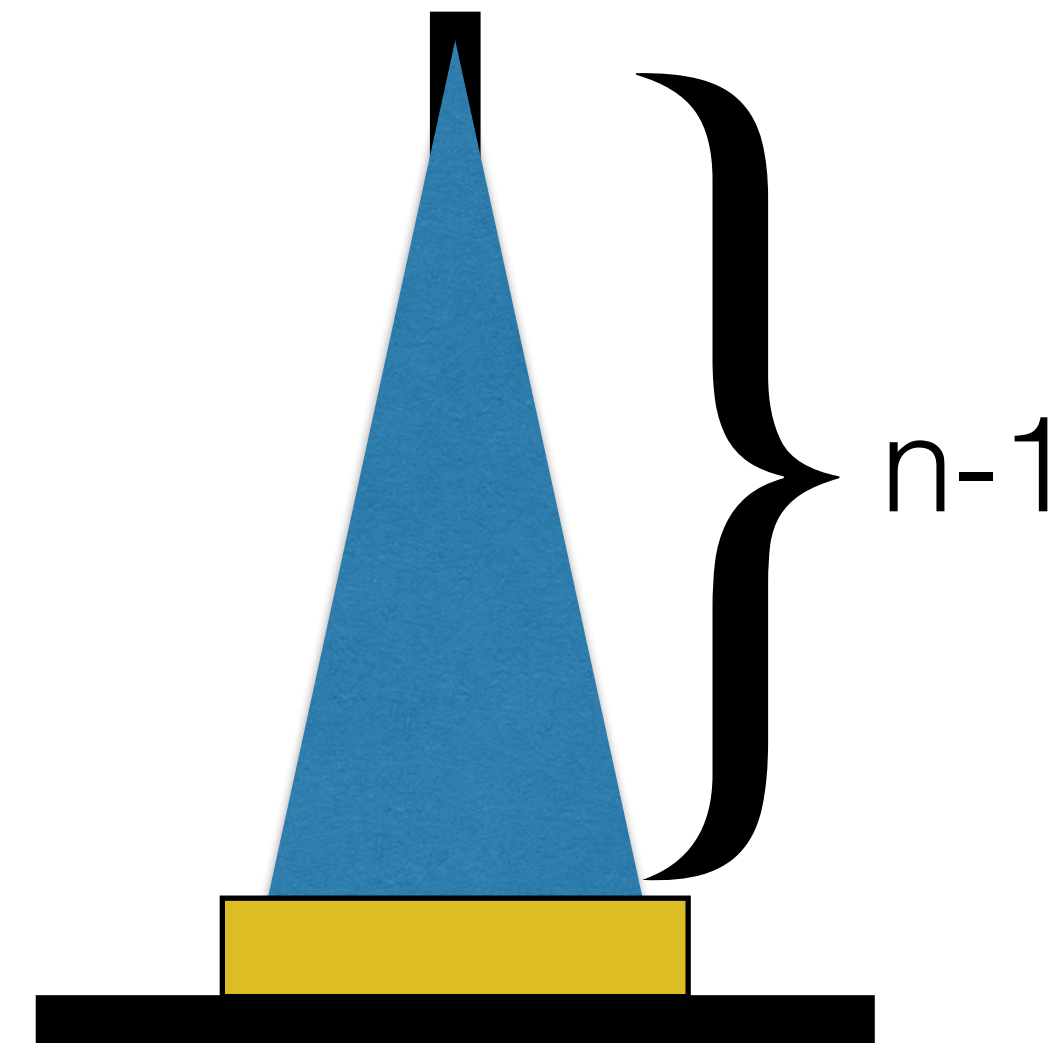
Sei also **n**>1.



Startstange



Zwischenstange



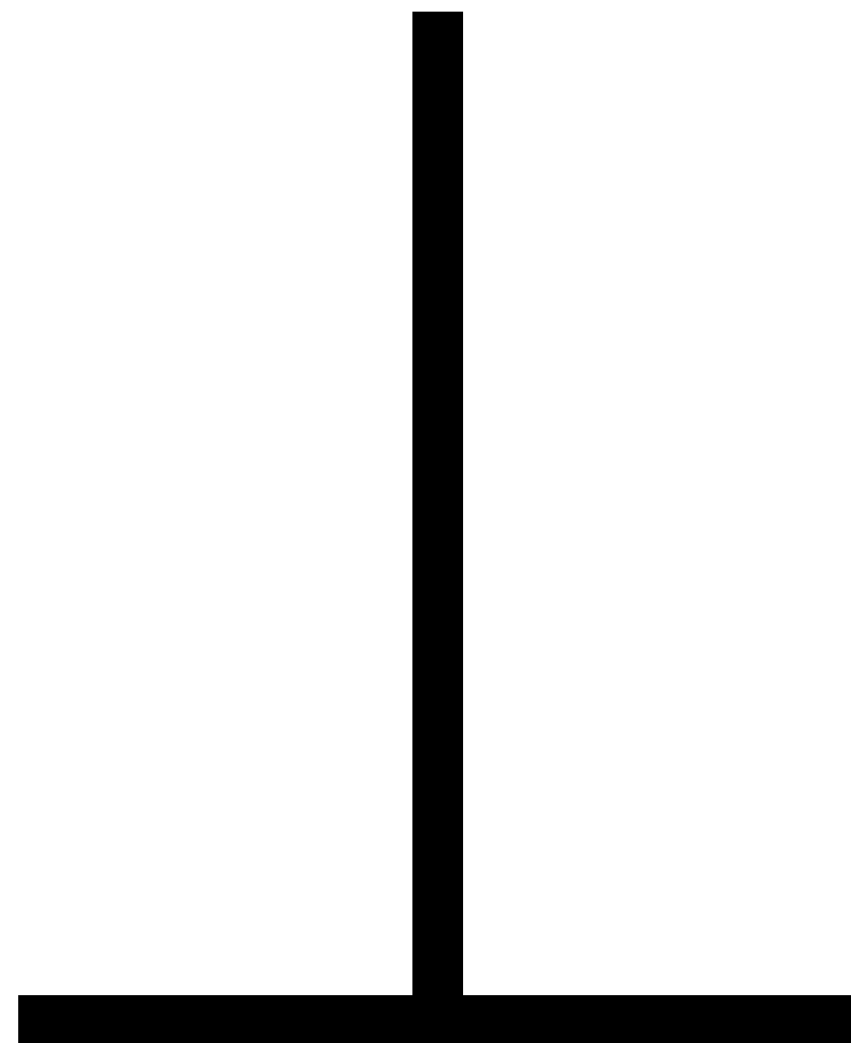
Zielstange

Verschiebe rekursiv  von **Zwischenstange** zur **Zielstange**

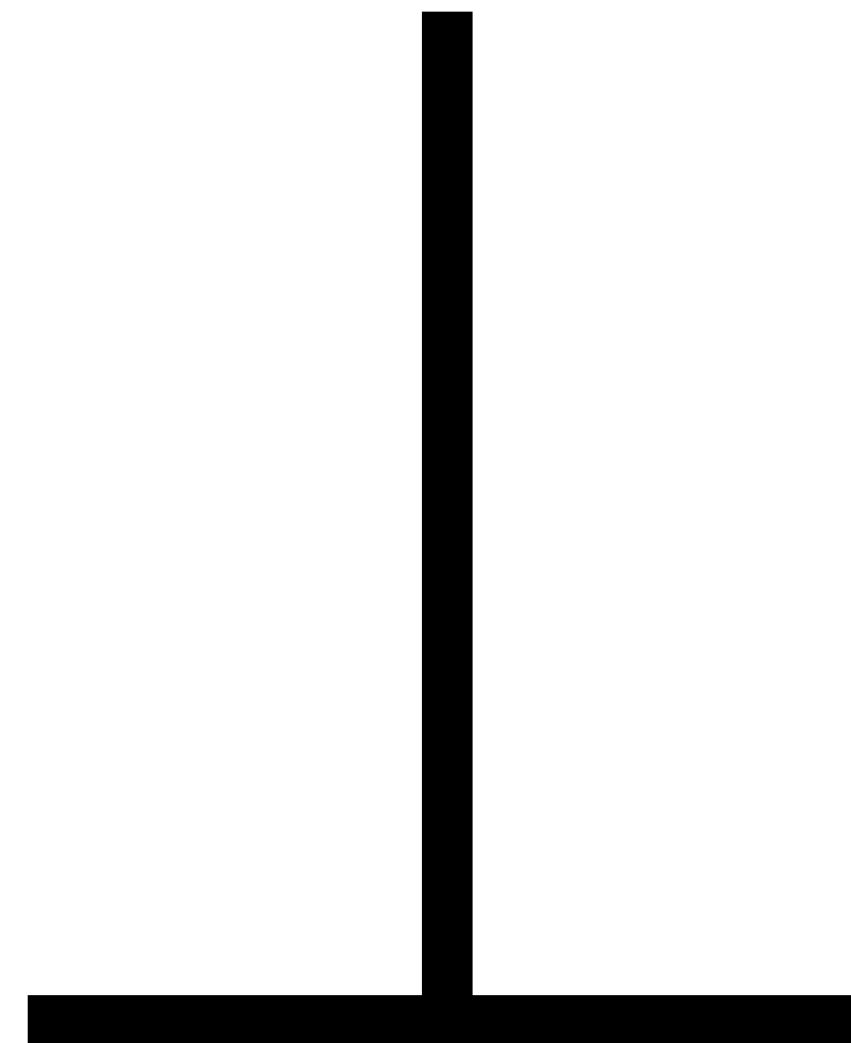
Leichte Verallgemeinerung

Die triviale Instanz **n**=1 haben wir bereits gelöst!

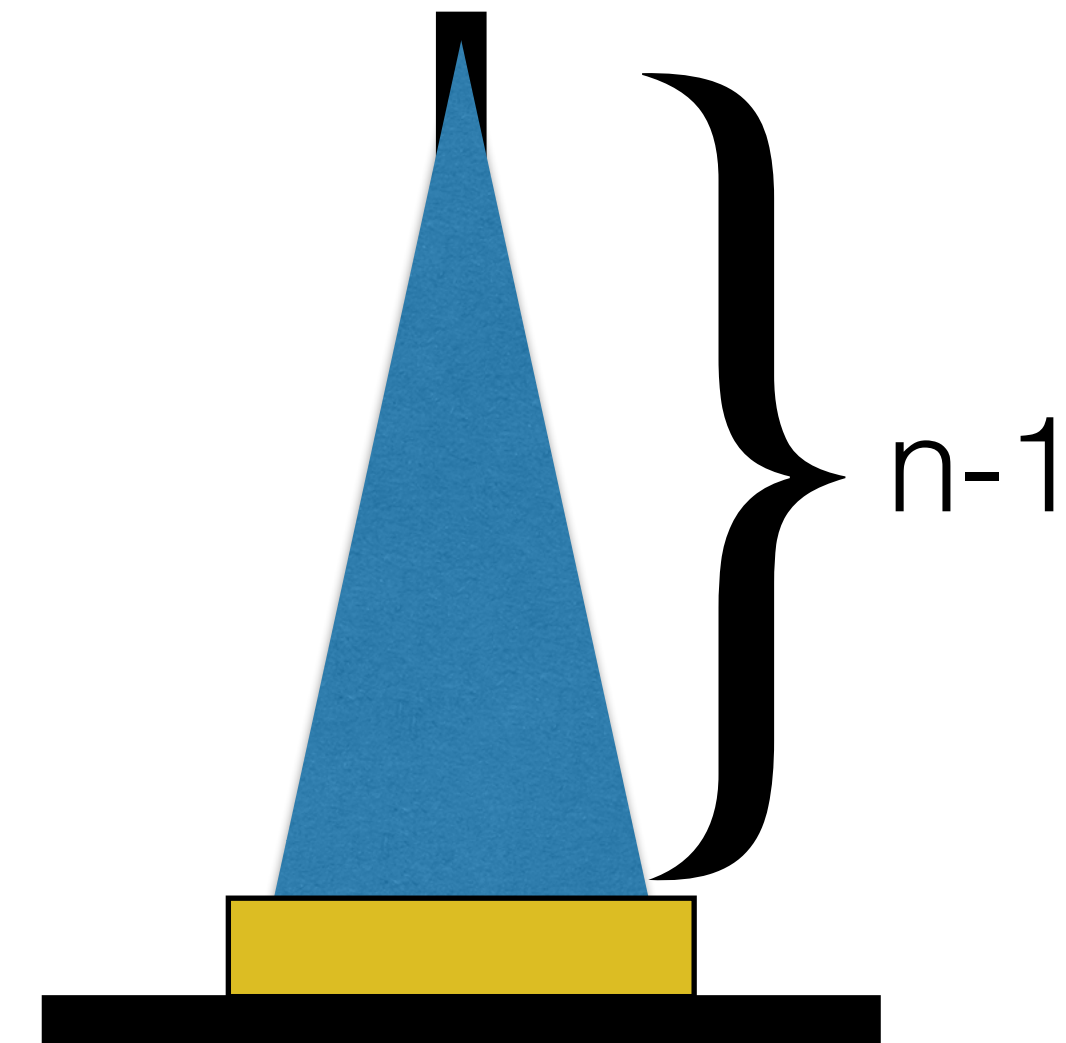
Sei also **n**>1.



Startstange



Zwischenstange



Zielstange

Wir sind fertig!

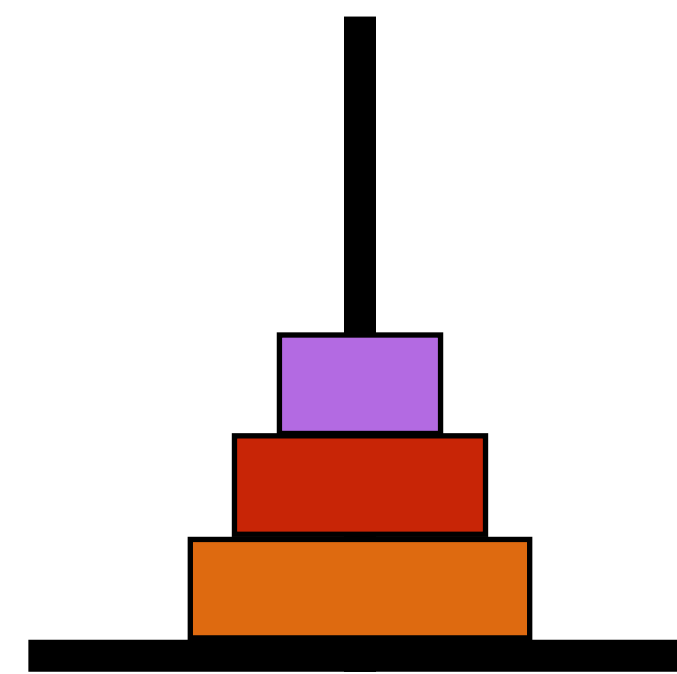
Lösung in Python

```
def Tuerme(n, von, nach, zwischen):  
    if n==1:  
        print(von+ " -> "+ nach)  
    else:  
        Tuerme(n-1, von, zwischen, nach)  
        Tuerme(1, von, nach, zwischen)  
        Tuerme(n-1, zwischen, nach, von)  
  
n=input("Geben Sie die Anzahl der Scheiben an ")  
Tuerme(int(n), "A", "C", "B")
```

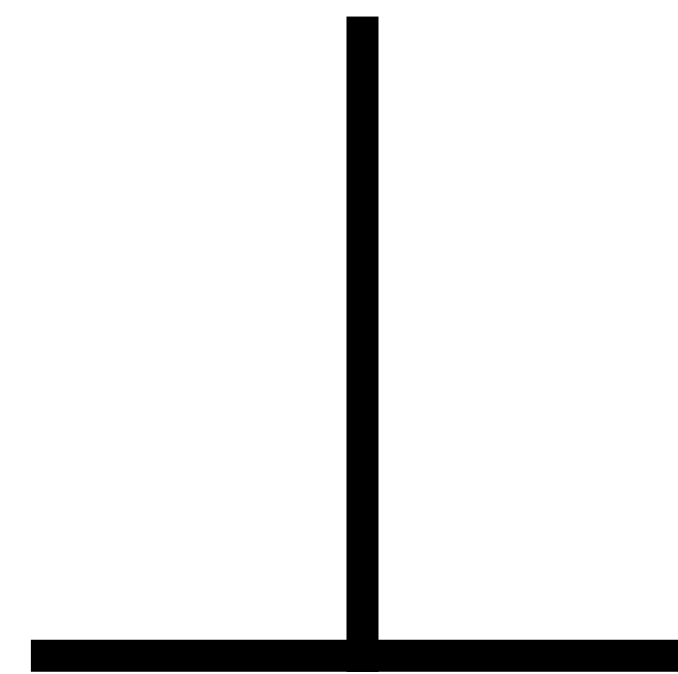
Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

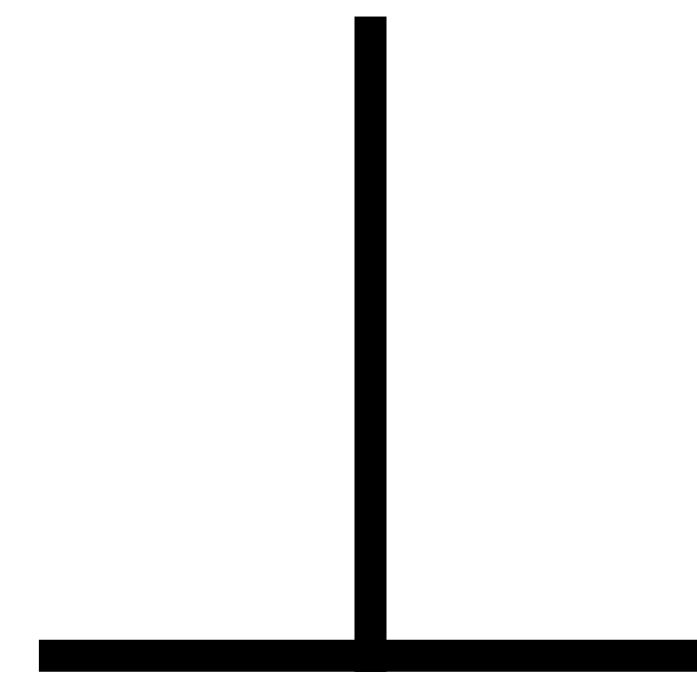
A -> C
A -> B
C -> B
A -> C
B -> A
B -> C
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

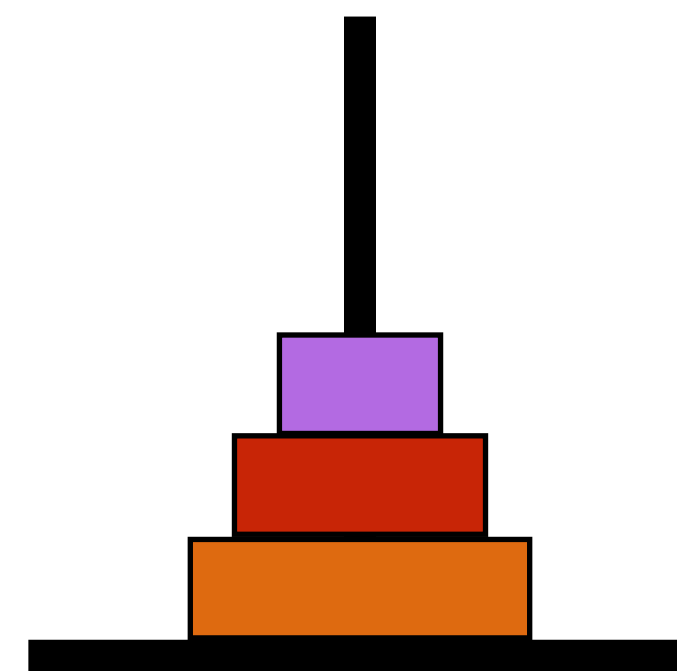
C -> B

A -> C

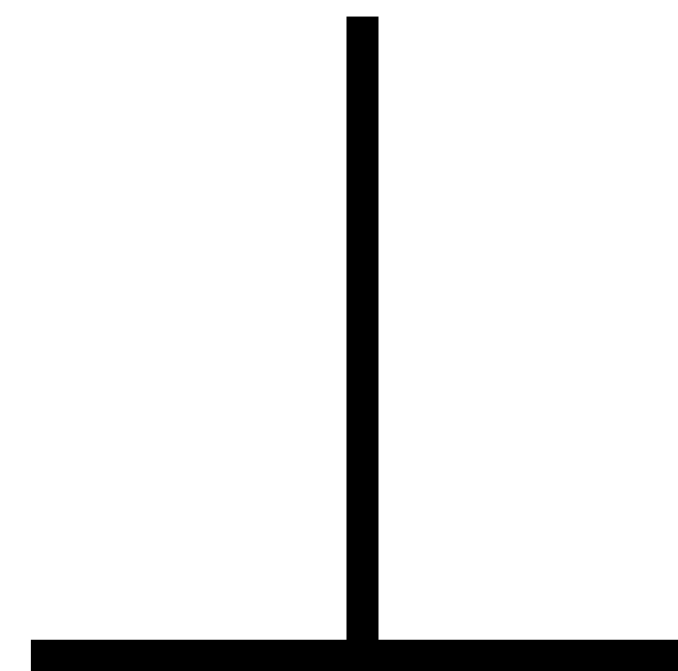
B -> A

B -> C

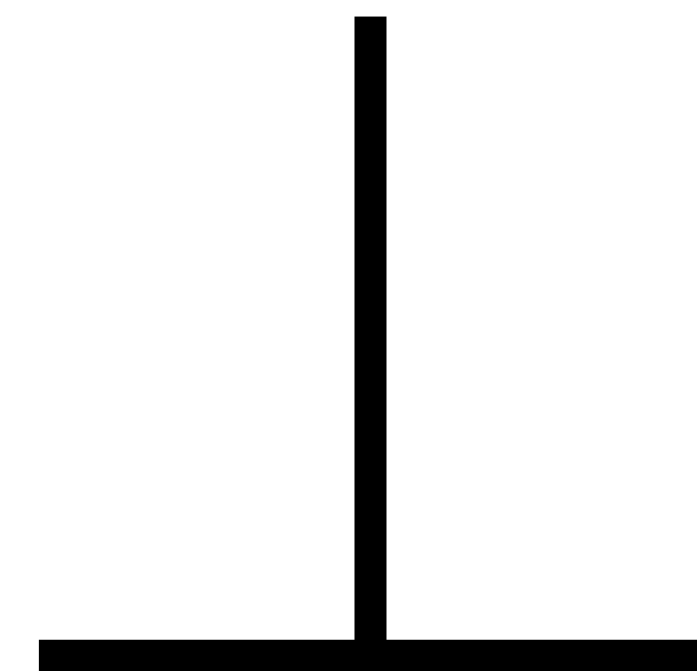
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

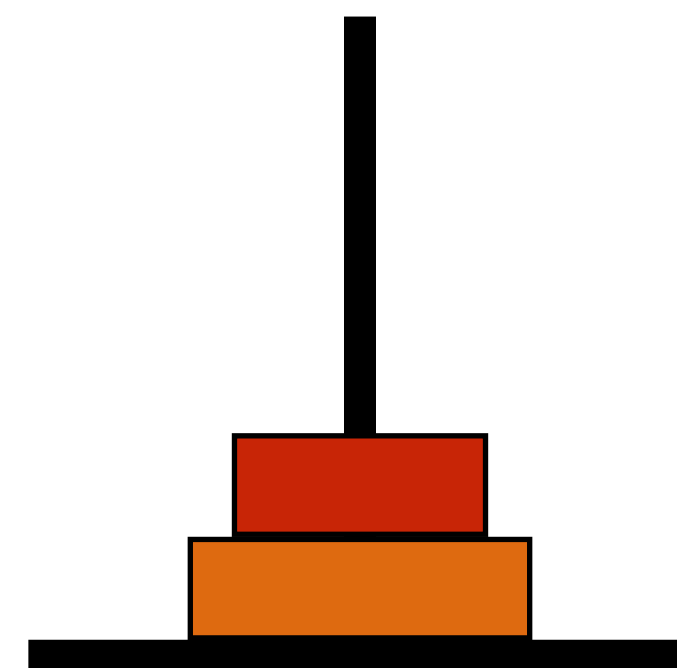
C -> B

A -> C

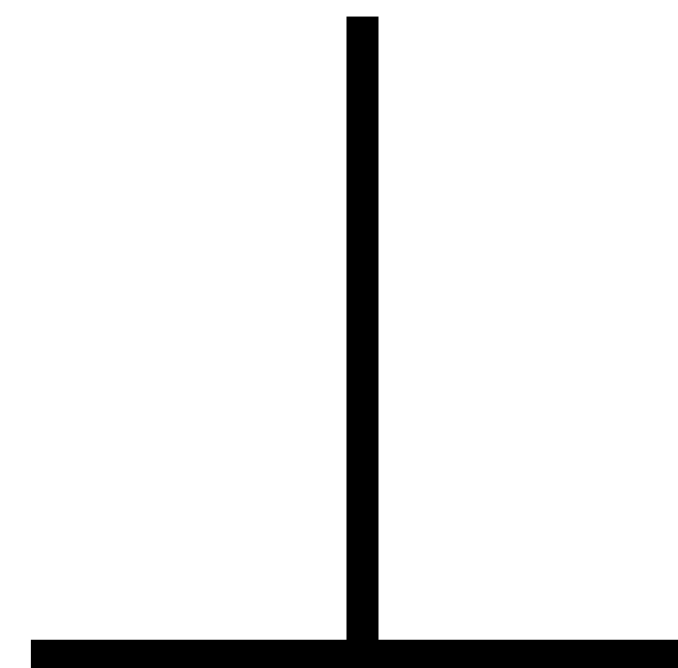
B -> A

B -> C

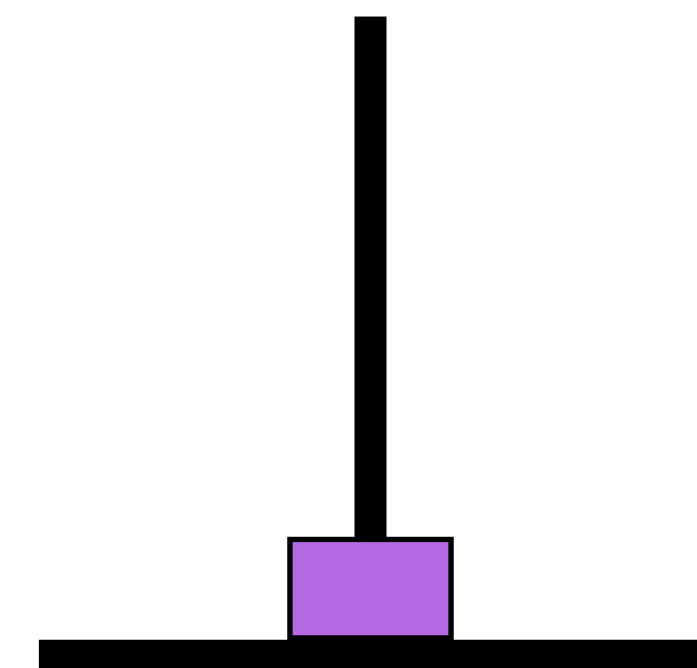
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

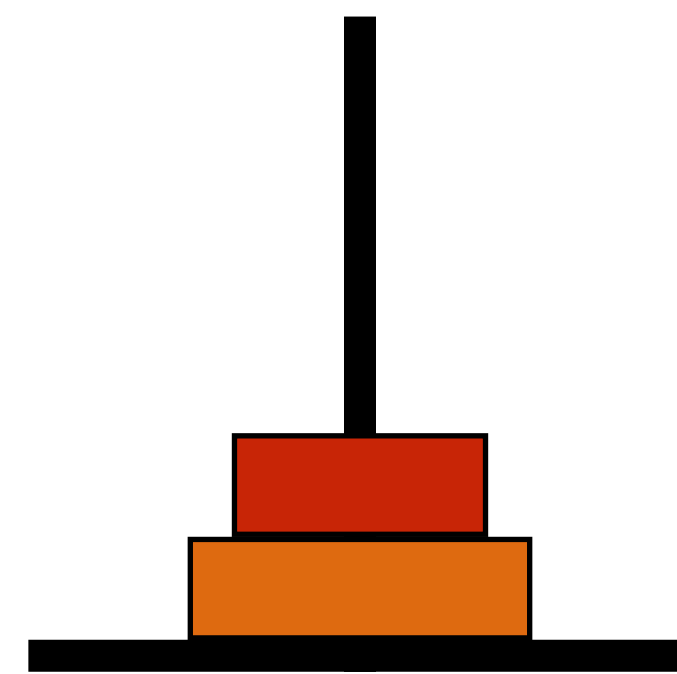
C -> B

A -> C

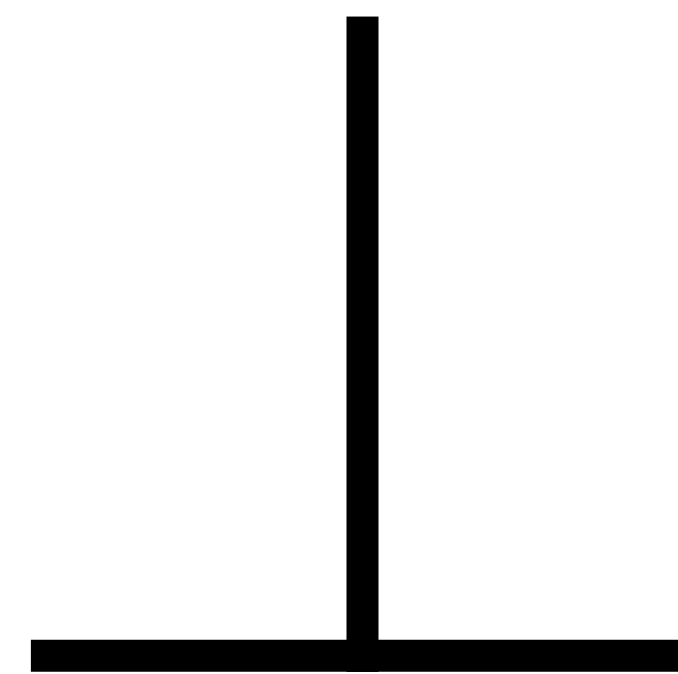
B -> A

B -> C

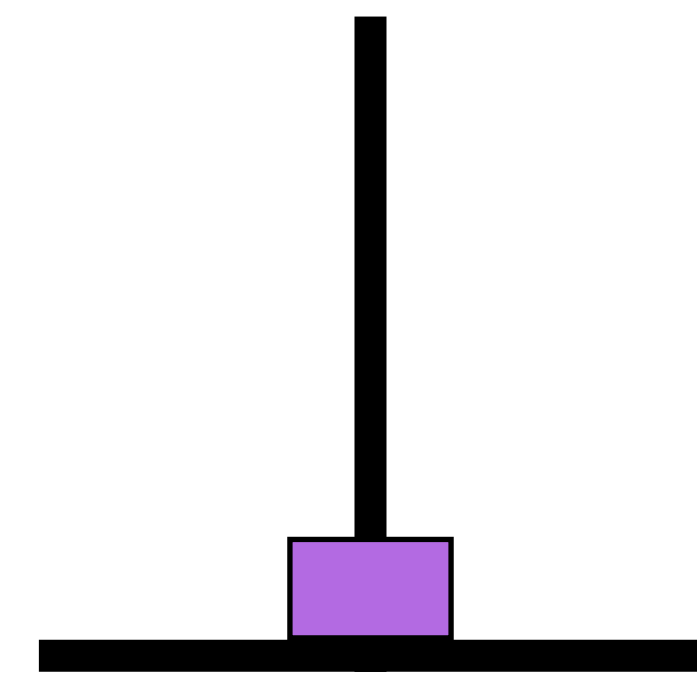
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

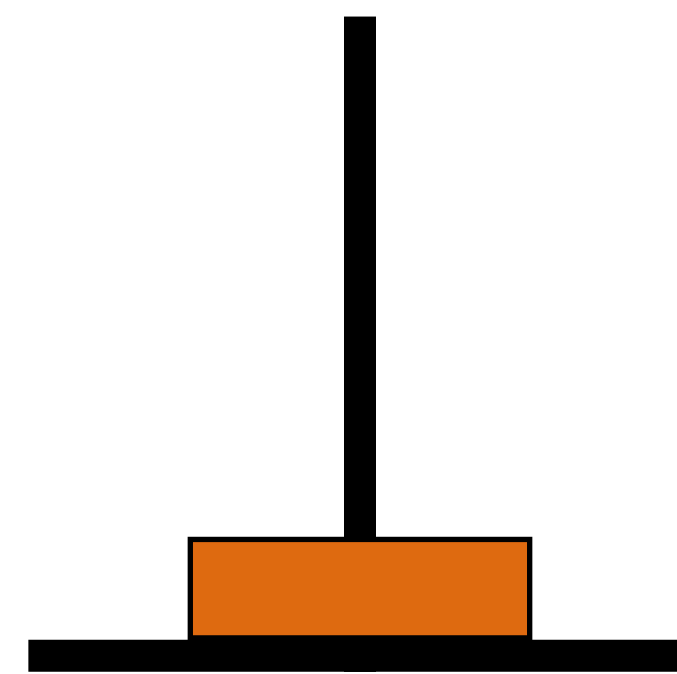
C -> B

A -> C

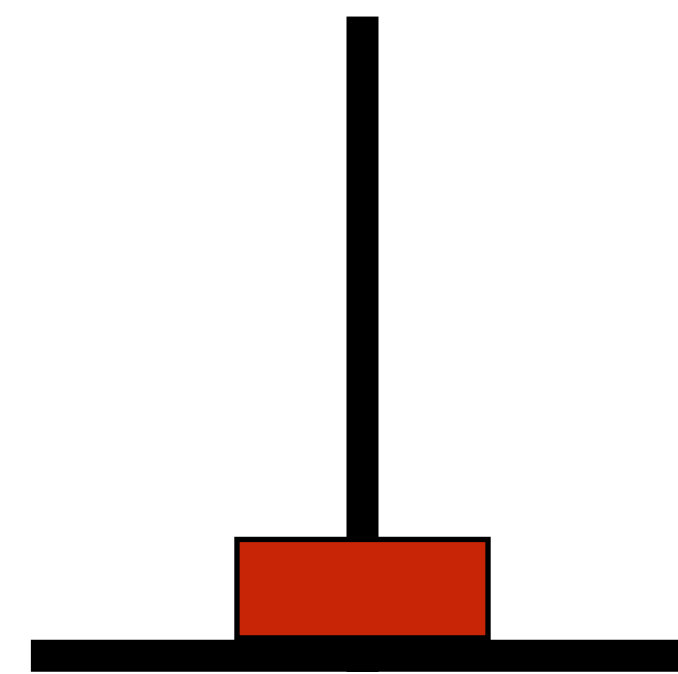
B -> A

B -> C

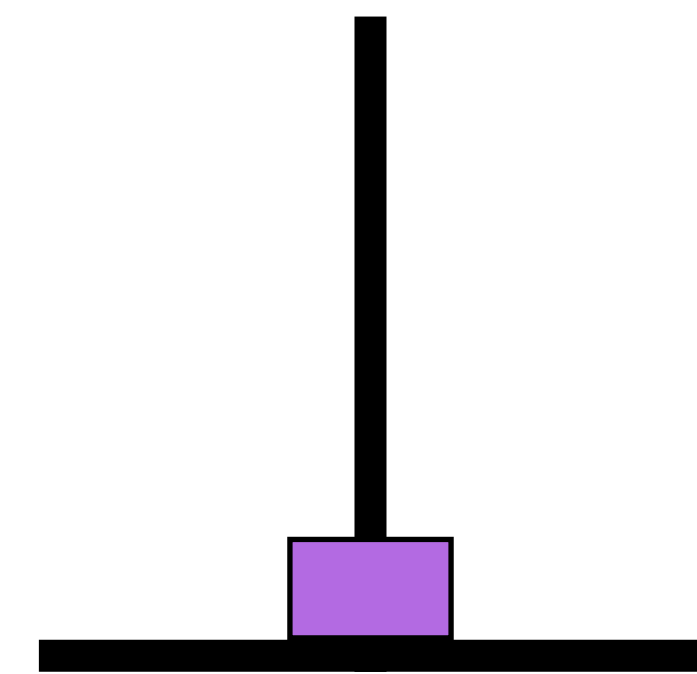
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

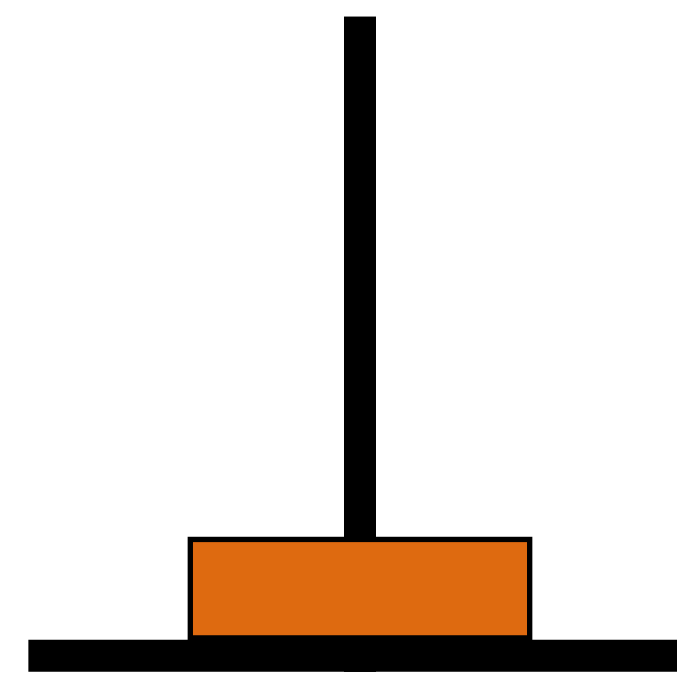
C -> B

A -> C

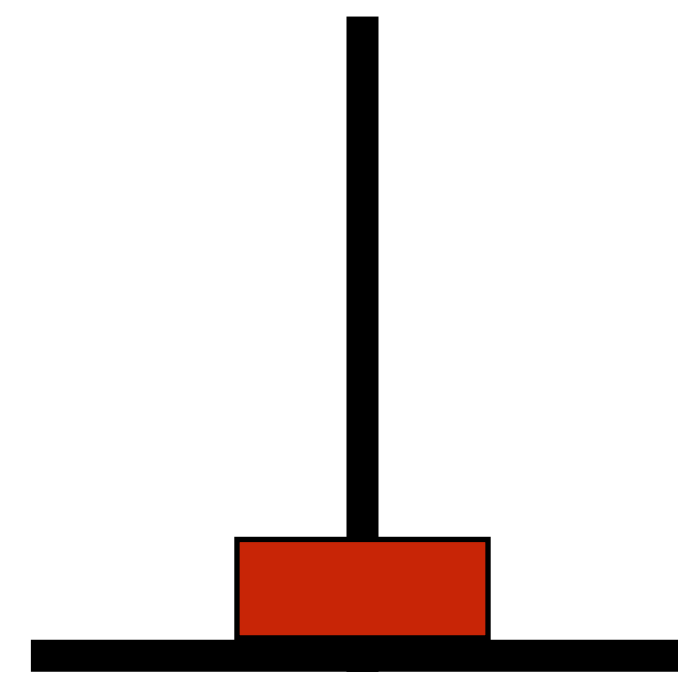
B -> A

B -> C

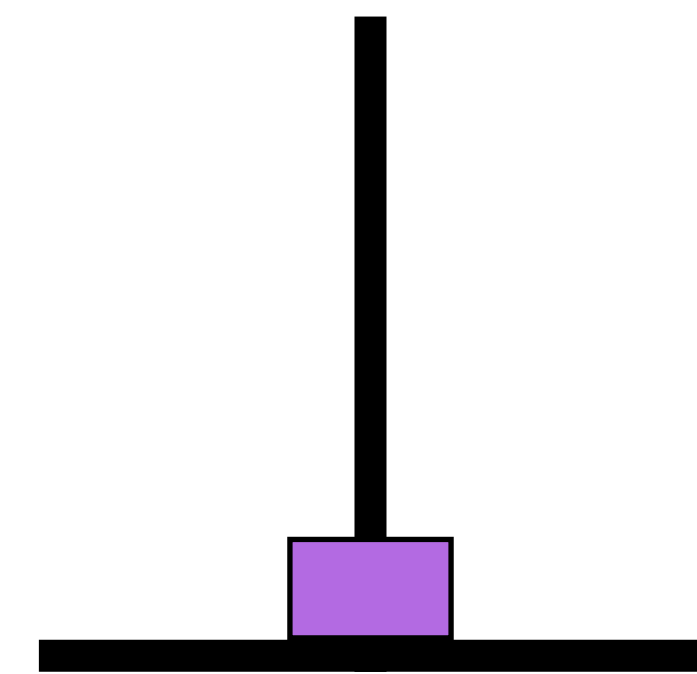
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

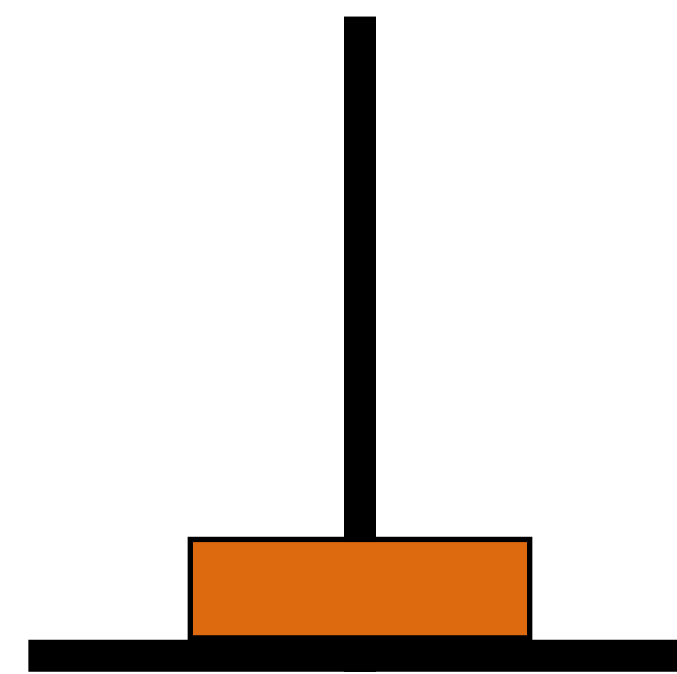
C -> B

A -> C

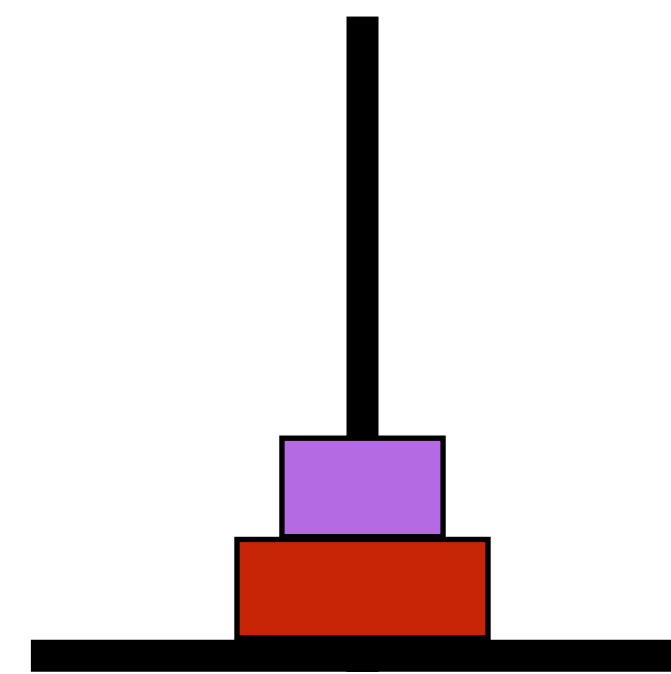
B -> A

B -> C

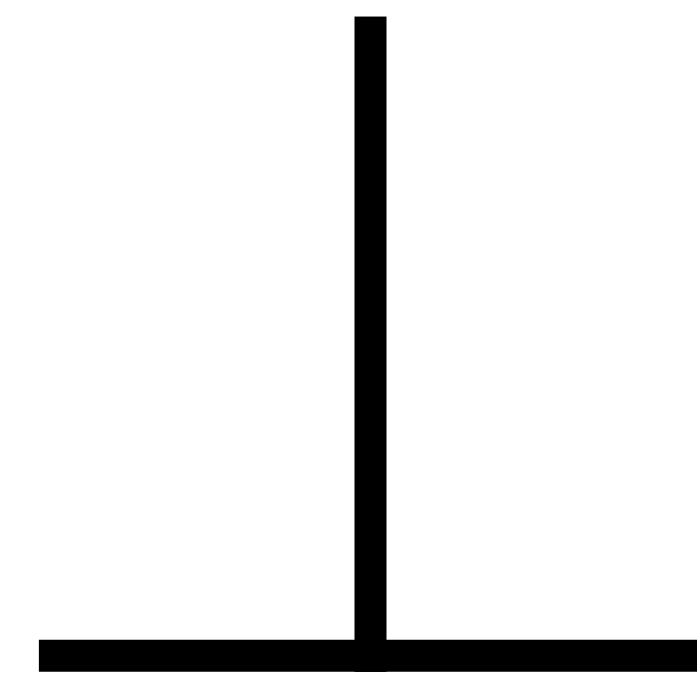
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

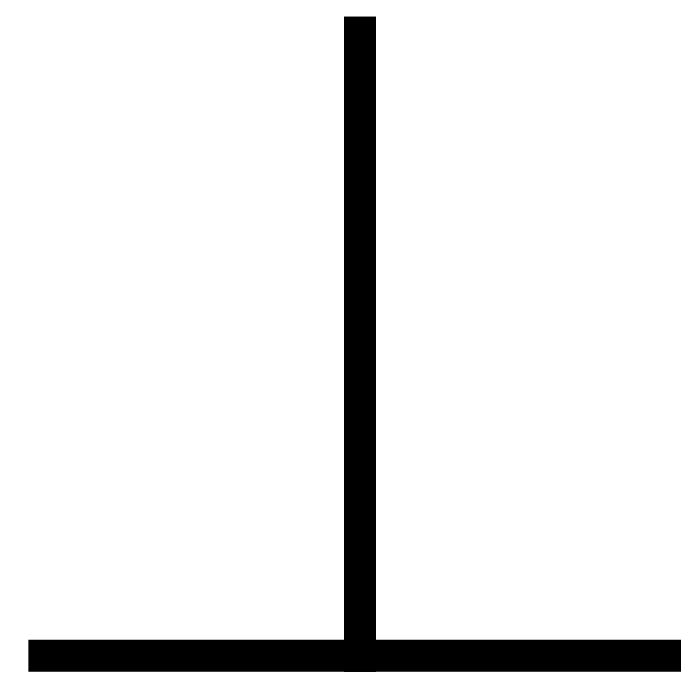
C -> B

A -> C

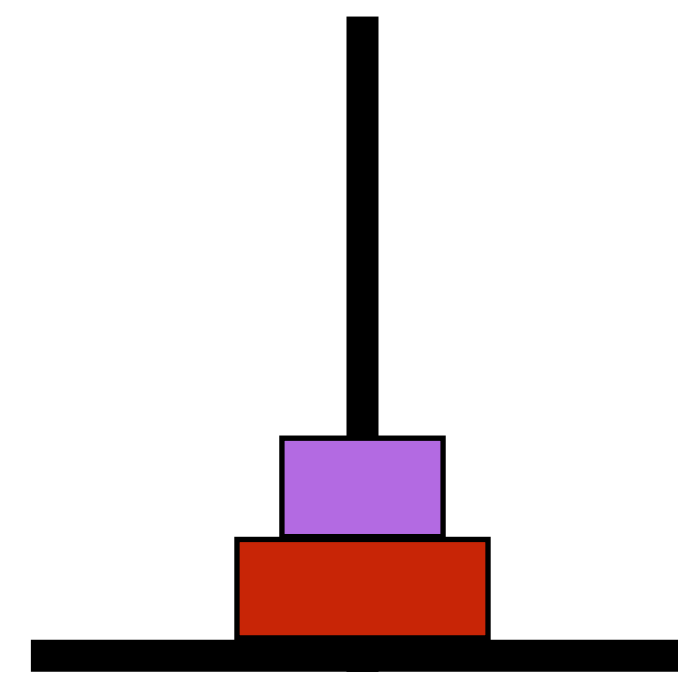
B -> A

B -> C

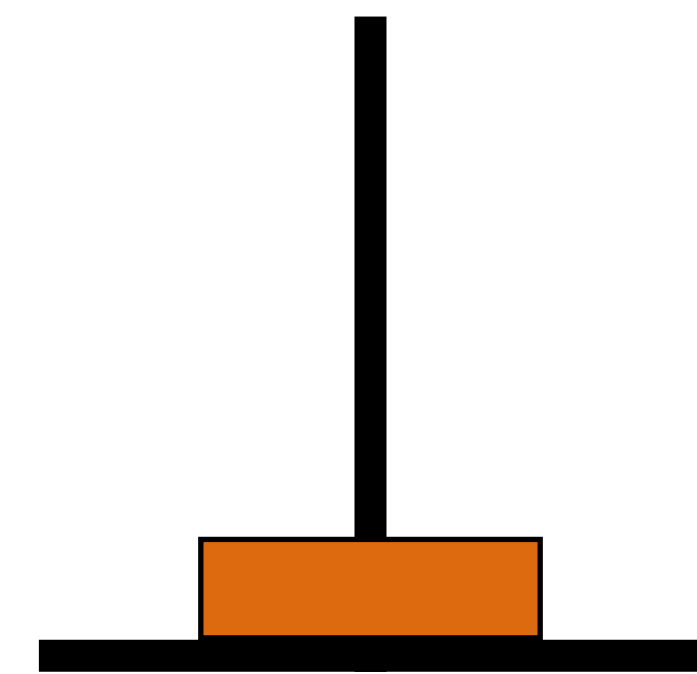
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

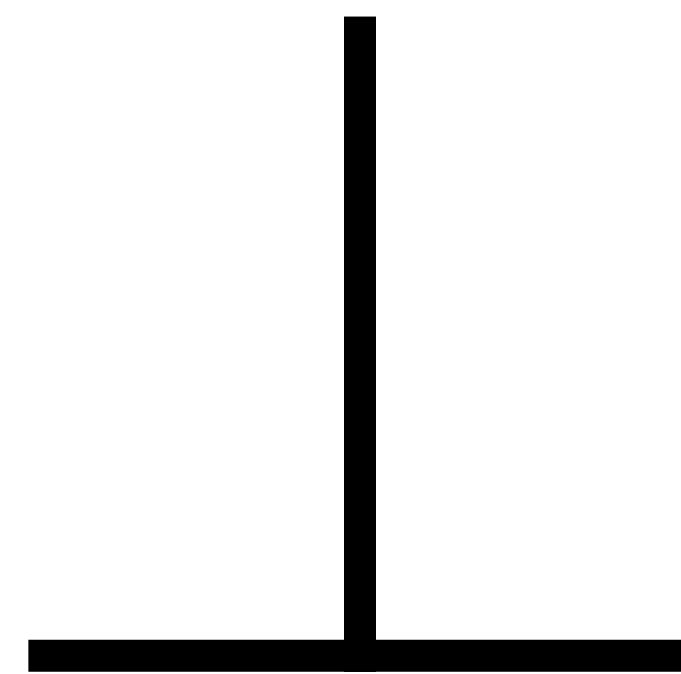
C -> B

A -> C

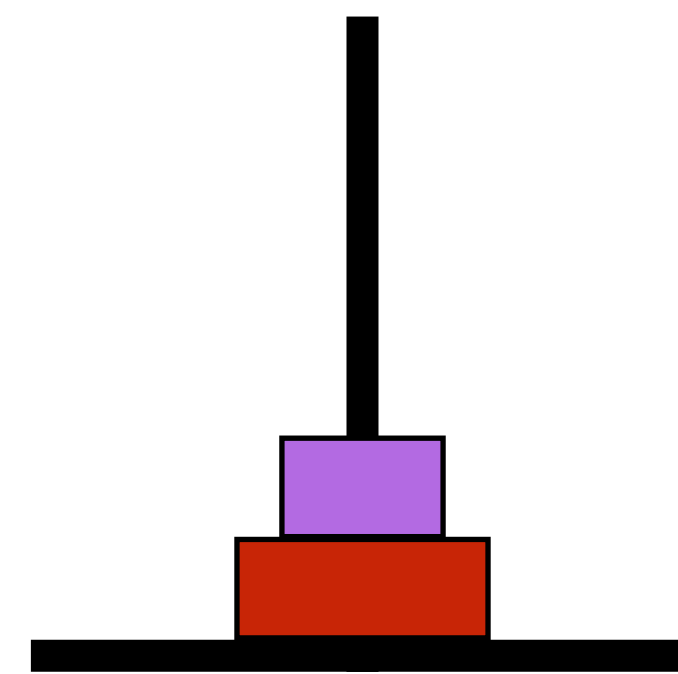
B -> A

B -> C

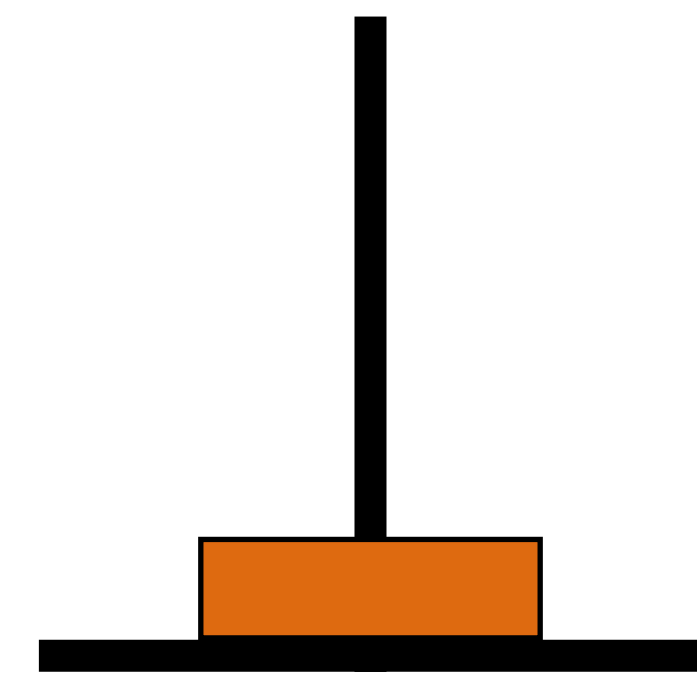
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

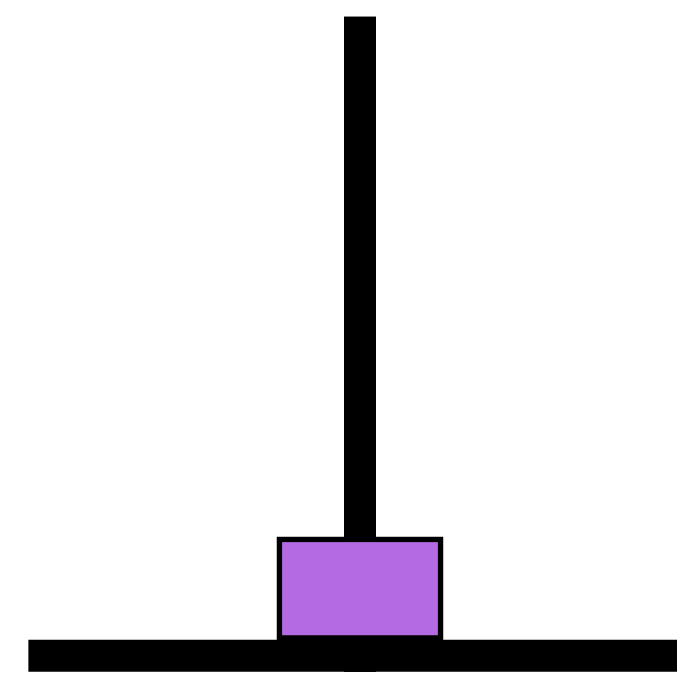
C -> B

A -> C

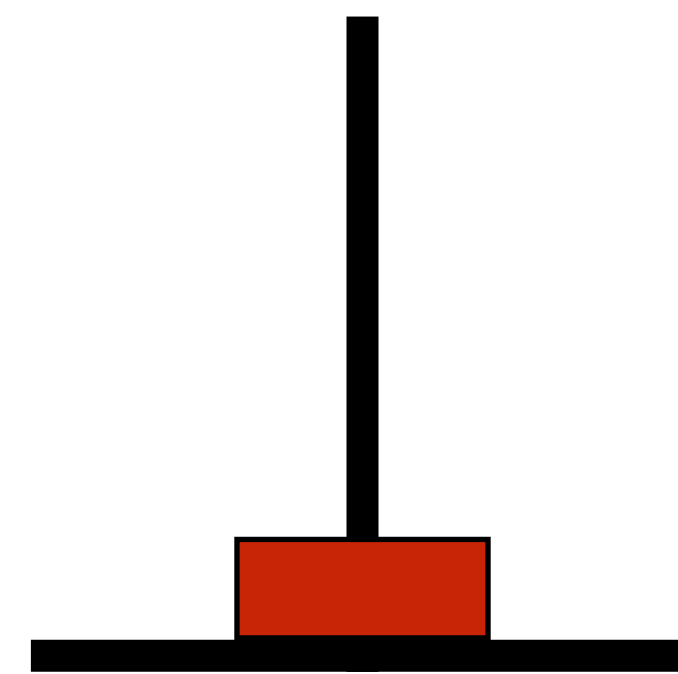
B -> A

B -> C

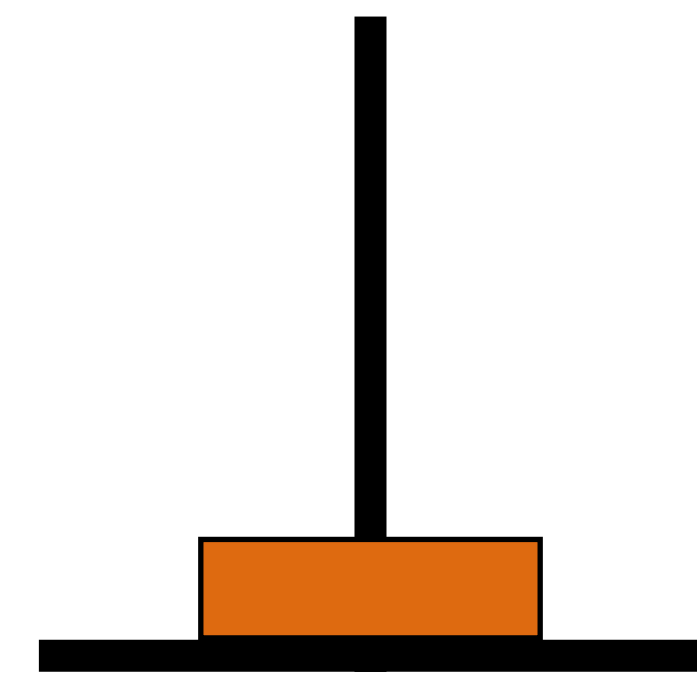
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

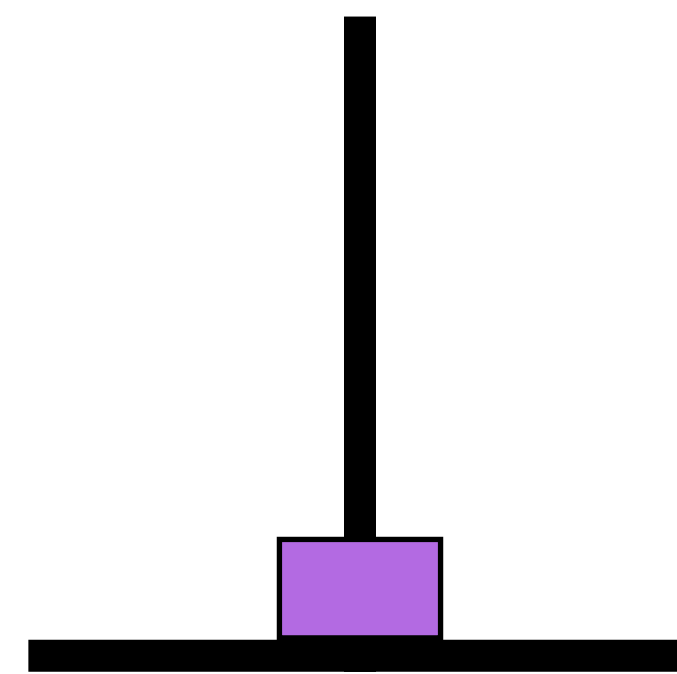
C -> B

A -> C

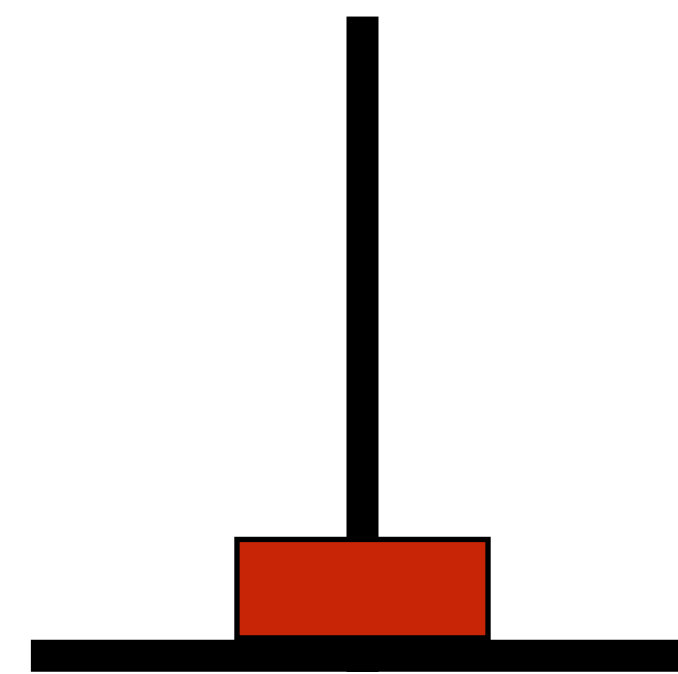
B -> A

B -> C

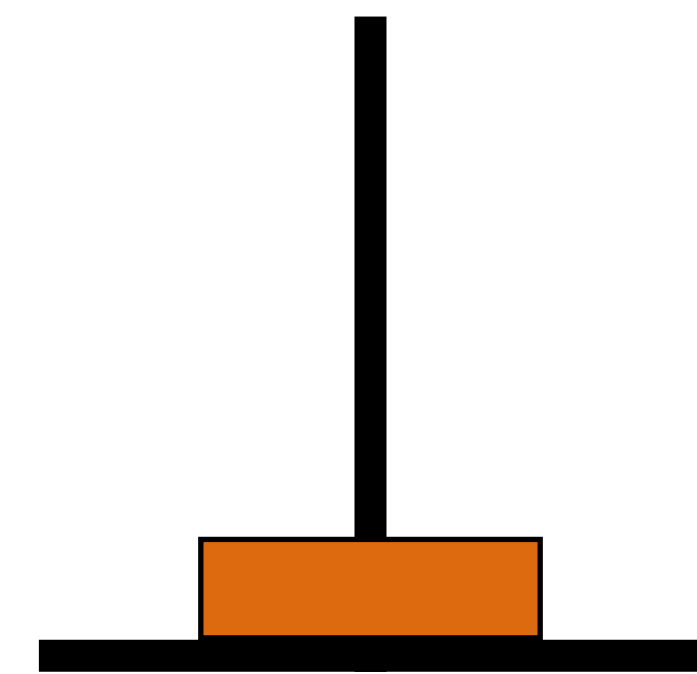
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

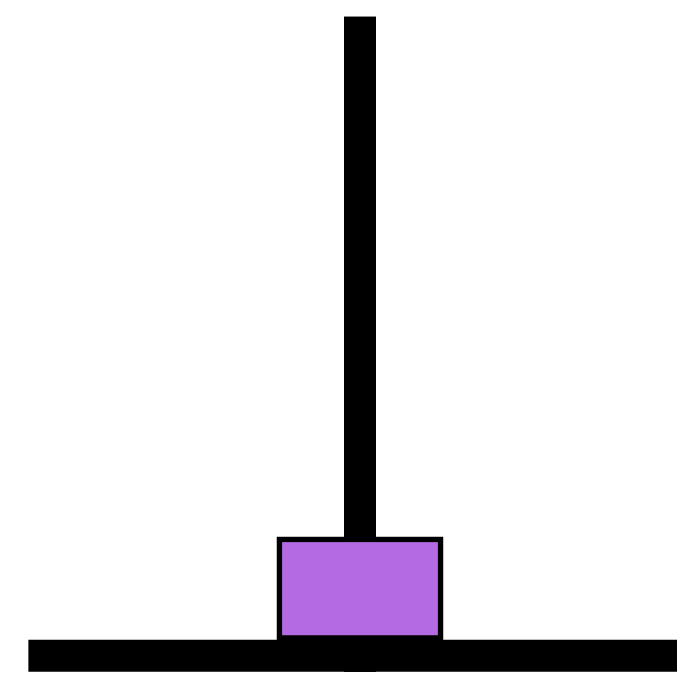
C -> B

A -> C

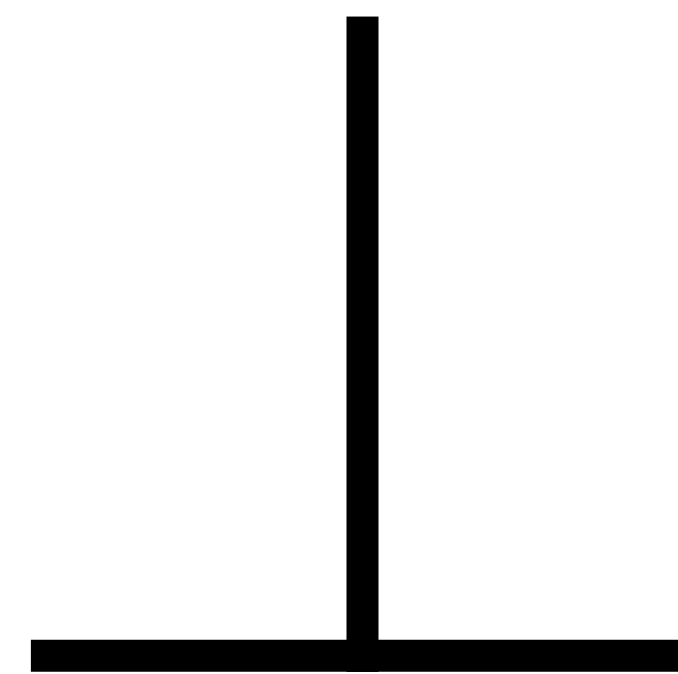
B -> A

B -> C

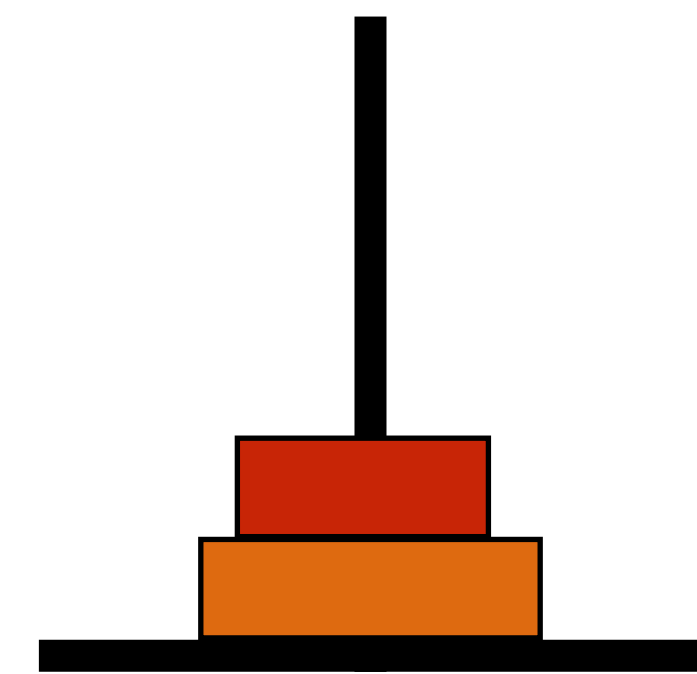
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

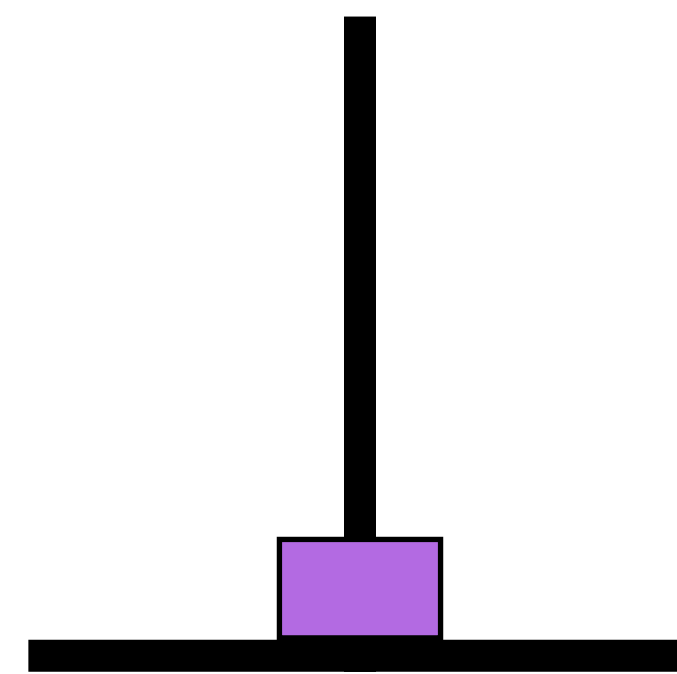
C -> B

A -> C

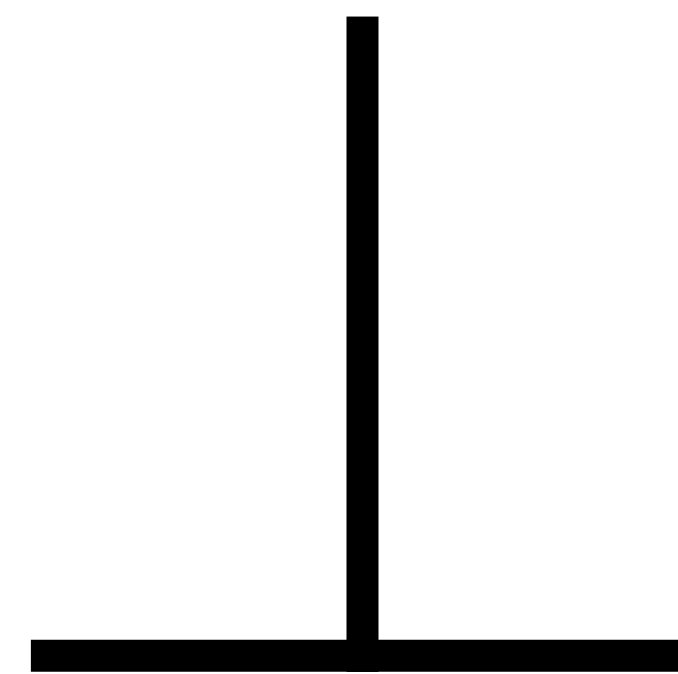
B -> A

B -> C

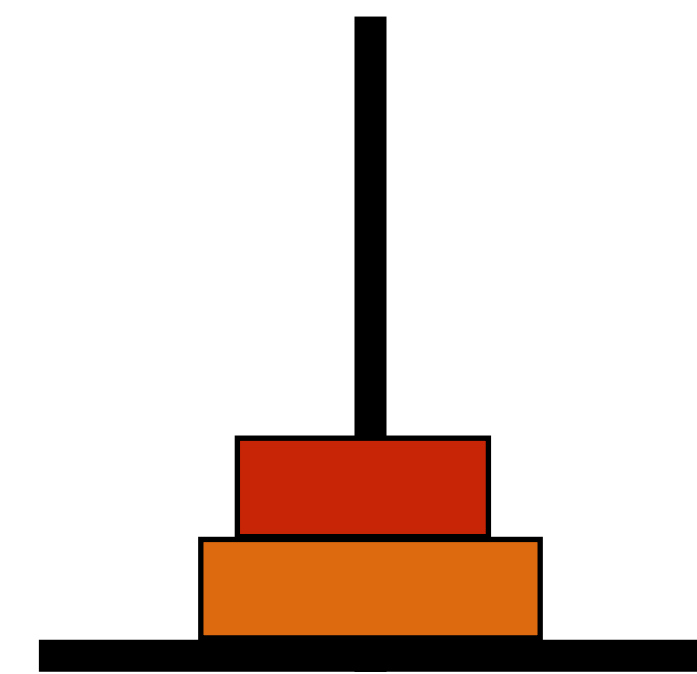
A -> C



Stange A



Stange B



Stange C

Ausgaben von TuermevonHanoi

Geben Sie die Anzahl der Scheiben an: 3

A -> C

A -> B

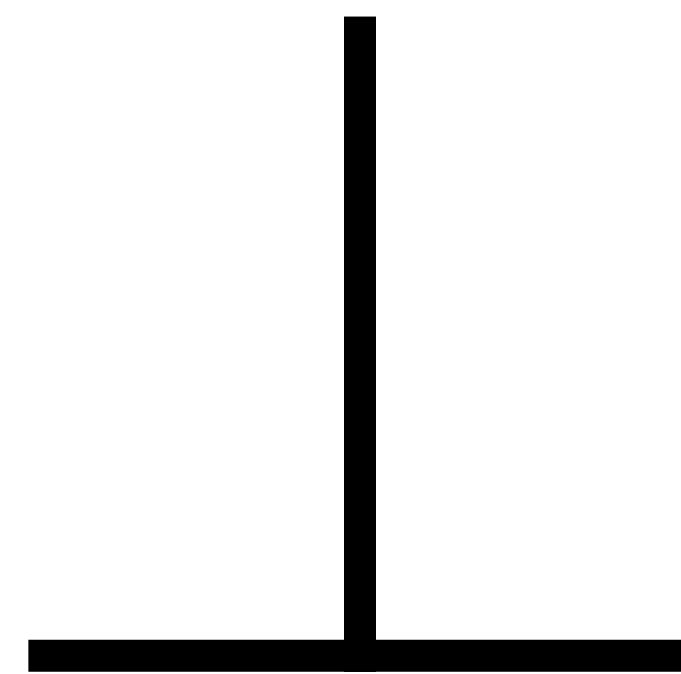
C -> B

A -> C

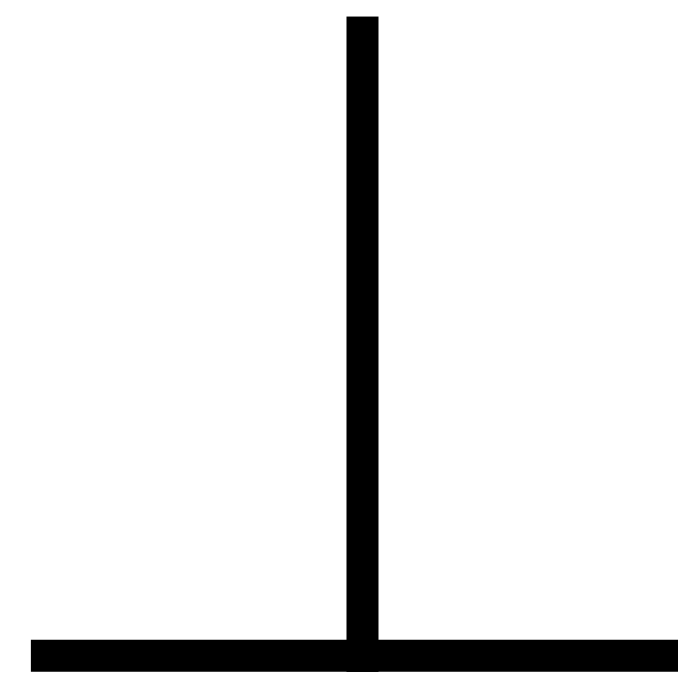
B -> A

B -> C

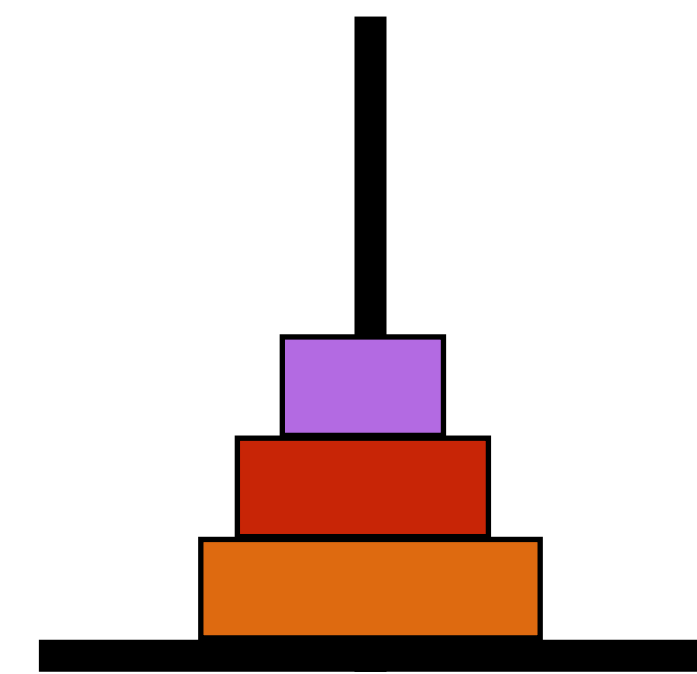
A -> C



Stange A

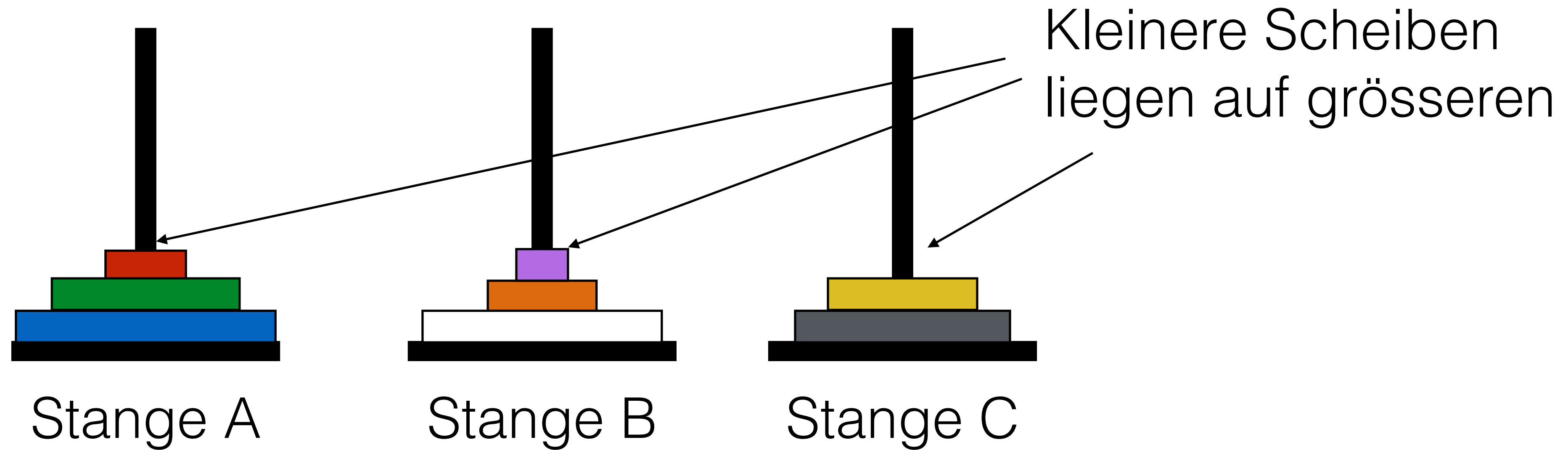


Stange B



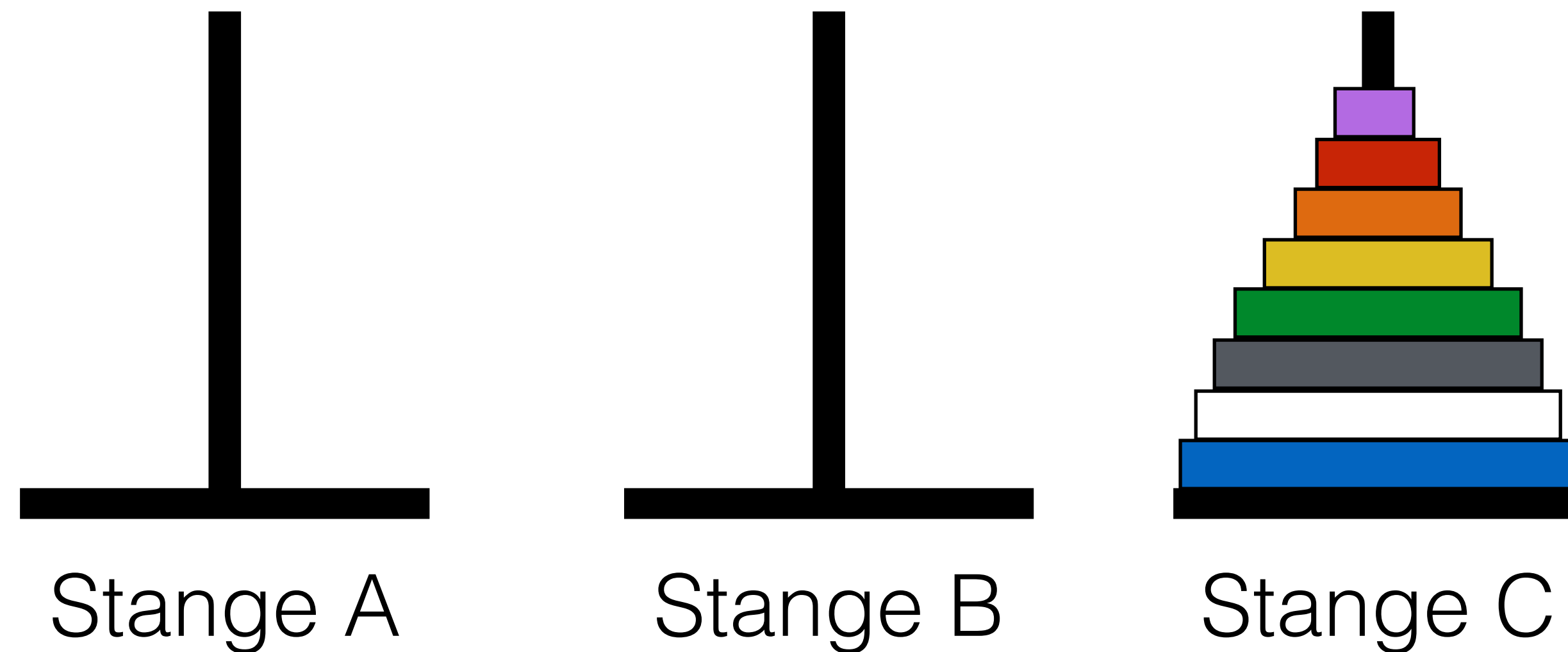
Stange C

Übungsaufgabe



Frage:

Können wir die
Scheiben auf
Stange C
bewegen?



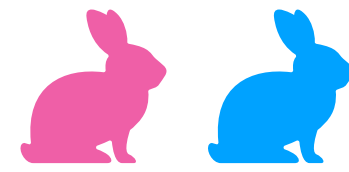
Rekursion mit mehreren Basisfällen



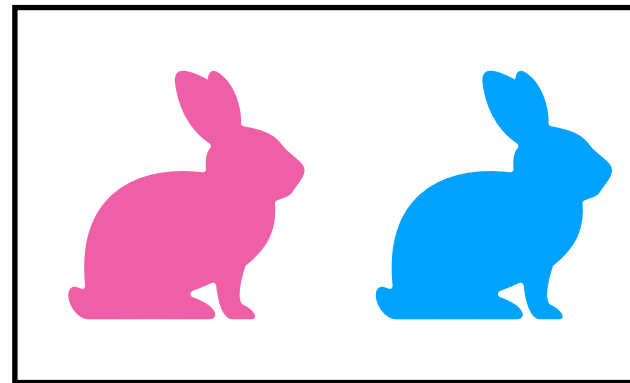
- Leonardo von Pisa (aka Fibonacci) hat folgendes Rätsel gestellt
 - Ein Paar neugeborener Hasen wird (ein Weibchen, ein Männchen) ins Gehege getan
 - Hasen sind nach einem Monat geschlechtsreif
 - Sie haben eine Trächtigkeitsdauer von einem Monat
 - Wir nehmen an, dass Hasen niemals (aus)sterben und jedes Weibchen (ab dem 2. Monat) monatlich zwei Hasen (ein Weibchen und ein Männchen) wirft.

Frage: Wieviele Hasen gibt es am Ende des Jahres?

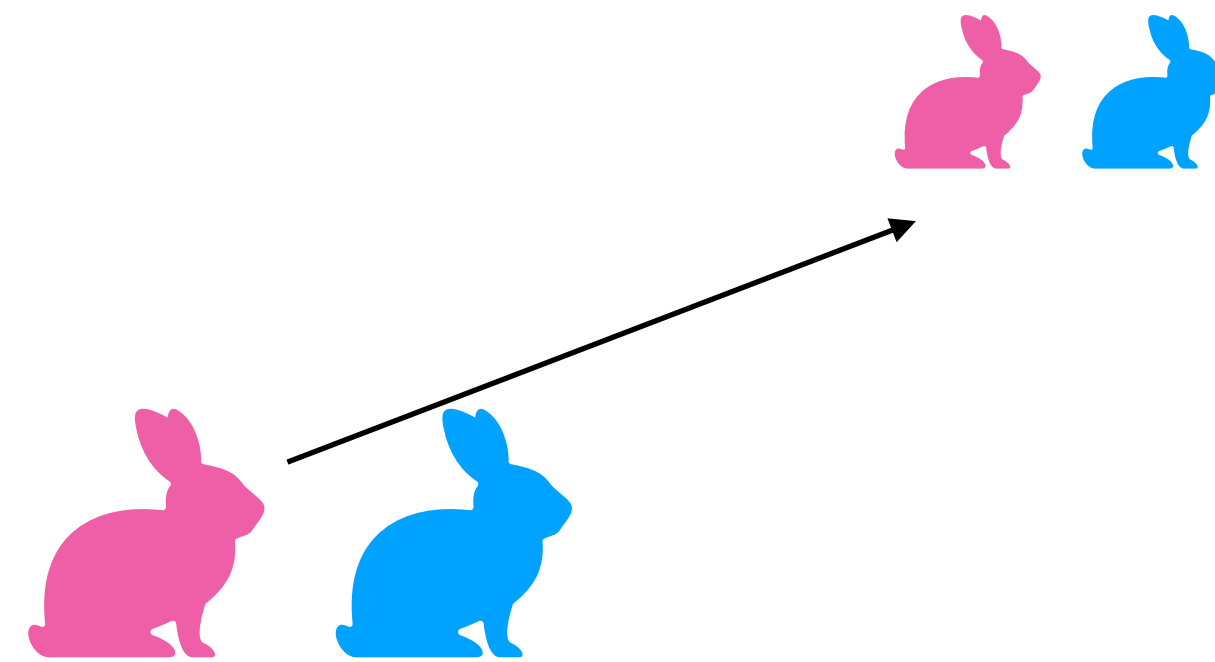
Vermehrung von Hasen



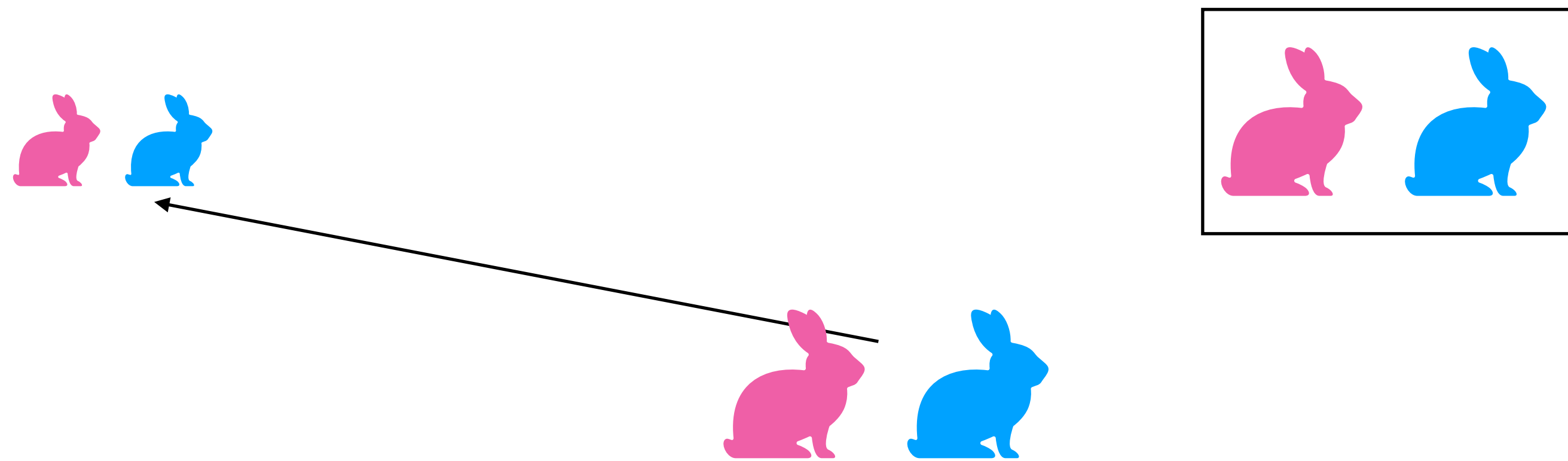
Vermehrung von Hasen



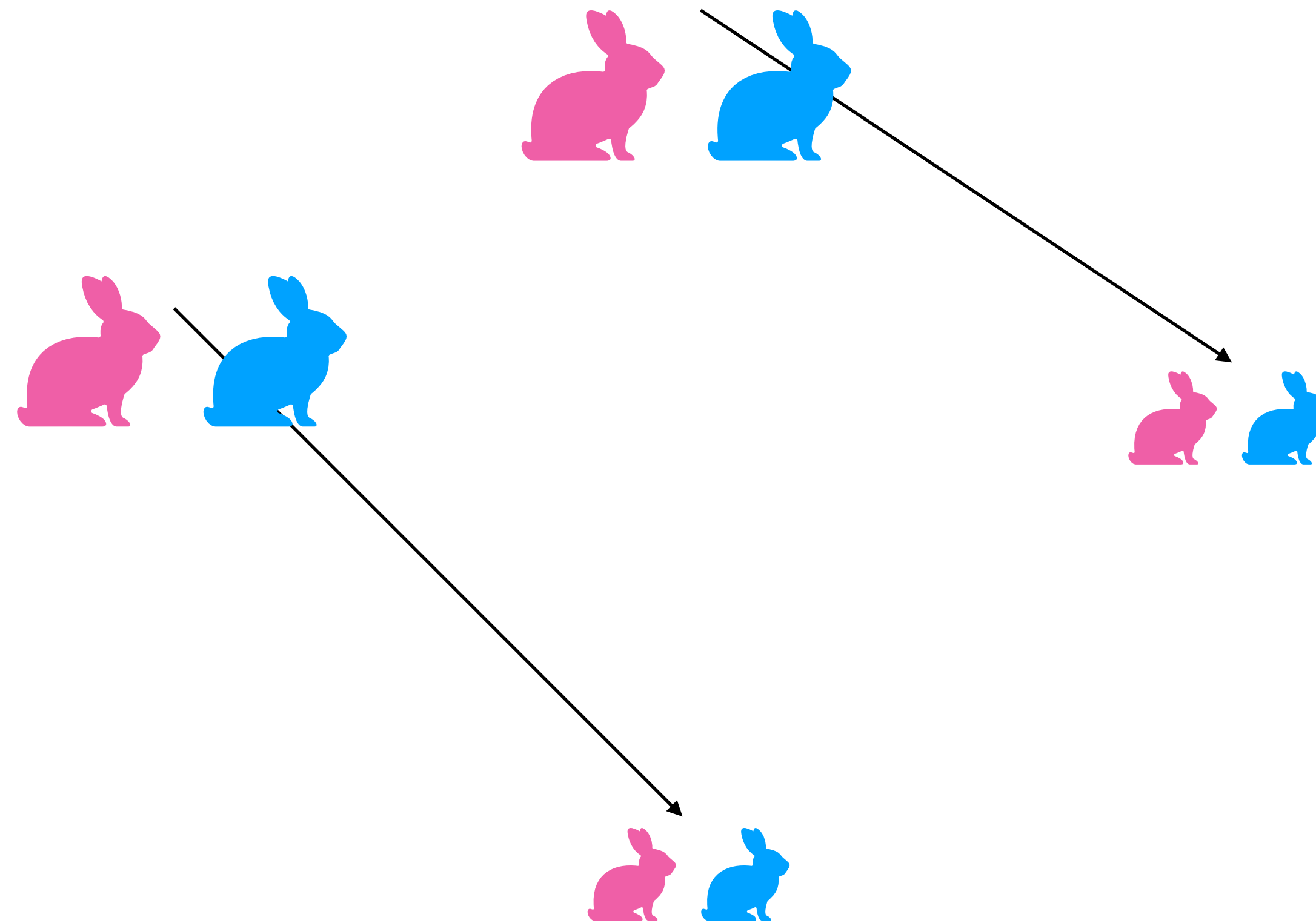
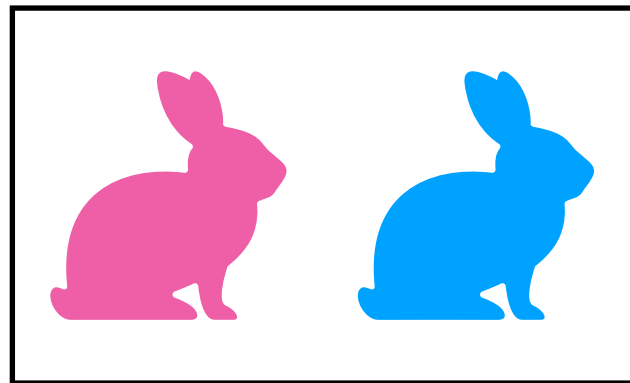
Vermehrung von Hasen



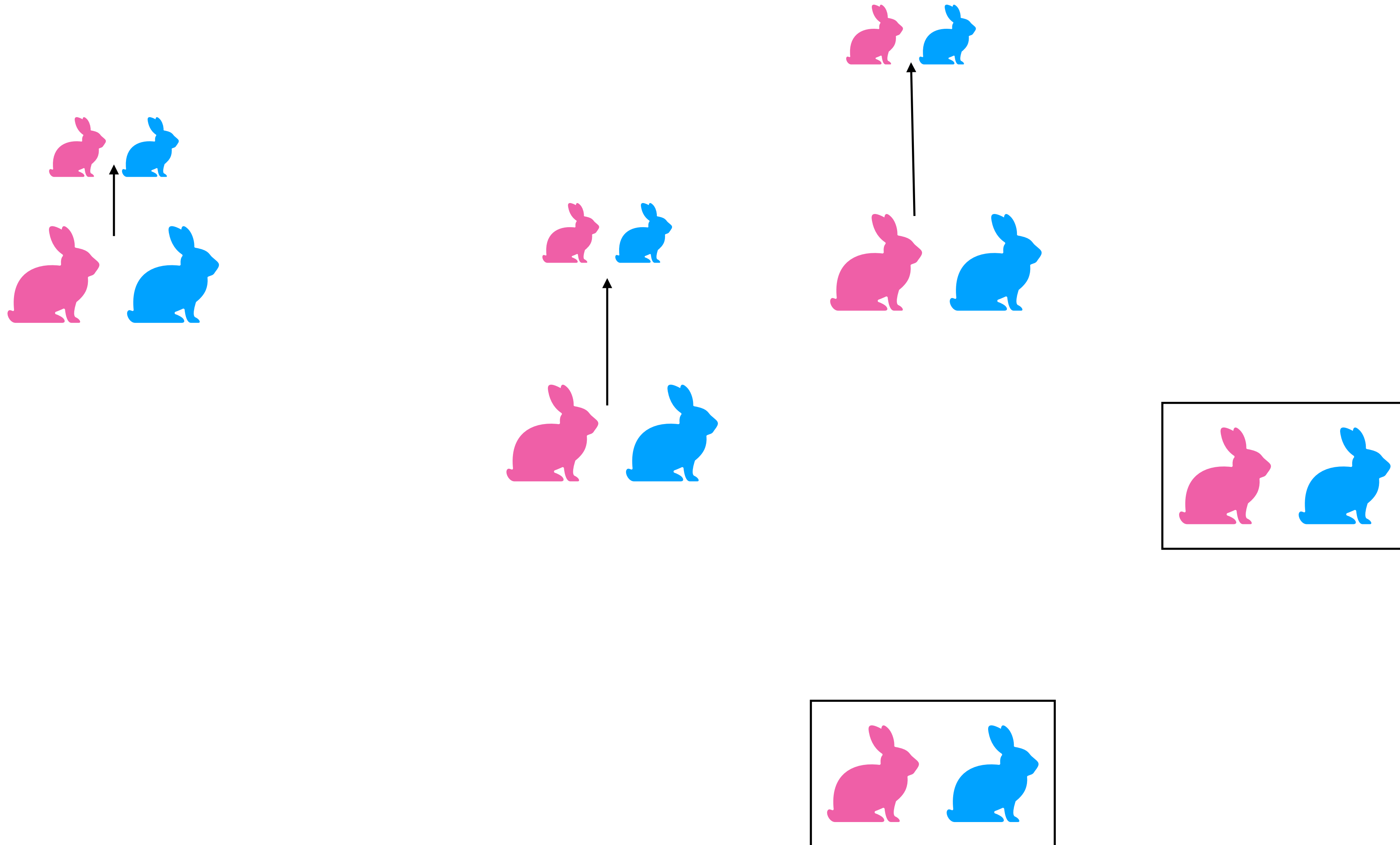
Vermehrung von Hasen



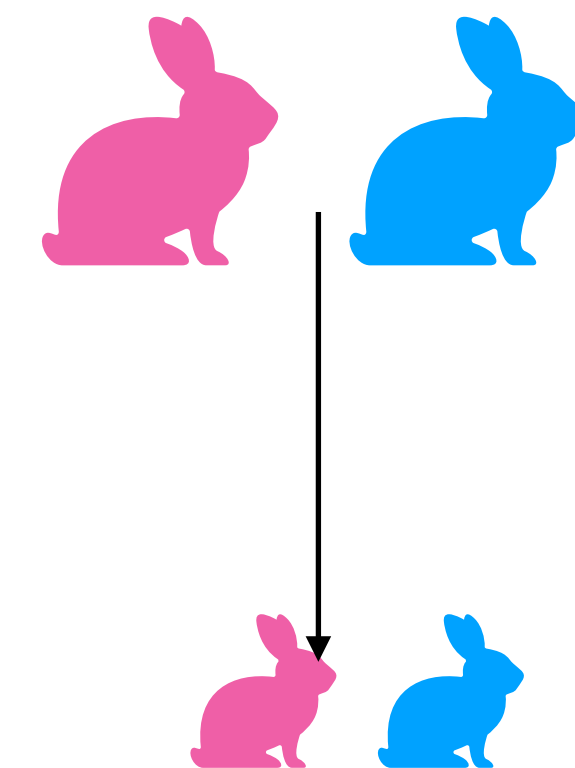
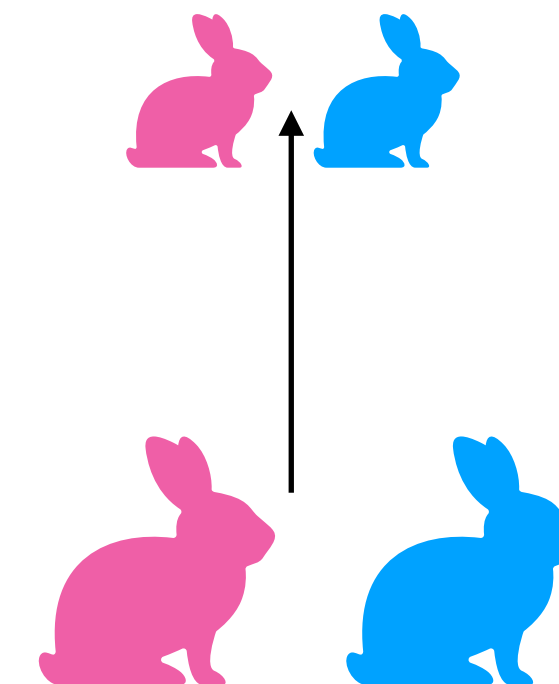
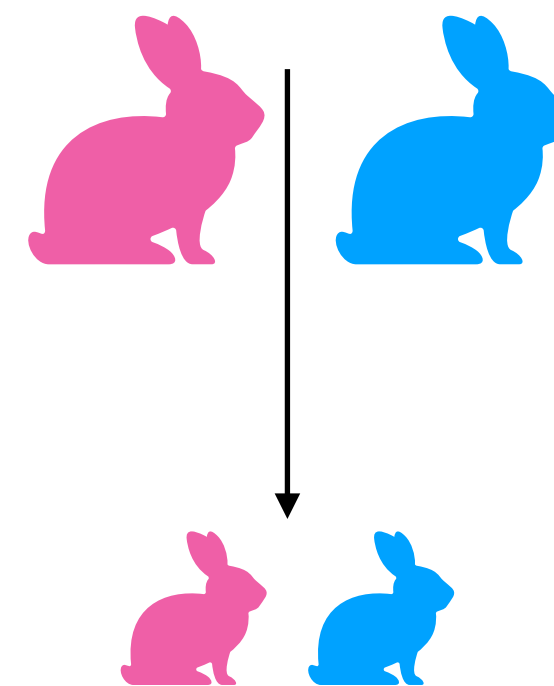
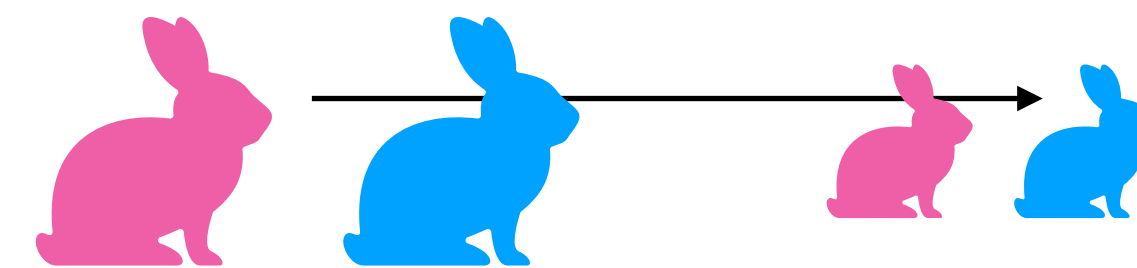
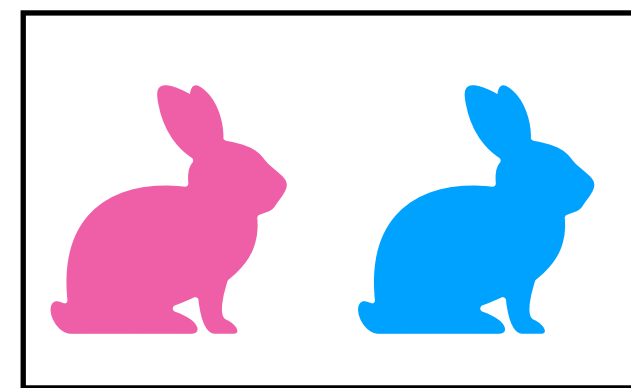
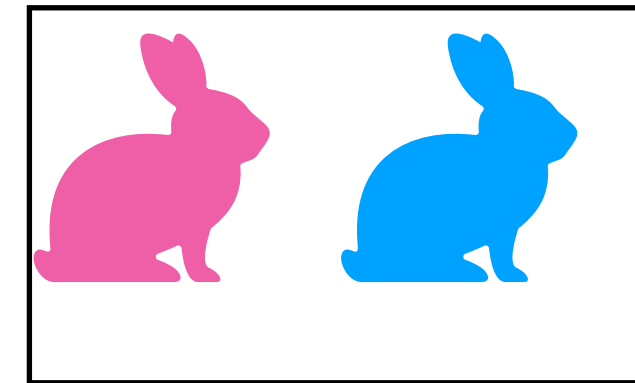
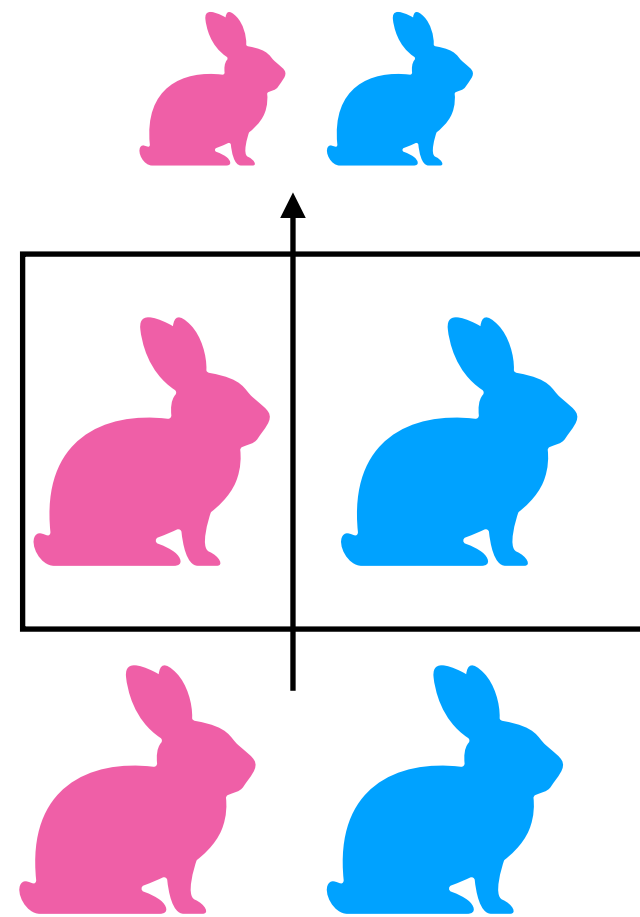
Vermehrung von Hasen



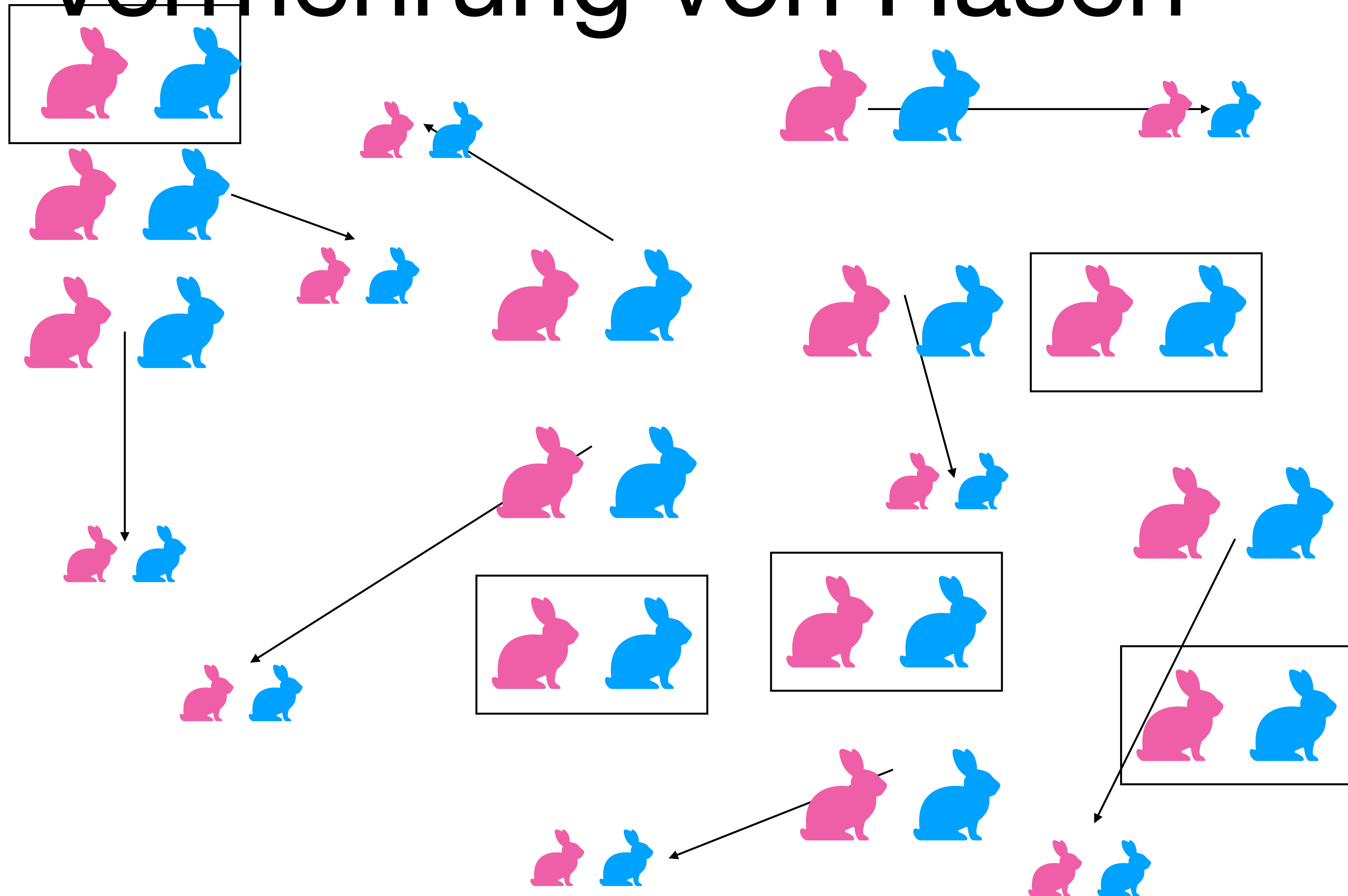
Vermehrung von Hasen



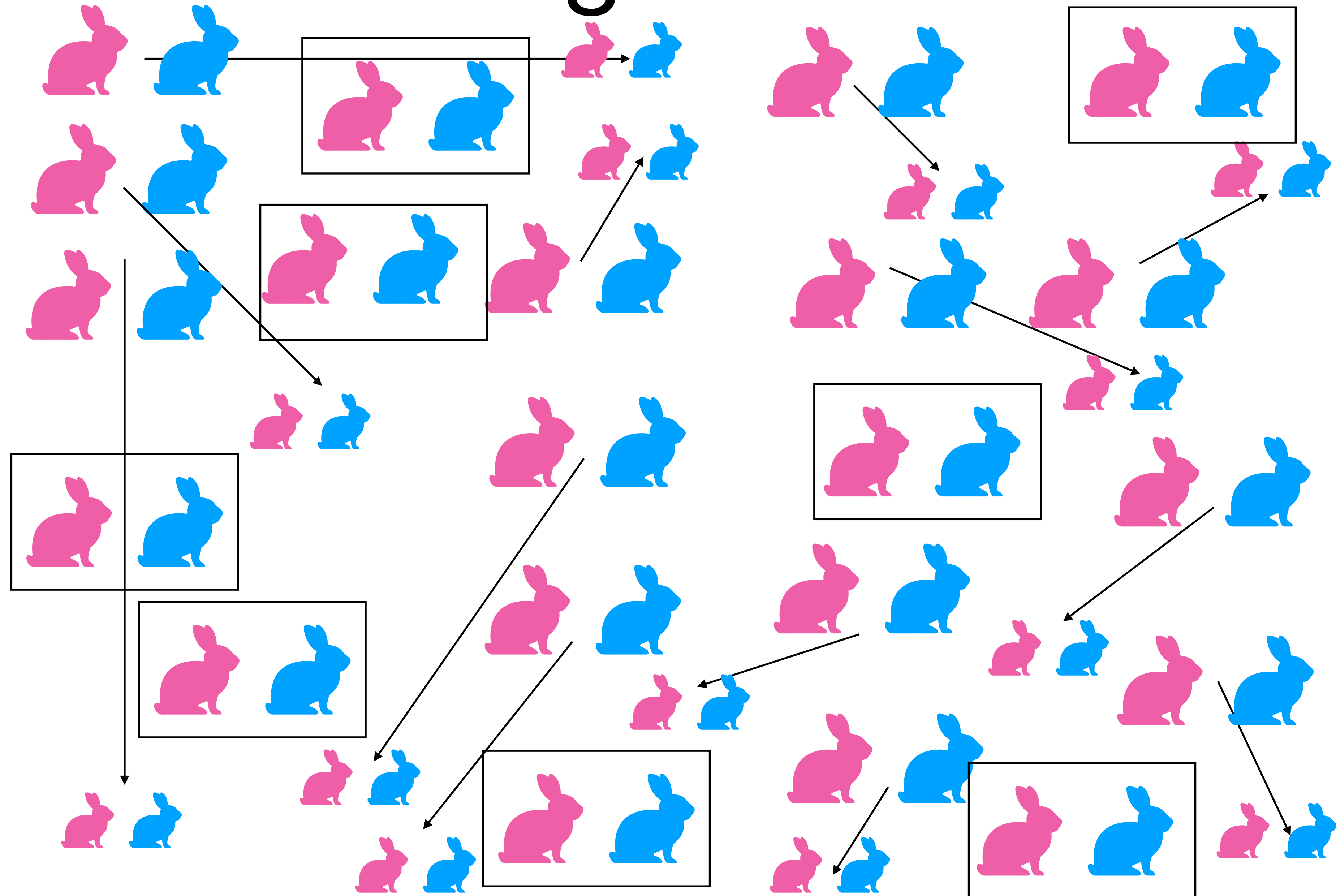
Vermehrung von Hasen



Vermehrung von Hasen



Vermehrung von Hasen



Fibonacci rekursiv

- Zu Beginn (nach 0 Monaten) gibt es ein Weibchen
- Nach einem Monat gibt es nach wie vor ein Weibchen (das nun trächtig ist)
- Nach dem zweiten Monat gibt es gibt es zwei Weibchen (eines das trächtig ist und eines das noch nicht trächtig ist)
- Bezeichne $\text{fib}(n)$ die Anzahl der Weibchen nach dem n -ten Monat
- Für alle $n > 1$ gilt
$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2), \quad \text{denn}$$
 - Jedes Weibchen, das im Monat $n-2$ lebt, wird ein junges Weibchen im Monat n werfen
 - Hinzu kommen die Weibchen, die bereits im Monat $n-1$ leben

Code Written By A CS 101 Student

```
public int fibonacci(int x) {  
    if (x == 1) {  
        return 1;  
    } else if (x == 2) {  
        return 1;  
    } else {  
        return fibonacci(x - 1) + fibonacci(x - 2);  
    }  
}
```


Code Written At A Hackathon

```
public int getFibonacciNumber(int n) {  
    switch(n) {  
        case 1: return 1;  
        case 2: return 1;  
        case 3: return 2;  
        case 4: return 3;  
        case 5: return 5;  
        case 6: return 8;  
        case 7: return 13;  
        default:  
            // good enough for the demo, lol  
            return -1;  
    }  
}
```

Code Written At A Startup

```
// TODO add Javadoc comments
/**
 * getFibonacciNumber
 */
// TODO Should we move this to a different file?
public int getFibonacciNumber(int n) {
    // TODO Stack may overflow with recursive implementation, switch over to
    // iteration approach at some point?
    if (n < 0) {
        // TODO This should probably throw an exception. Or maybe just print
        // a log message?
        return -1;
    } else if (n == 0) {
        // TODO Generalize the initial conditions?
        return 0;
    } else if (n == 1) {
        return 1;
    } else {
        // TODO Spend some time with my family and kids, I've been at work for
        // over 48 hours straight.
        return getFibonacciNumber(n - 1) + getFibonacciNumber(n - 2);
    }
}
```


}

Code Written At A Large Company

```
/**
 * getFibonacciNumber is a method that, given some index n, returns the nth
 * Fibonacci number.
 * @param n The index of the Fibonacci number you wish to retrieve.
 * @return The nth Fibonacci number.
 */
public CustomInteger64 getFibonacciNumber(CustomInteger64 n) {
    FibonacciDataViewBuilder builder =
        FibonacciDataViewBuilderFactory.createFibonacciDataViewBuilder(
            new FibonacciDataViewBuilderParams(n, null, null, 0, null));
    if (builder == FibonacciDataViewBuilderConstants.ERROR_STATE) {
        throw new FibonacciDataViewBuilderFactoryException();
    }
    FibonacciDataView dataView = builder.GenerateFibonacciDataView(this);
    if (dataView == FibonacciDataViewConstants.ERROR_STATE) {
        throw new FibonacciDataViewGenerationException();
    }
    return dataView.accessNextFibonacciNumber(null, null, null);
}
```


Code Written By A Math Ph.D.

```
public int getFibonacciNumber(int n) {  
    return (int) divide(subtract(exponentiate(phi(), n), exponentiate(psi(), n)),  
        subtract(phi(), psi()));  
}  
  
public double exponentiate(double a, double b) {  
    if (equal(b, zero())) {  
        return one();  
    } else {  
        return multiply(a, exponentiate(a, subtract(b, one())));  
    }  
}  
  
public double phi() {  
    return divide(add(one(), sqrt(add(one(), one(), one(), one(), one()))),  
        add(one(), one()));  
}  
  
public double psi() {  
    return subtract(one(), phi());  
}
```

Code Written By Your Cat

```
public static final int UNITE = 1;
public static final int UNITED = 2;

// meowwww meow
public int meow(int KITTENS_OF_THE_WORLD) {
    // MEOW
    if (KITTENS_OF_THE_WORLD < UNITED) {
        return KITTENS_OF_THE_WORLD;
    } else {
        // meeooowwwwwwww
        // meooooowwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwwww
        return meow(KITTENS_OF_THE_WORLD - UNITED)
            + meow(KITTENS_OF_THE_WORLD - UNITED);
    }
}
```

Fibonacci-Zahlen

Basisfälle

$\text{fib}(0)=1$

$\text{fib}(1)=1$

Rekursiver Fall, $n > 1$

$\text{fib}(n)=\text{fib}(n-1)+\text{fib}(n-2)$

n	fib(n)
0	1
1	1
2	2
3	3
4	5
5	8
6	13
7	21
8	34
9	55
10	89
11	144
12	233
13	377

Berechnung der Fibonacci-Zahlen

```
def fib(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)
```

Rekursion auf nicht-numerischen Daten

- Wie kann man überprüfen, ob eine Zeichenkette ein Palindrom ist, d.h. es liest sich von hinten wie von vorne gleich, wie z.B.
 - ANNA
 - RENTNER
 - REGALLAGER
 - RETSINAKANISTER
 - Geist ziert Leben, Mut hegt Siege, Beileid trägt belegbare Reue, Neid dient nie, nun eint Neid die Neuerer, abgelebt gärt die Liebe, Geist geht, umnebelt reizt Sieg.
 - Die liebe Tote Beileid

Wenn man Leerzeichen, Groß- und Kleinschreibung und Kommasetzung vernachlässigt

Palindrome rekursiv erkennen

- **Schritt 1:** Wir bereinigen die eingegebene Zeichenkette bzgl. Groß- und Kleinschreibung und Zeichen außerhalb des Alphabets:
 - entferne alle Zeichen, die keinem Buchstaben entsprechen
 - ersetze jeden verbleibenden Großbuchstaben in den entsprechenden Kleinbuchstaben
- **Schritt 2:** Entscheide ob gegebene bereinigte Zeichenkette ein Palindrom ist rekursiv:
 - **Basisfall:** Jede Zeichenkette der Länge 0 oder 1 ist ein Palindrom
 - **Rekursiver Fall** (Länge mindestens 2):
 - Die Zeichenkette ist genau dann ein Palindrom, falls
 - erster und letzter Buchstabe gleich sind, und
 - das mittlere Wort (Wort das durch Löschen des ersten und letzten Zeichens entsteht) selbst ein Palindrom ist

Palindrom Beispiel

- Sei die Eingabe die Zeichenkette

"<In Nagold legen Hähne Geld> log Anni."

- Nach dem Bereinigungsschritt entsteht

"innagoldlegenhähnegeldloganni"

- Wir definieren eine Funktion `istPalindrom`, die eine Zeichenkette entgegennimmt und ein `bool` zurückliefert

- Es gilt `istPalindrom("innagoldlegenhähnegeldloganni")` genau dann, wenn
 - `i == i` und
 - `istPalindrom("nnagoldlegenhähnegeldlogann")` gilt

==true

Palindromerkennung rekursiv

```
def Palindrom(s):  
    def bereinige(s):  
        s=s.lower()  
        antwort=""  
        for c in s:  
            if c in "abcdefghijklmnopqrstuvwxyzäüö":  
                antwort = antwort + c  
        return antwort  
    def istPalindrom(s):  
        if len(s)<=1:  
            return True  
        else:  
            return s[0]==s[-1] and istPalindrom(s[1:-1])  
    return istPalindrom(bereinige(s))
```


Der euklidische Algorithmus

- gilt als der älteste nicht-triviale Algorithmus
- Berechnung des größten gemeinsamen Teilers zweier positiver natürlicher Zahlen a und b mit $a \geq b$ eindeutig bestimmt
- Setze $r_{-1} = a$ und $r_0 = b$ und finde $q_1, r_1 \geq 1$ mit $r_1 < r_0$ so, dass

$$a = q_1 \cdot r_0 + r_1 \quad \text{Falls } r_1 = 0 \text{ dann sind wir fertig.}$$

$$r_0 = q_2 \cdot r_1 + r_2 \quad \text{für } q_2, r_2 \geq 0 \text{ mit } 0 \leq r_2 < r_1$$

$$r_1 = q_3 \cdot r_2 + r_3 \quad \text{für } q_3, r_3 \geq 0 \text{ mit } 0 \leq r_3 < r_2$$

bis $\vdots$

$$r_{n-1} = q_{n+1} \cdot r_n + r_{n+1} \quad \text{für } q_{n+1}, r_{n+1} \geq 0 \text{ mit } r_{n+1} = 0$$

für ein $n \geq 0$ =0

- Ergebnis $\text{ggT}(a, b) = r_n$



um 300 v. Chr.

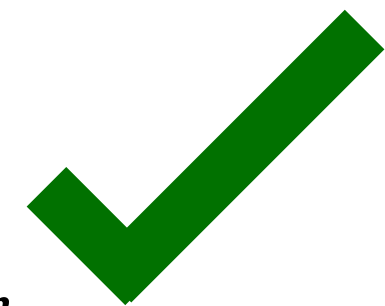
Terminierung/Korrektheit Euklidischer Alg.

- Terminierung klar wegen: $r_{i+1} < r_i$ für alle $1 \leq i < n$ Warum?
- In der $(i + 1)$ -ten Iteration werden $q_{i+1}, r_{i+1} \geq 0$ mit $0 \leq r_{i+1} < r_i$ gefunden so, dass $r_{i-1} = q_{i+1} \cdot r_i + r_{i+1}$

- Es gilt $\mathbf{ggT}(r_{i-1}, r_i) = \mathbf{ggT}(r_i, r_{i+1})$
für alle $0 \leq i < n$

Warum?

$$\mathbf{ggT}(a, b) = \mathbf{ggT}(r_{-1}, r_0) \boxed{=} \mathbf{ggT}(r_0, r_1) \boxed{=} \dots \boxed{=} \mathbf{ggT}(r_{n-1}, r_n) \boxed{=} r_n$$



- Es gilt sogar $r_{i+1} < r_{i-1}/2$ für alle $0 \leq i < n$

Warum?

Euklidischer Algorithmus rekursiv

```
def Euklid(a,b):  
    if a==0 or b==0:  
        return "undefiniert"  
    if a<b: return Euklid(b,a)  
    if a%b==0:  
        return b  
    else:  
        return Euklid(b,a%b)
```

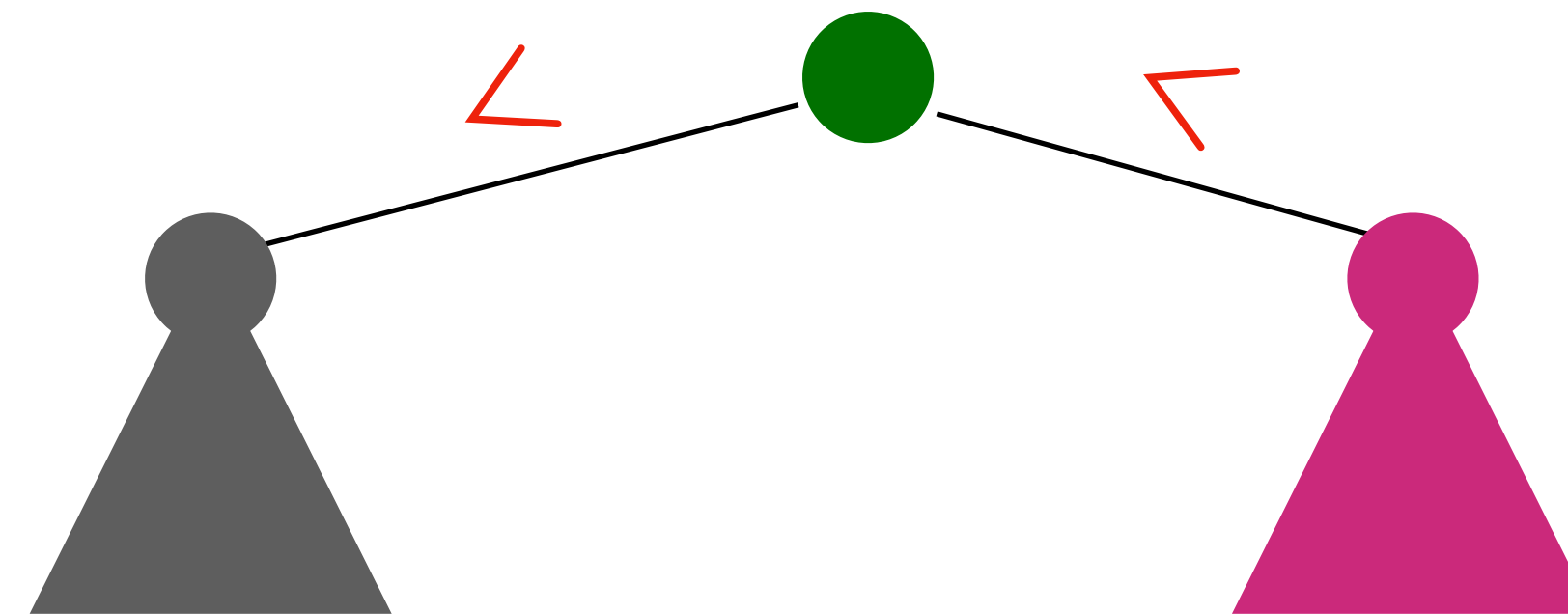

Betreiben einer Suchmaschine

- Stellen Sie sich vor Sie betreiben eine Suchmaschine
- Sie müssen sehr viele Suchbegriffe und die dazugehörigen Websites verwalten
- Sie müssen eine Datenstruktur anlegen, das diese angefragten Suchbegriffe verwaltet und jeweils mit Websites assoziiert
- Manche der Suchbegriffe kommen häufiger vor als andere und Sie kennen die Häufigkeit aller Suchbegriffe
- Die einem geg. Suchbegriff zugehörige Website muss so schnell wie möglich gefunden werden
- Sie ordnen die Suchbegriffe (mit Ihren Websites) in einem binären Suchbaum an (die einzige Möglichkeit, die Ihnen zur Verfügung steht)

Binäre Suchbäume

werden wir ab nun
vernachlässigen

- Sie verwalten Ihre Suchbegriffe in einem binären Suchbaum:
 - die Knoten in Ihrem Suchbaum entsprechen Ihren verwalteten Suchbegriffen (zugehörige Website ebenfalls im Knoten vorhanden)
 - alle Knoten im **linken Unterbaum** sind **kleiner oder gleich**, im **rechten Unterbaum größer oder gleich** dem **Wurzelwert** (für jeden Unterbaum)



kleiner bzw. größer
bzgl. String-Vergleich
oder jeder anderen
totalen Ordnung

- „je tiefer Sie den Baum nach unten traversieren um einen Suchbegriff zu suchen desto länger muss gewartet die dazugehörige Website aufzurufen“
- Suchbegriffe werden je nach angegebener Häufigkeit gesucht

Binäre Suchbäume: Gesamtgewicht

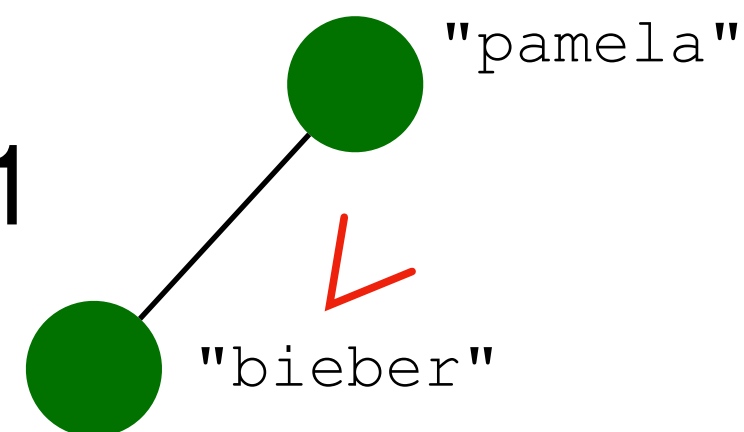
- nehmen wir an Sie haben folgende Begriffs- und Häufigkeitslisten:

begriffe = ["bieber", "pamela"]

Annahme:
nach String sortiert
"bieber" < "pamela"

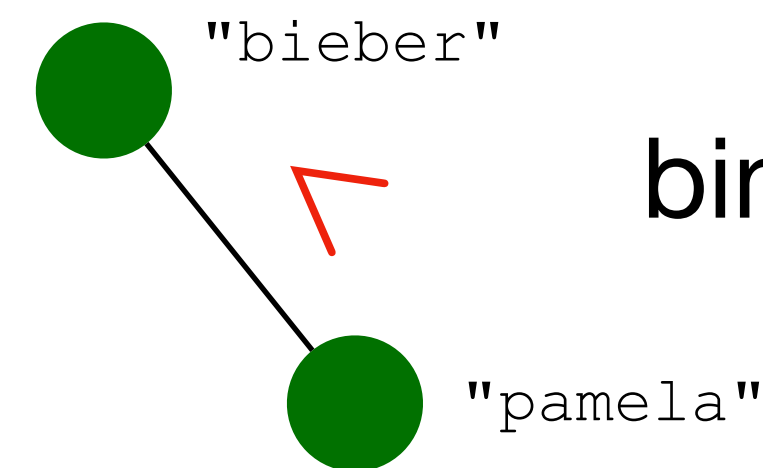
haeufigkeiten = [50, 34]

bin. Suchbaum 1



Gesamtgewicht $1 \cdot 34 + 2 \cdot 50 = 134$

bin. Suchbaum 2



Gesamtgewicht $1 \cdot 50 + 2 \cdot 34 = 118$
optimal!

- allgemeine Definition des **Gesamtgewicht (GG)** eines Suchbaums T

$$\mathbf{GG(T)} = \sum_{\substack{\text{Knoten } v \\ \text{in } T}} \text{Tiefe}(v) * \text{Häufigkeit}(\text{Beschriftung}(v))$$

Entfernung zur Wurzel + 1

hier: Zeichenkette, die an
Knoten v steht

Binäre Suchbäume: O(n) Zeit

- nehmen wir an Sie haben folgende

begriffe

haben

**Optimaler Suchbaum:
Finde binären Suchbaum
mit minimalem Gesamtgewicht**

Suchbaum 2

"anela"

Gesamtgewicht $1 \cdot 50 + 2 \cdot 34 = 118$

optimal!

Gesamtgewicht eines Suchbaums T

$$\text{Gewicht}(T) = \sum_{\text{Knoten } v \text{ in } T} \text{Tiefe}(v) * \text{Häufigkeit}(\text{Beschriftung}(v))$$

Knoten v
in T

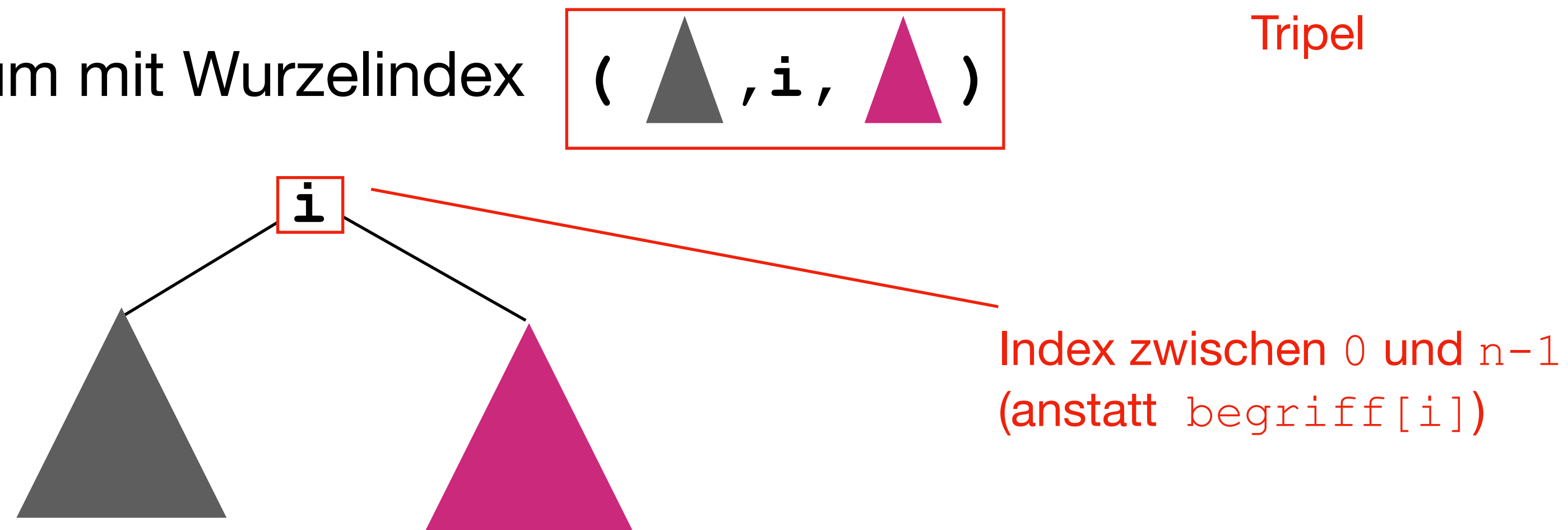
Entfernung zur Wurzel + 1

hier: Zeichenkette, die an
Knoten v steht

Präsentation von bin. Suchbäumen

begriffe = [ ,  , ... , ]
 haeufigkeiten = [ ,  , ... , ]

- Beschriftung der Knoten: Indizes anstatt Begriff an diesem Index
- Wir stellen binäre Suchbäume folgendermaßen rekursiv dar:
 - leerer Suchbaum: $()$
 - Suchbaum mit Wurzelindex $(\blacktriangle, i, \blacktriangle)$ Tripel



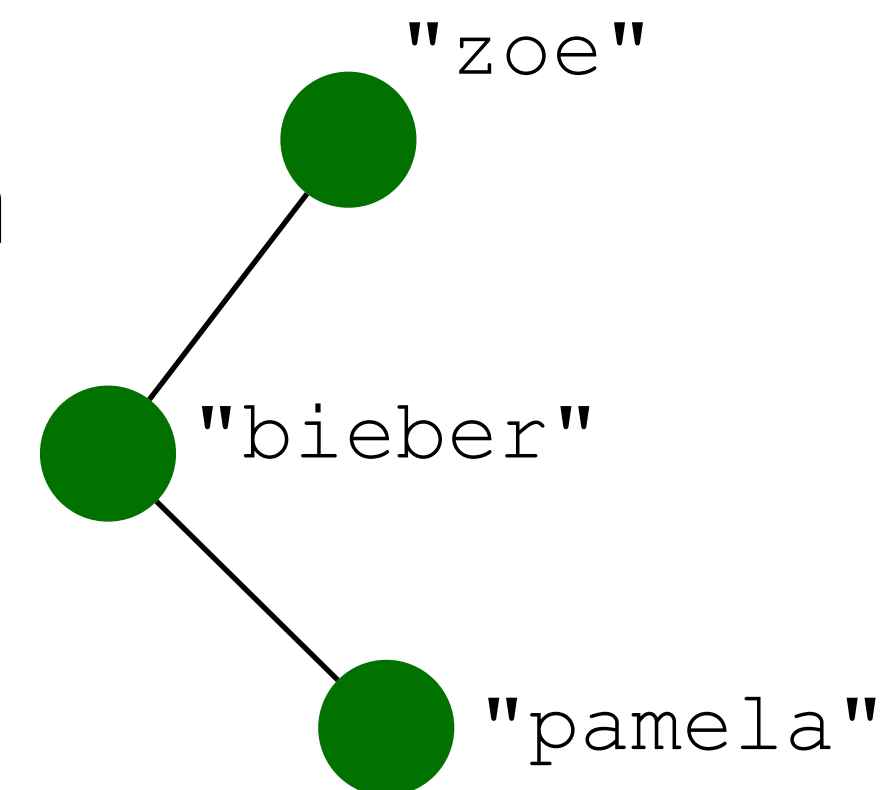
Darstellung von bin. Suchbäumen

Beispiel

begriffe = ["bieber", "pamela", "zoe"]

haeufigkeiten = [34, 8, 15]

Eigentlicher bin. Suchbaum



Interne Darstellung

(((), 0, ((), 1, ())), 2, ())

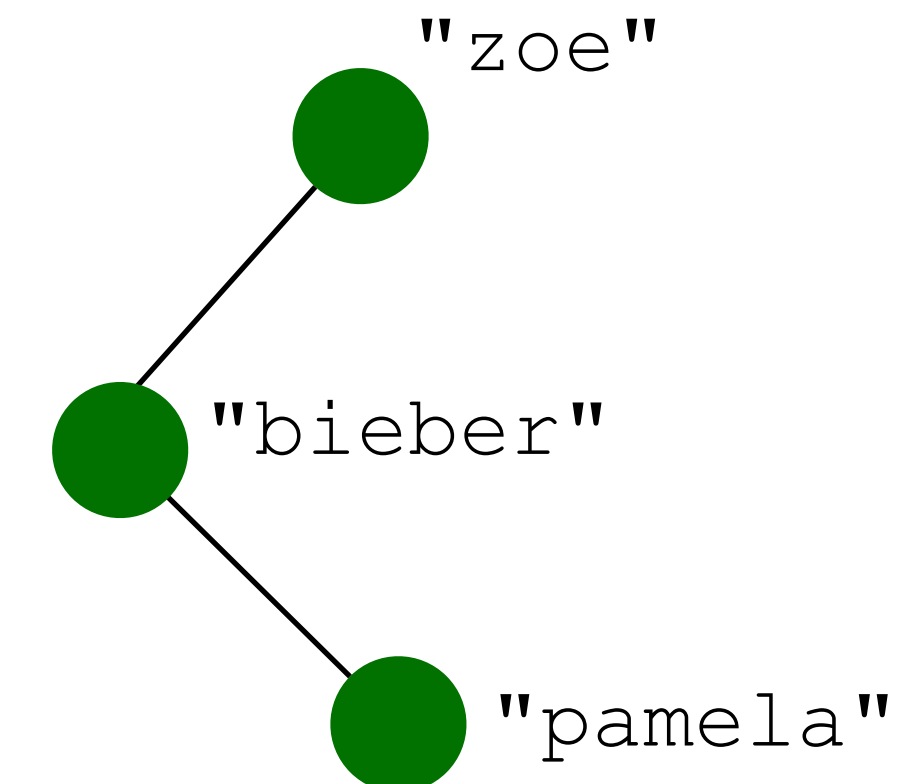
Index an der Wurzel

Geeignete Ausgabe der internen Präsentation

```
leerbaum=()
```

```
def ausgabe(baum):  
    if baum!=leerbaum:  
        x = "("  
        if baum[0] != leerbaum:  
            x += ausgabe(baum[0])  
        x += begriffe[baum[1]]  
        if baum[2] != leerbaum:  
            x += ausgabe(baum[2])  
    return x+")"
```

Eigentlicher bin. Suchbaum



Interne Darstellung:

```
(((), 0, (((), 1, ())), 2, ())
```

Rückgabe von ausgabe:

```
((bieber(pamela))zoe)
```

Berechnung des Gesamtgewichts

- allgemeine Definition des **Gesamtgewicht (GG)** eines Suchbaums T

$$\mathbf{GG(T)} = \sum_{\substack{\text{Knoten } v \\ \text{in } T}} \text{Tiefe}(v) * \text{Häufigkeit}(\text{Beschriftung}(v))$$

Entfernung zur Wurzel + 1

hier: Zeichenkette, die an Knoten v steht

def gg(baum) : Unterfunktion

```
def gewichtet(baum, faktor) :  
    if baum==leerbaum:  
        return 0  
    else:  
        return faktor*haeufigkeit[baum[1]] +\  
                gewichtet(baum[0], faktor+1) +\  
                gewichtet(baum[2], faktor+1)  
return gewichtet(baum, 1)
```

Wurzel des linken und rechten Unterbaums haben
einen um 1 erhöhten Abstand zur Wurzel des eigentlichen Baums

Problemstellung und Teillösungen

genau:

n-te Catalan-Zahl

Satz (ohne Beweis): Es gibt bei n Suchbegriffen rund $4^n / (n + 1) \sqrt{\pi n}$ verschiedene potentielle Suchbäume.

- Gegeben sei eine **Begriffsliste** und eine **Häufigkeitsliste** der Länge jeweils n
(Zeichenketten) (nicht-negative Integer)

	0	1		n-1	Indizes				
begriffe =	[	■	,	■	,	...	,	■	]
haeufigkeiten =	[	■	,	■	,	...	,	■	]

- Für jedes Unterintervall $[i, j]$ von $[0, n]$ bezeichne
 - $T[i, j]$ den (bzw. irgendeinen) optimalen bin. Suchbaum eingeschränkt auf $\{i, \dots, j-1\}$ und $haeufigkeiten[i, j]$
 - uns interessiert eigentlich nur $T[0, n]$

Rekursive Beschreibung

begriffe = [, , ..., ]
haeufigkeiten = [, , ..., ]

Basisfälle

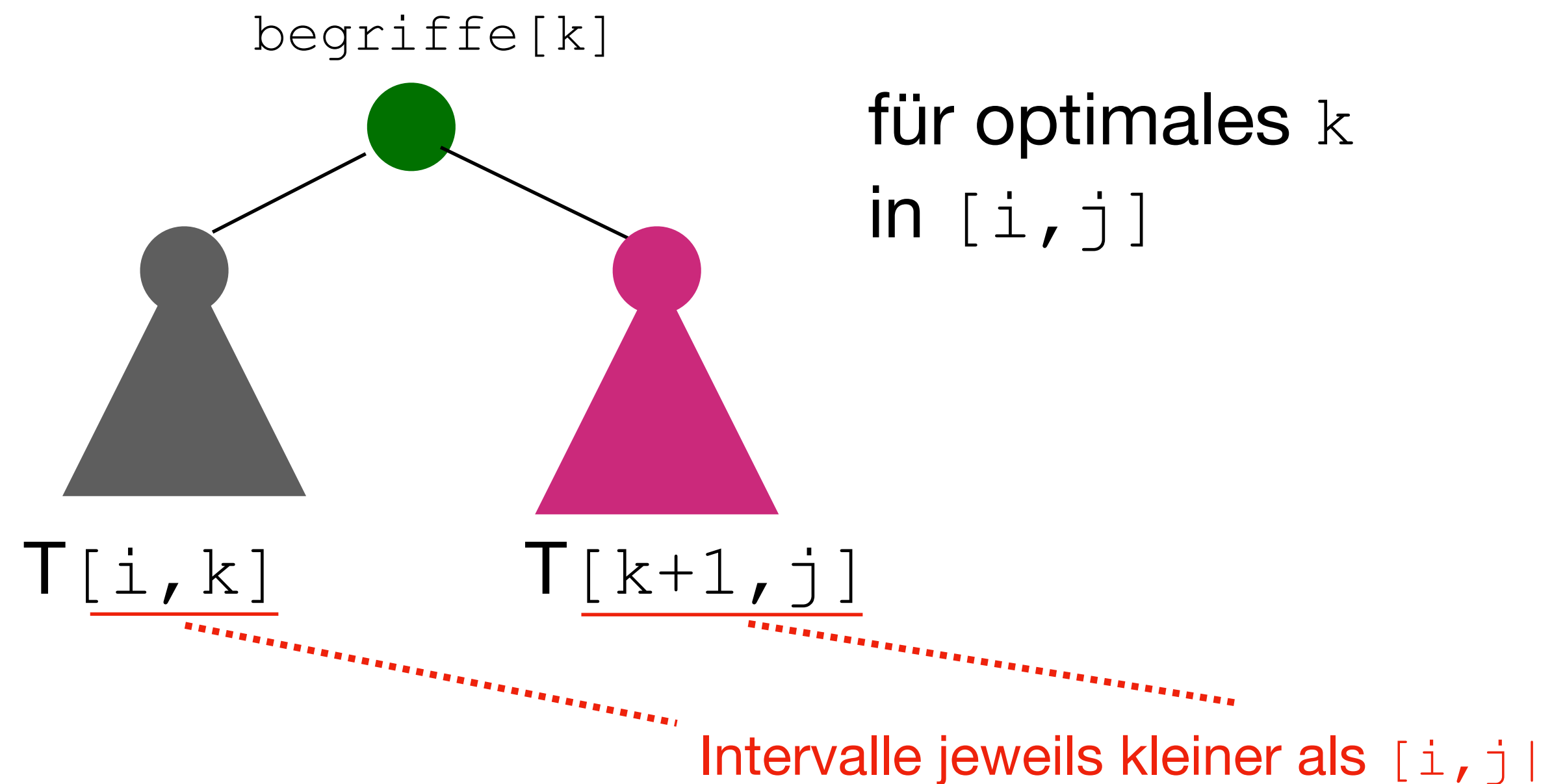
- $T[i, i] =$ **leerer Baum**
 $\parallel$
 j

- $T[i, i+1] =$ 
 $\parallel$
 j

Rekursive Fälle

- $T[i, j] =$

falls $j > i+1$



Rekursive Lösung

beginne mit erstem potentiellen optimalen Suchbaum für $[i, j]$: Nimm i als Wurzel, linker Teilbaum leer und rechter Teilbaum ist die beste Lösung für Intervall $[i+1, j]$

```
def loeseRekursiv(i, j):  
    if i==j:  
        return leerbaum  
    if j==i+1:  
        return (leerbaum, i, leerbaum)  
    else:  
        baum = (leerbaum, i, loeseRekursiv(i+1, j))  
        for k in range(i+1, j):  
            links = loeseRekursiv(i, k)  
            rechts = loeseRekursiv(k+1, j)  
            if gg((links, k, rechts)) < gg(baum):  
                baum = (links, k, rechts)  
    return baum
```

Ein Suchbaum im Paper von Donald Knuth



```
def init2():
    begriffe = ["a", "an", "and", \
                "by", "effects", "for", \
                "from", "high", "in", \
                "of", "on", "the", "to", \
                "with"]

    haeufigkeit=[32, 7, 69, 13, 6, 15,
                 10, 8, 64, 142, 22, 79, 18, 9]
    return (len(begriffe), begriffe, haeufigkeit)
```

Optimum Binary Search Trees*

D. E. KNUTH

Received June 22, 1970

One of the popular methods for retrieving information by its "name" is to store the names in a binary tree. To find if a given name is in the tree, we compare it to the name at the root, and four cases arise:

1. There is *no* root (the binary tree is empty): The given name is not in the tree, and the search terminates *unsuccessfully*.
2. The given name *matches* the name at the root: The search terminates *successfully*.
3. The given name is *less* than the name at the root: The search continues by examining the *left subtree* of the root in the same way.
4. The given name is *greater* than the name at the root: The search continues by examining the *right subtree* of the root in the same way.

Special cases of this method are the binary search and its variants (uncentered binary search; Fibonacci search) and the search-sort scheme of Wheeler-Berners Lee-Booth-Hibbard-Windley, *et al.* (see [1, 3, 7, 10]).

When all names in the tree are equally probable, it is not difficult to see that a best possible binary tree from the standpoint of average search time is one with minimum path length, namely the *complete binary tree* (see [9, pp. 400–401]). This is the tree which is implicitly present in one of the variants of the binary search method.

But when some names are known to be much more likely to occur than others, the best possible binary tree will not necessarily be balanced. For example, consider the following words and frequencies,

a	32
an	7
and	69
by	13
effects	6
for	15
from	10
high	8
in	64
of	142
on	22
the	79
to	18
with	9

* The research reported here was supported by IBM Corporation.

Das Hauptprogramm

```
leerbaum=()

def init2():
    ...

def ausgabe(baum):
    ...

def gg(baum):
    ...

def loeseRekursiv(i,j):
    ...

def loeseDP()
    ...

(n,begriffe,haeufigkeit)=init2()
baum1=loeseDP()
baum2=loeseRekursiv(0,n)
print(ausgabe(baum1))
print("Gesamtgewicht Baum 1: "+str(gg(baum1)))
print(ausgabe(baum2))
print("Gesamtgewicht Baum 2: "+str(gg(baum2)))
```

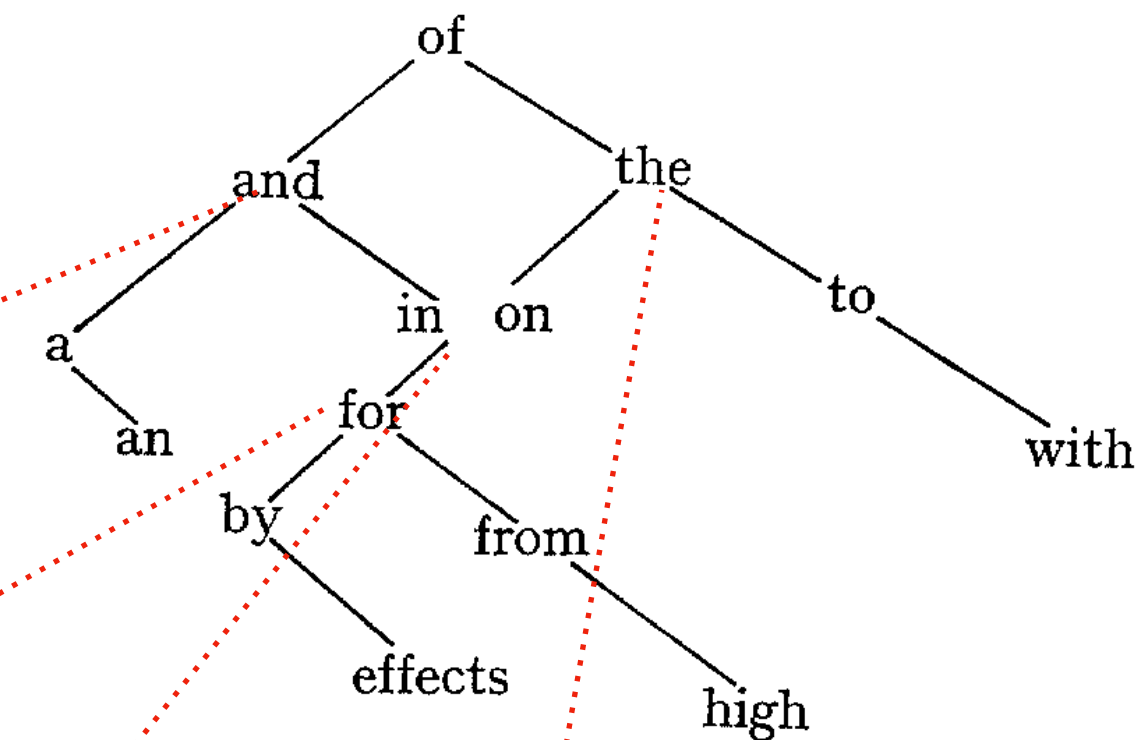

Ein Suchbaum im Paper von Donald Knuth



Optimum Binary Search Trees

15

showing words to be ignored in a certain KWIC indexing application [6, p.124].
The best possible tree in this case turns out to be



`((a(an))and(((by(effects))for(from(high)))in))of((on)the(to(with))))`

Gesamtgewicht dieses Baums:1169

Speichern von Studierendeninformation

- Bisher haben wir gelernt für jede einzelne Information eine Liste anzulegen

```
namen = ['Julia', 'Michael', 'Anna', 'Rose']
```

```
noten = ['1-', '2', '1', '4-5']
```

```
kurse = ['Mathe', 'Erdkunde', 'Deutsch', 'Kunst']
```

- Für jede Information einer Studierenden/eines Studierenden wird eine **separate Liste** angelegt
- Alle Listen müssen **diesselbe Länge** haben
- Information wird über mehrere Listen hinweg am **selben Index** gespeichert, jeder Index entspricht einer anderen Person

Zugriff/Modifikation von Studierendeninformation

```
def hole_note(studentIn, namenliste, notenliste, kursliste)
    i = namenliste.index(studentIn)
    note = notenliste[i]
    kurs = kursliste[i]
    return (kurs, note)
```

- recht **unübersichtlich** bis **choatisch** wenn man viel derartige Information zu verwalten hat
- **viele Listen** müssen verwaltet werden und viele Argumente müssen übergeben werden
- **stets** geschieht der Zugriff mittels **Integer-Indizes**
- mehrere Listen müssen angepasst werden (um bspw. dieselbe Längen zu garantieren), daher fehleranfällig

Die bessere Lösung: Wörterbücher/dictionaries

- **Indizierung allgemeiner**, nicht unbedingt mittels Integers
- angenehm eine einzelne Datenstruktur zu verwenden, anstatt separate Listen kompliziert zu verwalten

Eine Liste

0	Element 0
1	Element 1
2	Element 2
3	Element 3
...	...

Index

Element

Ein Wörterbuch

Schlüssel 1	Wert 1
Schlüssel 2	Wert 2
Schlüssel 3	Wert 3
Schlüssel 4	Wert 4
...	...

selbstdefinierter
Schlüssel

Element

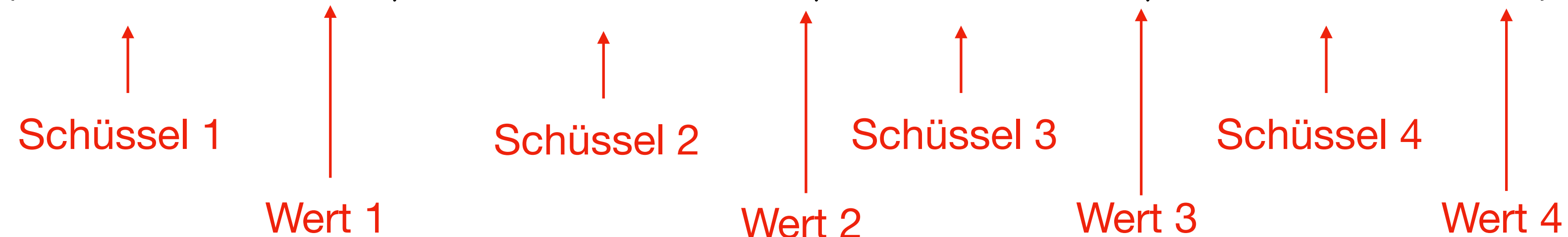
Ein Wörterbuch in Python

- speichert Paare von Daten, nämlich
 - einen Schlüssel
 - und seinen Wert

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'

```
mein_buch = {}
```

```
noten={'Julia':'1-','Michael':'2','Anna':'1','Rose':'4-5'}
```



Wörterbuchzugriff

- ähnlich zu Indizierung bei Listen
- entspricht Funktionen in der Mathematik (Urbildbereich=Schlüssel, Wertebereich=Werte)
- Zugriff mittels Schlüssel
- Wörterbuch gibt den, mit diesem Schlüssel assoziierten Wert zurück
- Fehlermeldung, wenn Schlüssel nicht gefunden werden kann

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'

```
noten={'Julia':'1-', 'Michael':'2', 'Anna':'1', 'Rose':'4-5'}
```

```
noten['Anna']  wertet sich zu '1' aus
```

```
noten['Horst']  liefert einen KeyError
```

Wörterbuch- operationen

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'
'Horst'	'3-'

```
noten={'Julia':'1-','Michael':'2','Anna':'1','Rose':'4-5'}
```

- **füge** einen Eintrag **hinzu**

```
noten['Horst']='3-'
```

- **teste** ob es einen Schlüssel im Wörterbuch gibt

'Elvira' in noten  wertet sich zu False aus

'Horst' in noten  wertet sich zu True aus

- **lösche** einen Eintrag

```
del(noten['Rose'])
```

Wörterbuch- operationen

'Julia'	'1-'
'Michael'	'2'
'Anna'	'1'
'Rose'	'4-5'
'Horst'	'3-'

```
noten={'Julia':'1-','Michael':'2','Anna':'1','Rose':'4-5'}
```

Ordnung nicht garantiert

- hole einen **Iterator**, der sich wie ein **Tupel aller Schlüssel** verhält

```
noten.keys() ————— liefert ['Rose', 'Michael', 'Anna', 'Julia']
```

- hole einen **Iterator**, der sich wie ein **Tupel aller Werte** verhält

```
noten.values() ————— liefert ['4-5', '2', '1', '1-']
```

Ordnung nicht garantiert

Wörterbuchschlüssel und -werte

- **Werte**
 - können beliebigen Typ haben (**veränderbar** oder **unveränderbar**)
 - können **mehrfach vorkommen**
 - können sogar Listen und weitere Wörterbücher sein!
- **Schlüssel**
 - müssen **eindeutig** sein
 - müssen von **unveränderbaren** Typ sein (also `int, float, string, tuple, bool`)
 - es reicht, wenn sie **hashable** sind, aber der Einfachheit halber denken wir unveränderbar entspricht hashable
- Weder auf Schlüsseln noch Werten muss eine Ordnung bestehen

```
wb = {4: (1, 0), (1, 3): 'zwölf', 'konst': [3.14, 2.7, 8.44]}
```

list vs dict

- **geordnete** Sequenz von Elementen
- Zugriff auf Elemente mittels Index
- Indizes haben eine **Ordnung**
- Index ist ein **Integer**
- **ordnet** Schlüsseln Wert **zu**
- Zugriff auf Elemente mittels Schlüssel
- **keine** garantierte **Ordnung**
- Schlüssel kann jeder **unveränderbare** Typ sein

Ein Wortzähler im Songtext

- Finde diejenigen Wörter, die in Datei `lied.txt` **mindestens x mal vorkommen**
 - Lies Zeilen der Datei `lied.txt` ein und bereinige diese
 - Benutzer gibt x selbst als **Parameter** an
 - Wir kreieren ein **Häufigkeitswörterbuch** (`str` nach `int`)
 - Gib eine **Liste von Paaren** der Form (`list, int`) zurück, wobei die erste Komponente eine Liste von Wörtern und die zweite Komponente die Häufigkeit aller in dieser Liste vorkommenden Wörter beschreibt
 - **Informelle Idee:** Ausgehend vom Lieder-Wörterbuch, **erhalte die größte Häufigkeit**, entferne alle solch häufigen Wörter, entferne sie aus Wörterbuch und füge Sie dem Ergebnis hinzu. Wiederhole dies. Funktioniert, da man Wörter aus Lieder-Wörterbuch entfernt.

Häufigkeitswörterbuch

```
def liedtext_haeufigkeit(liedtext):  
    '''  
    Erwartet einen Liste von Zeichenketten,  
    die am Besten um Sonderzeichen und Gross- und  
    Kleinschreibung bereinigt ist.  
    '''  
    meinwb = {}  
    for wort in liedtext:  
        if wort in meinwb:  
            meinwb[word] += 1  
        else:  
            meinwb[word] = 1  
    return meinwb
```

Häufigkeitswörterbuch

```
def haeufigste_woerter(haeufige) :  
    werte = haeufige.values()  
    maximum = max(werte)  
    woerter = []  
    for s in haeufige:  
        if haeufige[s] == maximum:  
            woerter.append(s)  
    return (woerter,maximum)
```


Häufigkeitswörterbuch

```
def haeufige_woerter(woerterbuch, mindestens):  
    ergebnis = []  
    fertig = False  
    while not fertig and woerterbuch:  
        temp = haeufigste_woerter(woerterbuch)  
        if temp[1] >= mindestens:  
            ergebnis.append(temp)  
            for w in temp[0]:  
                del (woerterbuch[w])  
        else:  
            fertig = True  
    return ergebnis
```

Hauptprogramm (1)

```

    lied=""
    datei = open("lied.txt")
    for line in datei:
        lied+=" " + line.rstrip()
    datei.close()

    lied=lied.lower()

    lied=lied.replace(",","")
    lied=lied.replace(")","")
    lied=lied.replace("(","")

    l=lied.split()

    x=input("Geben Sie eine Häufigkeit >= 1 ein : ")

    woerterbuch=liedtext_haeufigkeit(l)

    liste=haeufige_woerter(woerterbuch,int(x))

```

Öffne Datei `lied.txt`

entferne Zeilenumbruch `\n` am Ende der eingelesenen Zeile

Schließe Datei

Alle Grossbuchstaben werden in Kleinbuchstaben umgewandelt

Alle Kommata, "(" und ")" werden durch den leeren String ersetzt, also entfernt

Erzeuge Liste aller Wörter im bereinigten Liedtext, die nach Leerzeilen getrennt werden

Erzeuge Wörterbuch aller im Lied vorkommenen Wörter und ordne ihnen ihre Häufigkeit zu

Hauptprogramm (2)

Ausgabe der berechneten Liste (von `(list, integer)`-Paaren)

```
for paar in liste:
    if paar[1]== 1:
        w=paar[0].pop()
        print("Das Wort '"+w+"' kommt einmal vor")
    else:
        satz=str(paar[1])+" mal kommen vor: "+ "\n"
        for w in paar[0]:
            satz = satz + " "*20 +w+"\n"
        print(satz)
```

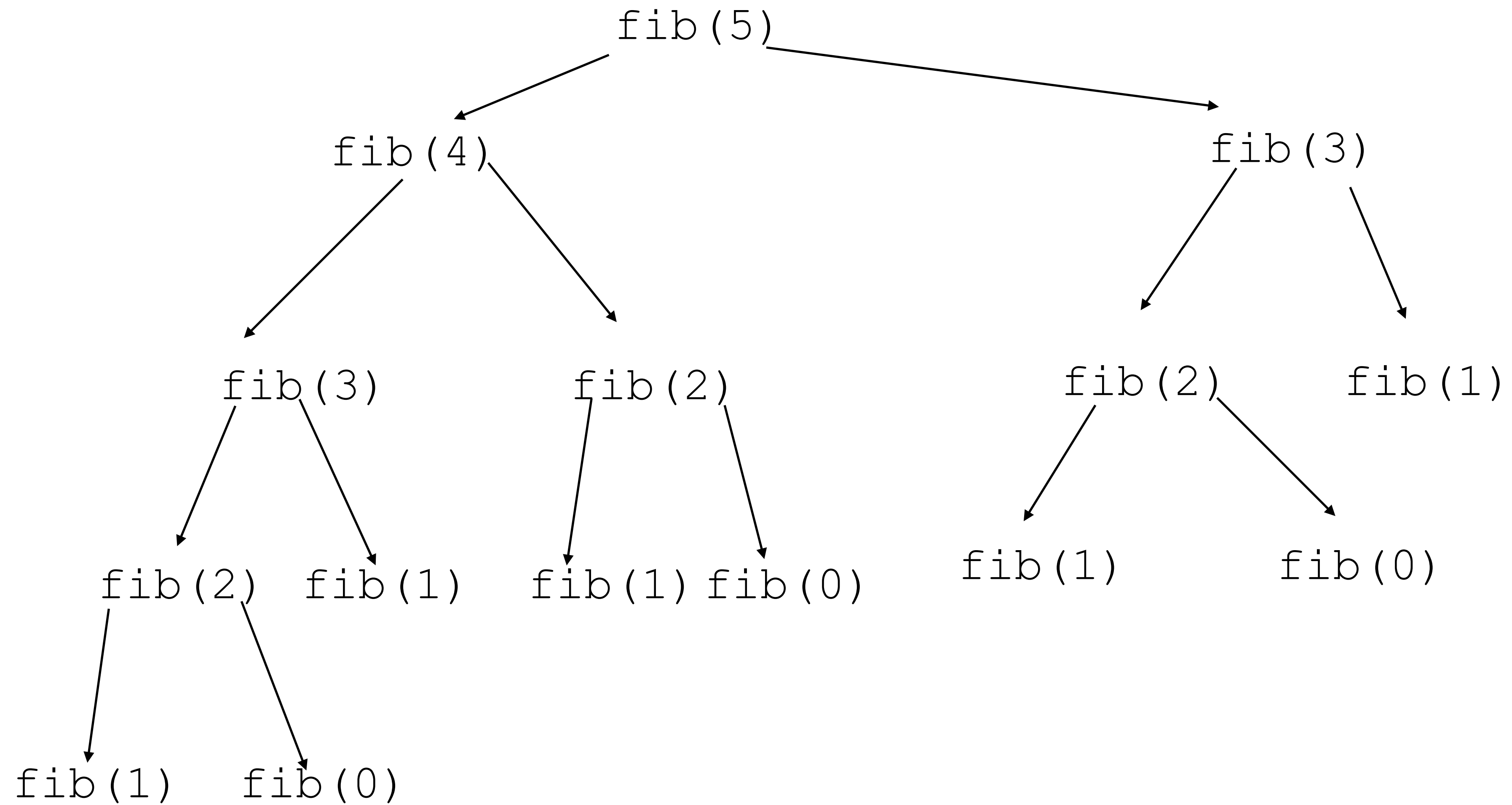
Kopiere Leerzeichen 20 Mal (für Darstellungszwecke)

Fibonacci-Zahlen rekursiv

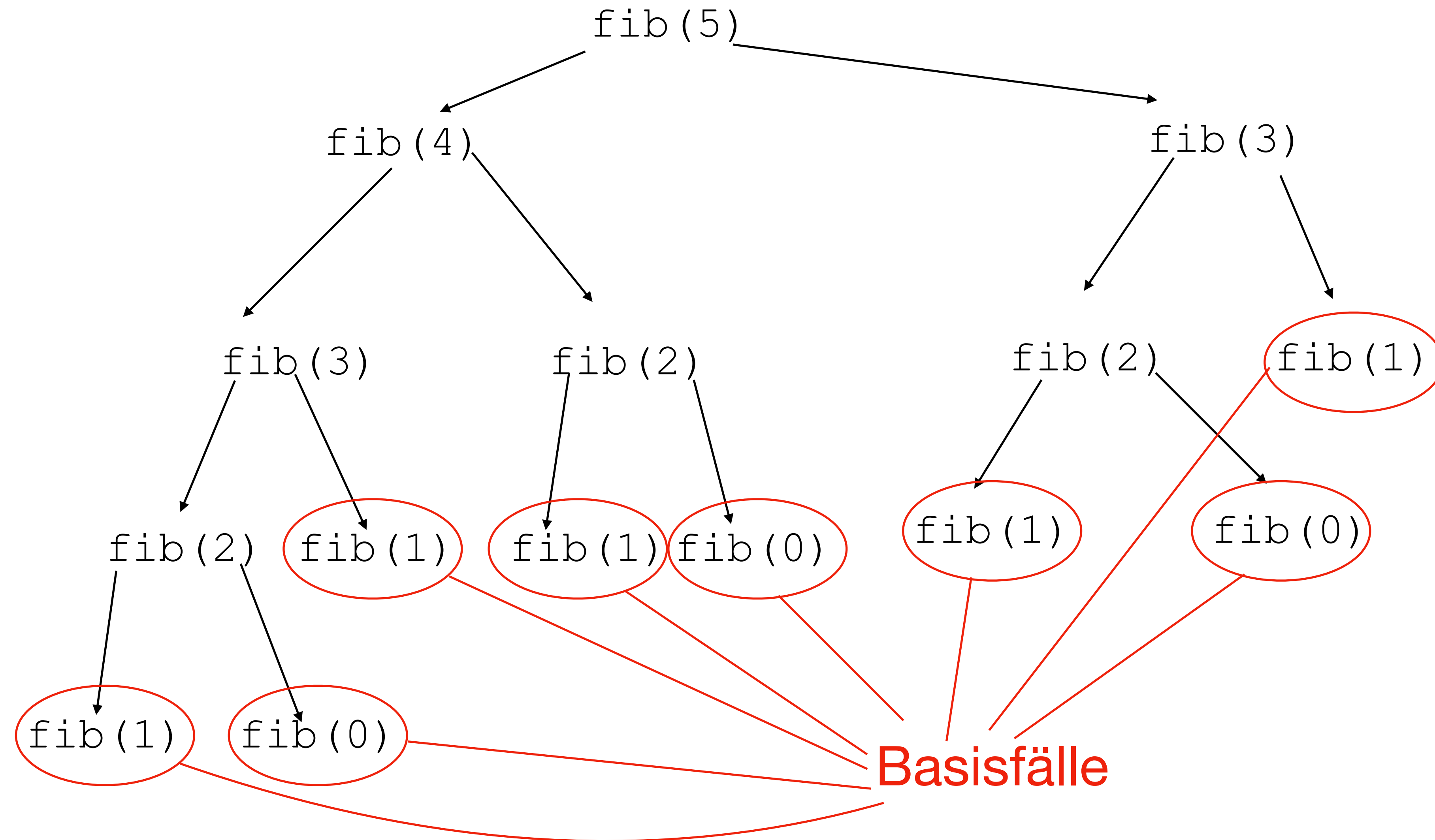
```
def fib(n):  
    if n==0 or n==1:  
        return 1  
    else:  
        return fib(n-1) + fib(n-2)
```

- zwei Basisfälle (n=0 und n=1)
- ruft sich zweimal rekursiv auf
- dieser Code ist ineffizient *Warum?*

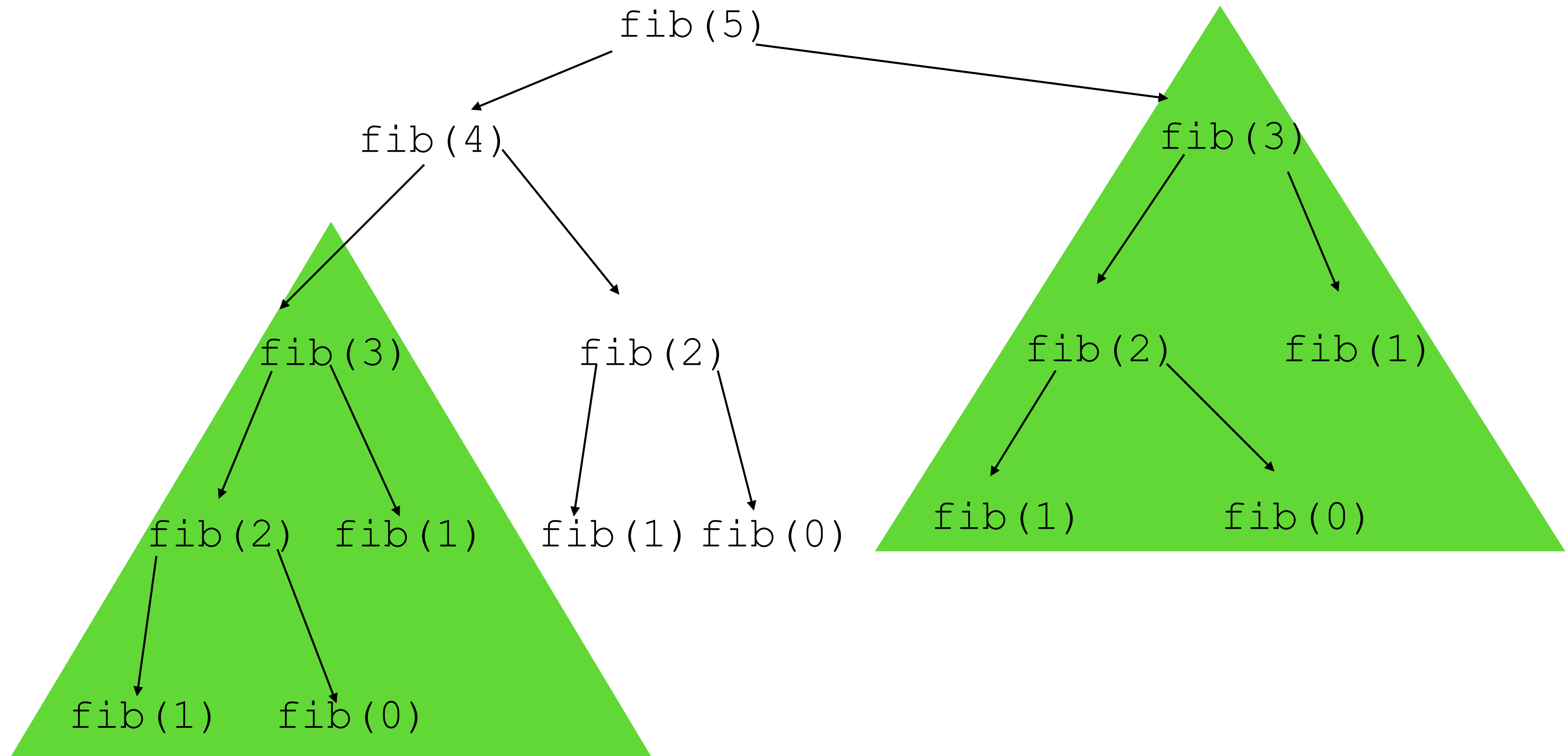
Ineffizienter Code



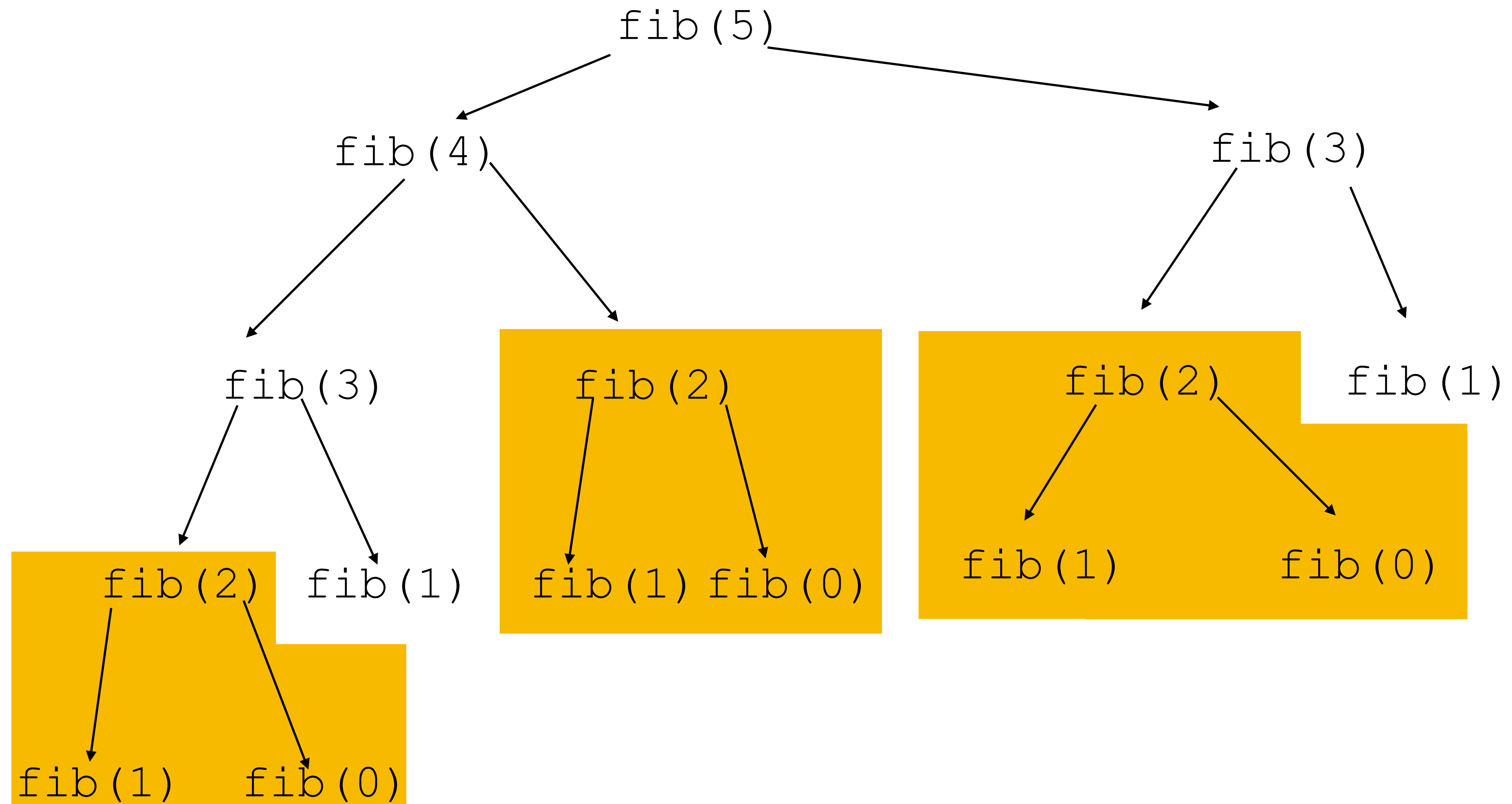
Ineffizienter Code



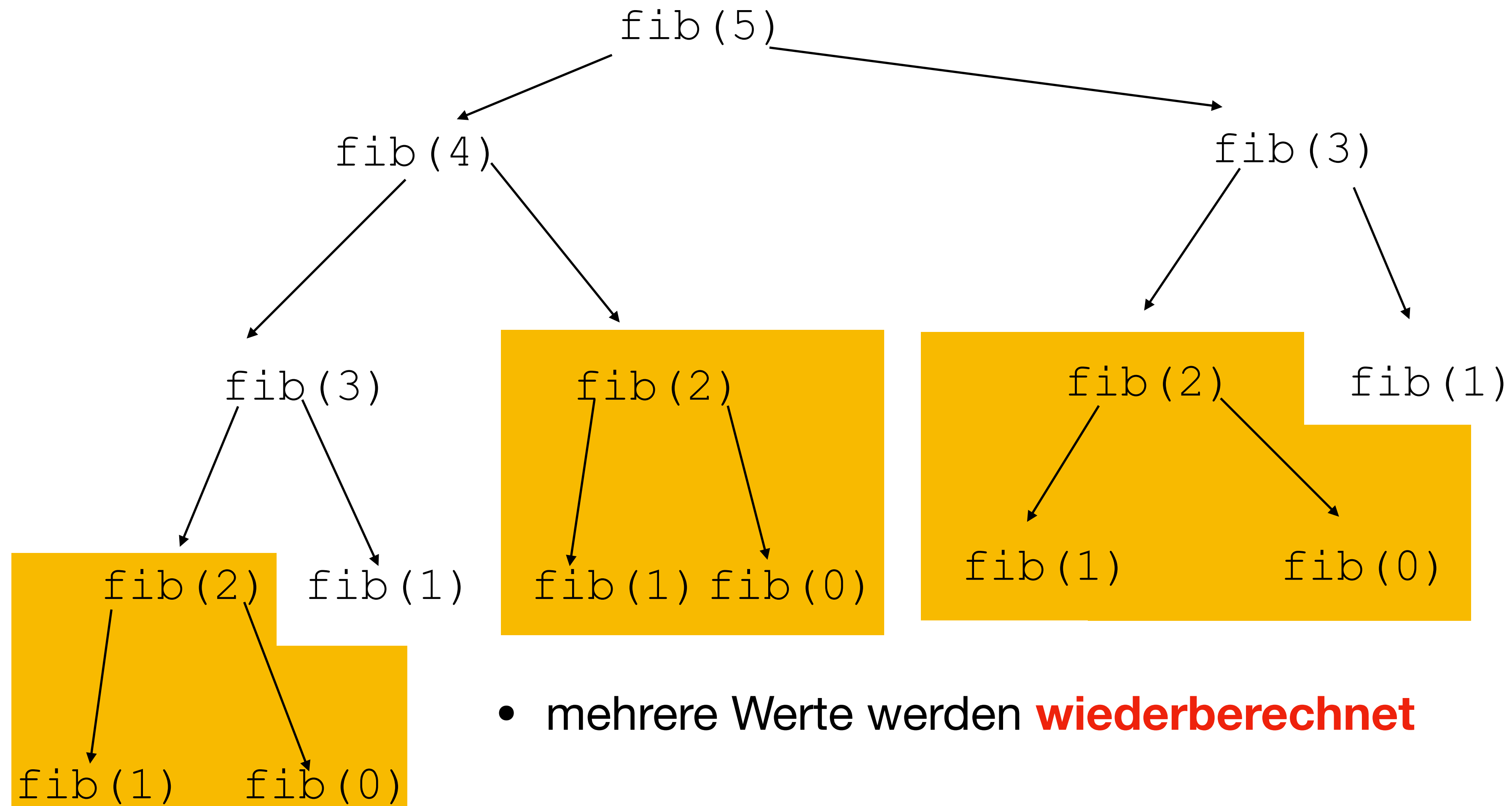
Ineffizienter Code



Ineffizienter Code



Ineffizienter Code



- mehrere Werte werden **wiederberechnet**
- besser wäre es zu **speichern** welche Werte bereits berechnet wurden

Fibonacci mit einem Wörterbuch

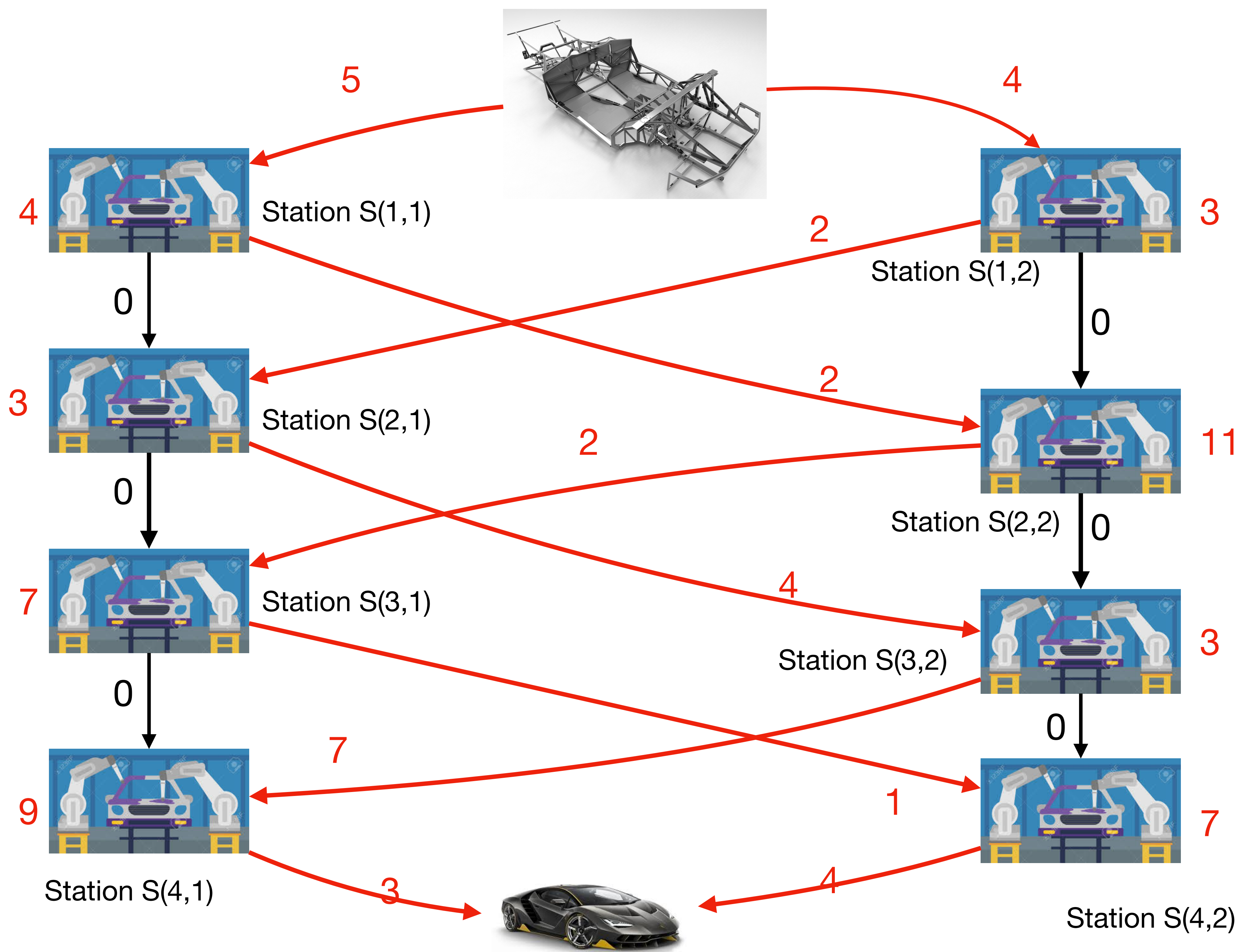
```
def fib_effizient(n,wb):  
    if n in wb:  
        return wb[n]  
    else:  
        antwort = fib_effizient(n-1,wb) + fib_effizient(n-2,wb)  
        wb[n] = antwort  
        return antwort
```

```
initial={0:1,1:1}  
n=input("Geben sie n ein: ")  
print(fib_effizient(int(n),initial))
```

Achtung: Da Wörterbücher veränderbar sind, hat diese Funktion Nebeneffekte auf das Wörterbuch *initial* im Hauptprogramm (aber genau das wollen wir!)

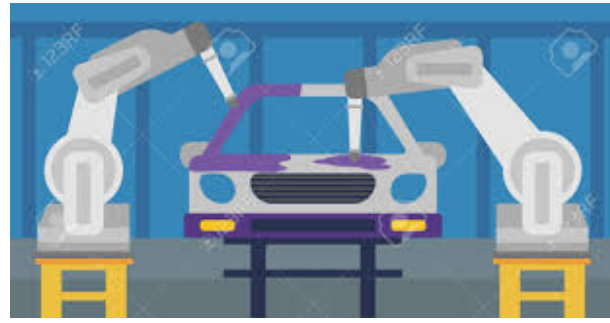
Initialisiere Wörterbuch

- **zuerst** im Wörterbuch **nachschlagen**
- falls Schlüssel nicht vorhanden, **trage** entsprechenden Wert **ins Wörterbuch ein**

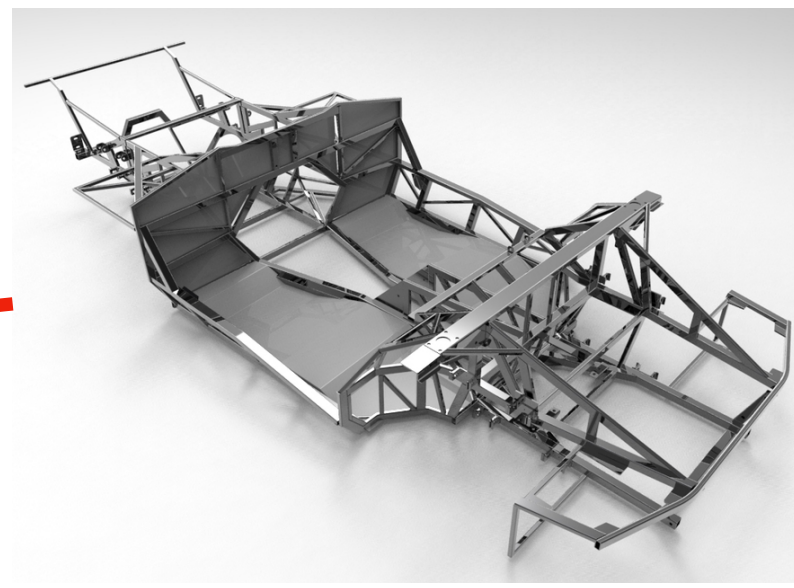




4



5

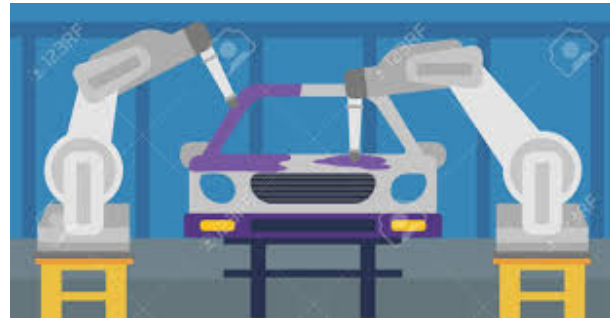


4



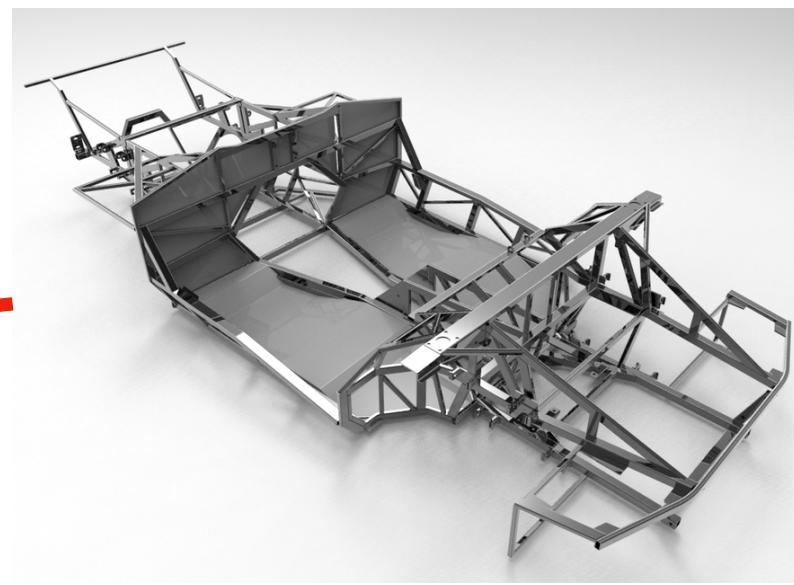
3

4



9

5

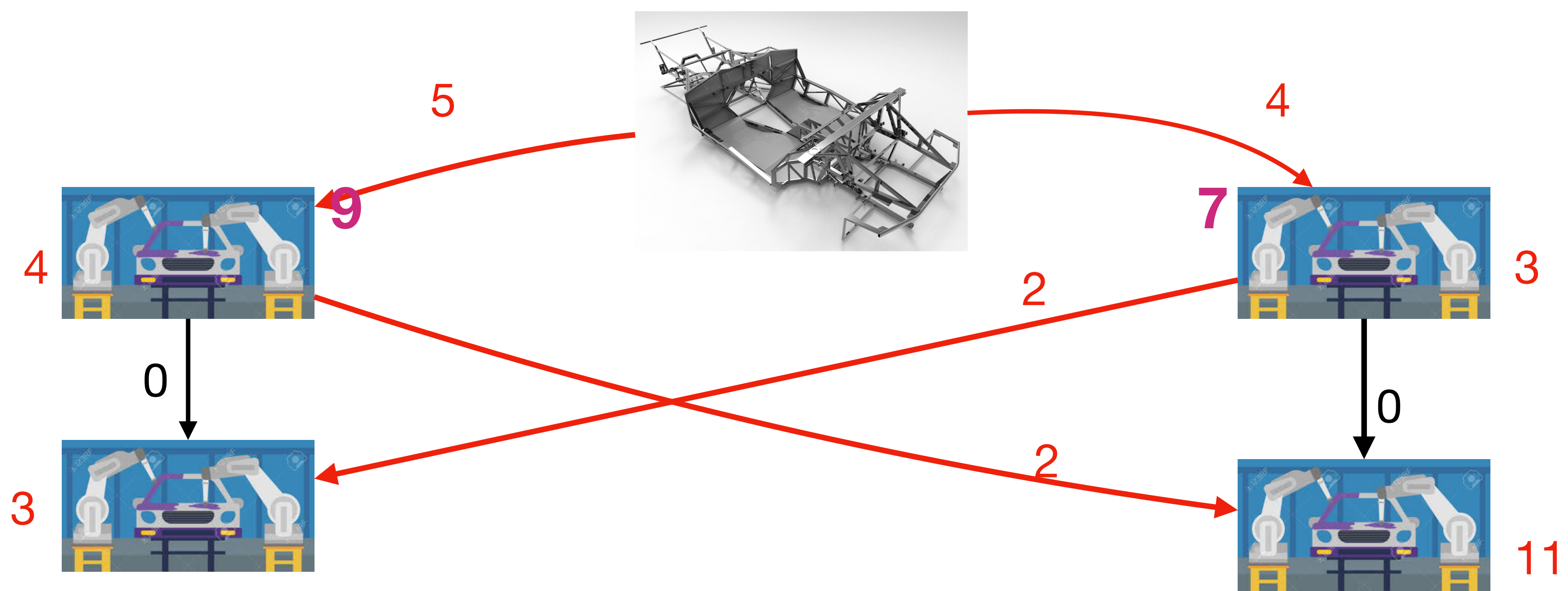


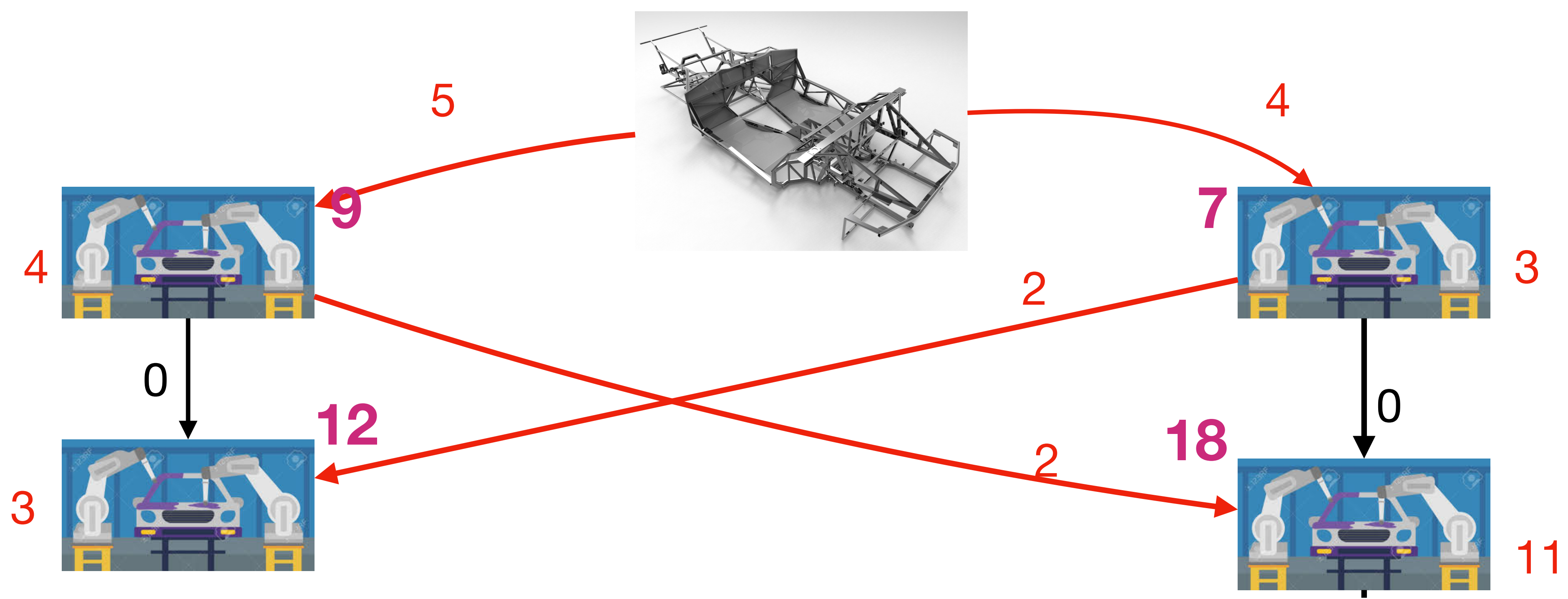
7

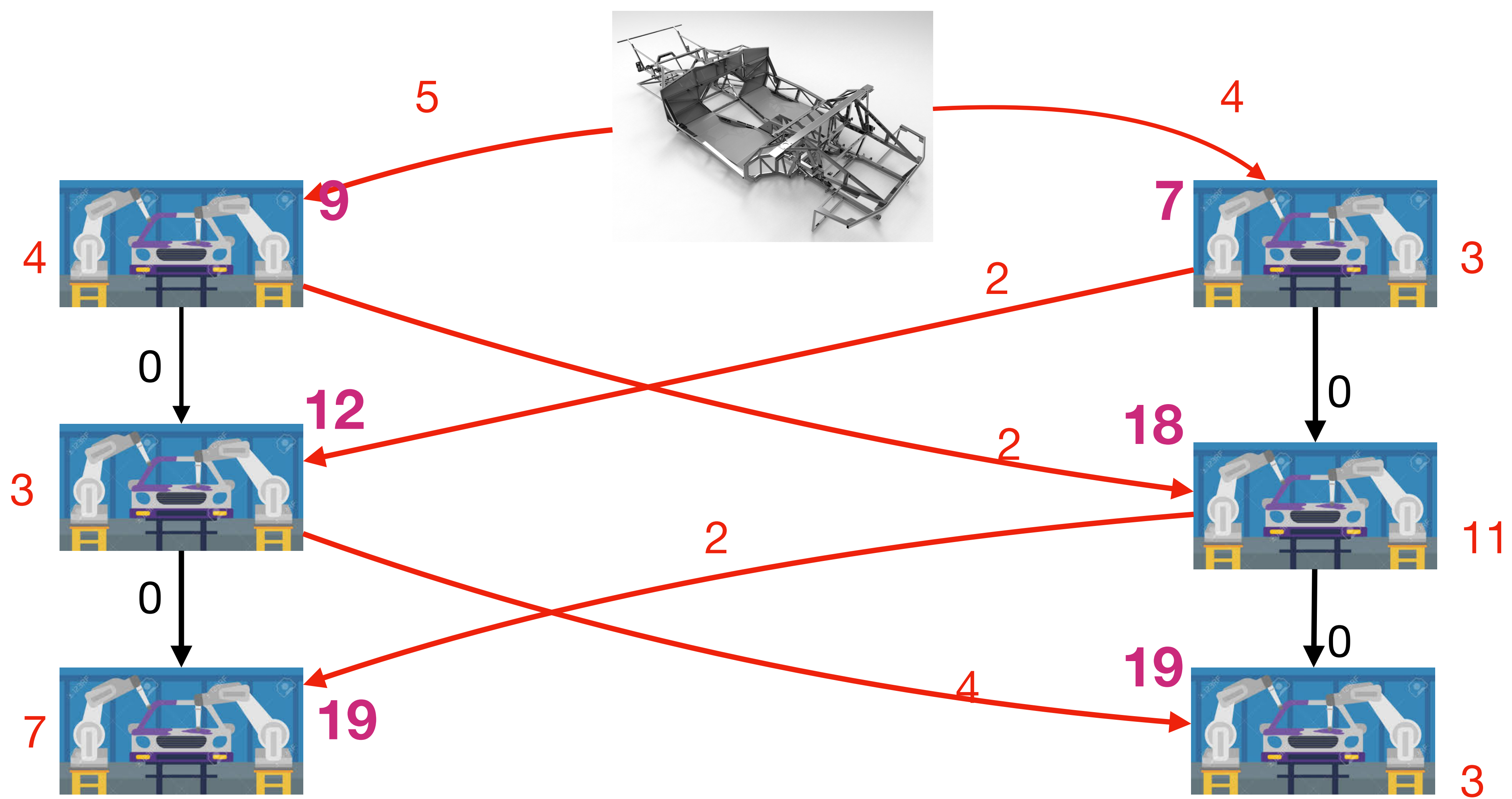


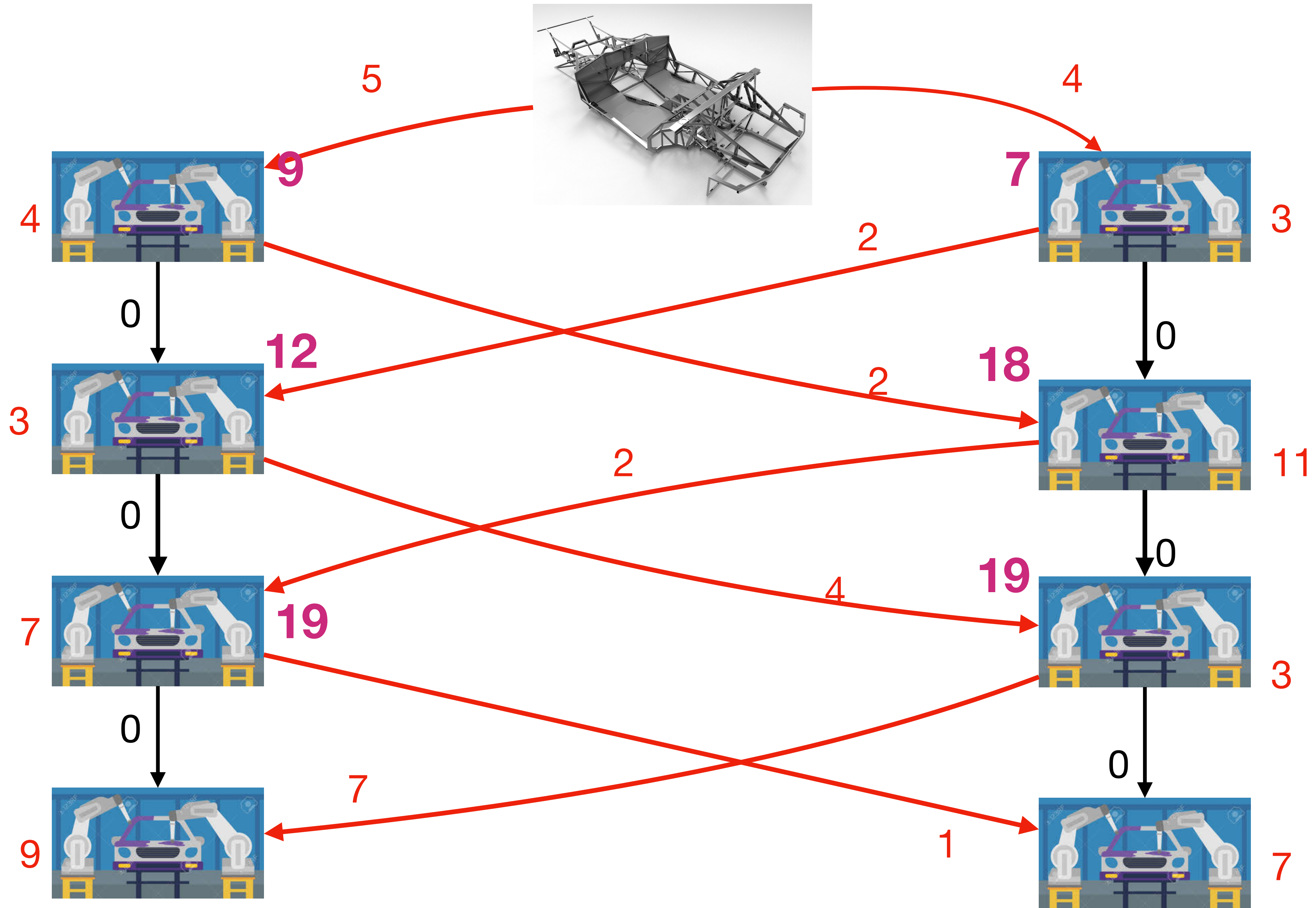
3

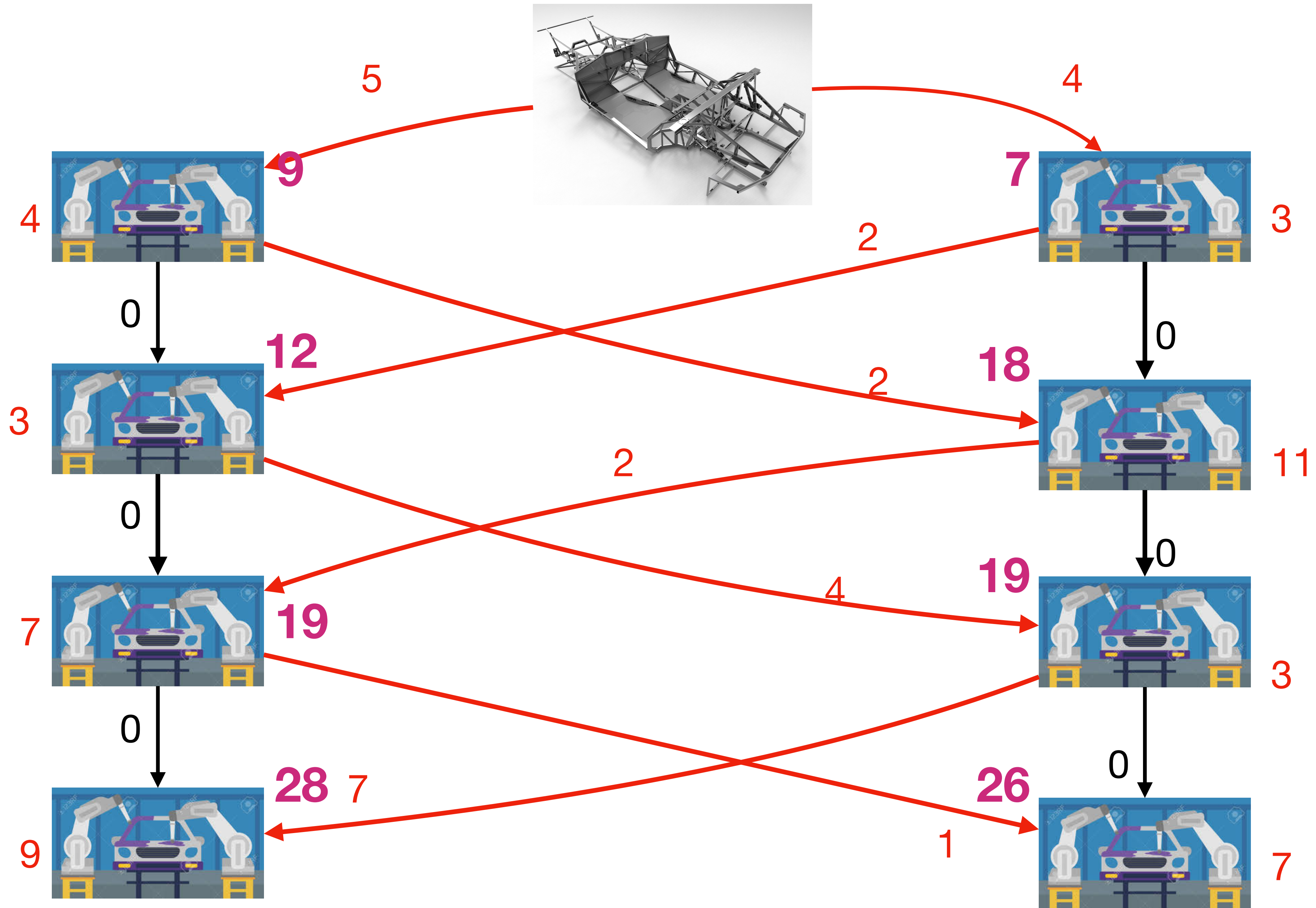
4

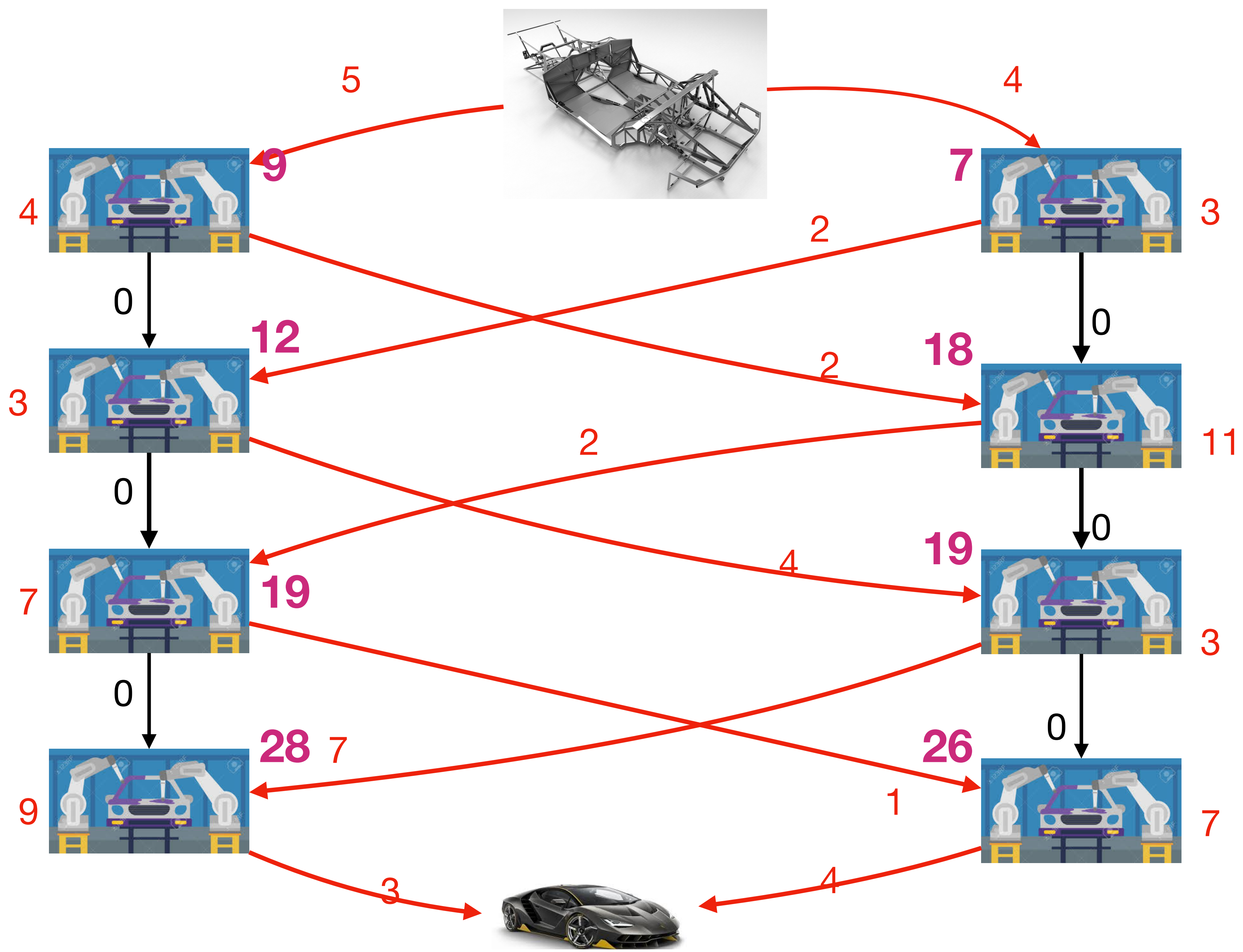


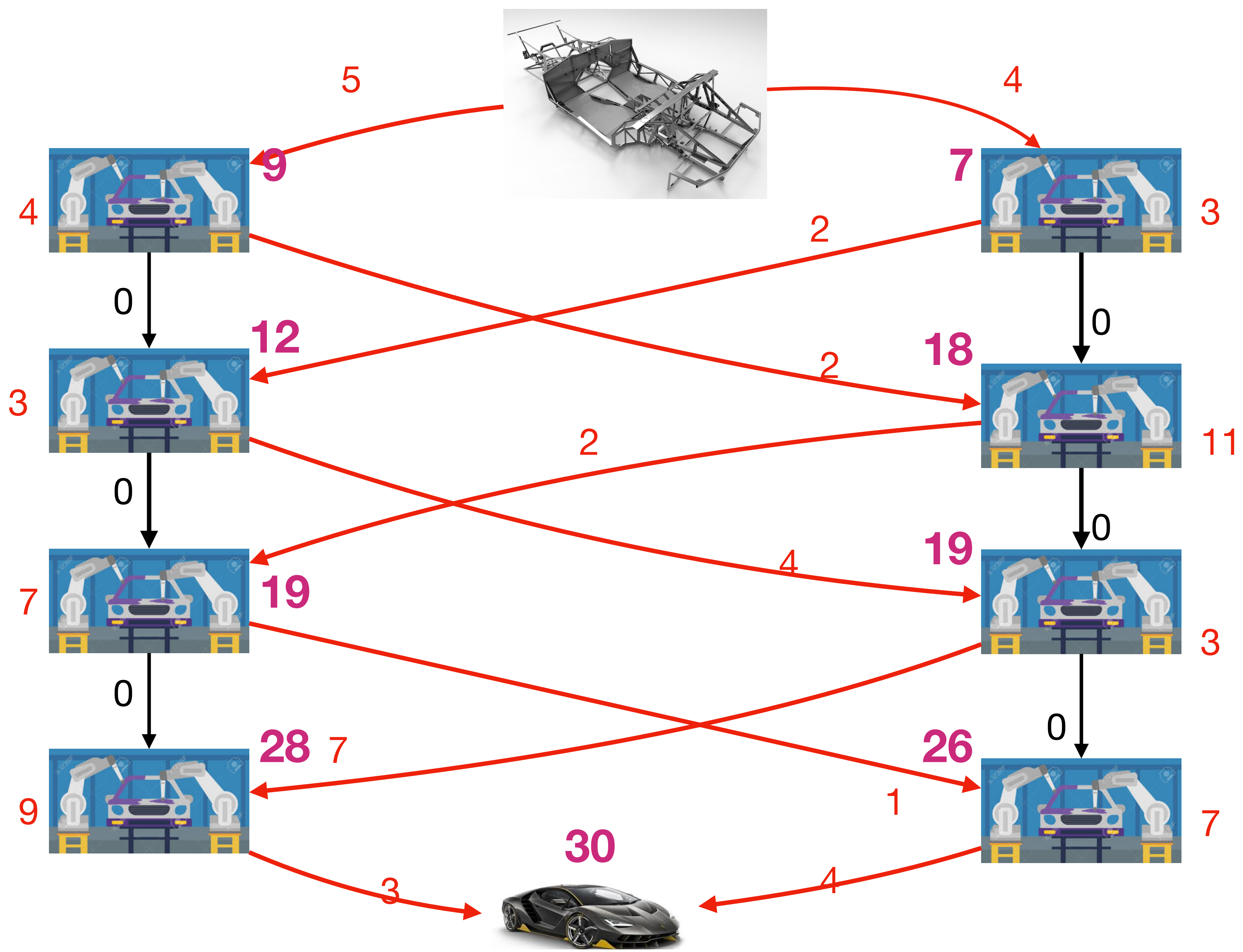


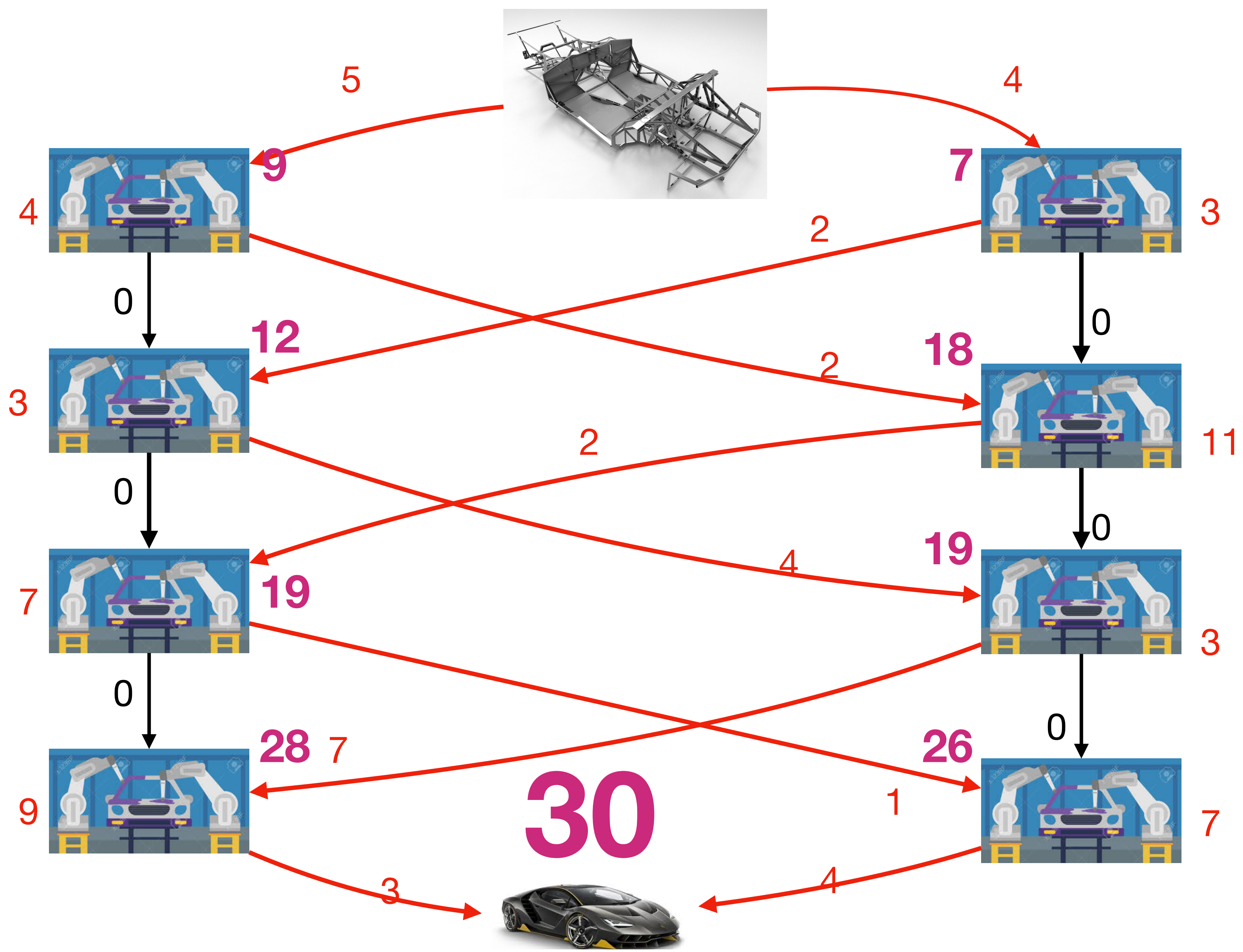






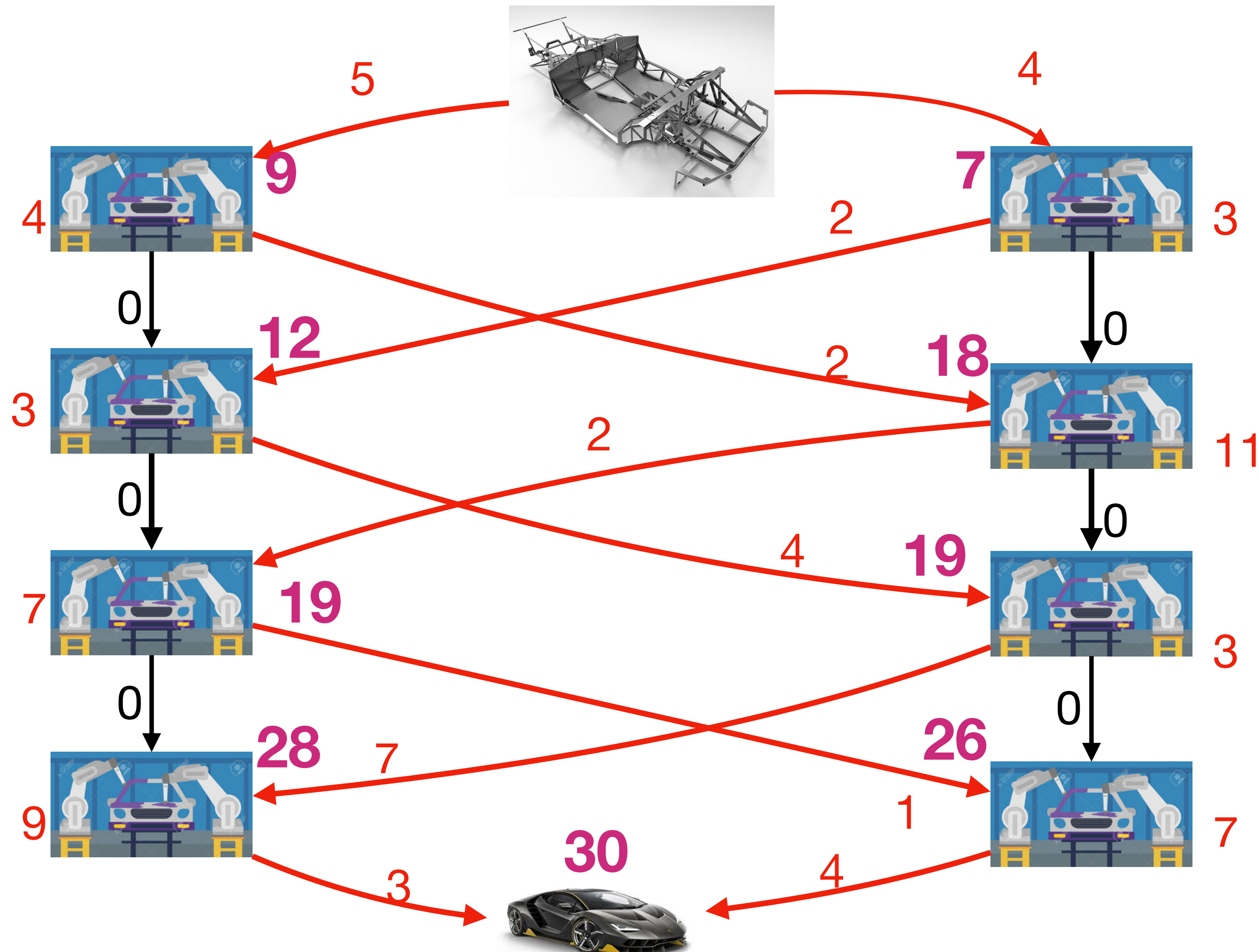






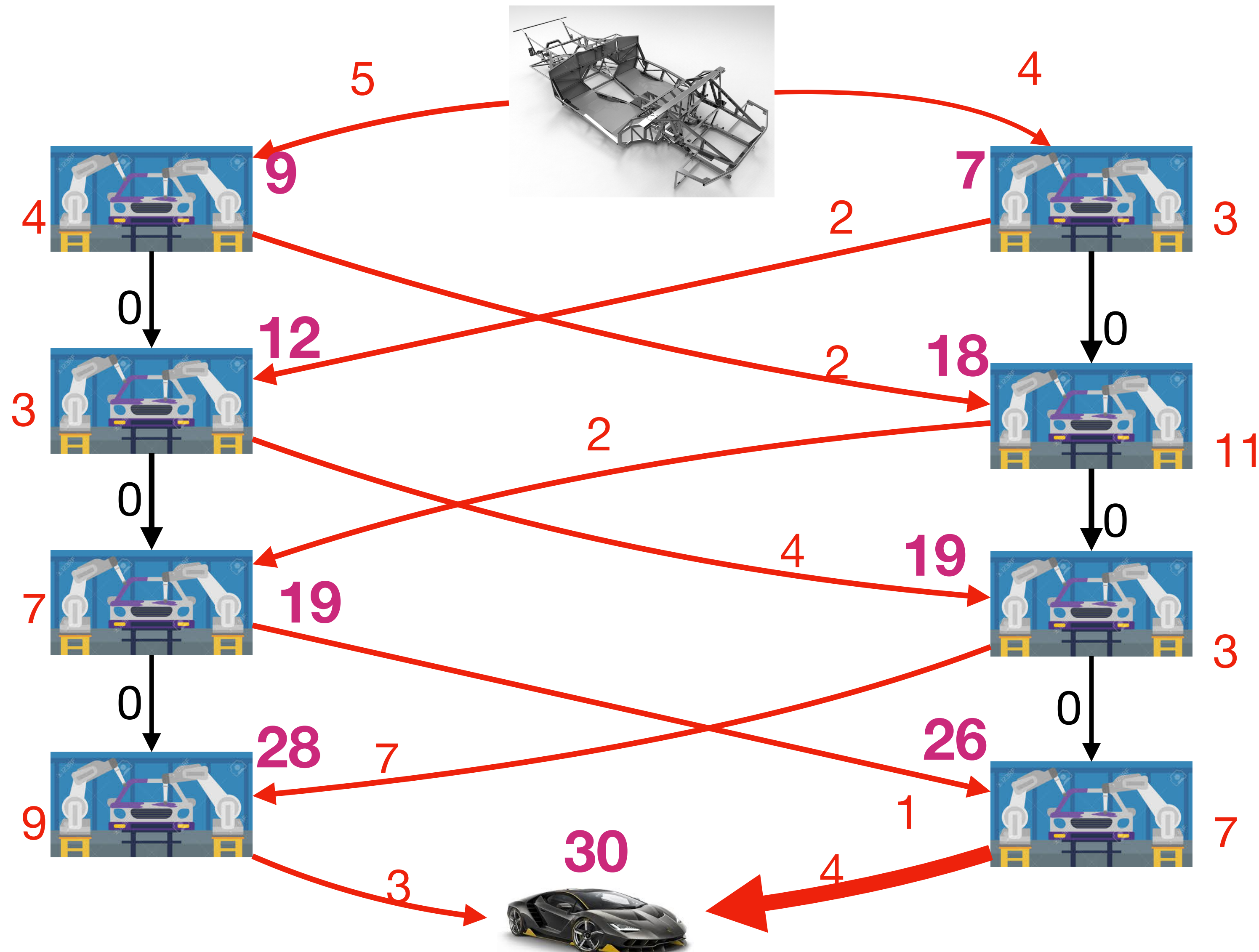
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



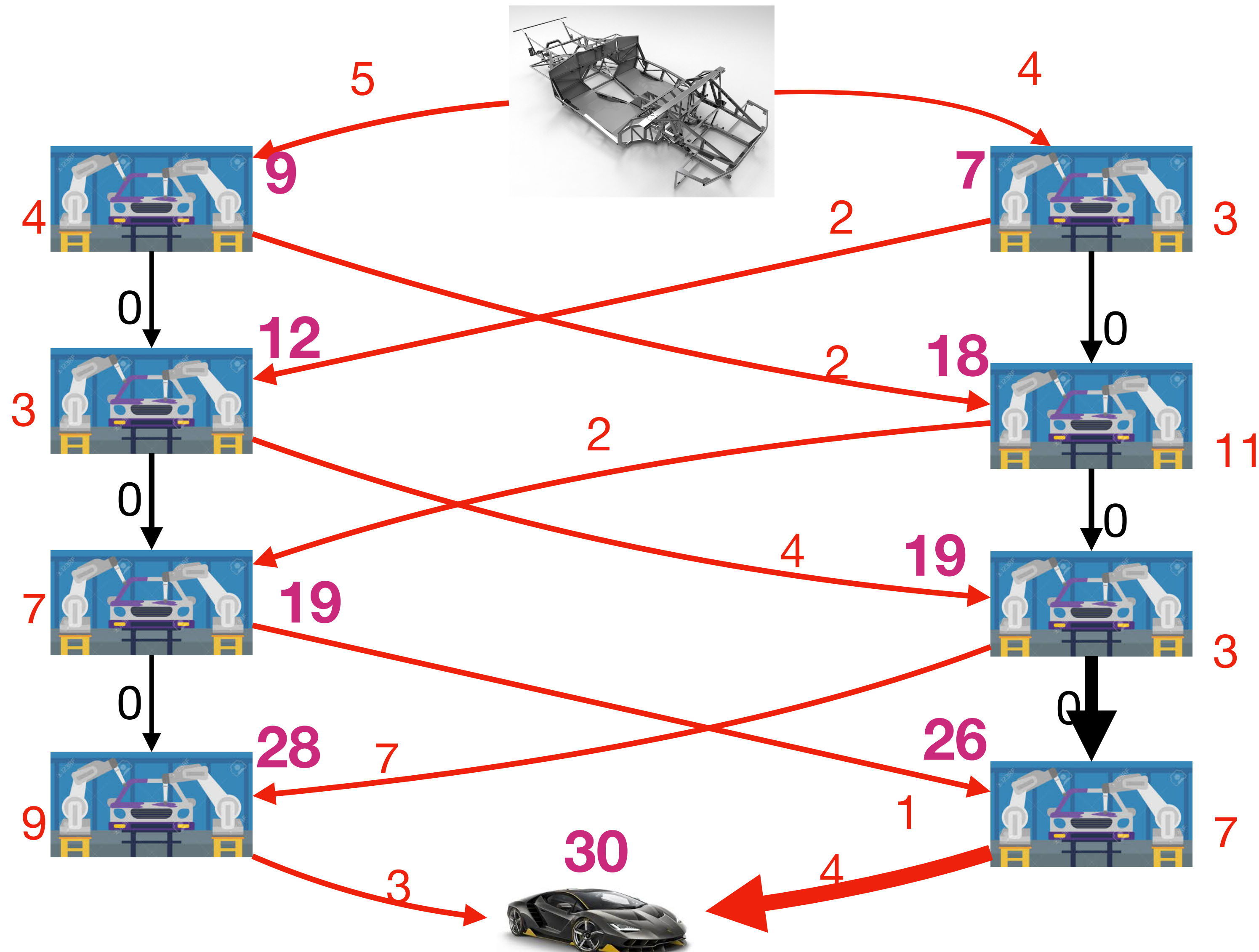
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



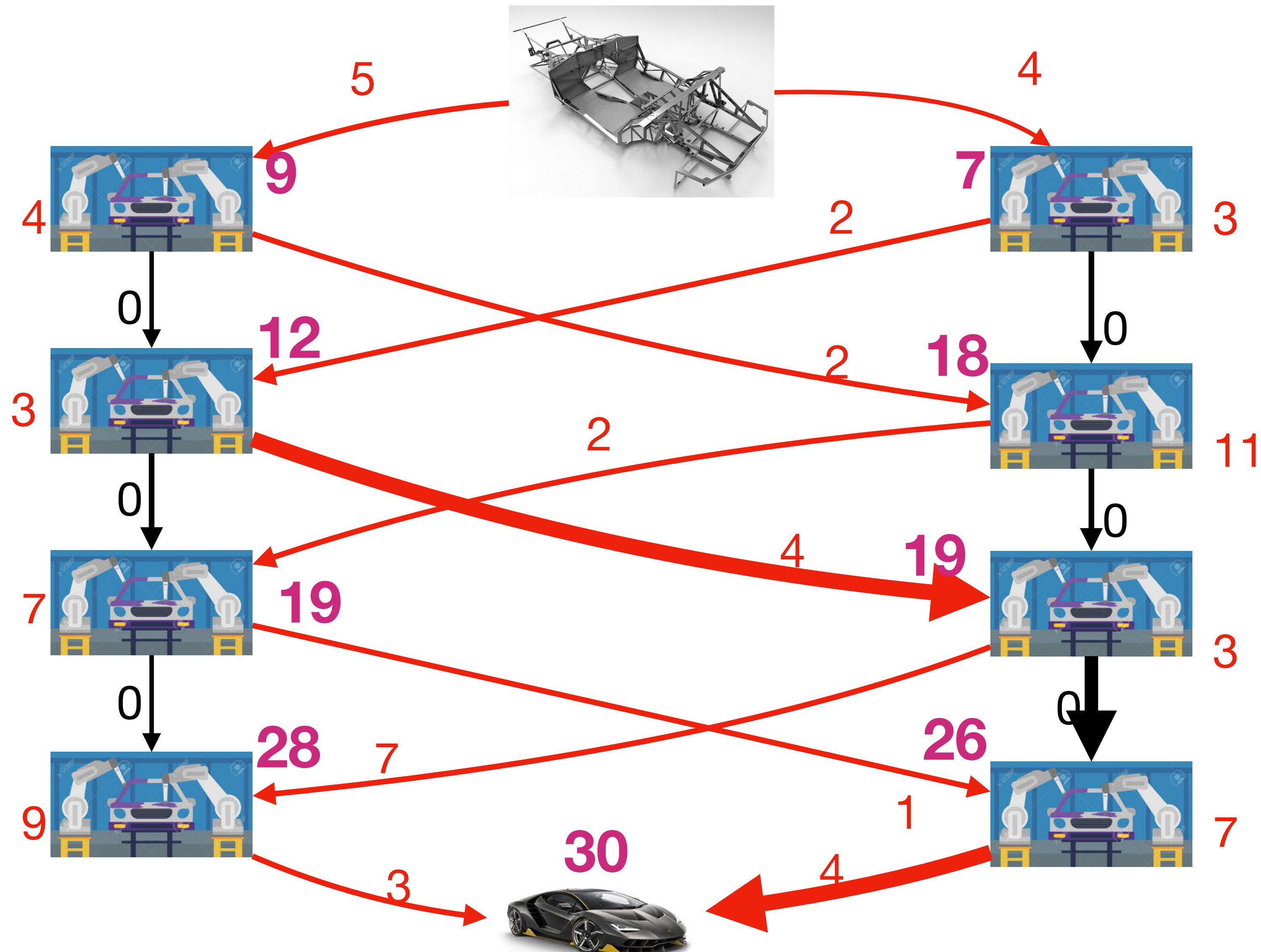
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



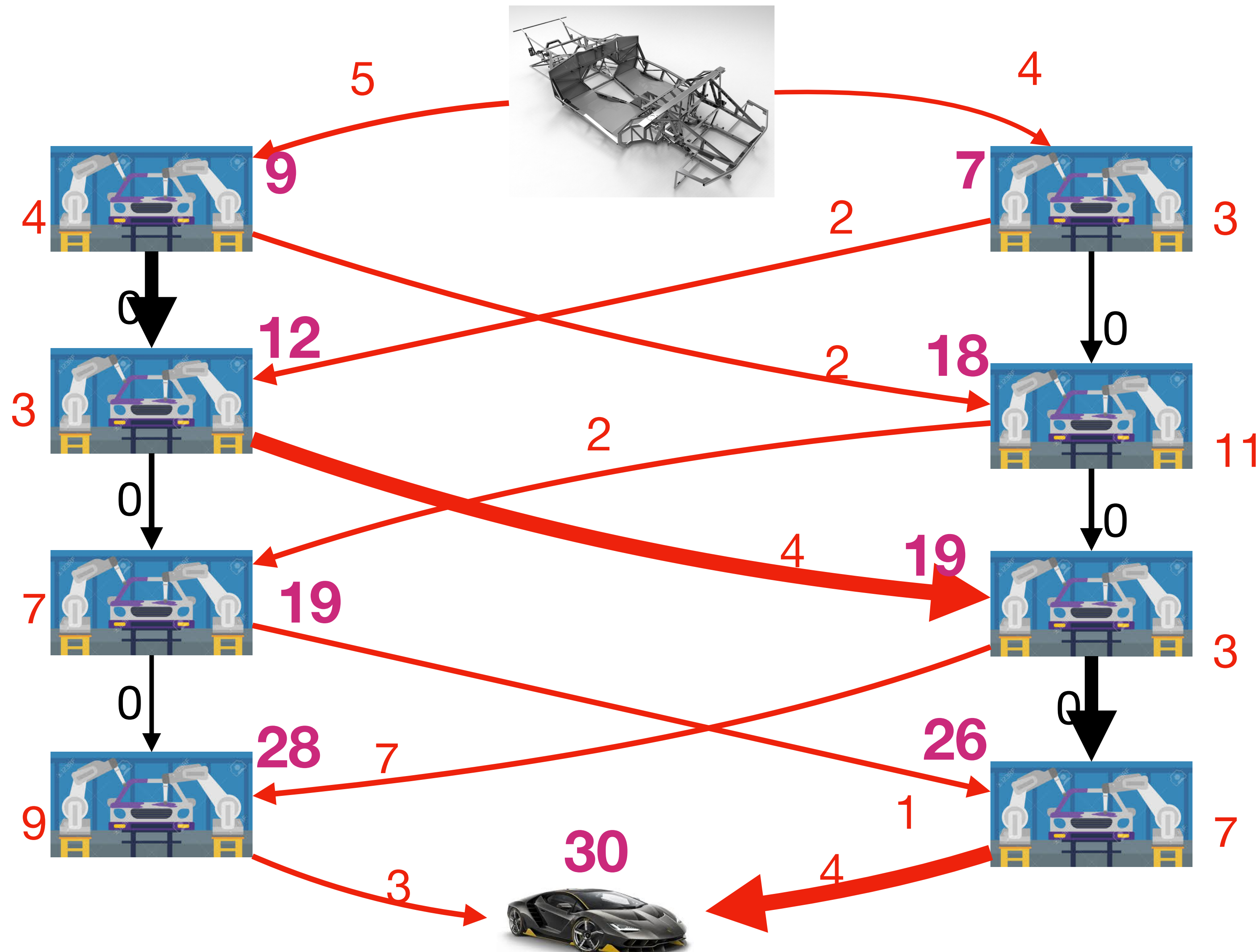
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



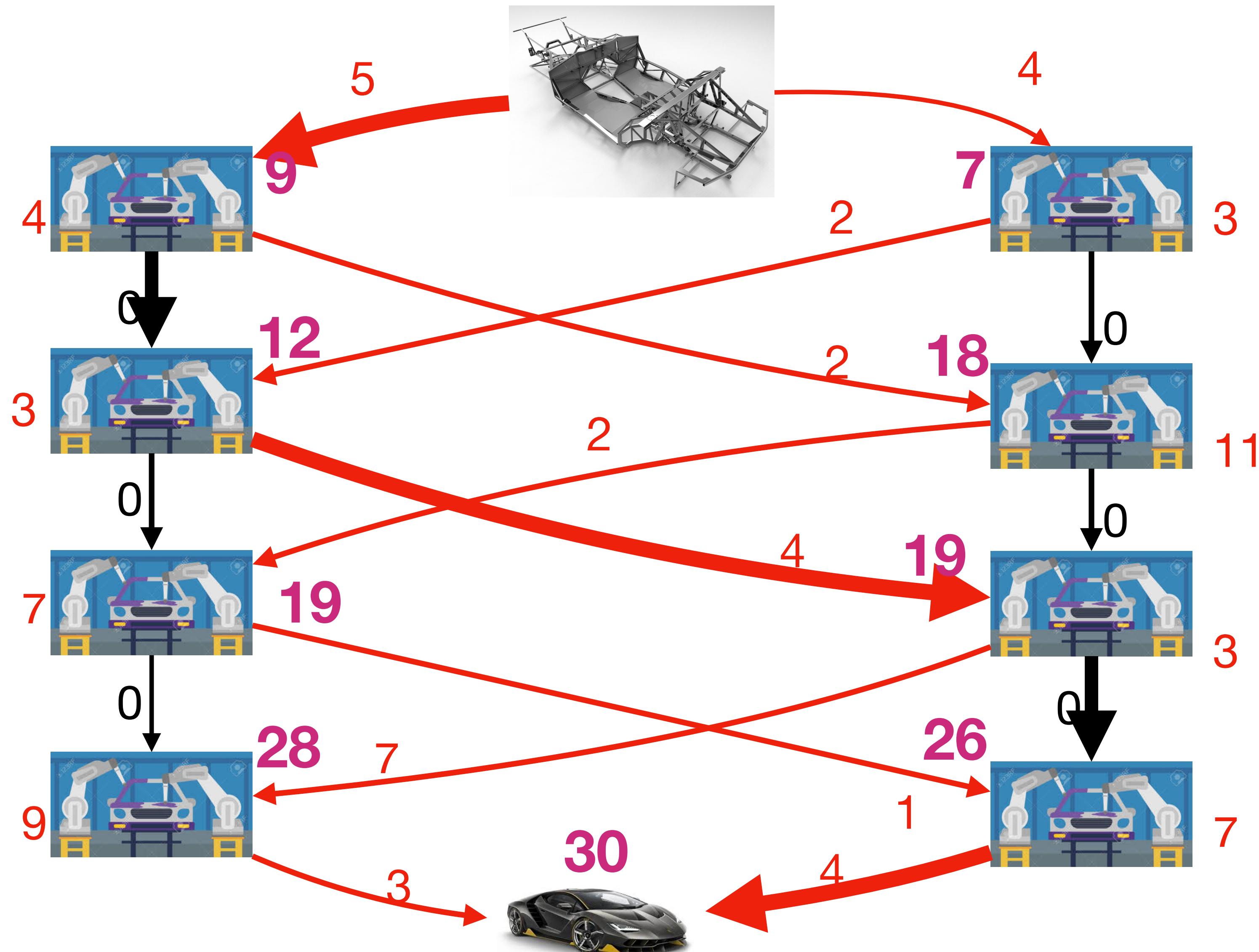
Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



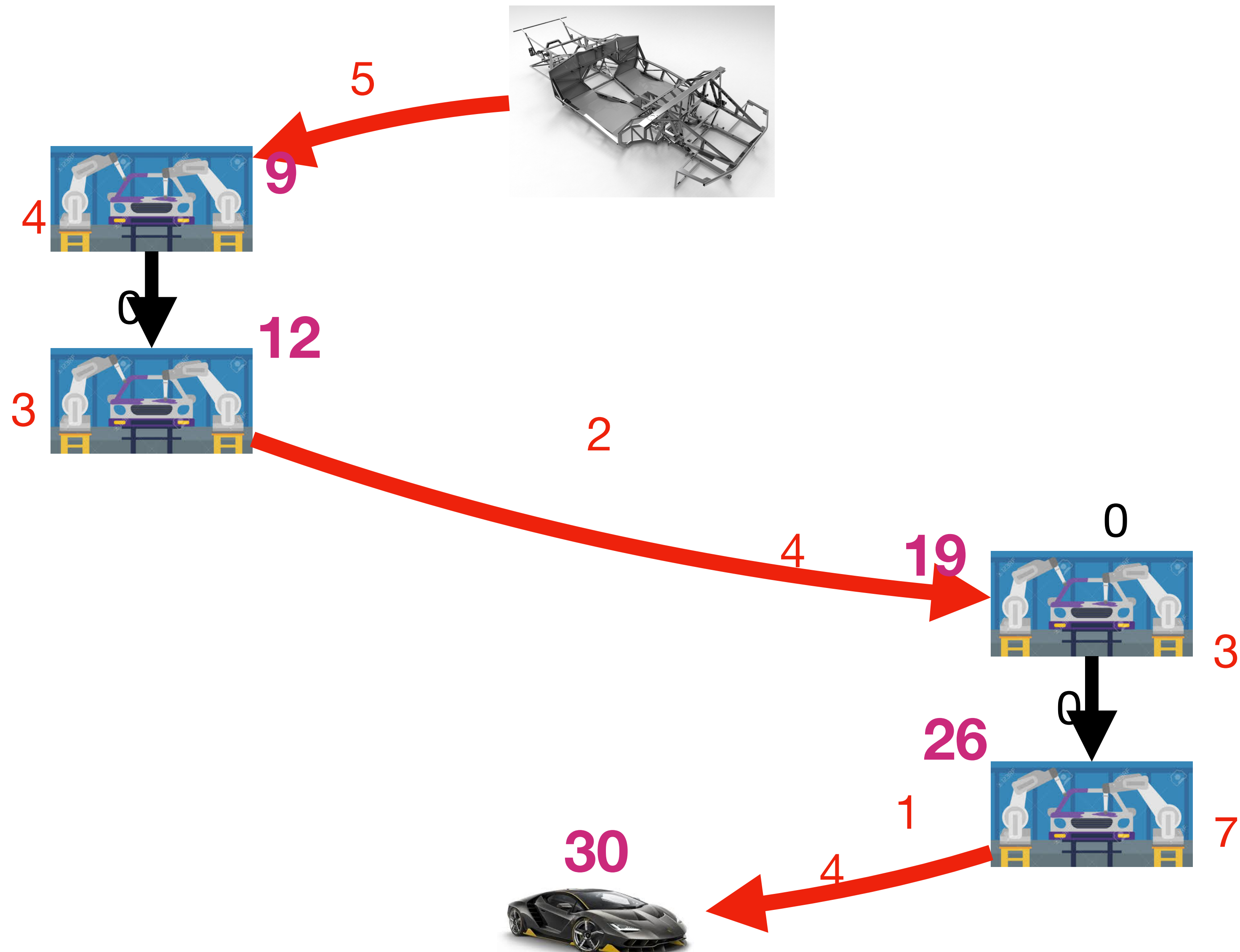
Struktur einer optimalen Lösung

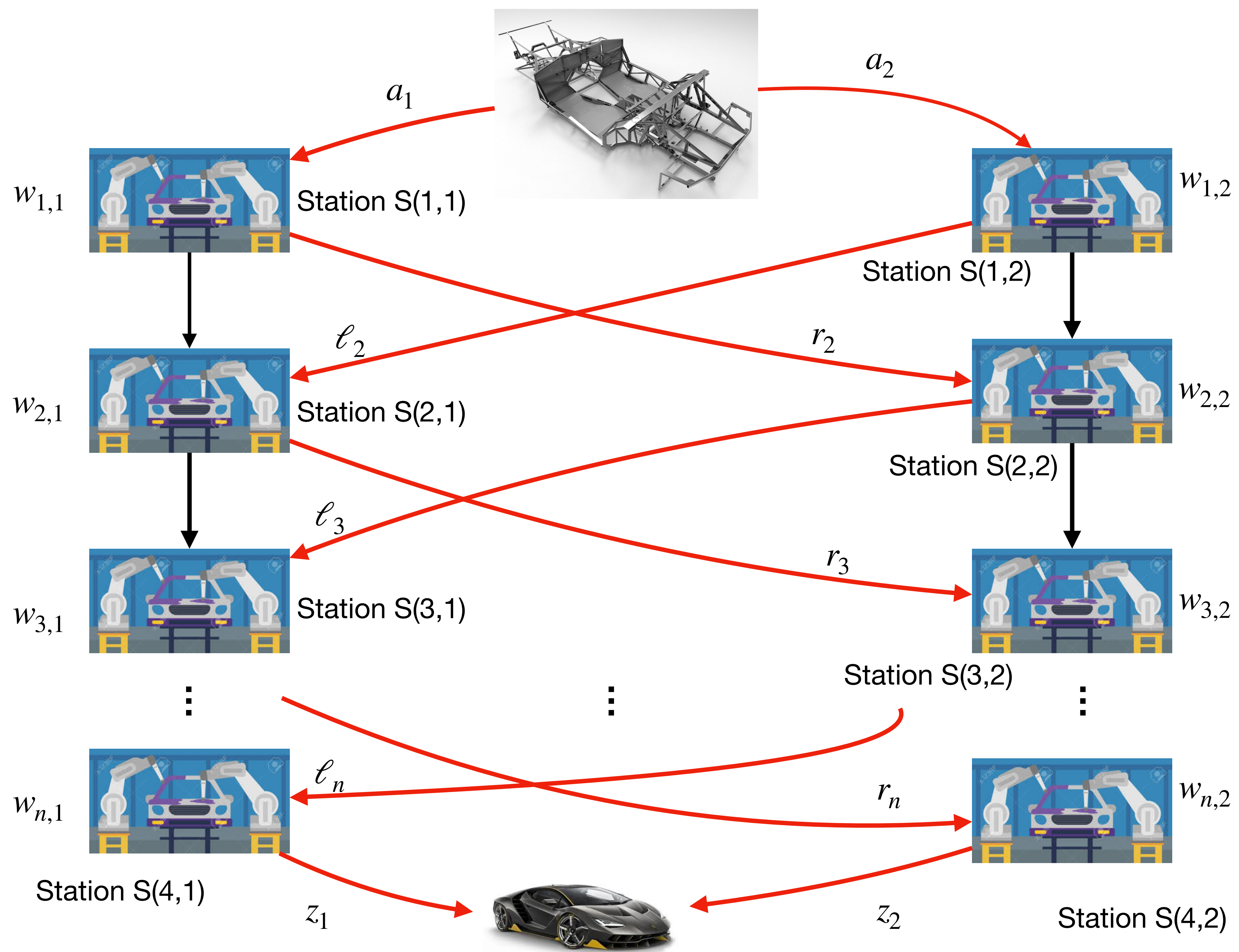
- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**



Struktur einer optimalen Lösung

- Hier: **Pfad durch Werkstatt mit minimaler Gesamtzeit**





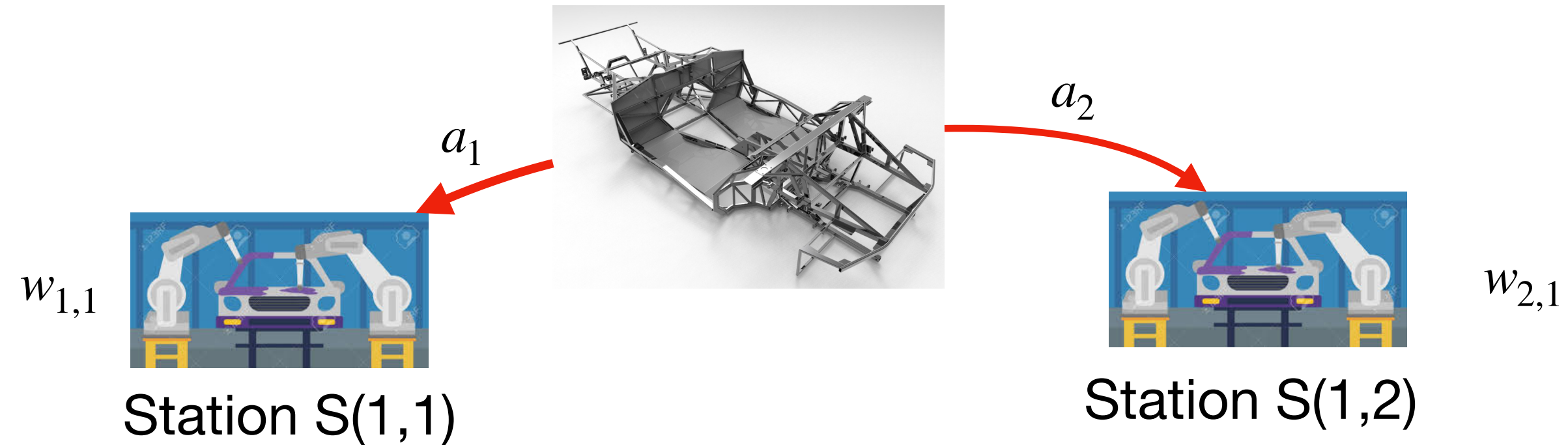
Lösung rekursiv beschreiben

- Bezeichne $m_{i,j}$ die Minimalzeit, bis Chassis an Stelle $S(i,j)$ zu fertig ist.

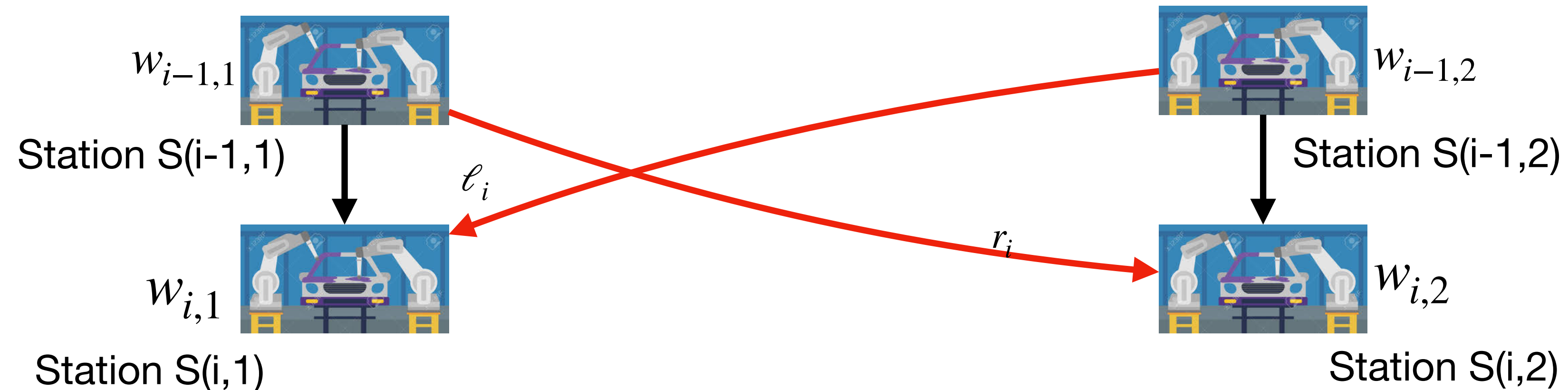
- Es gilt folgendes:

$$m_{1,1} = a_1 + w_{1,1}$$

$$m_{1,2} = a_2 + w_{1,2}$$



- Für $1 < i \leq n$ gilt:



$$m_{i,1} = \min (w_{i,1} + m_{i-1,1}, \quad w_{i,1} + \ell_i + m_{i-1,2})$$

$$m_{i,2} = \min (w_{i,2} + m_{i-1,2}, \quad w_{i,2} + r_i + m_{i-1,1})$$

Zufällige Eingabe erzeugen

m=10

n=3

Maximalwert für Zeitwerte

Anzahl der Stationen

a={ }

z={ }

l={ }

r={ }

w={ }

```
def init(a,z,l,r,w):  
    a={k:random.choice(range(1,m)) for k in range(1,3)}  
    z={k:random.choice(range(1,m)) for k in range(1,3)}  
    l={i:random.choice(range(1,m)) for i in range(1,n+1)}  
    r={i:random.choice(range(1,m)) for i in range(1,n+1)}  
    w={(i,k):random.choice(range(1,m)) for i in range(1,n+1) \  
        for k in range(1,3)}  
    return (a,z,l,r,w)
```


Zufällig erzeugte Eingabe ausgeben

```
def ausgabe():  
  
    print("a[1] = " + str(a[1]))  
    print("a[2] = " + str(a[2]))  
  
    for x in range(1,n+1):  
        print("l["+str(x)+"] = " + str(l[x]))  
        print("r["+str(x)+"] = " + str(r[x]))  
        print("w[("+str(x)+",1)] = " + str(w[(x,1)]))  
  
    print("z[1] = " + str(z[1]))  
    print("z[2] = " + str(z[2]))
```

Rekursive Berechnung

```
def loeseRekursiv(i,k):  
    if i==1:  
        return a[k] + w[(i,k)]  
    else:  
        h = l if k==1 else r  
        return w[i,k] + min(loeseRekursiv(i-1,k),  
                             h[i] + loeseRekursiv(i-1,3-k))
```

wir wollen entweder auf
Wörterbuch `l` oder auf
Wörterbuch `r` zugreifen

Beschleunigte rekursive Berechnung mit Wörterbuch

```
def loeseRekursivmitwb(wb,i,k):  
    if i==1:  
        return a[k] + w[(i,k)]  
    elif (i,k) in wb:  
        return wb[(i,k)]  
    else:  
        h = 1 if k==1 else r  
        wb[(i,k)] = w[i,k] + min(loeseRekursivmitwb(wb,i-1,k), \  
                                h[i] + loeseRekursivmitwb(wb,i-1,3-k))  
        return wb[(i,k)]
```